

SyncSort UNIX

Release 3

Reference Guide



SI-2605-5
3.6.2

© Syncsort Incorporated, 1999, 2001, 2002, 2003

All rights reserved. This document contains proprietary and confidential material, and is only for use by the lessees of the SyncSort proprietary software system. This publication may not be reproduced in whole or in part, in any form, except with written permission from Syncsort Incorporated. SyncSort and Visual SyncSort are trademarks of Syncsort Incorporated. No claim is made to the exclusive right to use Visual apart from the mark as shown. Windows NT and Windows 2000 are registered trademarks of Microsoft Corporation. All other company and product names used herein may be the trademarks of their respective companies.

Table of Contents

Chapter 1.	Introduction to SyncSort.....	1.1
	Performance	1.1
	Capabilities	1.1
	Invocation	1.6
Chapter 2.	Invoking SyncSort.....	2.1
	2.1 The syncsort Command.....	2.2
	2.2 The SyncSort Subroutines.....	2.7
	sync_load_application	2.19
	sync_set_source_file	2.21
	sync_add_source_file	2.23
	sync_set_target_file	2.25
	sync_set_source_buffer.....	2.27
	sync_set_target_buffer	2.29
	sync_set_input_data_bytes	2.30
	sync_set_input_record_count.....	2.32
	sync_run	2.34
	sync_put_record	2.36
	sync_put_record_end	2.38
	sync_get_record	2.39
	sync_clear.....	2.41
	sync_get_statistic	2.43
	sync_get_highest_severity	2.47
	sync_get_error_number.....	2.49
	sync_get_error_message	2.50
	sync_get_library_version	2.51
	sync_add_key.....	2.52
	sync_add_positional_key.....	2.55
	sync_add_delimited_key	2.58
	sync_add_datetime_format	2.61
	sync_set_numeric_value	2.63
	sync_set_text_value	2.65
	sync_select_join_side.....	2.67
	sync_set_source_reader	2.69
	sync_set_target_writer.....	2.72

	Program Invocation Examples.....	2.76
2.3	The SyncSort Korn Shell Interface	2.100
	SyncNew.....	2.104
	SyncSetApplicationFileName	2.105
	SyncSetEstimatedInputRecordCount	2.106
	SyncSetEstimatedInputDataBytes.....	2.107
	SyncSetSourceFile	2.108
	SyncAddSourceFile	2.109
	SyncSetTargetFile	2.110
	SyncSetNumericValue.....	2.111
	SyncSetTextValue	2.112
	SyncSelectJoinSide	2.113
	SyncRun	2.115
	SyncClear	2.116
	SyncSort Korn Shell Examples.....	2.117
2.4	The Micro Focus COBOL SORT and MERGE Verbs	2.120
	Micro Focus COBOL Program Example.....	2.123
2.5	The SAS Sort Procedure	2.126
2.6	The IBM DB2 Load Accelerator	2.127
2.7	Interrupting SyncSort	2.129
2.8	Disk Full Recovery.....	2.131

Chapter 3.	SyncSort Options.....	3.1
	/AGGREGATE.....	3.2
	/BEGIN.....	3.9
	/CENTURYWINDOWSTART	3.10
	/CHECKSEQUENCE.....	3.12
	/COLLATINGSEQUENCE.....	3.14
	/COLLECT	3.21
	/COLUMNS.....	3.23
	/CONDITION.....	3.28
	/COPY.....	3.36
	/DATADictionary	3.37
	/DATASIZE	3.39
	/DELETE.....	3.41
	/DELIMITEDRECORDLAYOUT	3.42
	/DERIVEDFIELD	3.48
	/DISPLAY.....	3.63
	/DOCUMENTATION.....	3.64
	/EDIT.....	3.67
	/END	3.68

/FIELDS	3.69
/IGNORE.....	3.77
/INCLUDE.....	3.79
/INFILE.....	3.82
/INSERT.....	3.91
/JOIN.....	3.94
/JOINKEYS	3.97
/KEYS	3.100
/MEMORY.....	3.103
/MERGE.....	3.105
/NOEXECUTE	3.106
/NOINTERRUPT.....	3.107
/NOPROMPT	3.109
/OMIT	3.111
/OUTFILE.....	3.113
/PADBYTE	3.125
/PADKEY	3.127
/PADRECORD.....	3.129
/PAGEFOOTER.....	3.131
/PAGEHEADER.....	3.136
/QUIT	3.140
/RECORDLAYOUT.....	3.141
/REFORMAT.....	3.148
/REPORT	3.154
/SAVE	3.158
/SECTION.....	3.160
/SECTIONFOOTER.....	3.163
/SECTIONHEADER.....	3.169
/SILENT	3.173
/SKIPBLANK.....	3.174
/SKIPRECORD	3.176
/STABLE	3.178
/STATISTICS	3.180
/STOPAFTER	3.183
/SUMMARIZE	3.185
/TITLE	3.188
/TRAILER.....	3.192
/WARNINGS	3.197
/WORKSPACE	3.199

Chapter 4.	The SyncSort Message Utility.....	4.1
Appendix A.	Data Types	A.1
	Data Types and Lengths Supported by SyncSort	A.1
	Description of Supported Data Types	A.4
Appendix B.	Constants and Identifiers.....	B.1
	Specifying Character String Constants	B.1
	Specifying Numeric Constants	B.4
	Specifying Bit String Constants	B.6
	Identifiers	B.7
	Specifying Field Names.....	B.9
Appendix C.	Collating Sequences.....	C.1
	ASCII Collating Sequence	C.1
	Folded ASCII Collating Sequence	C.10
	EBCDIC Collating Sequence.....	C.17
	Folded EBCDIC Collating Sequence.....	C.22
	Multinational Collating Sequence	C.26
Appendix D.	Record Formats	D.1
	Record Formats.....	D.1
	Description of Supported Record Formats	D.2
Appendix E.	Masks	E.1
	Edit Specifications for Numeric Fields	E.1
	Replacement.....	E.1
	Suppressing Leading Zeroes.....	E.3
	Sign Indicator	E.5
	Currency indicator.....	E.7
	Combining Sign and Currency Indicators	E.8
	Default Numeric Edit Mask	E.10
	Edit Specifications for Date Fields.....	E.11
	Default Date Edit Mask	E.14
	Masks to Extract Sub-fields.....	E.15
	Regular Expressions	E.15
	Single-character matching	E.15
	List	E.17
	Concatenation.....	E.19
	Grouping	E.20
	Repetition.....	E.20

	Alternation.....	E.22
	Anchoring.....	E.23
	Precedence.....	E.24
	Context for special characters	E.24
	Pattern matching.....	E.26
	Functions Library	E.27
	Source Date Patterns	E.27
Appendix F.	Arithmetic Expressions.....	F.1
	Arithmetic Operators.....	F.1
	Order of Applying the Operators	F.2
	Data Type and Precision of the Result of Arithmetic Expressions.....	F.4
	Constant.....	F.4
	Operation	F.4
Appendix G.	Technical Support.....	G.1

Documentation Conventions

The following conventions are used in the format statements in this manual.

Convention	Explanation	Example
Regular type	Items in regular type must be entered literally using either lowercase or uppercase letters. Items may be abbreviated. The names for SyncSort subroutines, which begin with the prefix sync_ , must always be entered in lowercase letters. The SyncSort constants which begin with the prefix SYNC_ must always be entered in uppercase letters.	ASCII ascending sync_run SYNC_SUCCESS
<i>Italics (non-bold)</i>	Items in italics (non-bold) represent variables. You must substitute an appropriate numerical or text value for the variable.	<i>file_name</i>
Braces { }	Braces indicate that a choice must be made among items contained in the braces.	<code>{"a" }</code> <code>{X"xx" }</code> <code>{O"ooo" }</code>
Brackets []	Brackets indicate that an item is optional. A choice may be made among multiple items contained in brackets.	[<i>alias</i>]
Slash /	A slash identifies a SyncSort option keyword. The slash must be included when an option keyword is specified.	/INFILE /infile
Double quotes " "	Double quotation marks that appear in a format statement must be specified literally.	"b"-"e"
Ellipsis ...	An ellipsis indicates that the preceding argument or group of arguments may be repeated.	[<i>proc_qual...</i>]

Convention	Explanation	Example
Sequence number	A sequence number indicates that a series of arguments or values may be specified. The sequence number itself must never be specified.	<i>field2</i>

Chapter 1. Introduction to SyncSort

SyncSort is a high performance sort/merge/copy/join utility. It handles a wide variety of file, record, and data formats, provides general purpose record level processing for file and data conversion, and minimizes sorting time for large volumes of data.

Performance

SyncSort reduces sort execution times while improving overall system throughput. Proprietary sorting algorithms, I/O optimization, parallel processing and dynamic environmental monitoring techniques result in significantly reduced CPU time, elapsed time and disk I/O activity.

Capabilities

SyncSort provides the following features:

Input and Output

- performs a sort, merge (/MERGE), join (/JOINKEY), or copy (/COPY)
- processes sequential or indexed files (/INFILE), records passed from an input procedure, records passed in a table in memory, or records passed in from standard input
- accepts records of up to 64K bytes with fixed, variable, stream, Fortran unformatted, Micro Focus line sequential, and Micro Focus variable format
- generates one or more sequential or indexed files or an ADDROUT file (/OUTFILE)
- returns records one at a time to an output procedure, returns records in a table in memory or returns records to standard output
- splits output to multiple files (**syncsplit** command)

-
- automatically generates record numbers for output processing (/OUTFILE).

Sort Key Processing

- sorts on an unlimited number of key fields (/FIELDS, /KEYS) with the following standard data formats:
 - character
 - unsigned decimal
 - leading and trailing separate sign decimal
 - leading and trailing embedded sign decimal (Micro Focus and alternative formats)
 - edited numeric
 - packed decimal
 - signed integer
 - unsigned integer
 - floating point
 - bit
 - date/time
- sorts character data using standard collating sequences and user-defined sequences with one-for-one, one-for-two, two-for-one, and two-for-two replacements (/COLLATINGSEQUENCE)
- sorts on different key fields based on selection criteria
- allows control of the output sequence of records with duplicate keys (/STABLE) and totalling of values in records with duplicate keys (/SUMMARIZE)
- performs record sequence checking (/CHECKSEQUENCE)

-
- pads short records and keys of short records with user-specifiable characters (/PADBYTE, /PADKEY, /PADRECORD)

Record Selection and Reformatting

- includes or omits records on the basis of comparisons between field contents and constants or on the basis of the file from which the records come (/INCLUDE, /OMIT, /CONDITION)
- edits, inserts and removes fields within records (/REFORMAT)
- reformats differently based on selection criteria
- assigns values to fields based on selection criteria and converts data in fields from one format to another (/DERIVEDFIELD).
- processes or skips a specified number of records (/COLLECT, /IGNORE, /STOPAFTER, /SKIPRECORD), and eliminates blank records /SKIPBLANK)
- selects records based on a pattern of data in a field (/CONDITION)
- extracts information based on a pattern in a field (/DERIVEDFIELD)
- manually generates record numbers specifying a form and a location within the output record (/DERIVEDFIELD, /REFORMAT)

Report Formatting

Generates reports, including:

- columns and column headings (/COLUMNS)
- sections and subsections (/SECTIONS)
- page and section headers (/PAGEHEADER, /SECTIONHEADER)
- page and section footers (/PAGEFOOTER, /SECTIONFOOTER)
- totals, subtotals, counts, minimum, maximum, and averages /AGGREGATE)

-
- title page and report trailer (/TITLE, /TRAILER)

Join Processing

The /JOIN and /JOINKEY options, combined with extensions to the REFORMAT option, give you the capability to join records from multiple source files. When you join the records from multiple files, each record from the first file(s) (the left side) with a given value in a named field (the join key) is joined to each record from the second file(s) (the right side) with the identical value in a named field in that record. Thus, if m records from the left side have a given join key value, and n from the right side have the same join key value, the join results in $m * n$ records with that join key value.

- option to control if data needs to be sorted or not using the SORTED argument on the /JOINKEYS
- assignment of specific fields to be used to join multiple inputs through /JOINKEYS
- field selection through the /REFORMAT option allows you to specify what information from the two records is placed in the joined record
- left, right, inner, and outer joins supported through the /JOIN option

Resource Control

- uses a specified directory, combination of directories, or a scratch directory for temporary files (/WORKSPACE)
- optionally uses specified amount of virtual memory (/MEMORY)
- optimizes through user-supplied information when system information is unavailable (/DATASIZE)

Ease of Use

- replaces other sort routines without changes to existing applications.

-
- creates application definition files and reads options from application definition files (/SAVE, /INSERT)
 - reads field definitions from COBOL data description files (/DATADictionary)
 - processes user-specified record layouts (/RECORDLAYOUT, /DELIMITEDRECORDLAYOUT)
 - allows online correction of errors without terminating and re-executing (/DELETE, /DISPLAY, /EDIT, /BEGIN)
 - validates the application options without executing (/NOEXECUTE)
 - allows the user either to interrupt processing and abort, suspend, or resume, or to suppress this feature (/NOINTERRUPT)
 - allows user control over the amount of informational output produced (/SILENT, /STATISTICS, /WARNINGS)
 - describes the command options online (**man syncsort** command)
 - displays SyncSort message texts, explanations and actions online (**syncmsg** command)

Invocation

SyncSort can be invoked from the Visual SyncSort graphical interface, from the command line through the shell commands **syncsort** or **ssort**, from Micro Focus COBOL programs through the SORT and MERGE verbs, from Software AG NATURAL sort statements, from SAS applications through the sort procedure, from IBM's DB2 through the DB2 load command, and from program calls in any system-supported language through the sync_ subroutines.

In addition, SyncSort can be invoked by the UNIX **sort** command, using UNIX system sort syntax, if the .../syncsort/bin directory precedes the directory containing the UNIX system sort in the UNIX PATH.

SyncSort allows you to build and run your UNIX sort applications through a graphical user interface component Visual SyncSort that runs on your Windows machine. It is no longer necessary to code options strings. Designing, developing, and then saving your application within the graphical interface allows for both command line and interactive execution using the graphical interface.

While developing your SyncSort application within the interface you can either browse Windows directories or remote UNIX system directories, provided they are running FTP (File Transfer Protocol), to locate source files, target files, workspace directories, and any COBOL copybooks used in the application. After building the application and saving it on a UNIX system, you can then run the application from within the graphical interface. To run the same application from either a shell script or the command prompt, you would enter:

```
syncsort /run applicationname.srt
```

where applicationname.srt is the name you gave the application when saving it within the Visual SyncSort component.

Chapter 2. Invoking SyncSort

You can invoke SyncSort in any of several ways:

- By using the Visual SyncSort graphical interface (see Invocation on page 1.6).
- By entering the **syncsort** command at the shell prompt (see “2.1 The syncsort Command” on page 2.2).
- By using the UNIX **sort** command. See “Invocation” on page 1.6.
- By calling the SyncSort **sync_** subroutines from a program (see “2.2 The SyncSort Subroutines” on page 2.7).
- By calling the SyncSort **syncset...** shell functions from a Korn Shell script (see “2.3 The SyncSort Korn Shell Interface” on page 2.100).
- By using the Micro Focus COBOL SORT and MERGE verbs (see “2.4 The Micro Focus COBOL SORT and MERGE Verbs ” on page 2.120).
- By using the IBM DB2 Sort Accelerator (see “2.6 The IBM DB2 Load Accelerator” on page 2.127)
- By using the SAS sort procedure (see “2.5 The SAS Sort Procedure ” on page 2.126).

When the **syncsort** command is used, the format of the options is that shown in Chapter 3.

You can interrupt SyncSort during execution by pressing an interrupt key (see “2.7 Interrupting SyncSort” on page 2.129).

If SyncSort needs more disk space during execution, it pauses and asks for directions (see “2.8 Disk Full Recovery ” on page 2.131).

This chapter contains sample programs which invoke SyncSort. These sample programs, together with input files, are available in the directory `.../syncsort/examples`.

2.1 The syncsort Command

You can use the **syncsort** command to invoke SyncSort from the shell prompt or from a shell script.

The format of the command is:

`syncsort [option ...]`

The options are described in Chapter 3. *SyncSort Options*.

You may enter all options directly after the syncsort command (at the command line or in a shell script).

In this case, they are first interpreted by the active UNIX shell. Thus, the treatment of double quotation marks, the method of continuing the list of SyncSort options from one line to the next, and the interpretation of environment variables depends on the UNIX shell. The following are suggestions for specifying the desired SyncSort options in a shell environment:

- To prevent the shell from removing double quotation marks, enclose the quoted text in single quotation marks (for example, "test").
- To continue SyncSort options from one line to the next, either use a back slash (\) at the end of the line to escape the newline character or include the options in a "here document" (using <<). The examples displayed in this book are assumed to be parts of here documents. Thus, no continuation characters are used when an option spans multiple lines.
- To facilitate running a "template" SyncSort job with a variety of different input files, use an environment variable to specify the filename on the /INFILE option and set and export the environment variable outside the SyncSort job. A similar approach may be used for /OUTFILE, /WORKSPACE, and other SyncSort options.

Alternatively, you can issue a partial syncsort command at the shell prompt. If the command does not terminate with the /END option, the SyncSort prompt (>) is

shown at the beginning of the next line to indicate that SyncSort is expecting you to enter options. For example:

```
syncsort /infile invoices/2001/jan
[SyncSort HP-UX Copyright (c) 2001 Syncsort Inc.]
[For use by ABC Incorporated.]
After entering options, press Ctrl-D or key /END
> _
```

All of the SyncSort keywords can be abbreviated. Any number of leading characters of a keyword can be used as an abbreviation as long as the abbreviation is not ambiguous. After keying a line of options, press <enter>. If you need to continue a keyword, character string or other token on another line, key the continuation character (\) before pressing <enter>:

```
syncsort /infile invoices/2001/jan
[SyncSort HP-UX Copyright (c) 2001 Syncsort Inc.]
[For use by ABC Incorporated.]
After entering options, press Ctrl-D or key /END
> /outfile invoices/2001/jan_out
> _
```

If you wish to comment your application, you can insert a comment anywhere in the options except where a file or directory name is expected. The text of the comment is preceded by '/' and followed by '*'.

SyncSort will continue prompting you for additional options until you terminate the definition with the /END option or Ctrl-D. If no errors are found in the options, SyncSort executes without further dialog.

```
syncsort /infile invoices/2001/jan
[SyncSort HP-UX Copyright (c) 2001 Syncsort Inc.]
[For use by ABC Incorporated.]
After entering options, press Ctrl-D or key /END
> /outfile invoices/2001/jan_out
> /fields invoice_number 7 character 20 /keys invoice_number /end
SyncSort options validated. Processing continues.
SyncSort has completed
```

After SyncSort validates the options, you can interrupt SyncSort by pressing the interrupt key, which is usually Ctrl-C. For more details, see “2.7 Interrupting SyncSort” on page 2.129.

Problem Correction

When SyncSort finds problems in the options, you are notified of each problem in turn. In many cases, you are advised of the problem immediately after you have entered the line of options containing the problem. SyncSort will not execute until you correct all problems.

Up to four pieces of information are displayed for each problem:

- If the error was found in the contents of an insert file or a COBOL data description file, a message indicating which line in the file is in error.
- The line of text from the application options containing the problem.
- An indicator pointing to the position in the line at which the problem was detected.
- A descriptive message.

For example,

```
syncsort /infile invoices/2001/jan /outfile invoices/2001/jan_out
[SyncSort HP-UX Copyright (c) 2001 Syncsort Inc.]
[For use by ABC Incorporated.]
/infile invoices/2001/jan /outfile invoices/1999/jan_out
^
SyncSort: (INFNF) input file not found
After entering options, press Ctrl-D or key /END
> _
```

SyncSort provides several methods of correcting problems in the options. These are the editing options /BEGIN, /DELETE, /DISPLAY, /EDIT, /QUIT and /SAVE, described in Chapter 3. SyncSort Options.

If SyncSort detects any problem after you terminate the definition, you are given the opportunity to amend the options before SyncSort executes them. This is achieved via a prompt requiring a yes or no response.

For example,

```
syncsort /infile invoices/2001/jan
[SyncSort HP-UX Copyright (c) 2001 Syncsort Inc.]
[For use by ABC Incorporated.]
After entering options, press Ctrl-D or key /END
> /outfile invoices/2001/jan_out /end
```

```
SyncSort: (DKEY) there is no key specification; the default will
be used
```

```
Warning(s) were issued. Do you want to continue without amending?
(Y/N): _
```

If you indicate that you wish to amend the options, SyncSort prompts you for additional options. You can add further options or you can amend the existing options by entering one of the editing options. If you do not wish to amend the options and there is no problem, SyncSort displays the following message and executes:

```
Minor exception(s) in options. Processing continues.
```

If there are uncorrected problems and you choose not to correct them, SyncSort terminates. All failed applications end with the message:

```
SyncSort has aborted
```

Statistics

When SyncSort completes, statistics are written to standard error (file descriptor 2) if you requested them using the /STATISTICS option.

For the statistics provided by SyncSort, see “/STATISTICS” on page 3.180

Completion Status of SyncSort

The value of the exit status indicates the completion status of SyncSort.

- | | |
|------------|--|
| 0 | SyncSort completed successfully. |
| 1 | SyncSort detected an out-of-sequence record when you specified /CHECKSEQUENCE. |
| 100 | SyncSort completed but output generated may be incorrect or incomplete. |
| 111 | SyncSort terminated prematurely. |

2.2 The SyncSort Subroutines

The SyncSort subroutines, invoked from a program written in a system-supported language, give you access to the full functionality of SyncSort. The SyncSort subroutines have been enhanced to include a new set of subroutine calls. The old subroutine interface will continue to execute and is fully supported. New programs should use the new SyncSort subroutine interface in conjunction with Visual SyncSort. All future enhancements will be implemented through the new SyncSort subroutine interface.

The SyncSort graphical component, Visual SyncSort, allows for applications to be developed without the coding of specific options strings. Once the application is created using Visual SyncSort and the new subroutine interface, future SyncSort application changes will usually not require a change to the program, avoiding a potential recompilation.

The SyncSort subroutines allow for multiple concurrent applications. In this way, you can run several sort, merge, and copy applications simultaneously without the restriction that one complete before another begins. Most subroutines have a context parameter that you should set to zero before the **sync_load_application** call initiating an application. SyncSort returns in this parameter a context to identify the application. You must include the same context in all further calls for the same application.

See “Subroutines to Run a Sort ” on page 2.10, “Subroutines to Run a Merge ” on page 2.13, and “Subroutines to Run a Copy” on page 2.15 to find which subroutines you need to run an application.

You can find several examples of working C and COBOL programs containing invocations of SyncSort in the directory **SyncSort\Examples\Invoked**.

Note:

- When you invoke a SyncSort subroutine prefixed with **sync_** you must code the routine name in lowercase letters. SyncSort subroutine names are language-independent and always contain underscores.
- When you reference a SyncSort constant prefixed with **SYNC_** you must code the constant name in uppercase letters. In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

SyncSort Subroutine Parameters

The SyncSort subroutines accept parameter types of character string, integer, unsigned integer, and double precision float. Table 1 shows how to represent the parameter types in C and Micro Focus COBOL.

Parameter Type	C Type	COBOL Type
character string	char [] , char*	PIC X(n)
integer	int	PIC S9(9) COMP-5
unsigned integer	unsigned int	PIC 9(9) COMP-5
double precision float	double	COMP-2

Table 1. Parameter Type Representation in C and COBOL

SyncSort provides files `sync.h`, `syncsort.cbl`, and `syncsort.f77`, which contain subroutine prototypes and constants for programs written in C, COBOL, and FORTRAN-77, respectively. You can find these files in the directory **SyncSort\Include**.

Compiling and Linking Programs that Call SyncSort

You can compile a C program, *prog.c*, that calls SyncSort subroutines with the following command:

```
cc prog.c -I syncsort_include_directory  
          -L syncsort_library_directory -lsyncsort
```

where *syncsort_include_directory* is the directory that contains the `sync.h` include file and the *syncsort_library_directory* is the directory where the SyncSort library is installed. For example, if SyncSort is installed in `/usr/local/syncsort`, specify **-L /usr/local/syncsort/lib** and **-I /usr/local/syncsort/include**.

The easiest way to invoke SyncSort from a Micro Focus COBOL program is by using the COBOL SORT or MERGE verbs. This method is described in “2.4 The Micro Focus COBOL SORT and MERGE Verbs” on page 2.120. However, if you use a different COBOL compiler you may wish to compile a COBOL program that

calls SyncSort routines. To do this, you may first need to change the COBCPY variable on your system to include the directory that contains the syncsort.cbl file. Then use the following command:

```
cob -x prog.cob -L syncsort_library_directory -lsyncsort
```

Completion Status of SyncSort Subroutines

Each subroutine (except **sync_get_library_version**, **sync_get_error_number** and **sync_get_error_message**) returns a completion status of type integer, the value of which indicates success or failure.

A completion status of **SYNC_SUCCESS** indicates that the call completed successfully or with minor exceptions. A minor exception does not preclude further calls to SyncSort subroutines. You can determine if a minor exception occurred by invoking **sync_get_error_number**. A zero return from **sync_get_error_number** indicates success and a non-zero return is the error number for the last minor exception encountered. You can find the severity of a minor exception by invoking **sync_get_highest_severity**. The severity level is as follows:

0	success
1	warning – minor exceptions occurred
2	severe – the target produced by SyncSort may be incorrect or incomplete

A completion status of **SYNC_FAILURE** indicates that the call failed. You should make no further calls to SyncSort subroutines, including **sync_clear**. You can find the cause of the failure by invoking **sync_get_error_number**.

To obtain the text message, explanation and action associated with the error number, see Visual SyncSort Help topic “Error codes from programs invoking SyncSort”.

Note: In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

Subroutines to Run a Sort

There are twenty-one subroutines available to run a sort.

sync_load_application	loads the SyncSort sort application from an .srt file
sync_set_source_file	overrides a source file name
sync_add_source_file	adds an additional source file
sync_set_target_file	overrides a target file name
sync_set_source_buffer	identifies the table holding source records
sync_set_target_buffer	identifies the table to hold target records
sync_set_input_data_bytes	overrides the estimated amount of data to be sorted
sync_set_input_record_count	overrides the estimated number of records to be sorted
sync_set_numeric_value	replaces the contents of a named number
sync_set_text_value	replaces the contents of a named piece of text
sync_add_key	selects a named field as an additional key for the sort application
sync_add_positional_key	defines a fixed-position field and uses it as an additional key for the sort application.
sync_add_datetime_format	defines a new date/time format when defining additional keys.
sync_add_delimited_key	defines a delimited field and uses it as an additional key for the sort application.
sync_set_source_reader	provides a reader object that supplies on demand a buffer filled with source records.

sync_set_target_writer	provides a writer object that receives on demand a target buffer filled with processed records.
sync_run	runs the SyncSort application
sync_put_record	supplies a record to the sort
sync_put_record_end	indicates that all records have been supplied to the sort
sync_get_record	retrieves a record from the sort
sync_clear	terminates the sort, releasing all resources

You do not need all the subroutines to run a sort, but you must call any subroutines you use in the order shown above. If you use more than one **sync_set**/**sync_add** routine, you can call them in any order.

The subroutines are mandatory as follows:

sync_load_application	for all sorts
sync_run	for all sorts
sync_put_record	only when you provide records one at a time in memory via a source procedure
sync_put_record_end	only when you provide records one at a time in memory via a source procedure
sync_get_record	only when records are returned one at a time in memory to a target procedure
sync_clear	for all sorts

In addition, there are five subroutines available to provide information on the run.

sync_get_statistic	obtains a statistic on the sort run
sync_get_highest_severity	obtains the highest severity for exceptions recorded for the sort run

sync_get_error_number	obtains the error number associated with the previous SyncSort subroutine call
sync_get_error_message	obtains the error message associated with the previous SyncSort subroutine call
sync_get_library_version	obtains the version number of the SyncSort library

You can call **sync_get_highest_severity** any time after calling **sync_load_application** and before calling **sync_clear**. You can call **sync_get_statistic** any time after calling **sync_run** and before calling **sync_clear**. You can call the others at any time.

Subroutines to Run a Merge

There are seventeen subroutines available to run a merge.

sync_load_application	loads the SyncSort merge application from an .srt file
sync_set_source_file	overrides a source file name
sync_add_source_file	adds an additional source file
sync_set_target_file	overrides a target file name
sync_set_source_buffer	identifies the table holding source records
sync_set_target_buffer	identifies the table to hold target records
sync_set_numeric_value	replaces the contents of a named number
sync_set_text_value	replaces the contents of a named piece of text
sync_add_key	selects a named field as an additional key for the merge application
sync_add_positional_key	defines a fixed-position field and uses it as an additional key for the merge application.
sync_add_datetime_format	defines a new date/time format when defining additional keys.
sync_add_delimited_key	defines a delimited field and uses it as an additional key for the merge application.
sync_set_source_reader	provides a reader object that supplies on demand a buffer filled with source records.
sync_set_target_writer	provides a writer object that receives on demand a target buffer filled with processed records.
sync_run	runs the SyncSort application

sync_get_record	retrieves a record from the merge
sync_clear	terminates the merge, releasing all resources

You do not need all the subroutines to run a merge, but you must call any subroutines you use in the order shown above. If you use more than one **sync_set_/sync_add_** routine, you can call them in any order.

The subroutines are mandatory as follows:

sync_load_application	for all merges
sync_run	for all merges
sync_get_record	only when records are returned one at a time in memory to a target procedure
sync_clear	for all merges

In addition, there are five subroutines available to provide information on the run.

sync_get_statistic	obtains a statistic on the merge run
sync_get_highest_severity	obtains the highest severity for exceptions recorded for the merge run
sync_get_error_number	obtains the error number associated with the previous SyncSort subroutine call
sync_get_error_message	obtains the error message associated with the previous SyncSort subroutine call
sync_get_library_version	obtains the version number of the SyncSort library

You can call **sync_get_highest_severity** any time after calling **sync_load_application** and before calling **sync_clear**. You can call **sync_get_statistic** any time after calling **sync_run** and before calling **sync_clear**. You can call the others at any time.

Subroutines to Run a Copy

There are fifteen subroutines available to run a copy.

sync_load_application	loads the SyncSort copy application from an .srt file
sync_set_source_file	overrides a source file name
sync_add_source_file	adds an additional source file
sync_set_target_file	overrides a target file name
sync_set_source_buffer	identifies the table holding source records
sync_set_target_buffer	identifies the table to hold target records
sync_set_numeric_value	replaces the contents of a named number
sync_set_text_value	replaces the contents of a named piece of text
sync_set_source_reader	provides a reader object that supplies on demand a buffer filled with source records.
sync_set_target_writer	provides a writer object that receives on demand a target buffer filled with processed records.
sync_run	runs the SyncSort application
sync_put_record	releases a record to the copy
sync_put_record_end	indicates that all records have been released to the copy
sync_get_record	retrieves a record from the copy
sync_clear	terminates the copy, releasing all resources

You do not need all the subroutines to run a copy, but you must call any subroutines you use in the order shown above. If you use more than one **sync_set_/sync_add_** routine, you can call them in any order.

You can use **sync_put_record** to supply records one at a time to a copy except when you want SyncSort to return the records one at a time to a target procedure via **sync_get_record**.

The subroutines are mandatory as follows:

sync_load_application	for all copies
sync_put_record	only when you provide records one at a time in memory via a source procedure
sync_put_record_end	only when you provide records one at a time in memory via a source procedure
sync_get_record	only when records are returned one at a time in memory to a target procedure
sync_clear	for all copies

In addition, there are five subroutines available to provide information on the run.

sync_get_statistic	obtains a statistic on the copy run
sync_get_highest_severity	obtains the highest severity for exceptions recorded for the copy run
sync_get_error_number	obtains the error number associated with the previous SyncSort subroutine call
sync_get_error_message	obtains the error message associated with the previous SyncSort subroutine call
sync_get_library_version	obtains the version number of the SyncSort library

You can call **sync_get_highest_severity** any time after calling **sync_load_application** and before calling **sync_clear**. You can call **sync_get_statistic** any time after calling **sync_run** and before calling **sync_clear**. You can call the others at any time.

Subroutines to Run a Join

There are nine subroutines available to run a join.

sync_load_application	loads the SyncSort join application from an .srt file
sync_set_source_file	overrides a source file name
sync_add_source_file	adds an additional source file
sync_set_target_file	overrides a target file name
sync_set_numeric_value	replaces the contents of a named number
sync_set_text_value	replaces the contents of a named piece of text
sync_select_join_side	to select a join side to amend
sync_run	runs the SyncSort application
sync_clear	terminates the merge, releasing all resources

You do not need all the subroutines to run a join, but you must call any subroutines you use in the order shown above. If you use more than one **sync_set/sync_add** routine, you can call them in any order.

The subroutines are mandatory as follows:

sync_load_application	for all joins
sync_run	for all joins
sync_clear	for all joins

In addition, there are five subroutines available to provide information on the run.

sync_get_statistic	obtains a statistic on the join run
sync_get_highest_severity	obtains the highest severity for exceptions recorded for the join run

sync_get_error_number	obtains the error number associated with the previous SyncSort subroutine call
sync_get_error_message	obtains the error message associated with the previous SyncSort subroutine call
sync_get_library_version	obtains the version number of the SyncSort library

You can call **sync_get_highest_severity** any time after calling **sync_load_application** and before calling **sync_clear**. You can call **sync_get_statistic** any time after calling **sync_run** and before calling **sync_clear**. You can call the others at any time.

sync_load_application

Purpose

To load the SyncSort application from an .srt file.

Format

sync_load_application *application_filename, application_filename_length,
log_filename,
log_filename_length, context*

Parameters

<i>application_filename</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the SyncSort application the .srt file created through the SyncSort graphical interface.

<i>application_filename_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the application filename.

sync_load_application

<i>log_filename</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the file to be used to log messages.

<i>log_filename_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the log filename.

<i>context</i>	type:	unsigned integer
	access:	write only
	mechanism:	by reference

Identifies one application from a number of concurrent applications. The returned value is used in further `sync_` calls to identify the application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_set_source_file

Purpose

To replace a source file name.

Format

sync_set_source_file *source_file_number, source_filename,
source_filename_length, context*

Parameters

<i>source_file_number</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The source file whose name you want to change. The files are numbered sequentially from 1 in the order in which they appear in the application file identified by the *application_filename* parameter for **sync_load_application**.

<i>source_filename</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the source file.

sync_set_source_file

<i>source_filename_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the source filename.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The file identified in *source_filename* is a source for the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_add_source_file

Purpose

To add another source file to the application.

Format

sync_add_source_file *source_filename, source_filename_length,
source_file_number, context*

Parameters

<i>source_filename</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the source file.

<i>source_filename_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the source filename.

<i>source_file_number</i>	type:	unsigned integer
	access:	write only
	mechanism:	by reference

The number of the source file as assigned by SyncSort.

sync_add_source_file

<i>context</i>	type:	unsigned integer
	access:	modify
	mechanism:	by reference

Identifies one application from a number of concurrent applications. The file identified in *source_filename* is a source for the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort ” on page 2.6.

sync_set_target_file

Purpose

To replace a target file name.

Format

sync_set_target_file

target_file_number, target_filename, target_filename_length, context

Parameters

target_file_number

type:

unsigned integer

access:

read only

mechanism:

by value

The target file whose name you want to change. The files are numbered sequentially from 1 in the order in which they appear in the application file identified by the *application_filename* parameter for **sync_load_application**.

target_filename

type:

character string

access:

read only

mechanism:

by reference

The name of the target file.

target_filename_length

type:

unsigned integer

access:

read only

mechanism:

by value

sync_set_target_file

The length in bytes of the target filename.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The file identified in *target_filename* is a target for the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort ” on page 2.6.

sync_set_source_buffer

Purpose

To identify the table holding the source records.

Format

sync_set_source_buffer *source_table, source_table_length, context*

Parameters

<i>source_table</i>	type:	character string
	access:	read only
	mechanism:	by reference

A table containing all source records.

<i>source_table_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of valid data in *source_table*.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The records in *source_table* are the source record for the identified application.

sync_set_source_buffer

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort ” on page 2.6.

sync_set_target_buffer

Purpose

To identify the table to hold the target records.

Format

sync_set_target_buffer *target_table, target_table_length, context*

Parameters

<i>target_table</i>	type:	character string
	access:	write only
	mechanism:	by reference

The target table. The table must be large enough to hold all the records issuing from SyncSort; otherwise, the excess records are lost. When the source records are supplied through a procedure, the table must still be available to SyncSort at the call to **sync_put_record_end**.

<i>target_table_length</i>	type:	unsigned integer
	access:	modify
	mechanism:	by value

Passed to SyncSort: The usable length in bytes of the *target_table* buffer. Returned by SyncSort: The length in bytes of valid data in *target_table*.

When you supply the source records through a procedure, the *target_table_length* storage must still be available to SyncSort at the call to **sync_put_record_end**. The value is not updated until returning from the **sync_put_record_end** call.

sync_set_input_data_bytes

Purpose

To override the estimated amount of data to be sorted.

Format

sync_set_input_data_bytes *data_size, context*

Parameters

<i>data_size</i>	type:	double precision float
	access:	read only
	mechanism:	by value

The amount of data in bytes that remain after applying any filter for all targets.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The data size provided in *data_size* is for the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort ” on page 2.6.

sync_set_input_record_count

sync_set_input_record_count

Purpose

To override the estimated number of records to be sorted.

Format

sync_set_input_record_count *record_count, context*

Parameters

<i>record_count</i>	type:	double precision float
	access:	read only
	mechanism:	by value

The number of records that remain after applying any filter for all targets.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The number of records provided in *record_count* is for the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_run

To run a SyncSort application.

sync_run	<i>context</i>
----------	----------------

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. SyncSort is to run the identified application.

<i>completion_code</i>	type:	integer
	mechanism:	by value

sync_run returns **SYNC_OUT_OF_ORDER_RECORD** when sequence checking is requested and an out-of-order record is detected.

Note: In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_put_record

To pass one record in memory to SyncSort from a source procedure for a sort or copy.

sync_put_record	<i>source_record, source_record_length, context</i>
-----------------	---

<i>source_record</i>	type:	character string
	access:	read only
	mechanism:	by reference

<i>source_record_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the record in *source_record*. The record should be of length **SYNC MAX REC LEN** or less.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The record in *source_record* is a source record for the identified application.

Note: In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

To indicate to SyncSort that the source procedure has passed all records.

Format

sync_put_record_end	<i>context</i>
---------------------	----------------

Parameters

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The source is complete for the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

To retrieve one record in memory from SyncSort to a target procedure.

Format

<code>sync_get_record</code>	<i>target_record</i> , <i>target_record_length</i> , <i>context</i>
------------------------------	--

Parameters

<i>target_record</i>	type:	character string
	access:	write only
	mechanism:	by reference

The target buffer. The buffer must be large enough to hold the longest record issuing from SyncSort; otherwise, the record is truncated.

<i>target_record_length</i>	type:	unsigned integer
	access:	modify
	mechanism:	by reference

Passed to SyncSort: The useable length in bytes of the *target_record* buffer.
Returned by SyncSort: The length in bytes of the record in *target_record*.

sync_get_record

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The record is to be returned from the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

sync_get_record returns **SYNC_END_OF_OUTPUT** when calls are made after the last record has been returned.

Note: In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

To end a SyncSort application and release resources.

Format

sync_clear	<i>context</i>
------------	----------------

Parameters

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. SyncSort is to end the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_clear

Notes

You can call **sync_clear** any time after calling **sync_load_application**. Do not call **sync_clear** after one of the other routines has returned a completion status of **SYNC_FAILURE**. Under such circumstances, SyncSort releases resources automatically.

When SyncSort ends, the *context* is no longer recognized.

Note: In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

sync_get_statistic

Purpose

To obtain one statistic concerning the running of a SyncSort application.

Format

sync_get_statistic *statistic_type, statistic_value, target_file, context*

Parameters

<i>data_size</i>	type:	double precision float
	access:	read only
	mechanism:	by value

The amount of data in bytes that remain after applying any filter for all targets.

The values you can assign to *statistic_type* are as follows:

SYNC_INPUT_RECORDS	the number of source records provided to SyncSort
SYNC_PROCESSED_RECORDS	the number of records sorted, merged, copied or joined before applying any per-target filter
SYNC_OUTPUT_RECORDS	the number of records sent to a target
SYNC_INPUT_DATA	the total size in bytes of source records provided to SyncSort
SYNC_PROCESSED_DATA	the total size in bytes of records sorted, merged, copied or joined before applying any per-target filter or reformatting

SYNC_OUTPUT_DATA	the total size in bytes of records sent to a target
SYNC_DELETED_INPUT	the number of source records deleted because of data errors
SYNC_DELETED_OUTPUT	the number of records not sent to a target because of data errors
SYNC_PROCESSED_LRL	the length in bytes of the longest record sorted, merged, copied or joined before applying any per-target reformatting
SYNC_INPUT_LRL	the length in bytes of the longest source record
SYNC_OUTPUT_LRL	the length in bytes of the longest record sent to a target
SYNC_WORK_SPACE	the allocated size in bytes of temporary work files
SYNC_OUTPUT_TARGETS	the number of targets provided to SyncSort
SYNC_JOIN_SIDES	the number of join sides in a join application or 0 if the application is not a join

The values you can assign to *statistic_type* for retrieving per side statistics in a join application are as follows:

SYNC_JOIN_SIDE_INPUT_RECORDS	the number of source records provided to SyncSort from one side of the join
SYNC_JOIN_SIDE_PAIRED_RECORDS	the number of records paired in one side of the join
SYNC_JOIN_SIDE_UNPAIRED_RECORDS	the number of records not paired in one side of the join

SYNC_JOIN_SIDE_DELETED_INPUT	the number of source records deleted from one side of the join because of data errors
SYNC_JOIN_SIDE_INPUT_DATA	the number of join sides in a join application or 0 if the application is not a join
SYNC_JOIN_SIDE_PAIRIED_DATA	the total size in bytes of records paired in one side of the join
SYNC_JOIN_SIDE_UNPAIRED_DATA	the total size in bytes of records not paired in one side of the join
SYNC_JOIN_SIDE_INPUT_LRL	the length in bytes of the longest source record provided to SyncSort from one side of the join

When the application is not a join, retrieval of the per-side statistics results in errors. Invoke subroutine **sync_select_join_side** to select the side in interest before invoking **sync_get_statistic**. Statistics SYNC_INPUT_RECORDS, SYNC_INPUT_DATA and SYNC_INPUT_LRL and the values associated with them are the combined values from all join sides.

Note: In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

<i>statistic_value</i>	type:	double precision float
	access:	write only
	mechanism:	by reference

The value of the statistic requested.

<i>target_file</i>	type:	unsigned integer
	access:	read only
	mechanism:	by reference

sync_get_statistic

The target file for which you require the statistic. **sync_get_statistic** ignores the parameter when the statistic you request is independent of a target file. The files are numbered sequentially from 1 in the order in which they appear in the application file identified by the *application_filename* parameter for **sync_load_application**.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The identified application is the one for which you want the statistic.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_get_highest_severity

Purpose

To obtain the highest severity of any exceptions recorded for the application.

Format

sync_get_highest_severity *highest_severity, context*

Parameters

<i>highest_severity</i>	type:	unsigned integer
	access:	write only
	mechanism:	by reference

The highest severity of exceptions recorded by SyncSort while running the application.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The highest severity returned is for the identified application.

sync_get_highest_severity

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_get_error_number

Purpose

To obtain the error number for the previous call to a sync_routine.

Format

sync_get_error_number

Returns

<i>error_number</i>	type:	integer
	mechanism:	by value

The error number for the previous call to a sync_routine.

sync_get_error_message

Purpose

To obtain the message associated with the error number for the previous call to a sync_routine.

Format

sync_get_error_message *error_message, error_message_length*

Parameters

<i>error_message</i>	type:	character string
	access:	write only
	mechanism:	by reference

The buffer to receive the error message for the previous call to a sync_ routine. The buffer should be of length **SYNC_ERROR_DESC_MAX_LEN** or greater; otherwise, the message may be truncated. The returned message is null (\0) terminated.

Note: In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

<i>error_message_length</i>	type:	unsigned integer
	access:	modify
	mechanism:	by reference

Passed to SyncSort: The useable length in bytes of the *error_message* buffer.

Returned by SyncSort: The length in bytes of the message in *error_message*, excluding the terminating null.

sync_get_library_version

Purpose

To obtain the version of the SyncSort library.

Format

sync_get_library_version *library_version, library_version_length*

Parameters

<i>library_version</i>	type:	character string
	access:	write only
	mechanism:	by reference

The buffer to receive the version of the SyncSort library. The buffer should be of length **SYNC_VERSION_MAX_LEN** or greater; otherwise, the text may be truncated. The returned text is null (\0) terminated.

Note: In COBOL all SyncSort constants use hyphen (-) instead of underscore (_) because of language constraints.

<i>library_version_length</i>	type:	unsigned integer
	access:	modify
	mechanism:	by reference

Passed to SyncSort: The useable length in bytes of the *library_version* buffer.

Returned by SyncSort: The length in bytes of the text in *library_version*, excluding the terminating null.

sync_add_key

sync_add_key

Purpose

To select a named field as an additional key for a sort or a merge application.

Format

sync_add_key *field_name, field_name_length, direction,
sequence_name, sequence_name_length, context*

Parameters

<i>field_name</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the field you want to add as an additional key. The field must be defined in the data dictionary of the application identified by the *application_filename* parameter for **sync_load_application**. You may not reference an array element in the field name.

<i>field_name_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the field name.

<i>direction</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The direction in which to order the key. See the SyncSort header file for the list of direction codes.

<i>sequence_name</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the collating sequence to use when ordering the key. The sequence can be any of the standard sequences or a customized sequence defined in the data dictionary of the loaded application. To use the default collating sequence, pass an empty string by setting the *sequence_name_length* parameter to 0.

The sequence name must be empty for a field with type other than character. SyncSort terminates with an error otherwise.

<i>sequence_name_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the collating sequence name.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

sync_add_key

Identifies one application from a number of concurrent applications. The key is added to the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

To define a fixed-position field and use it as an additional key for a sort or a merge application.

Format

sync_add_positional_key	<i>starting_byte, number_of_bytes, data_format, direction, sequence_name, sequence_name_length, context</i>
-------------------------	---

Parameters

<i>starting_byte</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The starting byte position of the fixed position field you want to use as the additional key. The first position within the data portion of the record is byte position 1.

<i>number_of_bytes</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the fixed-position key. Provide a value of 0 to indicate that the field extends to the end of the record.

sync_add_positional_key

<i>data_format</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The format of the data stored in the field. It must be one of the standard data format codes with the exception of SYNC_DATA_DATE_TIME, which is not currently supported for a fixed-position field. See the SyncSort header file for the list of standard data format codes.

<i>direction</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The direction in which to order the key. See the SyncSort header file for the list of direction codes.

<i>sequence_name</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the collating sequence to use when ordering the key. The sequence can be any of the standard sequences or a customized sequence defined in the data dictionary of the loaded application. To use the default collating sequence, pass an empty string by setting the *sequence_name_length* parameter to 0.

The sequence name must be empty for a field with *data_format* other than character. SyncSort terminates with an error otherwise.

<i>sequence_name_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the collating sequence name.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The key is added to the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_add_delimited_key

To define a delimited field and use it as an additional key for a sort or a merge application.

<code>sync_add_delimited_key</code>	<i>field_number, data_format, direction, sequence_name, sequence_name length, context</i>
-------------------------------------	---

<i>field_number</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The field number of the delimited field you want to use as an additional key. The fields in the source record are numbered from left to right, beginning with 1.

<i>data_format</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The format of the data stored in the field. Supported values for a delimited field are SYNC_DATA_CHARACTER, SYNC_DATA_DATE_TIME, SYNC_DATA_EDITED_NUMERIC and the code generated by a call to sync_add_datetime_format. See the SyncSort header file for a description of the standard data format codes.

<i>direction</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The direction in which to order the key. See the SyncSort header file for the list of direction codes.

<i>sequence_name</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the collating sequence to use when ordering the key. The sequence can be any of the standard sequences or a customized sequence defined in the data dictionary of the loaded application. To use the default collating sequence, pass an empty string by setting the *sequence_name_length* parameter to 0.

The sequence name must be empty for a field with *data_format* other than character. SyncSort terminates with an error otherwise.

<i>sequence_name_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the collating sequence name.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

sync_add_delimited_key

Identifies one application from a number of concurrent applications. The key is added to the identified application.

Returns

completion_code

type:

integer

mechanism:

by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_add_datetime_format

Purpose

To define a new date/time format.

Format

sync_add_datetime_format *datetime_pattern, datetime_pattern_length, custom_format_id, context*

Parameters

<i>datetime_pattern</i>	type:	character string
	access:	read only
	mechanism:	by reference

The pattern describing how the date/time data appear in a field. Refer to the section "Source Date Patterns" in the "Masks" appendix in the SyncSort UNIX reference guide for more information on specifying a date/time field pattern.

<i>datetime_pattern_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the date/time pattern.

sync_add_datetime_format

<i>custom_format_id</i>	type:	unsigned integer
	access:	write only
	mechanism:	by reference

A unique format code generated by SyncSort to identify the custom date/time format. You can use this code where a data format code is accepted.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The custom date/time format is added to the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_set_numeric_value

Purpose

To replace the contents of a named number.

Format

sync_set_numeric_value *value_name, value_name_length, new_value, context*

Parameters

<i>value_name</i>	type:	character string
	access:	read only
	mechanism:	by reference

The name of the value whose contents you want to change. The named value appears in the application file identified by the *application_filename* parameter for **sync_load_application**.

<i>value_name_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the value name.

sync_set_numeric_value

<i>new_value</i>	type:	double precision float
	access:	read only
	mechanism:	by value

The new value of the named number.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The named value identified in *value_name* is a value for the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

sync_set_text_value

Purpose

To replace the contents of a named piece of text.

Format

sync_set_text_value

value_name, value_name_length, new_value, new_value_length, context

Parameters

value_name

type:

character string

access:

read only

mechanism:

by reference

The name of the value whose contents you want to change. The named value appears in the application file identified by the *application_filename* parameter for **sync_load_application**.

value_name_length

type:

unsigned integer

access:

read only

mechanism:

by value

The length in bytes of the value name.

sync_set_text_value

<i>new_value</i>	type:	character string
	access:	read only
	mechanism:	by reference

The new value of the named piece of text.

<i>new_value_length</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The length in bytes of the new value.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications. The named value identified in *value_name* is a value for the identified application.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

To select the join side to work on. Once selected, any subsequently invoked sub-routines that are side dependent will be applied to that side until the selection is changed.

sync_select_join_side	<i>side_number, context</i>
-----------------------	-----------------------------

<i>side_number</i>	type:	unsigned integer
	access:	read only
	mechanism:	by reference

The join side that you want to access. The left side is assigned number 1 and the right side is assigned number 2.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications.

sync_select_join_side

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

Notes

Prior to the first call to this subroutine, the left side (side 1) is selected.

Once you select a side by calling **sync_select_join_side**, any subsequent calls to sync_ subroutines that are side dependent are applied to that side until the selection is changed again. The subroutines affected by this call are:

sync_set_source_file

sync_add_source_file

sync_set_input_data_bytes

sync_set_input_record_count

sync_get_statistic

You cannot call this subroutine when the application is a sort, merge or copy.

sync_set_source_reader

Purpose

To provide SyncSort a reader object which will read source records into buffers.

Format

sync_set_source_reader *source_number, reader, context*

Parameters

<i>source_number</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The source that you want to associate a reader with. The sources are numbered sequentially from 1 in the order in which they appear in the application. Since only one source is supported currently when source records are obtained one buffer at a time, you must pass 1 for this parameter.

<i>reader</i>	type:	object
	mechanism:	by reference

The reader object that you want SyncSort to use to obtain source records one buffer at a time. The construct of this object is described in the section below as well as in the sync.h header file located in the ../syncsort/include directory.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

sync_set_source_reader

Identifies one application from a number of concurrent applications.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

Reader object

Purpose

To read source records into buffers allowing SyncSort to process the records when requested.

Member

<i>buffer</i>	type:	character string
---------------	-------	------------------

The buffer that stores the source records that were just read which SyncSort needs to process. This member must point to a valid buffer if the value of the member *buffer_size* is not 0.

Member

<i>buffer_size</i>	type:	unsigned integer
--------------------	-------	------------------

The amount (in bytes) of source records included in the buffer.

Member

sync_set_target_writer

Purpose

To provide SyncSort a writer object which when requested will supply buffers for SyncSort (to fill with target records) and write the buffers out.

Format

`sync_set_target_writer` *target_number, writer, context*

Parameters

<i>target_number</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The target that you want to associate a writer with. The targets are numbered sequentially from 1 in the order in which they appear in the application. Since only one target is supported currently when target records are output one buffer at a time, you must pass 1 for this parameter.

<i>writer</i>	type:	object
	mechanism:	by reference

The writer object that you want SyncSort to use to output target records one buffer at a time. The construct of this object is described in the section below as well as in the sync.h header file located in the ../syncsort/include directory.

<i>context</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

Identifies one application from a number of concurrent applications.

Returns

<i>completion_code</i>	type:	integer
	mechanism:	by value

For a description, see “Completion Status of SyncSort Subroutines” on page 2.9.

Writer object

Purpose

To supply buffers for SyncSort (to fill with target records) and write the buffers out.

Member

<i>buffer</i>	type:	character string
---------------	-------	------------------

The buffer which SyncSort can fill with target records.

Member

<i>buffer_size</i>	type:	unsigned integer
--------------------	-------	------------------

The size of the buffer in bytes.

Method

write *this, amount_filled, all_records_output*

SyncSort will invoke this method whenever it has completed filling the current buffer and ready to fill another one. When invoking the method, SyncSort will pass the amount (in bytes) of target records that SyncSort has put into the current buffer and whether SyncSort has output all the target records. SyncSort will always put as many complete records as possible into the buffer. SyncSort will leave a portion toward the end of the buffer unfilled only if that portion is not large enough for the next complete record. This method will be responsible for flushing the buffer and providing SyncSort with another buffer when not all records have been output. Before returning, this method needs to update members *buffer* and *buffer_size* to correctly indicate the location and size of the new buffer.

Parameters

<i>this</i>	type:	object
	mechanism:	by reference

This object.

<i>amount_filled</i>	type:	unsigned integer
	access:	read only
	mechanism:	by value

The amount (in bytes) of target records that SyncSort has put into the current buffer.

<i>all_records_output</i>	type:	unsigned integer
	access:	read only

mechanism: by value

Whether SyncSort has output all the target records. Set to 0 to indicate FALSE and 1 to indicate TRUE.

Returns

continue_output type: integer

mechanism: by value

Whether SyncSort should continue to output target records if not all of them have been output. Set to 0 to indicate FALSE and 1 to indicate TRUE.

Once SyncSort has output all the target records, this function will not be called again regardless what value it returns.

Program Invocation Examples

C Programs

```

/*****
/*
/*      EXAMPLE ONE:   SORTING OF TWO FILES
/*
/*      SOURCE: Files on disk
/*      TARGET: File on disk
/*
*****/

#include <stdio.h>                /* Standard I/O */
#include <string.h>               /* String handling */
#include <stdlib.h>               /* Standard C runtime */

#include "sync.h"                 /* SyncSort constants */

int main()
{
    /* SyncSort application information */
    char *application_name_ptr;   /* SyncSort application file name */
    unsigned application_name_len; /* Length of application file name */
    int status;                   /* SyncSort return status */
    unsigned most_severe_status;   /* Most severe SyncSort status */
    unsigned context = 0;          /* SyncSort context */

    void CheckStatus(char *routine_name_ptr, int status);
                                /* to check SyncSort status */

    /* Set up the SyncSort application information */
    application_name_ptr = "example1.srt";
    application_name_len = strlen(application_name_ptr);

    /* Load the SyncSort application */
    status = sync_load_application(application_name_ptr, application_name_len,
                                0, 0, /* SyncSort application file */
                                /* no log file */
                                &context); /* SyncSort context */

    /* Check the return status of the sync_load_application invocation */
    CheckStatus("sync_load_application", status);

    /* Run the SyncSort application, checking the return status */
    status = sync_run(context);
    CheckStatus("sync_run", status);
    sync_get_highest_severity(&most_severe_status, context);

    /* Terminate the SyncSort application, checking the return status */
    status = sync_clear(context);
    CheckStatus("sync_clear", status);

    return (most_severe_status);
}

void CheckStatus(char *routine_name_ptr,
                int status)
{
    /* routine whose status to be checked */
    /* status to be checked */

```

```

/*****
/* PURPOSE:
/* This routine checks the state of SyncSort during execution. It is invoked
/* after every SyncSort API call to ensure SyncSort has not encountered any
/* problems. CheckStatus sends a message to standard output when the status
/* and/or error indicate a problem.
/*
/* After a fatal error, any SyncSort API call made is not guaranteed to
/* succeed. SyncSort execution must be ended immediately. CheckStatus exits
/* the program.
*****/

int error;
error = sync_get_error_number(); /* specific error returned by SyncSort */

if (status == SYNC_SUCCESS)
{
    if(error != 0)
    {
        printf("SyncSort routine %s completed successfully with ",
               routine_name_ptr);
        printf("exceptions.\n");
        printf("Exception is %d.\n", error);
    }
}
else
{
    /* Fatal error - cannot proceed any further */
    printf("SyncSort routine %s terminated prematurely. Error is %d.\n",
           routine_name_ptr, error);
    exit(status);
}
}

```

```

/*****
/*
/*      EXAMPLE TWO:      COPY WITH SELECTION
/*
/*      SOURCE:      File on disk
/*      TARGET:      File on disk
/*
/*
/*****

#include <stdio.h>                                /* standard I/O */
#include <string.h>                                /* string handling */
#include <stdlib.h>                                /* standard C runtime */

#include "sync.h"                                  /* SyncSort constants */

int main()
{
/* SyncSort application information */
char *application_name_ptr;                       /* SyncSort application file name */
unsigned application_name_len;                    /* length of application file name */
int status;                                       /* SyncSort return status */
unsigned most_severe_status;                     /* most severe SyncSort status */
unsigned context = 0;                            /* SyncSort context */

void CheckStatus(char *routine_name_ptr, int status); /* to check SyncSort status */

/* Set up the SyncSort application information */
application_name_ptr = "Example2.srt";

application_name_len = strlen(application_name_ptr);

/* Load the SyncSort application */
status = sync_load_application(application_name_ptr, application_name_len,
                                0, 0,                                /* SyncSort application file */
                                &context);                          /* no log file */
                                /* SyncSort context */

/* Check the return status of the sync_load_application invocation */
CheckStatus("sync_load_application", status);

/* Run the SyncSort application, checking the return status */
status = sync_run(context);
CheckStatus("sync_run", status);
sync_get_highest_severity(&most_severe_status, context);

/* Terminate the SyncSort application, checking the return status */
status = sync_clear(context);
CheckStatus("sync_clear", status);

return (most_severe_status);
}

void CheckStatus(char *routine_name_ptr,
int status)                                       /* routine whose status to be checked */
{                                                 /* status to be checked */

/*****

```

```

/* PURPOSE:                                                                    */
/* This routine checks the state of SyncSort during execution. It is invoked */
/* after every SyncSort API call to ensure SyncSort has not encountered any */
/* problems. CheckStatus sends a message to standard output when the status */
/* and/or error indicate a problem.                                          */
/*                                                                            */
/* After a fatal error, any SyncSort API call made is not guaranteed to     */
/* succeed. SyncSort execution must be ended immediately. CheckStatus exits */
/* the program.                                                              */
/*****                                                                    */

int error;                                                                    /* specific error returned by SyncSort */
error = sync_get_error_number();

if (status == SYNC_SUCCESS)
{
    if(error != 0)
    {
        printf("SyncSort routine %s completed successfully with ",
                routine_name_ptr);
        printf("exceptions.\n");
        printf("Exception is %d.\n", error);
    }
}
else
{
    /* Fatal error - cannot proceed any further */
    printf("SyncSort routine %s terminated prematurely. Error is %d.\n",
           routine_name_ptr, error);
    exit(status);
}
}

```

```

/*****
/*      EXAMPLE FIVE:  CONCURRENT APPLICATIONS      */
/*      */
/*      Application 1 ( Sort )
/*      */
/*      SOURCE      - Records one at a time in memory
/*      TARGET      - Records one at a time in memory
/*      */
/*      Application 2 ( Copy with Selection)
/*      */
/*      SOURCE      - Records one at a time in memory
/*      TARGET      - File on Disk
*****/

#include <stdio.h>          /* standard I/O */
#include <string.h>         /* string handling */
#include <malloc.h>         /* memory allocation */
#include <stdlib.h>         /* standard C runtime */

#include "sync.h"           /* SyncSort constants */

unsigned context1 = 0;      /* SyncSort context for sort */
unsigned context2 = 0;      /* SyncSort context for copy */

FILE      *infile;         /* file which supplies records */
FILE      *outfile         /* file to receive records from sort */

void abort_all();          /* to end both sort and copy */

#define MAX_RECORD_LEN 100 /* maximum source record length */

int main()
{
/* SyncSort application information - sort */
char  *application1_name_ptr; /* SyncSort application file name */
unsigned application1_name_len; /* length of application file name */
unsigned most_severe_status1; /* most severe SyncSort status */

/* SyncSort application information - copy */
char  *application2_name_ptr; /* SyncSort application definition */
unsigned application2_name_len; /* length of definition */
unsigned most_severe_status2; /* most severe SyncSort status */

int      status;           /* SyncSort return status */

void      CheckStatus(char *routine_name_ptr, int status);
/* to check SyncSort status */

/* Passing records to SyncSort one at a time - sort + copy */
void SupplySource();        /* to supply records */

/* Retrieving records from SyncSort one at a time - sort */
void RetrieveRecords();     /* to retrieve records */

/* Open file which is the source of the records */
if ((infile =
      fopen("ex5input1.dat", "r"))
    == NULL )
{
    printf("Error opening the source file.\n");
    exit (-1);
}

/* Set up the SyncSort sort application information */
application1_name_ptr =

```

```

        "Example5Sort.srt";
        application1_name_len = strlen(application1_name_ptr);

/* Set up the SyncSort copy application information */
        application2_name_ptr =
            "Example5Copy.srt";
        application2_name_len = strlen(application2_name_ptr);

/* Initiate the SyncSort sort application */
        status = sync_load_application(application1_name_ptr, application1_name_len,
                                        /* SyncSort application file */
                                        0, 0, /* no log file */
                                        &context1); /* SyncSort context */

/* Check the return status of the sync_load_application invocation */
        CheckStatus("sync_load_application for sort", status);

/* Run the sort application, checking the return status */
        status = sync_run(context1);
        CheckStatus("sync_run for sort", status);

/* Initiate the SyncSort copy application */
        status = sync_load_application(application2_name_ptr, application2_name_len,
                                        /* SyncSort application file */
                                        0, 0, /* no log file */
                                        &context2); /* SyncSort context */

/* Check the return status of the sync_load_application invocation */
        CheckStatus("sync_load_application for copy", status);

/* Run the copy application, checking the return status */
        status = sync_run(context2);
        CheckStatus("sync_run for copy", status);

/* Supply the records to the sort + copy */
        SupplySource();

/* Terminate the copy, checking the return status */
        sync_get_highest_severity(&most_severe_status2, context2);
        status = sync_clear(context2);
        CheckStatus("sync_clear for copy", status);

/* Close source file */
        fclose(infile);

/* Retrieve the records from the sort */
        RetrieveRecords();

/* Terminate the sort, checking the return status */
        sync_get_highest_severity(&most_severe_status1, context1);
        status = sync_clear(context1);
        CheckStatus("sync_clear for sort", status);

        return (most_severe_status1 > most_severe_status2 ?
                most_severe_status1 : most_severe_status2);
    }

void CheckStatus(char *routine_name_ptr,
                 int status) /* routine whose status to be checked */
{                          /* status to be checked */

/*****

```

```

/* PURPOSE:                                                                    */
/* This routine checks the state of SyncSort during execution. It is invoked  */
/* after every SyncSort API call to ensure SyncSort has not encountered any  */
/* problems. CheckStatus sends a message to standard output when the status  */
/* and/or error indicate a problem.                                          */
/*                                                                            */
/* After a fatal error, any SyncSort API call made is not guaranteed to      */
/* succeed. SyncSort execution must be ended immediately. CheckStatus exits */
/* the program.                                                              */
/*****                                                                    */

int error;                                                                    /* specific error returned by SyncSort */
error = sync_get_error_number();

if (status == SYNC_SUCCESS)
{
    if(error != 0)
    {
        printf("SyncSort routine %s completed successfully with ",
                routine_name_ptr);
        printf("exceptions.\n");
        printf("Exception is %d.\n", error);
    }
}
else
{
    /* Fatal error - cannot proceed any further */
    printf("SyncSort routine %s terminated prematurely. Error is %d.\n",
           routine_name_ptr, error);
    exit(status);
}
}

void SupplySource()
{
/*****                                                                    */
/* PURPOSE: This routine supplies records one at a time to SyncSort.      */
/*****                                                                    */

    int    status;                                                            /* SyncSort return status */
    char    *record_buffer_ptr = NULL;                                       /* buffer to hold next record */
    unsigned record_buffer_len = 0;                                           /* length of record buffer */
    unsigned record_len;                                                       /* length of next record */

    int     NextSourceRecord(char *buffer_ptr, unsigned *size_ptr);           /* to create single record */

/* Allocate storage for a single record */
record_buffer_len = MAX_RECORD_LEN + 3;
/* to allow for terminators + null */
record_buffer_ptr = (char *) malloc(record_buffer_len);
record_len = record_buffer_len;

/* Generate and send each record to SyncSort in turn */
while( NextSourceRecord(record_buffer_ptr, &record_len) )
{
    /* Send the record to the sort, checking the return status */
    status = sync_put_record(record_buffer_ptr, record_len, context1);
    CheckStatus("sync_put_record for sort", status);

    /* Send the record to the copy, checking the return status */
    status = sync_put_record(record_buffer_ptr, record_len, context2);
    CheckStatus("sync_put_record for copy", status);

    record_len = record_buffer_len;
}
}

```

```

/* Inform SyncSort that all records have been supplied - sort */
status = sync_put_record_end(context1);
CheckStatus("sync_put_record_end for sort", status);

/* Inform SyncSort that all records have been supplied - copy */
status = sync_put_record_end(context2);
CheckStatus("sync_put_record_end for copy", status);

/* Free the buffer storage */
if(record_buffer_ptr)
    free(record_buffer_ptr);
}

int NextSourceRecord(char *record_buffer_ptr, /* location of buffer to hold record */
                    unsigned *size_ptr) /* size:
                                         on input: available length of buffer
                                         on output: length of supplied record */
{
    /*******
    /* PURPOSE: This routine produces a single record to be supplied to SyncSort. */
    /* */
    /* The buffer to hold the requested record is provided by SupplySource and */
    /* must not be freed. NextSourceRecord is responsible for ensuring that */
    /* bounds of the buffer are respected. */
    /*******

    int status = 1; /* function return */

    /* Supply the next record */
    if ( fgets(record_buffer_ptr, *size_ptr, infile) != NULL )
        /* Perform individual record pre-processing here */
        *size_ptr = strcspn(record_buffer_ptr, "\n");
    else
    {
        *size_ptr = 0;
        if ( feof(infile) != 0 )
            status = 0;
        else
        {
            printf("Error reading record from source file.\n");
            abort_all();
        }
    }

    return (status);
}

void RetrieveRecords()
{
    /*******
    /* PURPOSE: This routine retrieves records one at a time from SyncSort. */
    /*******

    int status; /* SyncSort return status */
    char *record_buffer_ptr = NULL; /* buffer to hold next record */
    unsigned record_buffer_len = 0; /* length of record buffer */
    unsigned record_len; /* length of next record */

    void NextTargetRecord(char *buffer_ptr, unsigned size);

    /* to process a single record */

```

```

/* Allocate storage for a single record */
record_buffer_len = MAX_RECORD_LEN + 3;
record_buffer_ptr = (char *) malloc(record_buffer_len); /* to allow for terminators + null */

/* Open the file to hold the sorted records */
if ( (outfile = fopen("output_file1.dat", "w")) == NULL )
{
    printf("Error opening the output file.\n");
    abort_all();
}

/* Obtain each record from SyncSort in turn */
do
{
    /* Initialize the available buffer length */
    record_len = record_buffer_len;

    /* Retrieve the record from SyncSort */
    status = sync_get_record(record_buffer_ptr, &record_len, context1);

    if(status != SYNC_END_OF_OUTPUT)
    {
        /* Record returned by SyncSort; check the return status */
        CheckStatus("sync_return_record for sort", status);
        /* Process the returned record */
        NextTargetRecord(record_buffer_ptr, record_len);
    }
} while (status != SYNC_END_OF_OUTPUT);

/* Free the buffer storage */
if(record_buffer_ptr)
    free(record_buffer_ptr);

/* Close the output file */
fclose (outfile);
}

void NextTargetRecord(char *record_buffer_ptr,
                     unsigned size) /* location of buffer holding record */
                                   /* length of the record */
{
    /******
    /* PURPOSE: This routine post-processes a single record returned by SyncSort.*/
    /******
    /* Perform individual record post-processing here */
    *(record_buffer_ptr + size) = '\n'; /* add terminator */
    *(record_buffer_ptr + size + 1) = '\0'; /* add null */
    /* Write out record to file */
    if ( (fputs(record_buffer_ptr, outfile)) == EOF )
    {
        printf("Error writing record to output file.\n");
        abort_all();
    }
}

void abort_all()
{

```

```

/*****
/*  PURPOSE: This routine ends SyncSort and exits the program when an error is */
/*           encountered with an external file.                               */
/*                                           */
/*****
    if (context1 != 0)
        sync_clear(context1);
    if (context2 != 0)
        sync_clear(context2);
    exit(-1);
}

```

```

/*****
/*      EXAMPLE SIX:  Copy to Table      */
/*                                          */
/*      SOURCE:  File on disk             */
/*      TARGET:  Table in memory          */
*****/

#include <stdio.h>                /* standard I/O */
#include <malloc.h>               /* memory allocation */
#include <string.h>               /* string handling */
#include <stdlib.h>               /* standard C runtime */

#include "sync.h"                 /* SyncSort constants */

int main()
{
    /* SyncSort application information */
    char *application_name_ptr;    /* SyncSort application file name */
    unsigned application_name_len; /* length of application file name */
    int status;                   /* SyncSort return status */
    unsigned most_severe_status;   /* most severe SyncSort status */
    unsigned context = 0;          /* SyncSort context */

    void    CheckStatus(char *routine_name_ptr, int status);
                                   /* to check SyncSort status */

    /* Retrieving records from SyncSort in a table */
    void CreateTargetTable(char **target_table_ptr, unsigned *size_ptr);
    /* to create target table */
    void ProcessTargetTable(char *target_table_ptr, unsigned size);
                                   /* to process target table */
    char *target_table_ptr = NULL; /* table for sorted records */
    unsigned target_table_len = 0; /* length of target table */

    /* Set up the SyncSort application information */
    application_name_ptr = "Example6.srt";
    application_name_len = strlen(application_name_ptr);

    /* Load the SyncSort application */
    status = sync_load_application(application_name_ptr, application_name_len,
                                   0, 0, /* SyncSort application file */
                                   &context); /* no log file */
                                   /* SyncSort context */

    /* Check the return status of the sync_load_application invocation */
    CheckStatus("sync_load_application", status);

    /* Create the table to hold the records returned by SyncSort */
    CreateTargetTable(&target_table_ptr, &target_table_len);

    /* Inform SyncSort of the location of the target table */
    status = sync_set_target_buffer(target_table_ptr, &target_table_len,
                                    context);
    CheckStatus("sync_set_target_buffer", status);

    /* Run the SyncSort application, checking the return status */
    status = sync_run(context);
    CheckStatus("sync_run", status);
    sync_get_highest_severity(&most_severe_status, context);

    /* Terminate the SyncSort application, checking the return status */
    status = sync_clear(context);
    CheckStatus("sync_clear", status);
    /* Process the table of records returned by SyncSort */

```

```

    ProcessTargetTable(target_table_ptr, target_table_len);

    return (most_severe_status);
}

void CheckStatus(char *routine_name_ptr, /* routine whose status to be checked */
                 int status)           /* status to be checked */
{
    /******
    /*  PURPOSE:
    /*  This routine checks the state of SyncSort during execution. It is invoked
    /*  after every SyncSort API call to ensure SyncSort has not encountered any
    /*  problems. CheckStatus sends a message to standard output when the status
    /*  and/or error indicate a problem.
    /*
    /*  After a fatal error, any SyncSort API call made is not guaranteed to
    /*  succeed. SyncSort execution must be ended immediately. CheckStatus exits
    /*  the program.
    /******

    int error; /* specific error returned by SyncSort */
    error = sync_get_error_number();

    if (status == SYNC_SUCCESS)
    {
        if(error != 0)
        {
            printf("SyncSort routine %s completed successfully with ",
                   routine_name_ptr);
            printf("exceptions.\n");
            printf("Exception is %d.\n", error);
        }
    }
    else
    {
        /* Fatal error - cannot proceed any further */
        printf("SyncSort routine %s terminated prematurely. Error is %d.\n",
               routine_name_ptr, error);
        exit(status);
    }
}

void CreateTargetTable(char **record_table_ptr, /* location of table to hold records */
                      unsigned *size_ptr)      /* size of table */
{
    /******
    /*  PURPOSE: This routine creates the table to hold the records returned by
    /*  SyncSort.
    /******
    /* Allocate the memory for the table */
    #define OUT_TABLE_SIZE 400
    *record_table_ptr = (char *) malloc(OUT_TABLE_SIZE);
    *size_ptr = OUT_TABLE_SIZE;
}

void ProcessTargetTable(char *record_table_ptr, /* location of table holding records */
                       unsigned size)          /* size of table */

```

```

{
/*****
/* PURPOSE: This routine is used to post-process the records returned by */
/*          SyncSort. */
/*          The memory for the table is freed. */
/*****
/* Process the records in the table */
#define OUT_RECORD_SIZE 80
    char next_record[OUT_RECORD_SIZE + 1];
    unsigned length_used = 0;          /* buffer for next record */
                                     /* length of table processed */

    while (length_used < size)
    {
        memcpy(next_record, (record_table_ptr + length_used), OUT_RECORD_SIZE);
        next_record[OUT_RECORD_SIZE] = 0;
        printf("%s\n", next_record);
        length_used = length_used + OUT_RECORD_SIZE;
    }

/* Free the table storage */
if(record_table_ptr)
    free(record_table_ptr);
}

```

```

/*****

EXAMPLE NINE : SORT MULTIPLE TABLES IN MEMORY
SOURCE       - MULTIPLE TABLES IN MEMORY
TARGET       - FILE ON DISK
APPLICATION   - SORT

*****/

#include <stdio.h>           /* standard I/O */
#include <malloc.h>         /* memory allocation */
#include <string.h>         /* string handling */
#include <stdlib.h>         /* standard C runtime */

#include "sync.h"           /* SyncSort constants */

int main()
{
    /* SyncSort application information */
    char *application_name_ptr; /* SyncSort application file name */
    unsigned application_name_len; /* length of application file name */
    int status; /* SyncSort return status */
    unsigned most_severe_status; /* most severe SyncSort status */
    unsigned context = 0; /* SyncSort context */

    void CheckStatus(char *routine_name_ptr, int status);
                                /* to check SyncSort status */

    /* Supplying records to SyncSort in multiple tables */
    struct sync_source_reader source_reader;
                                /* object that can supply tables
                                of source records to SyncSort */

    void SYNCAPI NextSourceTable
        (struct sync_source_reader *source_reader_ptr);
                                /* to supply the next table.
                                SyncSort will invokes this routine
                                when it has completed processing
                                the current table and is ready for
                                the next table */

    /* Set up the SyncSort application information */
    application_name_ptr = "Example9.srt";
    application_name_len = strlen(application_name_ptr);

    /* Load the SyncSort application */
    status = sync_load_application(application_name_ptr, application_name_len,
                                0, 0, /* SyncSort application file */
                                /* no log file */
                                &context); /* SyncSort context */

    /* Check the return status of the sync_load_application invocation */
    CheckStatus("sync_load_application", status);

    /* Construct the source reader object that will supply the source tables */
    source_reader.all_records_read = 0;
    source_reader.read = &NextSourceTable;
    source_reader.buffer = NULL;
    source_reader.buffer_size = 0; /* Setting the buffer size to
                                0 causes SyncSort to invoke
                                the NextSourceTable routine
                                for the first source table.
                                Alternatively, the fields
                                buffer and buffer_size can
                                be set to the location of
                                the first table */

```

```

/* Pass the source reader object to SyncSort */
status = sync_set_source_reader( 1, &source_reader, context);
CheckStatus("sync_set_source_reader", status);

/* Run the SyncSort application, checking the return status */
status = sync_run(context);
CheckStatus("sync_run", status);
sync_get_highest_severity(&most_severe_status, context);

/* Terminate the SyncSort application, checking the return status */
status = sync_clear(context);
CheckStatus("sync_clear", status);

return (most_severe_status);
}

void CheckStatus(char *routine_name_ptr,
                 int status) /* routine whose status to be checked */
{ /* status to be checked */
    /******
    PURPOSE:
    This routine checks the state of SyncSort during execution. It is invoked
    after every SyncSort API call to ensure SyncSort has not encountered any
    problems. CheckStatus sends a message to standard output when the status
    and/or error indicate a problem.

    After a fatal error, any SyncSort API call made is not guaranteed to
    succeed. SyncSort execution must be ended immediately. CheckStatus exits
    the program.
    *****/

    int error; /* specific error returned by SyncSort */
    error = sync_get_error_number();

    if (status == SYNC_SUCCESS)
    {
        if(error != 0)
        {
            printf("SyncSort routine %s completed successfully with ",
routine_name_ptr);
            printf("exceptions.\n");
            printf("Exception is %d.\n", error);
        }
    }
    else
    {
        /* Fatal error - cannot proceed any further */
        printf("SyncSort routine %s terminated prematurely. Error is %d.\n",
routine_name_ptr, error);
        exit(status);
    }
}

void SYNCAPI NextSourceTable(struct sync_source_reader *source_reader_ptr)
{
    /******
    PURPOSE:
    This routine sets up a new table of source records.

    SyncSort will invoke this routine when it has completed processing the
    current table and is ready for the next table. This routine, before
    returning, needs to update the members buffer, buffer_size, and
    all_records_read of the source reader object to correctly indicate the
    location, size, and status of the new table.
    *****/

```

```

static int next_table_index = 1;                                /* the index of the next table to be
                                                                returned by this routine */

/* The first source table */
static char table_1[] =
"SEAMAN      JASON      250-78-5673      38000      Y      34      "
"ROQUEMORE   PETER      240-68-5783      39000      N      39      "
"REYNOLDS    ROBERT     230-34-1203      47000      N      31      ";

/* The second source table */
static char table_2[] =
"BOGART      REBECCA     240-36-1543      45000      Y      29      "
"GROSSMAN    HAROLD     230-04-1323      29000      Y      56      ";

/* The third source table */
static char table_3[] =
"PECK        JOHN       130-48-5563      36000      N      32      "
"STEVENSON   DENNIS     440-66-8785      40000      Y      59      "
"WESTALL     JAMES      930-34-1255      41000      N      38      "
"AMMONS      KARLEEN    220-31-4545      42000      N      24      "
"WRIGHT      TERRY      240-06-1587      27000      Y      28      ";

switch(next_table_index)
{
case 1:
    /* return the first table */
    source_reader_ptr->all_records_read = 0;
    source_reader_ptr->buffer = table_1;
    source_reader_ptr->buffer_size = sizeof(table_1)-1;
    next_table_index = 2;
    break;
case 2:
    /* return the second table */
    source_reader_ptr->all_records_read = 0;
    source_reader_ptr->buffer = table_2;
    source_reader_ptr->buffer_size = sizeof(table_2)-1;
    next_table_index = 3;
    break;
case 3:
    /* return the last table */
    source_reader_ptr->all_records_read = 1;
    source_reader_ptr->buffer = table_3;
    source_reader_ptr->buffer_size = sizeof(table_3)-1;
    next_table_index = 1;
    break;
}
}

```

Micro Focus COBOL Program Example

```
*****
*   EXAMPLE THREE: MERGE TWO FILES                               *
*   SOURCES      - FILES ON DISK                                *
*   TARGET       - FILE ON DISK                                 *
*   APPLICATION  - MERGE                                         *
*****

*****
* Micro Focus compiler directives                               *
*****

      IDENTIFICATION DIVISION.
      PROGRAM-ID.          MAIN-SYNCSORT.

      ENVIRONMENT DIVISION.

      DATA DIVISION.
      WORKING-STORAGE SECTION.

*****
* SyncSort mnemonics                                           *
*****
      COPY "syncsort.cbl".

*****
* SyncSort application information                               *
*****
      *   name of SyncSort application file
      01 APPLICATION-FILE-NAME      PIC X(12) VALUE
         "Example3.srt".
      *   length of file name
      01 APPLICATION-FILE-LENGTH    PIC 9(9) COMP-5 VALUE 12.
      *   SyncSort context
      01 CONTEXT                    PIC 9(9) COMP-5 VALUE 0.
      *   return status from SyncSort
      01 RETURN-STATUS              PIC S9(9) COMP-5.

*****
* Error handling                                               *
*****
      *   SyncSort specific error
      77 ERROR-NUMBER                PIC S9(9) COMP-5.
      *   subroutine name string
      77 SUBROUTINE-STRING           PIC X(30).
      *   most severe status encountered
      77 MOST-SEVERE-STATUS          PIC 9(9) COMP-5.

      PROCEDURE DIVISION.

      INIT-SYNCSORT.
      *   Initiate the SyncSort application
      CALL SYNCSORT-API "sync_load_application"
      USING
         BY REFERENCE APPLICATION-FILE-NAME,
         BY VALUE APPLICATION-FILE-LENGTH,
      *       SyncSort application definition
         BY VALUE 0, 0,
      *       no log file
         BY REFERENCE CONTEXT
      RETURNING
         RETURN-STATUS.

      *   Check the return status of the call
      MOVE "sync_load_application" TO SUBROUTINE-STRING.
```

```
PERFORM CHECK-STATUS THRU END-CHECK-STATUS.
```

```
RUN-SYNCSORT.
```

```
* Run the SyncSort application
CALL "sync_run"
  USING
    BY VALUE CONTEXT
  RETURNING
  RETURN-STATUS.
```

```
* Check the return status of the call
MOVE "sync_run" TO SUBROUTINE-STRING.
PERFORM CHECK-STATUS THRU END-CHECK-STATUS.
```

```
END-SYNCSORT.
```

```
* Obtain the most severe status for the application
CALL "sync_get_highest_severity"
  USING
    BY REFERENCE MOST-SEVERE-STATUS,
    BY VALUE CONTEXT
  RETURNING
  RETURN-STATUS.
```

```
* Terminate the SyncSort application
CALL "sync_clear"
  USING
    BY VALUE CONTEXT
  RETURNING
  RETURN-STATUS.
```

```
* Check the return status of the call
MOVE "sync_clear" TO SUBROUTINE-STRING.
PERFORM CHECK-STATUS THRU END-CHECK-STATUS.
```

```
END-MAIN.
```

```
STOP RUN
  RETURNING
  MOST-SEVERE-STATUS.
```

```
CHECK-STATUS.
```

```
*****
* PURPOSE:
* This routine checks the state of SyncSort during execution. It
* is invoked after every SyncSort API call to ensure SyncSort has
* not encountered any problems. CHECK-STATUS displays a message
* when the status and/or ERROR-NUMBER indicate a problem.
*
* After a fatal error, any SyncSort API call made is not
* guaranteed to succeed. SyncSort execution must be ended
* immediately. CHECK-STATUS exits the program.
*****
```

```
CALL "sync_get_error_number"
  RETURNING
  ERROR-NUMBER.
```

```
IF RETURN-STATUS NOT EQUAL SYNC-SUCCESS
  DISPLAY "SyncSort subroutine ", SUBROUTINE-STRING
  IF RETURN-STATUS EQUAL SYNC-ERROR
    DISPLAY " has completed but output may be incorrect
    " or incomplete. Error is ", ERROR-NUMBER
    MOVE SYNC-ERROR TO MOST-SEVERE-STATUS
  ELSE
    DISPLAY " has terminated prematurely. Error is ",
    ERROR-NUMBER
```

```
                STOP RUN
                RETURNING
                RETURN-STATUS
ELSE
    IF ERROR-NUMBER NOT EQUAL 0
        DISPLAY "SyncSort subroutine ", SUBROUTINE-STRING
        DISPLAY " has completed successfully with minor
        " exceptions. Exception is ", ERROR-NUMBER.
END-CHECK-STATUS.
EXIT.

END PROGRAM MAIN-SYNCSORT.
```

```

*****
*   EXAMPLE SEVEN: SORT TABLE TO PROCEDURE                               *
*   SOURCE          - TABLE IN MEMORY                                   *
*   TARGET          - RECORDS ONE AT A TIME IN MEMORY                   *
*   APPLICATION     - SORT                                              *
*****

*****
* Micro Focus compiler directives                                         *
*****

IDENTIFICATION DIVISION.
PROGRAM-ID.          MAIN-SYNSORT.

ENVIRONMENT DIVISION.

DATA DIVISION.
WORKING-STORAGE SECTION.

*****
* SyncSort mnemonics                                                     *
*****
COPY "syncsort.cbl".

*****
* SyncSort application information                                         *
*****
*   name of SyncSort application file
01 APPLICATION-FILE-NAME PIC X(12) VALUE
   "Example7.srt".
*   length of file name
01 APPLICATION-FILE-LENGTH PIC 9(9) COMP-5 VALUE 12.
*   SyncSort context
01 CONTEXT PIC 9(9) COMP-5 VALUE 0.
*   return status from SyncSort
01 RETURN-STATUS PIC S9(9) COMP-5.

*****
* Error handling                                                         *
*****
*   SyncSort specific error
77 ERROR-NUMBER PIC S9(9) COMP-5.
*   subroutine name string
77 SUBROUTINE-STRING PIC X(30).
*   most severe status encountered
77 MOST-SEVERE-STATUS PIC 9(9) COMP-5.

*****
* Supplying records to SyncSort in a table                               *
*****
*   record table
01 SOURCE-TABLE.
05 FILLER PIC X(60) VALUE
   "SEAMAN JASON 250-78-5673 38000 Y 34".
05 FILLER PIC X(60) VALUE
   "ROQUEMORE PETER 240-68-5783 39000 N 39".
05 FILLER PIC X(60) VALUE
   "REYNOLDS ROBERT 230-34-1203 47000 N 31".
05 FILLER PIC X(60) VALUE
   "BOGART REBECCA 240-36-1543 45000 Y 29".
05 FILLER PIC X(60) VALUE
   "GROSSMAN HAROLD 230-04-1323 29000 Y 56".
05 FILLER PIC X(60) VALUE
   "PECK JOHN 130-48-5563 36000 N 32".
05 FILLER PIC X(60) VALUE
   "STEVENSON DENNIS 440-66-8785 40000 Y 59".

```

```

05 FILLER    PIC X(60) VALUE
"WESTALL    JAMES      930-34-1255      41000    N      38".
05 FILLER    PIC X(60) VALUE
"AMMONS     KARLEEN    220-31-4545      42000    N      24".
05 FILLER    PIC X(60) VALUE
"WRIGHT     TERRY      240-06-1587      27000    Y      28".
* length of record table
01 SOURCE-TABLE-LENGTH          PIC 9(9) COMP-5 VALUE 600.

*****
* Retrieving records from SyncSort one at a time *
*****
* record storage
01 TARGET-RECORD                PIC X(60).
* length of record passed in storage
01 TARGET-RECORD-LENGTH         PIC 9(9) COMP-5.
* length of record storage
77 TARGET-STORAGE-LENGTH        PIC 9(9) COMP-5 VALUE 60.

PROCEDURE DIVISION.

INIT-SYNCSORT.
* Initiate the SyncSort application
CALL "sync_load_application"
  USING
    BY REFERENCE APPLICATION-FILE-NAME,
    BY VALUE APPLICATION-FILE-LENGTH,
* SyncSort application definition
    BY VALUE 0, 0,
* no log file
    BY REFERENCE CONTEXT
  RETURNING
    RETURN-STATUS.

* Check the return status of the call
MOVE "sync_load_application" TO SUBROUTINE-STRING.
PERFORM CHECK-STATUS THRU END-CHECK-STATUS.

* Indicate the location of the source table to SyncSort
CALL "sync_set_source_buffer"
  USING
    BY REFERENCE SOURCE-TABLE,
    BY VALUE SOURCE-TABLE-LENGTH,
* source table
    BY VALUE CONTEXT
  RETURNING
    RETURN-STATUS.

* Check the return status of the call
MOVE "sync_set_source_buffer" TO SUBROUTINE-STRING.
PERFORM CHECK-STATUS THRU END-CHECK-STATUS.

RUN-SYNCSORT.
* Run the SyncSort application
CALL "sync_run"
  USING
    BY VALUE CONTEXT
  RETURNING
    RETURN-STATUS.

* Check the return status of the call
MOVE "sync_run" TO SUBROUTINE-STRING.
PERFORM CHECK-STATUS THRU END-CHECK-STATUS.

GET-RECORDS.
* Retrieve the records from SyncSort
PERFORM RETRIEVE-RECORDS THRU END-RETRIEVE-RECORDS.

```

```

END-SYNCSORT.
*   Obtain the most severe status for the application
    CALL "sync_get_highest_severity"
        USING
            BY REFERENCE MOST-SEVERE-STATUS,
            BY VALUE CONTEXT
        RETURNING
        RETURN-STATUS.

*   Terminate the SyncSort application
    CALL "sync_clear"
        USING
            BY VALUE CONTEXT
        RETURNING
        RETURN-STATUS.

*   Check the return status of the call
    MOVE "sync_clear" TO SUBROUTINE-STRING.
    PERFORM CHECK-STATUS THRU END-CHECK-STATUS.

END-MAIN.
    STOP RUN
    RETURNING
    MOST-SEVERE-STATUS.

CHECK-STATUS.
*****
* PURPOSE:
* This routine checks the state of SyncSort during execution. It
* is invoked after every SyncSort API call to ensure SyncSort has
* not encountered any problems. CHECK-STATUS displays a message
* when the status and/or ERROR-NUMBER indicate a problem.
*
* After a fatal error, any SyncSort API call made is not
* guaranteed to succeed. SyncSort execution must be ended
* immediately. CHECK-STATUS exits the program.
*****
    CALL "sync_get_error_number"
        RETURNING
        ERROR-NUMBER.

    IF RETURN-STATUS NOT EQUAL SYNC-SUCCESS
        DISPLAY "SyncSort subroutine ", SUBROUTINE-STRING
        DISPLAY " has terminated prematurely. Error is ",
            ERROR-NUMBER
        STOP RUN
        RETURNING
        RETURN-STATUS
    ELSE
        IF ERROR-NUMBER NOT EQUAL 0
            DISPLAY "SyncSort subroutine ", SUBROUTINE-STRING
            DISPLAY " has completed successfully with minor
                " exceptions. Exception is ", ERROR-NUMBER.

END-CHECK-STATUS.
EXIT.

RETRIEVE-RECORDS.
*****
* PURPOSE: This routine retrieves records one at a time from SyncSort.*
*****
    MOVE SYNC-SUCCESS TO RETURN-STATUS.
    PERFORM RETRIEVE-NEXT-RECORD THRU END-RETRIEVE-NEXT-RECORD
        UNTIL RETURN-STATUS EQUAL SYNC-END-OF-OUTPUT.

END-RETRIEVE-RECORDS.

```

```

EXIT.

RETRIEVE-NEXT-RECORD.
*****
* PURPOSE: This routine obtains a record from SyncSort. *
*****
      MOVE TARGET-STORAGE-LENGTH TO TARGET-RECORD-LENGTH.
      CALL SYNC SORT-API "sync_get_record"
      USING
          TARGET-RECORD,
          TARGET-RECORD-LENGTH,
          BY VALUE CONTEXT
      RETURNING
          RETURN-STATUS.

*      Check the return status of the call
      IF RETURN-STATUS NOT EQUAL SYNC-END-OF-OUTPUT
          MOVE "sync_get_record" TO SUBROUTINE-STRING
          PERFORM CHECK-STATUS THRU END-CHECK-STATUS
          PERFORM NEXT-TARGET-RECORD THRU END-NEXT-TARGET-RECORD.

END-RETRIEVE-NEXT-RECORD.
EXIT.

NEXT-TARGET-RECORD.
*****
* PURPOSE: This routine post-processes a single record returned by Syncsort.*
*****

*      Process the next record
*      Do individual record post-processing here
      DISPLAY TARGET-RECORD.

END-NEXT-TARGET-RECORD.
EXIT.
END PROGRAM MAIN-SYNC SORT.

```

2.3 The SyncSort Korn Shell Interface

The SyncSort Korn Shell interface gives you access to the full functionality of the application you develop with Visual SyncSort in a UNIX Korn Shell scripting environment. The SyncSort Korn Shell interface provides a set of shell functions for modifying applications developed with the SyncSort Graphical Interface Component, Visual SyncSort. This interface allows the application generated by the graphical interface to be modified at execution time with minimal development effort.

Visual SyncSort can be used to generate the template for a UNIX Korn shell script with all the necessary invocations of SyncSort Korn shell functions to execute the sort, merge, or copy. The template can form a basis for the script needed for your application suite, or you can extract sections of the template for inclusion in your script. To generate a script template, select the Generate Program command from the Tools menu. After you generate the template you may also wish to dynamically change the application when it is run, which is useful for automating frequently executed tasks. The following functions will allow you to change your application dynamically from within your UNIX Korn Shell script.

The SyncSort shell functions allow for multiple concurrent invocations. In this way, you can run several SyncSort applications simultaneously without the restriction that one completes before another begins. Most functions provide an *appId* parameter that identifies the particular invocation. SyncSort returns the *appId* to identify the invocation from the initial shell function **SyncNew** that initializes the invocation. You must provide the same *appId* to all subsequent functions for the same invocation.

You can find several examples of working Korn Shell scripts using the SyncSort shell functions in the directory **SyncSort\Examples\KornShell**.

Notes

- When using the shell functions to modify your application you must include the following line in your shell script:

```
. SyncSort.ksh
```

-
- If SyncSort.ksh is not in your path, you need to specify the complete pathname of the file. The leading dot is significant. It defines the shell functions under the current shell.
 - After calling each shell function, the return code should be checked for success or failure before calling subsequent shell functions.
 - Any value returned by a function is sent to standard output.
 - Temporary files will be created with file names prefixed with **sync** in the directory defined by the environment variable **TMPDIR** or **TEMP**. If neither of these environmental variables are defined, /tmp will be used. A call to the shell function **SyncClear** will remove all temporary files.

Return code of Korn Shell Functions

Each shell function returns an integer return code, which indicates the success or failure of the function. For a description of return codes, see “Completion Status of SyncSort” on page 2.6. If a shell function encounters an error, messages will be written to standard error. To obtain additional information on an error message, see “Error Code Explanation” in the Visual SyncSort Help.

Functions to Run a Sort, Merge, Copy or Join

There are twelve Korn shell functions available to run a sort, merge, copy or join application.

SyncNew	create a new SyncSort invocation
SyncSetApplicationFileName	to set the application .srt file name
SyncSetSourceFile	overrides a source file name
SyncAddSourceFile	adds an additional source file name
SyncSetTargetFile	overrides a target file name
SyncSetEstimatedInputDataBytes	overrides the estimated amount of data to be sorted
SyncSetEstimatedInputRecordCount	overrides the estimated amount of records to be sorted
SyncSetNumericValue	replaces the contents of a named number
SyncSetTextValue	replaces the contents of a named piece of text
SyncSelectJoinSide	to select a join side to amend
SyncRun	runs the SyncSort application
SyncClear	terminates the application, releasing all resources

You do not need all the functions to run a sort, but you must call any functions you use in the order shown above. If you use more than one **SyncSet.../SyncAdd...** function, you can call them in any order. You cannot call the **SyncSelectJoin-Side** if the application is a sort, merge or copy application.

The functions are mandatory as follows:

SyncNew	for all sort, merge, copy and join applications
SyncSetApplicationFileName	for all sort, merge, copy and join applications
SyncRun	for all sort, merge, copy and join applications
SyncClear	for all sort, merge, copy and join applications

SyncNew

Purpose

To create a new SyncSort invocation.

Usage

appId = 'SyncNew'

Return code

0 or 111. For a description, see “Completion Status of SyncSort ” on page 2.6.

Return value

A unique string that identifies the SyncSort application. The return value is used in subsequent **Sync...** calls to identify the invocation.

SyncSetApplicationFileName

Purpose

To set the application .srt file name.

Usage

SyncSetApplicationFileName *appId* *appFile*

appId the application id returned by **SyncNew** that identifies the invocation.

appFile a string with the name of the SyncSort application **.srt** file created through the SyncSort graphical interface.

Return code

0 or 111. For a description, see “Completion Status of SyncSort” on page 2.6.

Notes

This function should be invoked once after creating the invocation via **SyncNew**. This call should be made before any **SyncSet.../SyncAdd...** functions and before running the application via **SyncRun**.

SyncSetEstimatedInputRecordCount

Purpose

To override the estimated number of records to be sorted.

Usage

SyncSetEstimatedInputRecordCount *appId count*

<i>appId</i>	the application id returned by SyncNew that identifies the invocation.
<i>count</i>	an integer or double value which represents the number of records that remain after applying any filter for all targets.

Return code

0 or 111. For a description, see “Completion Status of SyncSort ” on page 2.6.

SyncSetEstimatedInputDataBytes

Purpose

To override the estimated amount of data to be sorted.

Usage

SyncSetEstimatedInputDataBytes *appId datasize*

appId the application id returned by **SyncNew** that identifies the invocation.

datasize an integer or double value that represents the amount of data in bytes that remain after applying any filter for all targets.

Return code

0 or 111. For a description, see “Completion Status of SyncSort ” on page 2.6.

SyncSetSourceFile

Purpose

To override a source file name.

Usage

SyncSetSourceFile *appId filename [fileno]*

<i>appId</i>	the application id returned by SyncNew that identifies the invocation.
<i>filename</i>	a string with the name of the source file.
<i>fileno</i>	an integer with the number of the source file whose name you want to change. The files are numbered sequentially from 1 in the order in which they appear in the application file identified by the <i>appFile</i> argument for SyncSetApplicationFileName . The argument is optional and defaults to 1.

Return code

0 or 111. For a description, see “Completion Status of SyncSort” on page 2.6.

Notes

You can invoke this function at any time after specifying the application via **SyncSetApplicationFileName** and before running the application via **SyncRun**.

SyncAddSourceFile

Purpose

To add an additional source file name.

Usage

fileno = 'SyncAddSourceFile *appId* *filename*'

appId the application id returned by **SyncNew** that identifies the invocation.

filename a string with the name of the source file.

Return code

0 or 111. For a description, see “Completion Status of SyncSort” on page 2.6

Return value

An integer which represents number of the source file added to the SyncSort application.

Notes

You can invoke this function at any time after specifying the application via **Sync-SetApplicationFileName** and before running the application via **SyncRun**.

When SyncAddSourceFile adds a new source file, it assigns to the new file all the properties of the first source file, other than its name.

SyncSetTargetFile

Purpose

To override a target file name.

Usage

SyncSetTargetFile *appId filename [fileno]*

<i>appId</i>	the unique application id returned by a previous call to SyncNew .
<i>filename</i>	a string with the name of the target file.
<i>fileno</i>	an integer with the number of the target file whose name you want to change. The files are numbered sequentially from 1 in the order in which they appear in the application file identified by the <i>appFile</i> argument for SyncSetApplicationFileName . The argument is optional and defaults to 1.

Return code

0 or 111. For a description, see “Completion Status of SyncSort” on page 2.6.

Notes

You can invoke this function at any time after specifying the application via **SyncSetApplicationFileName** and before running the application via **SyncRun**.

SyncSetNumericValue

Purpose

To replace the contents of a named number.

Usage

SyncSetNumericValue *appId name value*

<i>appId</i>	the application id returned by SyncNew that identifies the invocation.
<i>name</i>	a string with the name of the value whose contents you want to change. The named value appears in the application file identified by the <i>appFile</i> parameter for the SyncSetApplicatonFileName .
<i>value</i>	the new value of the number.

Return code

0 or 111. For a description, see “Completion Status of SyncSort” on page 2.6.

Notes

You can invoke this function at any time after specifying the application via **SyncSetApplicationFileName** and before running the application via **SyncRun**.

SyncSetTextValue

Purpose

To replace the contents of a named piece of text.

Usage

SyncSetTextValue *appId name value*

<i>appId</i>	the application id returned by SyncNew that identifies the invocation.
<i>name</i>	a string with the name of the value whose contents you want to change. The named value appears in the application file identified by the <i>appFile</i> parameter for SyncSetApplicationFileName .
<i>value</i>	a string with the new value for the text.

Return code

0 or 111. For a description, see “Completion Status of SyncSort” on page 2.6.

Notes

You can invoke this function at any time after specifying the application via **SyncSetApplicationFileName** and before running the application via **SyncRun**.

SyncSelectJoinSide

Purpose

To select a join side to amend for a join application.

Usage

SyncSelectJoinside *appId side*

AppId the application id returned by **SyncNew** that identifies the invocation.

Side an unsigned integer indicating the join side that you want to access. The left side is assigned number 1 and the right side is assigned number the application id returned by **SyncNew** that identifies the invocation.

Return code

0, 1, 100 or 111. For a description, see “Completion Status of SyncSort ” on page 2.6.

Notes

Prior to the first call to this function, the left side (side 1) is selected.

Once you select a side by calling **SyncSelectJoinSide**, any subsequent calls to **Sync** functions that are side dependent are applied to that side until the selection is changed again. The functions affected by this call are:

SyncSetSourceFile

SyncAddSourceFile

SyncSetEstimatedInputRecordCount

SyncSetEstimatedInputDataBytes

You cannot call this function when the application is a sort, merge or copy.

SyncRun

Purpose

To run the SyncSort application.

Usage

SyncRun *appId*

appId the application id returned by **SyncNew** that identifies the invocation.

Return code

0, 1, 100 and 111. For a description, see “Completion Status of SyncSort ” on page 2.6.

Notes

You can invoke this function at any time after specifying the application via **SyncSetapplicationFileName** and before releasing resources via **SyncClear**.

This function will not complete until the application has completed. When this function completes all output should be generated. If your application is designed to read from standard input or write to standard output, data can be piped into or out of the application when invoking this function.

Example

```
cat infile | SyncRun $appId | cat >outfile
```

SyncClear

Purpose

To terminate the application, releasing all resources.

Usage

SyncClear *appId*

appId the application id returned by **SyncNew** that identifies the invocation.

Return code

0 and 111. For a description, see “Completion Status of SyncSort ” on page 2.6.

Notes

This function should be the last one to be called for an application. Do not call **SyncClear** if a call to any of the other functions fail with an error. Under such circumstances, SyncSort releases resources automatically.

SyncSort Korn Shell Examples

```
# This example uses a simple Korn shell script(SimpleScript.ksh) to run the srt
# application SimpleScript.srt.
#
#!/bin/ksh
# Import SyncSort shell functions
. SyncSort.ksh

# Create a new application
applicationID=`SyncNew` &&

# Set the application filename
SyncSetApplicationFileName $applicationID "SimpleScript.srt" &&

# Run the application
SyncRun $applicationID

# Print error code
echo "Status from running SyncSort application = $"

# End SyncSort application and clear resources
SyncClear $applicationID
```

```
# This example uses a Korn shell script (DyanmicDataSize.ksh) to change the
# estimated size of the data to be sorted before piping the data to the
# SyncSort application defined in DynamicDataSize.srt.

#!/bin/ksh
# Import SyncSort shell functions
. SyncSort.ksh

# Create a new application
applicationID=`SyncNew` &&

# Set the application filename
SyncSetApplicationFileName $applicationID "DynamicDataSize.srt" &&

# Specify input data size
SyncSetEstimatedInputDataBytes $applicationID 55702 &&

# Pipe input data and run the application
cat today.log | SyncRun $applicationID

# Print error code
echo "Status from running SyncSort application = $?"

# End SyncSort application and clear resources
SyncClear $applicationID
```

```
# This example uses a Korn Shell script (DynamicSourceandTarget.ksh) to change the
# source and target files of the srt application DynamicSourceandTarget.srt at
# execution time.
#
#
#!/bin/ksh
# Import SyncSort shell functions
. SyncSort.ksh

# Create a new application

applicationID=`SyncNew` &&
# Set the application filename
SyncSetApplicationFileName $applicationID "DynamicSourceAndTarget.srt" &&

# Add a new source file
filenum=`SyncAddSourceFile $applicationID "newinput.log"` &&

# Modify first target file name
SyncSetTargetFile $applicationID "newoutput.log" &&

# Run the application
SyncRun $applicationID

# Print error code
echo "Status from running SyncSort application = $?"

# End SyncSort application and clear resources
SyncClear $applicationID
```

2.4 The Micro Focus COBOL SORT and MERGE Verbs

Compiling and Linking Programs Using SORT and MERGE Verbs

When you use the SORT and MERGE verbs in a Micro Focus COBOL program, an internal sort/merge function is invoked when the program executes. To invoke SyncSort to perform the sort or merge instead, you need to compile your program with a directive specifying that you want an external sort/merge, and link your COBOL program or run time system with the SyncSort library. This directive is available at release 3.1 and higher of Micro Focus COBOL.

To invoke SyncSort, add the directive CALLSORT or CALLSORT=EXTSM to your compiler directives.

Compiler directives can be specified in cob options on the cob command, in the file \$COBDIR/cobopt, and in several other ways. For information about the different methods for specifying COBOL compiler directives, see the chapter "COBOL System Interface (cob)" in *Micro Focus COBOL for UNIX: COBOL System Reference*. The following example shows the cob command line method.

You will also need to link SyncSort to your COBOL program. To compile and link a program named prog.cbl, use the following:

```
cob -x prog.cbl -C CALLSORT -L syncsort_library_directory -lmfsyncsort -lsyncsort
```

In the command above, *syncsort_library_directory* refers to the subdirectory /lib of the syncsort directory created when you installed SyncSort. For example, if SyncSort is installed in the directory /usr/local/syncsort, specify **-L /usr/local/syncsort/lib**.

To link a run time system, use a command similar to the following:

```
cob -xe "" -o testrts prog.cbl -L syncsort_library_directory -lmfsyncsort -lsyncsort
```

Reading Special Input Files

When running a sort or merge from a COBOL program via the SORT or MERGE verb, the environment variable SYNCSORT_USE_EXTFH must be set to 1 and exported for either of the following cases:

- One or more of the input or output files is specified with an external file assignment.
- One or more of the input files is a LINE SEQUENTIAL file with null padding before non-ASCII characters. (For more information on line sequential files, see Appendix I of *Micro Focus COBOL for UNIX: COBOL System Reference*.)

For the best sort performance in the above cases, use the environmental variable SYNCSORT_INSERT to specify the amount of data released to SyncSort (see the /INSERT and /DATASIZE options). For the best sort performance in other cases, SYNCSORT_USE_EXTFH should **not** be set.

Using Local Collating Sequences

If the environment variable LANG or LC_COLLATE is set, SyncSort uses the collating sequence specified by either of these environment variables. (Note that LC_COLLATE overrides LANG.)

However, the Micro Focus COBOL SORT/MERGE uses a non-ASCII collating sequence only if the logical file name of the file to be sorted begins with the character string "%NLS%" and the program which opens the file is compiled with the NLS directive. (See the chapter "National Language Support (NLS) for UNIX" in *Micro Focus COBOL for UNIX: COBOL System Reference*.)

If you have the LANG environment variable set, but you want character keys to be sorted in ASCII order, set the LC_COLLATE environment variable to C for the process in which SyncSort is invoked by the COBOL SORT or MERGE verb.

Micro Focus COBOL Program Example

```
*****
*      EXAMPLE EIGHT: Micro Focus COBOL invokes SyncSort      *
* This program, if linked with libmfsyncsort and libsyncsort, *
* invokes SyncSort to perform SORT.                          *
*****

IDENTIFICATION DIVISION.
PROGRAM-ID. EXAMPLE8.

ENVIRONMENT DIVISION.
CONFIGURATION SECTION.

INPUT-OUTPUT SECTION.
FILE-CONTROL.

    SELECT IN-FILE
        ASSIGN TO "ex8input.dat"
        FILE STATUS IS IN-FILE-STATUS.

    SELECT OUT-FILE
        ASSIGN TO "ex8output.dat"
        FILE STATUS IS OUT-FILE-STATUS.

    SELECT SORT-FILE
        ASSIGN TO "ex8sorwork.dat"
        SORT STATUS IS SORT-STATUS.

DATA DIVISION.
FILE SECTION.
*****
* All files are record sequential fixed length of 10 bytes    *
*****
FD IN-FILE
    LABEL RECORDS ARE STANDARD
    BLOCK CONTAINS 0 RECORDS
    RECORD CONTAINS 10 CHARACTERS.

01 INPUT-RECORD          PIC X(10).

FD OUT-FILE
    LABEL RECORDS ARE STANDARD
    BLOCK CONTAINS 0 RECORDS
    RECORD CONTAINS 10 CHARACTERS.

01 OUTPUT-RECORD        PIC X(10).

SD SORT-FILE
    LABEL RECORDS ARE STANDARD
    BLOCK CONTAINS 0 RECORDS
    RECORD CONTAINS 10 CHARACTERS.
```

```

01  SORT-RECORD.
    05  SORT-KEY                      PIC X(9) .
    05  FILLER                        PIC X(1) .

WORKING-STORAGE SECTION.
*****
*   Variables to store file status                                     *
*****
01  IN-FILE-STATUS.
    05  IN-FILE-STATUS-1              PIC X(1) .
    05  IN-FILE-STATUS-2              PIC X(1) .

01  OUT-FILE-STATUS.
    05  OUT-FILE-STATUS-1             PIC X(1) .
    05  OUT-FILE-STATUS-2             PIC X(1) .

01  SORT-STATUS.
    05  SORT-STATUS-1                 PIC X(1) .
    05  SORT-STATUS-2                 PIC X(1) .

01  SORT-STATUS-DISPLAY.
    05  SORT-S1                       PIC S9(4) SIGN IS LEADING SEPARATE.
    05  SORT-S2                       PIC S9(4) SIGN IS LEADING SEPARATE.

PROCEDURE DIVISION.
MAIN-RTN.
*****
*   Call SORT with input file and output file                         *
*****
    SORT SORT-FILE
      ON ASCENDING KEY SORT-KEY
      USING IN-FILE GIVING OUT-FILE.
    PERFORM CHECK-SORT-STATUS.
    STOP RUN.

*****
*   Check sort status and display message                             *
*****
    CHECK-SORT-STATUS.

*****
*   Convert SORT status to displayable format                         *
*****
    IF SORT-RETURN NOT EQUAL 0
      DISPLAY "SORT has terminated with error.  The error codes
        " are: "
      MOVE SORT-STATUS-1 TO SORT-S1
      DISPLAY "SORT-STATUS-1 = " SORT-S1
      MOVE SORT-STATUS-2 TO SORT-S2
      DISPLAY "SORT-STATUS-2 = " SORT-S2
*****
*   Set return code                                                  *

```

```
        MOVE SORT-RETURN TO RETURN-CODE
        DISPLAY "SORT-RETURN  = " RETURN-CODE
ELSE
        DISPLAY "SORT has completed successfully".
```

2.5 The SAS Sort Procedure

When you use a PROC SORT ... statement in a SAS application, SAS invokes an internal sort function. To have SAS invoke SyncSort instead of the internal sort, you need to make SyncSort available and to choose whether to use SyncSort conditionally or unconditionally.

To make SyncSort available to a SAS application, follow the platform specific instructions supplied by SAS. To make SyncSort available conditionally, use the SORTPGM=BEST option on the SAS application. To make SyncSort available unconditionally, use the SORTPGM=HOST option on the SAS application.

Detailed instructions can be found in "Appendix G, Using SyncSort with the SAS System for UNIX Environments" in the SAS document *Installation Instructions*.

2.6 The IBM DB2 Load Accelerator

DB2 Load command performance can be significantly improved using SyncSort's DB2 Load Accelerator. The SyncSort DB2 Load Accelerator supports IBM's DB2 versions 5, 6, and 7, running on AIX releases 4.2 and higher, as well as on Windows NT releases 4.0 and higher. The SyncSort DB2 Load Accelerator will reduce elapsed time, CPU time, and temporary workspace used during the load process. Your savings on elapsed time and CPU time will increase as the number of indexes required to be built during the load process increases.

To use the SyncSort Accelerator for the DB2 Load command, do the following:

For DB2 version 5:

Set DB2 variable DB2SORT to the full pathname of the SyncSort library.

On AIX:

```
db2set DB2SORT=.../syncsort/lib/libdb2v5syncsort.a
where .../syncsort is the directory where SyncSort is installed.
```

On Windows:

```
db2set DB2SORT=...\\SyncSort\\Programs\\db2v5syncsort.dll
where ...\\SyncSort is the directory where SyncSort is installed.
```

For DB2 versions 6 and 7:

Set DB2 variable DB2SORT to the full pathname of the SyncSort library.

On AIX:

```
db2set DB2SORT=.../syncsort/lib/libdb2syncsort.a
where .../syncsort is the directory where SyncSort is installed.
```

On Windows:

```
db2set DB2SORT=...\SyncSort\Programs\db2syncsort.dll
```

where ... \SyncSort is the directory where SyncSort is installed.

After setting the appropriate environment variable, restart the DB2 instance.

Performance Considerations

For optimal performance during the load process, specify the USING parameter for the DB2 load command. If possible, the USING parameter should specify more than one directory name, each directory should be on a different file system, and each file system on a different physical disk drive to optimize performance. As of DB2 version 6, the USING parameter is no longer supported and is ignored if SyncSort's DB2 Load Accelerator is not specified. A warning message is returned stating that the parameter is no longer supported. This warning can be ignored.

Example of the USING parameter for the DB2 Load command:

```
db2 load from ascfile1 of asc
using /filesystem1/workspacel, /filesystem2/tmp
insert into table1
```

2.7 Interrupting SyncSort

You can interrupt SyncSort anytime after the options have been validated by pressing the interrupt key, which is usually Ctrl-C. SyncSort is suspended immediately and the following menu of options is displayed:

```
SyncSort : (CTRLY) You have interrupted execution of SyncSort
Enter one of the following options:
  [S]uspend SyncSort temporarily
  [A]bort
  [C]ontinue
Choice? _
```

If you choose the "suspend" option, SyncSort attempts to stop the current job. The following message is displayed:

```
SyncSort is attempting to place the current application in a
suspended state  Once suspended, a job control command such
as fg or bg resumes the application
(A system message will also be displayed to inform you
about the stopped job)
```

If there is any problem in stopping the current job, an error message is displayed:

```
SyncSort: (NSUSP) current application cannot be suspended
(Another message will follow to indicate the actual error
encountered)
```

The menu of options is redisplayed, but with the "suspend" option removed

If you choose the "abort" option, the following prompt is displayed:

```
Do you really want to ABORT ([Y]es)? _
```

If you enter "Y", "y", or just press <enter>, SyncSort is aborted. Any other response causes SyncSort to redisplay the menu of options.

If you choose the "continue" option, the following continuation message is displayed and the execution of SyncSort proceeds.

```
SyncSort is continuing ...
```

Note: The SyncSort interrupt trap can be disabled with /NOINTERRUPT. If you press the interrupt key when the SyncSort interrupt trap is disabled, control is returned directly to the operating system or to the program invoking SyncSort.

2.8 Disk Full Recovery

Upon encountering a disk full condition, SyncSort displays a menu of options and prompts you for further actions. For instance, if SyncSort encounters a disk full condition while writing to work files, it displays the following information and menu:

```
SyncSort : (SWEXH) all available work space is exhausted
           (IUWFS) The work files currently in use are:
           (SSMSG) /usr/tmp/T$0000085E_0000.TMP
           (SWRCV) additional disk space must be made available
                    for SyncSort to proceed
```

Enter one of the following options:

```
[S]uspend SyncSort temporarily
[A]bort
[C]ontinue
```

Choice? _

If you want SyncSort to complete the application, choose the "suspend" option. This will give you the opportunity to free additional disk space for use by SyncSort. SyncSort attempts to stop the current job. The following message is displayed:

```
SyncSort is attempting to place the current application in a
suspended state  Once suspended, a job control command such
as fg or bg resumes the application
(A system message will also be displayed to inform you
about the stopped job)
```

If there is any problem in stopping the current job, an error message is displayed:

```
SyncSort: (NSUSP) current application cannot be suspended
(Another message will follow to indicate the actual error
encountered)
```

The menu of options is redisplayed but with the "suspend" option removed.

If you choose the "abort" option, the following prompt is displayed:

Do you really want to ABORT ([Y]es)? _

If you enter "Y", "y", or just press <enter>, SyncSort is aborted. Any other response causes SyncSort to redisplay the menu of options.

If you choose the "continue" option, the following continuation message is displayed and the execution of SyncSort proceeds.

SyncSort is continuing ...

Note: If the /NOPROMPT option is specified, all prompting for a user action will be disabled and control will be returned directly to the operating system or to the program invoking SyncSort.

Chapter 3. SyncSort Options

This chapter describes the options that can be used when SyncSort is invoked through the **syncsort** command (see “2.1 The syncsort Command” on page 2.2).

For typographical conventions, please see “Documentation Conventions” on page ix.

/AGGREGATE

/AGGREGATE

Purpose

To describe a value computed from the aggregates of values in a field in multiple records.

Format

```
/AGGREGATE name      [num_agg [num_format] ]  
                     [char_agg [char_format]]
```

where

```
num_agg              =  [TOTAL fieldname  ]  
                        [RUNTOTAL fieldname]  
                        [AVERAGE fieldname ]  
                        [MAXIMUM fieldname ]  
                        [MINIMUM fieldname ]  
                        [COUNT            ]  
                        [RUNCOUNT           ]  
  
char_agg             =  [MAXIMUM fieldname]  
                        [MINIMUM fieldname]  
  
num_format           =  [num_datatype [length]]  
                        [length num_datatype ]  
                        [(edit-spec)         ]  
  
char_format          =  [CHARACTER length]  
                        [length CHARACTER]
```


Arguments

<i>name</i>	the name of the aggregate, which you will use to reference the field in other SyncSort options. The name assigned to an aggregate must adhere to the rules described for an identifier in “Identifiers ” on page B.7.
<i>fieldname</i>	any numeric field defined by /FIELDS, /RECORDLAYOUT, /DELIMITEDRECORDLAYOUT, /DERIVEDFIELD or /DATADictionary. The named field is the source of the data to be aggregated for the report, a page, or a section. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.
<i>num_datatype</i>	the type of numeric data in the aggregate. A list of acceptable values for the data type argument can be found in “Table 1. `Data Types Supported by SyncSort” on page A.1. CHARACTER and BIT are not valid for an aggregate.
<i>length</i>	the length of the aggregate, in bytes. Acceptable values for the length associated with one of the recognized data types can be found in “Table 1. `Data Types Supported by SyncSort” on page A.1. The length may be omitted only when the data type is DFLOAT or FLOAT.
<i>edit-spec</i>	an edit mask to convert the aggregate value to a printable format, with masking. Edit masks and defaults are described in “Edit Specifications for Numeric Fields” on page E.1

Location

The option may appear anywhere in the application definition.

Notes

Usage

An aggregate can be specified in a /PAGEFOOTER, a /SECTIONFOOTER, or the /TRAILER for a report. The value of the aggregate is computed for the report entity (page, section or report) to which the aggregate applies.

Totals and Counts

An aggregate containing the TOTAL of a field contains the total of the values in that field in all records which appear in the report entity for which the aggregate applies. An aggregate appearing in a page footer contains the total of the values in the summarized field for all records whose detail lines appear on the page. An aggregate in a section footer contains the total of the values in the summarized field for all records appearing in the section. An aggregate in a report trailer contains the total of the values in the summarized field for all records appearing in the report.

An aggregate containing the RUNTOTAL of a field contains the total of the values in that field in all records which appear up to the point where the report entity for which the aggregate applies is produced. An aggregate appearing in a page footer contains the total of the values in the summarized field for all records whose detail lines appear on the page and all preceding pages. An aggregate in a section footer contains the total of the values in the summarized field for all records appearing in the report up to the end of the section. An aggregate in a report trailer contains the total of the values in the summarized field for all records appearing in the report.

An aggregate containing the AVERAGE of a field contains the average of the values in that field in all records appearing in the report entity for which the aggregate applies. An aggregate appearing in a page footer contains the average of the values in the summarized field for all records whose detail lines appear on the page. An aggregate in a section footer contains the average of the values in the summarized field for all records appearing in the section. An aggregate in a report trailer contains the average of the values in the summarized field for all records appearing in the report.

An aggregate containing a COUNT contains the number of records which appear in the report entity for which the aggregate applies. An aggregate appearing in a page footer contains the count of the records whose detail lines appear on the page. An aggregate in a section footer contains the count of the records appearing in the section. An aggregate in a report trailer contains the count of the records appearing in the report.

An aggregate containing a RUNCOUNT contains the number of records which appear up to the point where the report entity for which the aggregate applies is produced. An aggregate appearing in a page footer contains the number of records whose detail lines appear on the page and all preceding pages. An aggregate in a section footer contains the number of records appearing in the report up to the end of the section. An aggregate in a report trailer contains the number of records appearing in the report.

Maximum and Minimum An aggregate containing the MAXIMUM of a numeric field contains the largest of the values in that field in all records appearing in the report entity for which the aggregate applies. A MAXIMUM aggregate of a character field contains the largest value in that field when compared alphabetically. An aggregate appearing in a page footer contains the maximum of the values in the summarized field for all records whose detail lines appear on the page. An aggregate in a section footer contains the maximum of the values in the summarized field for all records appearing in the section. An aggregate in a report trailer contains the maximum of the values in the summarized field for all records appearing in the report.

An aggregate containing the MINIMUM of a numeric field contains the smallest of the values in that field in all records appearing in the report entity for which the aggregate applies. A MINIMUM aggregate of a character field contains the smallest value in that field when compared alphabetically. An aggregate appearing in a page footer contains the minimum of the values in the summarized field for all records whose detail lines appear on the page. An aggregate in a section footer contains the minimum of the values in the summarized field for all records appearing in the section. An aggregate in a report trailer contains the minimum of the values in the summarized field for all records appearing in the report.

When you want the maximum or minimum of a character field of indeterminate length, you should supply the length of the aggregate through the CHARACTER length argument. SyncSort assumes a length of 20 characters when you do not supply a length. The following fields are of indeterminate length:

- The variable length field at the end of the record (e.g. specified in /FIELDS with a starting position but no length).
- A field containing one or more delimited fields (other than a single delimited field with an end byte location).

Overflow

When the value resulting from an aggregation is too large to be contained in the final version of the aggregate, you are warned of the overflow and the aggregate appearing in the report is filled with pound signs (#).

The Aggregated Field

The field which you specify to be aggregated must be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on the field which is aggregated. However, when you specify input reformatting, you must not discard a field which you want to aggregate. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want to aggregate **fielda**.

Aggregating an Edited Numeric Field

When you wish to aggregate an edited numeric field, you need to indicate to SyncSort the precision with which the aggregate is to be calculated. You can do this either by supplying an edit mask with the aggregate definition in /AGGREGATE, or by providing the precision with the definition of the field to be aggregated in the /FIELDS option. Providing the edit mask is the simpler of the two ways of indicating the precision.

If you do not provide either an edit mask for the aggregate or a precision for the aggregated field, SyncSort uses the default precision for the aggregated field.

- For a delimited field of indeterminate length (e.g. 2: - 2: EN), the default precision is 15 integral digits (digits before the decimal point) and 2 fractional digits (digits after the decimal point).

- For a fixed length field, the precision depends on the length of the field. An attempt is made to use the default precision of 15 integral digits and 2 fractional digits. If the locale has a thousands separator, the number of characters needed to display a value with the default precision is one for the sign, plus one for the decimal point, plus four for the thousands separators, plus 15 for the integral digits, plus 2 for the fractional digits, or a total of 23 bytes. If the thousands separator is null in the locale, 19 characters are needed. If the number of characters needed to display a value with the default precision is greater than the field length, the precision is reduced. First the fractional precision is reduced, and, if required, the integral precision is also reduced.

When SyncSort aggregates a delimited edited numeric field, the aggregated value appearing in the report contains a number of integral digits up to the integral precision. Sufficient spaces are added before the value to account for missing integral digits and associated embedded thousand's separators. The aggregated value contains a number of fractional digits equal to the fractional precision.

A delimited edited numeric field which spans field separators cannot be aggregated. For example, fields defined as 2: - 3: EN, 3:4 - 4:1 EN and 7: EN cannot be aggregated, but fields defined as 2: - 2: EN, 3:4 - 3: EN and as 7: - 7:8 EN can be aggregated.

Examples

The following examples show the use of edit masks with /AGGREGATE. For more information on edit masks, see “Edit Specifications for Numeric Fields” on page E.1.

```
/aggregate rpt_count count (ZZZ9)
```

The aggregate named **rpt_count** contains the number of records which appear in the entity (page, section, or report) where the aggregate is specified. For example, when **rpt_count** is specified in a section footer, the **rpt_count** is the number of records in the section to which the section footer belongs. The number of records appears in the section footer as a four digit integer with leading zeroes in the first three positions replaced by spaces.

```
/aggregate total_amt total sale_amount (SSS,SS9V99)
```

The aggregate named **total_amt** contains the total of the values of **sale_amount** in the records which appear in the entity (page, section, or report) where the aggregate is specified. For example, when **total_amt** is specified in a page footer, the **total_amt** in the footer of a specific page is the total of the values of **sale_amount** of all records which appear in the page. The total appears in the footer as a ten character quantity (when the currency symbol is a single character), the result of applying the floating currency symbol mask SSS,SS9V99 to the total amount.

```
/aggregate runttotal_amt runttotal sale_amount ls 10
```

The aggregate named **runttotal_amt** contains the running total of the values of **sale_amount** in the records which appear prior to the entity (section, page, or report) where the aggregate is specified. For example, when **runttotal_amt** is specified in a page footer, **runttotal_amt** in the footer of a specific page is the total of the values in the field **sale_amount** of all records which appear in the page and all prior pages. The total appears in the footer as a ten character quantity consisting of a leading sign and nine digits.

```
/aggregate average_amt average sale_amount (ZZ,ZZ9V99)
```

The aggregate named **average_amt** contains the average of the values of **sale_amount** in the records appearing in the entity (page, section, or report) where the aggregate is specified. For example, when **average_amt** is specified in the trailer, **average_amt** is the average of the values of **sale_amount** of all records appearing in the report. The average appears in the trailer as a nine character quantity, the result of applying the zero suppression mask ZZ,ZZ9V99.

```
/aggregate maximum_amt maximum sale_amount (++,++9)
```

The aggregate named **maximum_amt** contains the largest **sale_amount** value in the records appearing in the entity (page, section, or report) where the aggregate is specified. For example, when **maximum_amt** is specified in a page footer, the **maximum_amt** in the footer of a specific page is the largest **sale_amount** value from all records appearing in the page. The maximum appears in the footer as a six character quantity, the result of applying the floating sign mask ++,++9.

/BEGIN

Purpose

To discard all previous options in an application definition.

Format

`/BEGIN`

Location

The option may appear anywhere in the application definition.

Notes

When the `/BEGIN` option appears in an application definition, SyncSort ignores it and all options which precede it.

Use the `/BEGIN` option if, because of an error, for instance, it becomes necessary to restart the application definition you have been entering at a terminal.

If `/BEGIN` is an application option in an `/INSERT` file, all options before `/BEGIN` in the file and all options before `/INSERT` in the application definition are ignored.

Since an application definition is terminated by the first `/END` option, `/BEGIN` is not recognized if it occurs after `/END`.

Example

```
/fields company 17 char 23 /begin /fields company 20 char 30
```

The definition of the input record field **company** preceding **/begin** will be ignored in favor of the field definition following **/begin**.

Purpose

To define a century window to be applied to two-digit years.

Format

/CENTURYWINDOWSTART *start*

Arguments

start a 4-digit year for the start of the century window. For example, a century window start of 1970 defines the 2-digit years from 70 to 99 as 1970 to 1999 and years 00 to 69 as 2000 to 2069.

Location

The option may appear anywhere in the application definition.

Notes

Year Translations

You may specify through this option a century-window to be applied to all 2-digit years in your application. For example, you may want to compare a field containing a 2-digit year to a field containing a 4-digit year. By applying a century window, you can control how the 2-digit year is translated before comparison. If no century window is specified, an implicit window from 1970 to 2069 is assumed.

Example

`/CENTURYWINDOWSTART 1930`

The option specifies that all 2-digit years should be translated to be in the range 1930 to 2029.

/CHECKSEQUENCE

/CHECKSEQUENCE

Purpose

To request that SyncSort check the order of the records in the input files to a sort or merge.

Format

/CHECKSEQUENCE [OUTPUT]

Location

The option may appear anywhere in the application definition.

Notes

/CHECKSEQUENCE is not valid for a copy.

Sort

When you want to determine if the records in a file are already in sorted order, you can request that SyncSort perform a sort and specify the ***/CHECKSEQUENCE*** option. When you specify ***/CHECKSEQUENCE*** (but not ***/MERGE***) and the records in the input file are in sorted order (according to the sort keys) no output is generated by the sort. However, if SyncSort detects that the records are not in the correct sequence, you are given a warning. If you have specified the ***OUTPUT*** argument the sort continues and output is produced with the records correctly sorted. If you do not add the ***OUTPUT*** argument the sort terminates on detection of an out-of-order record and there is no resulting output.

If you specify ***/SUMMARIZE*** for a sort in addition to ***/CHECKSEQUENCE***, SyncSort will check that there are no records with equal keys and confirm that the

records are in sorted order. Two records with duplicate keys cause a similar response to an out-of-order record.

Only a single input file can be supplied when the /CHECKSEQUENCE option is specified for a sort. /REFORMAT, /INCLUDE and /OMIT cannot be specified with /CHECKSEQUENCE. No arguments can be specified with /SUMMARIZE when /CHECKSEQUENCE is used for a sort.

Merge

By default, SyncSort does not check the sequence of records input to a merge. It is assumed that the records in each input file are sorted in key order.

If you want SyncSort to check that each input file to a merge is sorted correctly, specify the /CHECKSEQUENCE option. When you specify /MERGE and /CHECKSEQUENCE and SyncSort detects that the records are not in the correct sequence, you are given a warning and the merge continues. Note that output is produced whether or not you specify the OUTPUT argument. When any out-of-order records have been detected the output records are not in sorted order.

Example

```
/checksequence
```

This is a request to check the sequence of records in the input files.

/COLLATINGSEQUENCE

/COLLATINGSEQUENCE

Purpose

To define your own collating sequence.

Format

```
/COLLATINGSEQUENCE [name ]  
[DEFAULT] base_seq [seq_mod...] [invalid_chars]
```

where

```
base_seq          = [next_spec [, next_spec ...]]  
                   [LOCAL  
                   [ASCII  
                   [EBCDIC  
                   [MULTINATIONAL
```

```
seq_mod           = [MODIFICATION pos_changes]  
                   [NULLCHAR null_chars ]  
                   [FOLDED
```

```
next_spec         = [single_char]  
                   [double_char]  
                   [char_range ]
```

```
single_char       = ["a" ]  
                   [X"xx" ]  
                   [O"ooo"]
```

```
double_char       = ["aa" ]  
                   [X"xxxx" ]  
                   [O"oooooo"]
```

```
char_range      =  "b" - "e"

pos_changes     =  pos_change[,pos_change ...]

pos_change      =  moved_char  $\begin{matrix} [=] \\ [>] \\ [<] \end{matrix}$  comp_pos

moved_char      =   $\begin{bmatrix} \text{single\_char} \\ \text{double\_char} \end{bmatrix}$ 

comp_pos        =   $\begin{bmatrix} \text{single\_char} \\ \text{double\_char} \end{bmatrix}$ 

null_chars      =   $\begin{bmatrix} \text{next\_spec[,next\_spec ...]} \\ \text{NONPRINTABLE} \\ \text{NONDICTIONARY} \end{bmatrix}$ 

invalid_chars   =   $\begin{bmatrix} \text{HIGH} \\ \text{LOW } \end{bmatrix}$ 
```

Arguments

name the name of the alternative collating sequence. This alternative collating sequence takes effect only for those fields which reference it (see /FIELDS, /DERIVEDFIELD, /DELIMITEDRECORDLAYOUT, and /RECORDLAYOUT).

The name assigned to the sequence must adhere to the rules described for an identifier in “Identifiers” on page B.7.

a the character representation of a single character.

xx the hexadecimal representation of a single character.

ooo the octal representation of a single character.

/COLLATINGSEQUENCE

<i>aa</i>	the character representation of a double character, i.e. a 'character' which takes up two bytes of storage, but which is to be collated as if it occupies only a single byte, e.g. "CH".
<i>xxxx</i>	the hexadecimal representation of a double character. The first two hexadecimal digits represent the first half of the double character while the last two hexadecimal digits represent the second half of the double character, e.g. x"4348".
<i>oooooo</i>	the octal representation of a double character. The first three octal digits represent the first half of the double character while the last three octal digits represent the second half of the double character, e.g. o"103110".
<i>b</i>	the first character in a range (double character not allowed).
<i>e</i>	the last character in a range (double character not allowed).

Location

The option may appear anywhere in the application definition.

Notes

You may define an alternative collating sequence when one of the standard sequences does not provide the sequencing of values in a character field that you require.

Default Collating Sequence

You can specify a default collating sequence by using DEFAULT instead of the collating sequence name. The default collating sequence takes effect for those character fields for which you do not assign an explicit collating sequence (see /FIELDS, /DERIVEDFIELD, /DELIMITEDRECORDLAYOUT, and /RECORDLAYOUT).

Defining an Alternative Collating Sequence

There are two major steps to defining an alternative collating sequence. The first is to specify the base sequence. The second, which may not be necessary, is to specify modifications to the base sequence:

Base Sequence Definition.

To specify the base sequence for your alternative sequence, you start with one of the standard sequences (i.e. LOCAL, ASCII, EBCDIC, MULTINATIONAL) or you can define your own collating sequence. To define your own base sequence, simply specify the characters (single, double, range) in the order in which you want them to collate, low to high. For example,

```
/collatingsequence special_seq1 "a" - "l", "ll", "m" - "z",  
"A" - "L", "LL", "M" - "Z", "0" - "9"
```

The collating sequence **special_seq1** defined above includes no special or non-printable characters nor does it have any modifications to the base sequence. The characters excluded from it are regarded as invalid if any are encountered when collating fields according to this sequence. You can specify, through the *invalid_chars* argument of /COLLATINGSEQUENCE, where you want invalid characters to collate. The choices are for the invalid characters to appear either below (LOW) or above (HIGH) the characters that are specified in the collating sequence definition. If you leave the *invalid_chars* argument unspecified, invalid characters will be treated as null characters (for the definition of a null character, see below).

Collating values in a field according to the alternative sequence of the above example results in the following (assuming ascending ordering and padding with spaces):

```
a***** affect almond allure amends *****de ladder lambda llamas Affect
```

Base Sequence Modifications.

There are three different types of changes which you can make to your base sequence. First, characters can be repositioned within the collating sequence. Second, characters can be ignored during the collating process by defining them as null. Third, the distinction between the uppercase and lowercase characters may be removed.

Character Repositioning. Character repositioning is handled through the clause `MODIFICATION pos_changes` in the `/COLLATINGSEQUENCE` option. This is the only way to make two characters (other than upper- and lowercase equivalents) collate as if they have the same value. The "=" operator is used to define a pair of characters as equivalent. You can also insert a character between two adjacent positions in your base sequence by using the ">" or "<" operators. The following `/COLLATINGSEQUENCE` statement presents a case of character repositioning:

```
/collatingsequence special_seq2 ascii  
modification "ll" = "l", "LL" = "L", "0" > "9"
```

special_seq2 modifies the standard ASCII collating sequence. Its effect consists in allowing the double "ll" character to collate equal to the singleton "l" and in repositioning the "0" character so that it collates after "9" but before ":". Here is an example of collating values according to the alternative sequence **special_seq2** (assuming ascending ordering and padding with spaces):

```
Affect abend1 abend9 abend0 llamas lamase lambda
```

There are a few rules you must obey when specifying reposition modifications to a base sequence:

- The character used as the comparison position (on the right of the "=", ">" or "<" operator) must already be assigned a place in the collating sequence either as part of a standard base sequence or by a prior modification to the sequence. When the comparison character is a double character, the requirement is satisfied if the double character has been assigned a place in the sequence or the two characters that comprise the double character have both been assigned places separately.
- A character can be repositioned only once.

Null Character Definition. The second type of modification which you can apply to a base sequence is to specify "null" characters. These are established through the clause `NULLCHAR null_chars` in the `/COLLATINGSEQUENCE` option. A character defined as null is ignored when ordering values; null characters have the effect of reducing the number of significant characters in a field.

You either specify a predefined set of characters as null (i.e. NONPRINTABLE, NONDICTIONARY) or you specify your own list of null characters. The specification of a list of null characters is similar to the definition of a base sequence. For example,

```
/collatingsequence special_seq3 ascii nullchar "*",  
"0" - "9"
```

modifies the standard ASCII collating sequence by nullifying the asterisk and all digits. Collating values according to the alternative sequence **special_seq3** results in the following (assuming ascending ordering and padding with spaces):

```
****7 ol0de older olive ol5ve ****p
```

The NONDICTIONARY set of characters includes all those which are not alphabetic, digits, spaces, or tab characters. For all base sequences other than LOCAL, the NONDICTIONARY characters are those for 8-bit ASCII See "Appendix C. Collating Sequences" on page C.1. For the LOCAL base sequence, the NONDICTIONARY characters are those that are not defined as alphanumeric in the current locale. NONDICTIONARY can not be used with multi-byte character locales.

Case Folding. The third method of modifying a base sequence is to use the FOLDED argument to fold all lowercase characters into their uppercase equivalents. Each lowercase character then collates equal to its uppercase counterpart.

Examples

```
/collatingsequence seq1 "0" - "9", "A" - "Z", "a" - "z"  
modification "*" = "0" nullchar "(" , ")" low
```

seq1 defines a collating sequence that orders the numeric characters (0 - 9) before the alphabetic characters. Among the alphabetic characters, the uppercase characters (A - Z) are ordered before their lowercase counterparts (a - z). The asterisk (*) has the same collating value as zero (0). The opening and closing parentheses are defined as null and will therefore be ignored in the collating process. The invalid characters, i.e. those characters that are excluded from the definition of **seq1**, will be ordered low in the collating sequence.

/COLLATINGSEQUENCE

```
/collatingsequence seq2 ascii modification "SS" = "B"  
folded
```

seq2 modifies the standard ASCII collating sequence in two ways. Firstly, the uppercase double s (SS) is given the same collating value as the uppercase b (B). Secondly, all lowercase characters are folded into their uppercase equivalents.

```
/collatingsequence seq3 ascii nullchar nondictionary
```

seq3 defines a collating sequence that excludes the set of NONDICTIONARY characters from the ASCII sequence.

/COLLECT

Purpose

To limit the number of records to be processed after input selection.

Format

/COLLECT *number*

Arguments

number the number of records to be released to SyncSort after applying all input selection.

Location

The option may appear anywhere in the application definition.

Notes

You can specify **/COLLECT** for a sort or copy if the input records originate from files.

There are several options which you can use to limit the number of records to be released to SyncSort. These are:

/SKIPRECORD

/SKIPBLANK

/STOPAFTER

/INCLUDE and **/OMIT**

/IGNORE

/COLLECT

/COLLECT

The order in which the options appear in the above list is the order in which SyncSort applies them. Thus the `/COLLECT` option determines the number of records to be released after all other selection criteria have been applied.

The number of records which are finally released to SyncSort may be less than the number you request in the `/COLLECT` option, but it will not exceed that number. This will happen, for example, when the number of records in your file is too small to provide the correct `/COLLECT` number. It may also happen when you combine `/COLLECT` with `/STOPAFTER`. `/STOPAFTER` determines how many input records SyncSort reads from the input file. If this number is less than the `/COLLECT` number, the `/COLLECT` number will not be reached. Similarly, if a sufficient percentage of the records are discarded because of applying `/OMIT`, the `/COLLECT` number will not be reached.

Example

```
/collect 100
```

The option specifies that 100 records are to be released to SyncSort after all other input selection criteria have been applied.

/COLUMNS

Purpose

To describe the columns which make up a report.

Format

```
/COLUMNS          column1 [, column2 ... ]
```

where

```
column             =  fieldname [column_qual ...]
```

```
column_qual       =  [COLUMNSPACING before_space [after_space]]
                   [WIDTH max_width
                   [header_line1 [+ header_line2 ... ] ]
```

Arguments

<i>fieldname</i>	the name of a field defined through /DATADictionary, /DERIVEDFIELD, /FIELDS, /DELIMITEDRECORDLAYOUT, or /RECORDLAYOUT. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.
<i>before_space</i>	the number of spaces to be inserted prior to the column. The default is provided in the /REPORT option.
<i>after_space</i>	the number of spaces to be inserted after the column. The default is provided in the /REPORT option.
<i>max_width</i>	the maximum number of characters to appear across a column which holds a field of indeterminate length. The default is 20 characters. This width does not include any spaces required by before and after spacing.

/COLUMNS

header_line a character string which makes up the portion of the column header to appear on one line of the column headings. The string can be specified in any of the formats described in “Specifying Character String Constants” on page B.1.

Location

The */COLUMNS* option must appear after the */OUTFILE* option which defines the report file. When there are multiple output files, any */COLUMNS* option for a particular file must appear between the */OUTFILE* option specifying that file and the next */OUTFILE* option, if any.

Notes

There can be only one */COLUMNS* option for a report.

Using delimited fields in a report

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When you use a field from such a record as a column, the field delimiter does not appear in the column.

Enclosed delimited fields

Each field in a field-delimited text record may be enclosed by two enclosing characters. When you use a single or partial delimited field as a column, any enclosing characters do not appear in the column. When you use a multiple delimited field as a column, the field appears in the column with all enclosing characters.

Compatibility with Other SyncSort Options

/COLUMNS and */REFORMAT* cannot both be specified for the same report.

Column Headings

Column headings will appear in a report only when you specify at least one column header. The default column header is the field name in upper case.

If you want an individual column to appear without a header, specify the null string ("") as the header for that column. You can request a column header to be spread over more than one line of the column headings by providing the portions to appear on individual lines separated by plus signs (+).

The column headings consist of the maximum number of lines needed to accommodate all headers. The last portion of each header appears on the last line of the column headings.

Each portion of the column header is left aligned within the space allocated for the value in the column.

Column Width

The width of a column is the number of spaces which appear before and after the value in the column plus the greater of:

- the longest column header portion

and either of the following, depending on whether the field length is fixed or indeterminate.

- the length of the column field when the field length is fixed
- the value of the **width** argument when the field length is indeterminate. A field is of indeterminate length when it is the variable length field at the end of the record (e.g. specified in /FIELDS with a starting position but no length), or when it includes one or more delimited fields (other than a single delimited field with an end byte location).

A Column Field

Every field in the /COLUMNS option must be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on a column field. However, when you specify input reformatting, you must not discard a field which you want to appear in a column. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want the **fielda** value to appear in a column.

/COLUMNS

Examples

```
/columns sales_rep "Sales representative",  
state "State" columnspacing 3 3,  
city "City", company_name "Company",  
total_ytd "Y-T-D"
```

This /COLUMNS options creates the following five column headings on the report to which it applies.

Sales representative	State	City	Company	Y-T-D
Justin Cooper	GA	Atlanta	Bakers Inc	7,307.85
Alexandra Soon	NC	Raleigh	Crib and Cradle	704.00
Mike Drew	MA	Boston	Nature's Way	6,234.95
Alan Kessler	NY	New York	Cornell Labs	8,756.25

The report to which the /COLUMNS applies has five columns.

- The first column holds the contents of the **sales_rep** field and has "Sales representative" as its heading.
- The second column holds the value from the **state** field. Its heading is "State".
- The third column holds the **city** field value and has "City" as its heading.
- The fourth column contains the **company_name** field value and has the heading "Company".
- The fifth column is headed "Y-T-D" and holds the contents of the field **total_ytd**.

The columns, except for the **state** column, are spaced according to any spacing specified on the /REPORT option (or are evenly spaced within the output record length when /REPORT COLUMNSPACING is not defined). When /REPORT COLUMNSPACING 0,2 is specified, there are 5 spaces between the **sales_rep** and **state** columns (2 after the **sales_rep** column and 3 before **state**), 3 spaces between the **state** and **city** columns (3 after the **state** column, 0 before **city**) and 2 spaces after each of the remaining columns.


```
/columns department "Dept", salary "Monthly" + "Salary",  
employee_count "Number of" + "Employees",  
comments "Comments" width 40
```

This /COLUMNS options creates the following four column headings on the report to which it applies.

Dept	Monthly Salary	Number of Employees	Comments
------	-------------------	------------------------	----------

- The first column holds the contents of the **department** field and has "Dept" as its heading.
- The second column holds the **salary** field value and has "Monthly Salary" as its heading, spread over two lines.
- The third column holds the value from the **employee_count** field. Its heading is "Number of Employees", also spread over two lines.
- The fourth column is headed "Comments" and holds the contents of the field **comments**, to a maximum of 40 characters.

/CONDITION

/CONDITION

Purpose

To define a condition for selection of records or for assignment of different values to a field.

Format

```
/CONDITION      name condition
```

where

```
condition      = expression      [ [AND] expression ... ]  
                                   [ [OR ] expression ... ]
```

For use of parentheses within the condition, see Notes below (section “Evaluating a Condition”).

```
expression      = [ field_compare ]  
                  [ file_compare ]
```

```
field_compare = [ numeric_value numeric_operator numeric_value      ]  
                [ character_value character_operator character_value ]  
                [ bit_value bit_operator bit_value                  ]
```

```
numeric_value = [ + | - ] [ numeric_field      ]  
                  [ number                    ]  
                  [ ( arithmetic_expression ) ]
```

```
arithmetic_expression = numeric_value [ arithmetic_operator numeric_value ... ]
```

```
arithmetic_operator = [ * ]  
                      [ / ]  
                      [ + ]  
                      [ - ]
```

		[EQ]	equals	
		[NE]	does not equal	
		[LT]	less than	
		[GT]	greater than	
		[LE]	less than or equal to/not greater than	
		[GE]	greater than or equal to/not less than	
		[=]	equals	
<i>numeric_operator</i>	=	[!=]	does not equal	
		[<]	less than	
		[>]	greater than	
		[<=]	less than or equal to/not greater than	
		[!>]	not greater than	
		[>=]	greater than or equal to/not less than	
		[!<]	not less than	
<i>character_value</i>	=	[<i>character_field</i>]		
		[<i>text_string</i>]		
		[/ <i>pattern_match0</i>]		
		[<i>numeric_operator</i>]		
<i>character_operator</i>	=	[CT]	] contains	
		[NC]	] does not contain	
		[MT]	] matches	
		[NM]	] does not contain	
<i>bit_value</i>	=	[<i>bit_field</i>]		
		[<i>bit_string</i>]		
<i>file_compare</i>	=	ALIAS	[EQ]	<i>source_alias</i>
			[NE]	
			[=]	
			[!=]	

Arguments

<i>name</i>	the name of the condition, which you use to reference the condition in other SyncSort options. The name you give to a condition must adhere to the rules described for an identifier in “Identifiers ” on page B.7.
<i>numeric_field</i>	the name of a numeric field defined through /FIELDS, /RECORDLAYOUT, /DELIMITEDRECORDLAYOUT,

	/DATADictionary, or /DERIVEDFIELD. See “Specifying Field Names ” on page B.9 for details on specifying a field name. The value in the field is used in the comparison. The data type and length of the field must be the same as, or convertible to, the data type and length of the field or constant to which it is compared. See “Table 2. Valid Comparisons between Data Types” on page A.2 for a summary of valid comparisons. See “Table 3. Valid Conversions between Data Types” on page A.3 for a summary of valid conversions.
<i>number</i>	a numeric quantity. You can specify the number in any of the formats described in “Specifying Numeric Constants” on page B.4, provided the chosen format is compatible with the data type of the field to which it is compared.
<i>character_field</i>	the name of a character (text) field defined through /FIELDS, /RECORDLAYOUT, /DELIMITEDRECORDLAYOUT, /DATADictionary, or /DERIVEDFIELD. See “Specifying Field Names ” on page B.9 for details on specifying a field name. The value in the field is used in the comparison.
<i>text_string</i>	a character string. You can specify the string in any of the formats described in “Specifying Character String Constants” on page B.1.
<i>pattern_match</i>	a pattern to match against the contents of the source field. See “Regular Expressions” on page E.15 for details. Pattern matching is only available with the <i>matches</i> and <i>does not match</i> operators. If you need to include a slash (/) in the pattern other than in a bracketed list, precede it with a backslash (\).
<i>bit_field</i>	the name of a bit field defined through /FIELDS, /RECORDLAYOUT, /DELIMITEDRECORDLAYOUT, or /DERIVEDFIELD. See “Specifying Field Names ” on page B.9 for details on specifying a field name. The value in the field is used in the comparison.
<i>bit_string</i>	a bit string. The format of a bit string is described in “Specifying Bit String Constants ” on page B.6.

<i>source_alias</i>	the alias you have assigned to one of the input files or pipes. The expression <code>ALIAS EQ <i>alias</i></code> yields true for all records originating from the source with alias <i>alias</i> and false for records from all other sources.
---------------------	---

Location

The option may appear anywhere in the application definition.

Notes

Evaluating a Condition

A condition consists of one of the following:

- a single logical expression, a statement that is either true or false for a particular source record
- a compound expression, several logical expressions connected by AND and OR

You may include both single and compound logical expressions in a pair of parentheses. There is no limit to the level of nesting of parentheses.

When evaluating a condition for a source record, SyncSort evaluates each logical expression separately, yielding a true or false result. Expressions within parentheses are evaluated first. When there is nesting of parentheses (parentheses enclosed within parentheses), the expressions are evaluated outward from the one within the innermost pair of parentheses.

When the order of evaluation is not indicated by parentheses, AND is applied before OR. Multiple ANDs or ORs are applied from left to right. A compound expression consisting of two (or more) logical expressions connected by AND is true when both (all) of its component expressions are true. A compound expression consisting of logical expressions connected by OR is true when at least one of its component expressions is true.

The final result of the evaluation for a record is that the condition is either true or false.

Character String Comparisons

/CONDITION

You may use the *contains* (**CT**) and *does not contain* (**NC**) operators to do substring matching. For example, the condition **field1 ct "abcd"** evaluates as true if the string **abcd** occurs anywhere within the length of **field1**. The result of evaluating this condition for several records is as follows:

field1 contents	Result of evaluation
abcdefghijkl	T
lkjihgfedcba	F
defgabcdabcd	T
abcabcabcabc	F

You may match a pattern in a field by following the *matches* or *does not match* operator with a regular expression. For example, the condition **field1 mt /b.*a/** evaluates as true if the pattern **b.*a** matches **field1**, i.e. the letter **b** occurs anywhere within **field1** and the letter **a** also occurs somewhere to the right of the **b**. See "Regular Expressions" in Appendix E for full details. The result of evaluating this condition for several records is as follows:

field1 contents	Result of evaluation
abcdefghijkl	F
lkjihgfedcba	T
defgabcdabcd	T
abcabcabcabc	T

When a field and a string whose lengths are not equal are compared, the string is truncated or padded on the right with the key pad byte to the length of the field. When two character fields of different lengths are compared, the shorter is padded with the key pad byte. Truncation and padding are not done for the *contains*, *does not contain*, *matches* and *does not match* operators.

Numeric Comparisons

When a comparison involves two numeric values, each of the values can be a numeric field, a number or an arithmetic expression containing fields and numbers. When two operands are combined via an arithmetic operation (addition, subtraction, multiplication or division), the result has a data type that is dependent on the data type of the two operands. See "Table 1. Data Types

Resulting from Binary Arithmetic Operation” on page F.5 for a summary of the resultant data types. For example, when **(amount / 5) < discount** is evaluated where **amount** is a binary integer field, the result of dividing **amount** by 5 is also an integer. If you want the division to result in a number with a fractional part, define the result of the arithmetic expression as a derived field, for example, **/derivedfield max-discount (amount / 5) en 5/2 /condition max-discount < discount**.

Pattern Matching for a Numeric Field

You may match a pattern in a numeric field by following the *matches* or *does not match* operator with a regular expression. For example, the condition **field1 mt /\. /** evaluates as true if the pattern `\.` matches **field1**, i.e. the decimal point occurs within **field1**. See "Regular Expressions" in Appendix E for full details. Prior to matching a pattern to a numeric field, SyncSort converts the contents of the field to edited numeric type.

Bit String Comparisons

When a bit field and a bit string whose lengths are not equal are compared, the string is truncated on the right or padded on the right with 0 bits to the length of the field. When two bit fields of different lengths are compared, the shorter is padded on the right with 0 bits to the length of the longer.

Retaining Data for Output Selection

Fields involved in evaluating a condition used for the selection of records for a specific target (through /INCLUDE and /OMIT options) must be available in some form in the records leaving the sort, merge, or copy stage of the application. If you do not specify reformatting for all targets, there is no limitation on the fields appearing in the /CONDITION specification. However, if you specify reformatting for all targets, do not discard any of the data needed for selection for a specific target.

Examples

```
/condition USA country ct "USA" or country = "United States"
```

The option names a condition, **USA**, defined by a compound logical expression, **country ct "USA" or country = "United States"**. The truth of a compound expression depends on the truth of its components, which in this case are two

simple logical expressions, **country ct "USA"** and **country = "United States"**. The former is true if the field **country** contains (**ct**) the character string **USA**. The latter is true if the field **country** is equal to (**=**) the character string **United States**. Since these two expressions are connected by **or**, at least one of them must be true for the compound expression to be true. If the compound expression evaluates as true, then the condition **USA** is true. On the other hand, if both simple expressions are false, then the compound expression and the condition are false.

```
/condition reorder stock_num eq 0 or order_num gt stock_num
```

The name of the condition is **reorder**. It is true if at least one of the compounded expressions, **stock_num eq 0** and **order_num gt stock_num**, evaluates as true. The expression **stock_num eq 0** is true if the value of the field **stock_num** is zero; the expression **order_num gt stock_num** is true if the value in the field **order_num** is greater than (**gt**) the value in the field **stock_num**. If both expressions evaluate as false, the named condition, too, is false.

```
/condition extract (state = "MA" or state = "NH") and  
rep_id = 215
```

For the condition **extract** to be true, both expressions connected by **and** must evaluate as true. The first expression, **state = "MA" or state = "NH"**, is true if the **state** field equals the character string **"MA"** or **"NH"**. The second expression is true if the value in the **rep_id** field is 215. If either expression fails to evaluate as true, then also their combination and, consequently, the condition **extract** are false.

```
/condition null_amount check_total = 0 or check_print  
5"*
```

The condition **null_amount** is true if either the **check_total** field is zero or the **check_print** field contains five asterisks.

```
/condition april_receipts alias eq april_trans and  
total_received > 0
```

If a record originates from the input file whose alias is **april_trans** and the value in the **total_received** field in the record is greater than zero, then the condition **april_receipts** is true for the record. Otherwise, the condition is false. Note that the name **april_trans** originates from an **/INFILE** statement such as **/INFILE trans.dat april_trans**, where **trans.dat** is the input file and **april_trans** its alias.

```
/condition bit2only bit_fl = b"010"
```

The condition **bit2only** is true if the first and third bits in the **bit_fl** field are '0' bits and the second bit in the field is a 1 bit. `/condition has_time date_field mt /[0-9]+:[0-9]+:[0-9]+/`

The condition **has_time** is true if **date_field** has, somewhere within its length, the following pattern: one or more digits (as indicated by the sub-pattern **[0-9]+**), followed by a colon, followed by one or more digits, followed by another colon, followed by one or more digits. For full details on patterns, see "Regular Expressions" in Appendix E.

```
/condition overdrawn (balance - minimum-balance) < (credit-card-balance + other-debt)
```

The option names a condition, **overdrawn**, that compares the results of two arithmetic operations. The contents of the **minimum-balance** field are subtracted from the contents of **balance** to yield one number. The contents of the **credit-card-balance** and **other-debt** fields are added to yield a second number. When the first number is less than the second number, the condition is true. When the first number is greater than or equal to the second number, the condition is false.

/COPY

/COPY

Purpose

To specify that SyncSort is to perform a copy (rather than a sort, join, or merge).

Format

/COPY

Location

The option may appear anywhere in the application definition.

Notes

The */COPY* option allows you to use the other features of SyncSort (e.g. selection, reformatting) without changing the order of the records.

Example

/copy

This is a request for SyncSort to copy records without reordering them.

/DATADictionary

Purpose

To specify the location of an input record layout.

Format

/DATADictionary *file_name* COBOL

Arguments

<i>file_name</i>	the pathname of a file which contains at least one complete COBOL data description with a level 01 entry. A complete COBOL program is not accepted. If the pathname contains a space, surround the name with double quotation marks.
------------------	--

Location

The option may appear anywhere in the application definition.

Notes

If a file you want to sort is also used by a COBOL program, it is likely that the records in the file are described through a COBOL data description. You can make the contents of the COBOL data description available to SyncSort through the **/DATADictionary** option. Fields which are part of the data description can then be used in other SyncSort options without further specification. Details of the data description portion of a COBOL program can be found in *Micro Focus COBOL Language Reference*.

An elementary item which is subordinate to group items can be referred to by its unqualified data name (such as **product_code**), if it identifies the field unambig-

uously, or by its data name qualified by the data names of its superior group items (such as **invoice.line_item.product_code**).

An elementary or group item which is defined as a table through the **OCCURS** clause can only be referred to by its data name followed by a subscript enclosed in brackets, for example, **group[7]**, **month.day[28]**, **group_code.emp_id.hours_spent[5,4]**.

To refer to a complete table as a single field, use the elementary or group item without the index.

SyncSort supports 88 levels. You can use a condition name from an 88 level as a condition name in a SyncSort option.

SyncSort currently supports the **OCCURS a TO b TIMES DEPENDING ON c** clause only when it appears at the end of a data description.

The following clauses are accepted but ignored:

- GLOBAL
- BLANK WHEN ZERO
- EXTERNAL

See “Specifying Field Names ” on page B.9, for more details on referencing elementary and group items (subfields) and tables (arrays).

Example

```
/datadictionary company.cbl cobol
```

This option specifies that the file **company.cbl**, which is in ANSI standard format, defines the layout of the input records for the application.

/DATASIZE

Purpose

To specify the amount of data released to SyncSort after all input record processing.

Format

```
                [r_number RECORDS  ]  
/DATASIZE      [b_number BYTES    ]  
                [k_number KILOBYTES]  
                [m_number MEGABYTES]
```

Arguments

<i>r_number</i>	the number of records which will be released to the application.
<i>b_number</i>	the amount of data in bytes which will be released to SyncSort.
<i>k_number</i>	the amount of data in kilobytes (1,024 bytes) which will be released to SyncSort.
<i>m_number</i>	the amount of data in megabytes (1,048,576 bytes) which will be released to SyncSort.

Location

The option may appear anywhere in the application definition.

Notes

It is recommended that you provide the /DATASIZE option for a sort application when:

/DATASIZE

- SyncSort cannot obtain the information from the file system for the input files.
- you request input selection which omits a sizeable percentage of the records.
- you are running a Micro Focus COBOL application with a SORT or MERGE verb and the environment variable SYNC_SORT_USE_EXTFH=1 is specified.

/DATASIZE overrides any size values you specified for */INFILE*.

Examples

```
/datasize 256 kilobytes
```

This informs SyncSort that 256 kilobytes of data will be released to SyncSort after input record processing is done.

```
/datasize 700 records
```

This informs SyncSort that 700 records of data will be released to SyncSort after input record processing is done.

```
/datasize 70000 bytes
```

This informs SyncSort that 70000 bytes of data will be released to SyncSort after input record processing is done.

/DELETE

Purpose

To discard the last two lines of options keyed in.

Format

/DELETE

Location

The option may appear anywhere in the application definition.

Notes

The **/DELETE** option is valid only for applications invoked through the **syncsort** command in a line-by-line interactive session.

Use the **/DELETE** option when the previous line of keyed text needs to be removed. At the same time that the option erases the previous line it also removes the current line, including the **/DELETE** option. If you want to see the corrected definition before continuing, use the **/DISPLAY** option.

Example

```
/delete
```

This causes the current and preceding lines of application options to be removed.

/DELIMITEDRECORDLAYOUT

Purpose

To describe a layout of the input records in terms of named consecutive delimited fields.

Format

```
/DELIMITEDRECORDLAYOUT field [, field ... ]
```

where

```
field                =    [composite_field ]  
                        [elementary_field]
```

```
composite_field     =    name {field [, field ... ]} [REPEAT number]
```

Note that the braces ({ }) in *composite_field* must appear when shown. They do not indicate a choice of argument in this case.

```
elementary_field    =    [name [format] [precision|sequence] [REPEAT number] ]  
                        [FILLER [filler_count]]
```

```
format              =    [CHARACTER                ]  
                        [numeric_format            ]  
                        [DATETIME (datetime_pattern)]
```

```
precision          =    integral_digits/fractional_digits
```

Arguments

<i>name</i>	the name of a field in the input record layout, which you will use to reference the field in other SyncSort options.
-------------	--

When naming a field, you must adhere to the rules described for an identifier in “Identifiers ” on page B.7.

<i>number</i>	the number of times a field is repeated consecutively in the input record.
<i>filler_count</i>	the number of delimited fields that make up an unnamed portion of the record. You can omit the filler count to indicate a variable number of fields at the end of the record.
<i>sequence</i>	the sequence in which values are to be ordered when the field is involved in comparisons. The sequence is used when the field is defined as a sort key, when the field appears in a condition, and when the field is used as the control for a section of a report. An acceptable value for the sequence argument is one of the following: • a standard collating sequence. See “Table 6. Standard Collating Sequences Supported by SyncSort” on page 3.70. • the name of an alternative collating sequence defined through /COLLATINGSEQUENCE.
<i>numeric_format</i>	the type of numeric data held in the field. The default data type is character unless the position is a bit position, in which case the default data type is bit. A list of acceptable values for the data type argument can be found on “Table 1. `Data Types Supported by SyncSort” on page A.1.
<i>integral_digits</i>	the maximum number of significant digits to appear before any decimal point in an edited numeric value. The minimum number is 0. The maximum number is the maximum length supported for edited numeric fields. See “Table 1. `Data Types Supported by SyncSort” on page A.1. The default is 15 digits for a delimited field. The default for a positional field is dependent on the field length.
<i>fractional_digits</i>	the maximum number of digits to appear after any decimal point in an edited numeric value. The minimum number is 0. The maximum number is one less than the maximum length supported for edited numeric fields. See “Table 1. Data Types Resulting from Paired Data Types” on page A.1.

date_pattern the pattern describing the contents of a date/time field. Source field date patterns are described in “Source Date Patterns” on page E.27.

Location

This option may appear anywhere in the application definition.

Notes

Describing the record layout

So that SyncSort can extract key fields, reformat records, make selections based on field values, etc., it is necessary for you to provide a map of the data in the records input to your application. One of the ways you can provide such a map when the record is composed of delimited fields is through the /DELIMITEDRECORDLAYOUT option. The basic building block of the map is an *elementary field*, which describes one delimited field in an input record. An elementary field is not subdivided further. In many cases, your input record layout will consist simply of a list of elementary fields.

When an elementary field in the input record is repeated several times in the record without any intervening fields or fillers, you can conveniently specify the field only once and add the REPEAT argument to indicate how many times it is repeated. A repeated field is referred to as an array. Referencing fields in an array in a SyncSort option is described in “Specifying Field Names ” on page B.9. When all elements of an array are referenced as a single field, it is treated as a character field.

When two or more consecutive elementary fields in your input record are subdivisions of a larger field, it is sometimes convenient to name this larger field and refer to it in other options. A field made up of several elementary fields is referred to as a *composite field*. For example, a composite field **student_name** may consist of three elementary fields: **first_name**, **middle_initial** and **last_name**. In another option such as /REFORMAT you may be able to refer to one field **student_name** rather than its three elementary fields. In this example, you are not required to specify the composite field in addition to the elementary fields, but doing so may save you time.

A composite field can have other composite fields as components, as well as elementary fields. For example, a composite field **home_address** may consist of four elementary fields: **address**, **city**, **state** and **zip**. Another composite field, **personal_info**, may consist of three fields: **student_name** (a composite field), **home_address** (a composite field) and **telephone** (an elementary field).

The elementary fields that make up a composite field do not have to have the same data types. You cannot assign a data type to a composite field. A composite field containing one or more elementary fields is treated as if it is of character type, irrespective of the data types of its component elementary fields.

When a composite field in the input record is repeated several times in the record without any intervening fields or fillers, you can conveniently specify the field only once and add the REPEAT argument to indicate how many times it is repeated.

Date/time fields

A date/time elementary field consists of a calendar date component and/or a time component. The contents are in printable form. For example, the date component may appear as ***month/day/year*** with the month as two digits, the day as 2 digits and the year as 4 digits. The time component may appear as ***hour:minute:second*** with the hour as two digits using the 24-hour clock, the minutes as 2 digits and the seconds as 4 digits. 5 minutes and 9 seconds after 8 o'clock in the morning of August 4, 2000 appears as 08/04/2000 08:05:09 for this format. You would represent this date/time *date_pattern* as **(MM0/DD0/YEAR HH0:MI0:SE0)**. See “Source Date Patterns” on page E.27 for a description of how to describe a date in a source field.

Examples

```
/delimitedrecordlayout filler 1, emp_id en, filler 3,  
emp_name
```

This option describes an input record layout. **filler 1** and **filler 3** cover the input record fields that will not be referenced in other options. The option identifies by name only those fields that will be needed elsewhere in the application. The first of these, **emp_id**, is the second delimited field in the record and is numeric. The second named field, **emp_name**, is the sixth delimited field and contains character data.

/DELIMITEDRECORDLAYOUT

```
/delimitedrecordlayout location char fascii, filler 3,  
store_code char, manager_code char, emp_count en, filler 4,  
p_ind char
```

The option describes an input record layout with five named fields. The first field, **location**, contains character data. The values in **location** are collated according to the folded ASCII sequence. **filler 3** covers the next 3 delimited fields, fields 2 - 4. The fifth field is the character field **store_code**. Another character field, **manager_code**, follows. The seventh field, **emp_count**, contains numeric data. **filler 4** identifies an unnamed stretch of 4 fields. **p_ind** is the final field, the twelfth.

```
/delimitedrecordlayout cust_code en, regular_payment en,  
month_amount en repeat 12, month_paid repeat 12,  
date_received repeat 12, reminder_sent repeat 12
```

The record layout described by this option consists of 50 fields. The first field, **cust_code**, is a numeric field. The second field, **regular_payment**, is also numeric. The remaining fields define arrays in the record layout. The **month_amount** array defines an array containing twelve delimited fields, as indicated by repeat 12. Each field in the array is numeric. The first field in the array immediately follows **regular_payment**. The complete array occupies fields 3 through 14. This is followed by the **month_paid** array, which consists of the character fields 15 - 26. Fields 27 through 38 of the record are occupied by the **date_received** array. The record layout is completed by the **reminder_sent** array, which contains 12 character fields.

```
/delimitedrecordlayout book-id en, author {first_name,  
middle_initial, last_name}, title, publisher, date_published  
datetime (mm/dd/year)
```

This /DELIMITEDRECORDLAYOUT option describes an input record containing information on one book in a library. The record starts with a unique identifier for the book (**book_id**, a number). Starting at the second field of the record is the author information. This consists of the character **first_name** field, the **middle_initial** field and the **last_name** field. The author information can also be referenced as a multiple-delimited field, **author**, which is a composite field consisting of the first name, middle initial and last name. The character fields **title** and **publisher** and the date field **date_published** complete the record layout. **date_published** contains a date consisting of the month with any leading zero removed, the day with suppressed leading zero and the 4-digit year. The three components are separated by slashes.

```
/delimitedrecordlayout    cust_code en, regular_payment en,
                           monthly_account
                           { month_amount en,
                             date_received datetime
(mm0/dd0/year),
                           month_paid char,
                           reminder {reminder_sent char,
                                     date_sent datetime
(mm0/dd0/year)
                           }repeat 2
                           }repeat 12
```

The option in this example describes a record containing one year's payment records for a customer. The first field in the record is a customer's identifier (**cust_code**, a number). The amount of the customer's regular payment amount (**regular_payment**, also a number) is held in the second field.

Starting at the third field is the 12-entry array **monthly_account**. Each **monthly_account** entry contains the payment record for a single month. Each month's entry is 7 fields long and consists of the amount received for the monthly payment (**month_amount**, a number), the date payment was received (**date_received**), a flag indicating if the monthly payment is completely paid (**month_paid**), and a 2-entry **reminder** array. Each of the **reminder** array entries has 2 fields and consists of a **reminder_sent** flag and the **date_sent** field. The record represented by this layout has 86 fields.

/DERIVEDFIELD

/DERIVEDFIELD

Purpose

To define a field whose value is derived from constants and the values of other fields.

Format

```

                                     [source_value [derived_type]]
/DERIVEDFIELD name    =  [PAGENUMBER numeric_type   ]
                        [TODAY (date_mask)           ]
                        [RECORDNUMBER numeric_type   ]

```

where

```

                                     [field            ]
                                     [number            ]
source_value    =  [text_string          ]
                  [bit_string            ]
                  [conditional_value     ]
                  [arithmetic_expression]

```

```
conditional_value = IF condition THEN source_value ELSE source_value
```

```
arithmetic_expression = operand [arithmetic_operator operand...]
```

```

                                     [field            ]
operand          =  [+|-] [number            ]
                  [(arithmetic_expression)]

```

```

                                     [*]
arithmetic_operator =  [/]
                  [+ ]
                  [- ]

```

<i>derived_type</i>	=	[[<i>character_type</i> [<i>character_edit</i>] [<i>sequence</i>]]
<i>character_type</i>	=	[<i>CHARACTER</i> <i>length</i>] [<i>length</i> [<i>CHARACTER</i>]] [<i>EXTRACT</i> / <i>pattern_match</i> /[<i>fmt_string</i>] [<i>CHARACTER</i>]]
<i>character_edit</i>	=	[<i>COMPRESS</i>] [<i>TRUNCATE</i>]
<i>numeric_type</i>	=	[<i>numeric_format</i> [<i>length</i> [<i>COMPRESS</i>]]] [<i>length</i> <i>numeric_format</i> [<i>COMPRESS</i>]] [<i>EN</i> <i>length</i> [<i>precision</i>] [<i>COMPRESS</i>]] [<i>length</i> <i>EN</i> [<i>precision</i>] [<i>COMPRESS</i>]] [<i>COMPRESS</i>] [(<i>numeric_edit_mask</i>)] [<i>EXTRACT</i> / <i>pattern_match</i> /[<i>fmt_string</i>] [<i>EN</i>] [<i>COMPRESS</i>]]
<i>precision</i>	=	<i>Integral_digits</i> / <i>fractional_digits</i>
<i>bit_type</i>	=	[<i>BIT</i> <i>length</i>] [<i>length</i> <i>BIT</i>]

Arguments

<i>name</i>	the name of the derived field, which you use to reference the field in other SyncSort options. The name you give to a field must adhere to the rules described for an identifier in “Identifiers ” on page B.7.
<i>date_mask</i>	an edit mask to define the format of the date in the derived field. See “Edit Specifications for Date Fields ” on page E.11.
<i>field</i>	the field that is the source of the derived field. The field can be a field in the source record, or another derived field. See “Specifying Field Names ” on page B.9 for details on specifying a field name.

<i>number</i>	a numeric quantity. You can specify the number in any of the formats described in “Specifying Numeric Constants” on page B.4, provided the chosen format is compatible with the usage of the number.
<i>text_string</i>	a character string. You can specify the string in any of the formats described in “Specifying Character String Constants” on page B.1.
<i>bit_string</i>	a bit string. The format of a bit string is described more in “Specifying Bit String Constants ” on page B.6.
<i>condition</i>	the name of a condition defined through /CONDITION or a level 88 in a COBOL copybook supplied through /DATADictionary.
<i>sequence</i>	the sequence used to order values in the field when it is involved in comparisons. The sequence is applied when you use the field as a key, or in a condition, or as the control for a section of a report.

An acceptable value for the sequence argument is one of the following:

- A standard collating sequence. See Table 6. on page 3.70.
- The name of an alternative collating sequence defined through /COLLATINGSEQUENCE.

Only a character field can have an associated sequence. The default sequence is the collating sequence of the current locale unless you define a default through /COLLATINGSEQUENCE DEFAULT.

<i>length</i>	the length of the derived field. The length is in bytes for all data types except BIT, when the length is in bits. See “Table 1. `Data Types Supported by SyncSort” on page A.1 for the length of a field with one of the supported data types.
---------------	---

You may omit the length for DFLOAT and FLOAT.

<i>pattern_match</i>	a mask to extract one or more sub-fields from the source field. See “Masks to Extract Sub-fields” on page E.15 for details. If you need to include a slash (/) in the mask other than in a bracketed list, precede it with a backslash (\).
<i>fmt_string</i>	a character string. Specify the string in the single-quoted format described in “Specifying Character String Constants” on page B.1. The string can contain ASCII characters, hexadecimal characters, characters in escaped notation, and sub-field replacements. A sub-field is defined by a parenthesized portion of <i>pattern_match</i> . The maximum sub-field number is 9. See “Specifying Character String Constants” on page B.1 for details. A sub-field number that does not correspond to a group in <i>pattern_match</i> results in an empty sub-field.
<i>numeric_format</i>	the type of numeric data in the derived field. See “Table 1. `Data Types Supported by SyncSort” on page A.1 for the length of a field with one of the supported data types.
<i>integral_digits</i>	the maximum number of significant digits to appear before any decimal point in an edited numeric value. The minimum number is 0. The maximum number is the maximum length supported for edited numeric fields (see “Table 1. `Data Types Supported by SyncSort” on page A.1).
<i>functional_digits</i>	the maximum number of significant digits to appear after any decimal point in an edited numeric value. The minimum number is 0. The maximum number is one less than the maximum length supported for edited numeric fields (see “Table 1. `Data Types Supported by SyncSort” on page A.1).
<i>numeric_edit_mask</i>	an edit mask to convert the derived field value to a printable format. See “Edit Specifications for Numeric Fields” on page E.1 for details.

Location

The option may appear anywhere in the application definition.

Notes

Deriving a Field Value

When the condition in an **IF** statement evaluates as true, the derived field takes the value of the **THEN** clause. When the condition evaluates as false, the derived field takes the value of the **ELSE** clause.

A derived field specified with the **PAGENUMBER** argument contains the current page number for a report.

A derived field specified with the **TODAY** argument contains the current record number for a target. Such a field can be used in an output reformat only.

A derived field specified with the **RECORDNUMBER** argument contains the current record number for a target. Such a field can be used in an output reformat only.

Derived fields of character type are aligned left. Derived fields of edited numeric format and those of numeric data types with an applied edit mask are aligned right.

Nested If Statements

The **THEN** clause and/or **ELSE** clause of an **IF** statement can be another **IF** statement. The level of nesting of **IF** statements is not limited.

Compatibility of THEN and ELSE

The data types of the fields and constants which appear in the **THEN** and **ELSE** clauses of an **IF** statement used to define the value of a derived field must be compatible with each other. They must also be compatible with the data type which you may assign to the derived field itself. The compatibility of data types is summarized in the following table. Incompatible data types are indicated by **n/a**.

	AN	BIT	CHAR	DFLOAT	EN	FLOAT	INT	LP	LS	LZ	PD	TP	TS	UINT	ZD
AN	an	n/a	n/a	n/a	en	n/a	pd	lp	ls	lz	pd	tp	ts	an	zd
BIT	n/a	bit	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
CH	n/a	n/a	ch	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
DFLOAT	n/a	n/a	n/a	dfloat	en	dfloat	dfloat	n/a	n/a	n/a	n/a	n/a	n/a	dfloat	n/a
EN	en	n/a	n/a	en	en	en	en	en	en	en	en	en	en	en	en
FLOAT	n/a	n/a	n/a	dfloat	en	float	float	n/a	n/a	n/a	n/a	n/a	n/a	float	n/a
INT	pd	n/a	n/a	dfloat	en	float	int	lp	ls	lz	pd	tp	ts	int	zd
LP	lp	n/a	n/a	n/a	en	n/a	lp	lp	lp	lp	pd	tp	ts	lp	lp
LS	ls	n/a	n/a	n/a	en	n/a	ls	lp	ls	ls	pd	tp	ts	ls	ls
LZ	lz	n/a	n/a	n/a	en	n/a	lz	lp	ls	lz	pd	tp	ts	lz	zd
PD	pd	n/a	n/a	n/a	en	n/a	pd	pd	pd	pd	pd	pd	pd	pd	pd
TP	tp	n/a	n/a	n/a	en	n/a	tp	tp	tp	tp	pd	tp	tp	tp	tp
TS	ts	n/a	n/a	n/a	en	n/a	ts	ts	ts	ts	pd	tp	ts	ts	ts
UINT	an	n/a	n/a	dfloat	en	float	int	lp	ls	lz	pd	tp	ts	uint	zd
ZD	zd	n/a	n/a	n/a	en	n/a	zd	lp	ls	zd	pd	tp	ts	zd	zd

Table 1. Data Types Resulting from Paired Data Types

When you use fixed position fields and/or delimited fields and/or constants in the **THEN** and **ELSE** clauses of an **IF** statement, the types of **THEN** and **ELSE** references must be compatible. See Table 3 below for a summary of reference type compatibility. Incompatible reference types are indicated by the **n/a**.

/DERIVEDFIELD

	Fixed Length or Constant	Variable Length Positional Field	Fixed Number of Delimited Fields	Unknown Number of Delimited Fields
Fixed Length or Constant	fixed length	variable length	n/a	n/a
Variable Length Positional Field		variable length	n/a	n/a
Fixed Number of Delimited Fields			fixed number of delimited fields	unknown number of delimited fields
Unknown Number of Delimited Fields				unknown number of delimited fields

Table 2. Types Resulting from Paired Reference Types

Examples of reference types are shown in the following table:

ReferenceType	Examples
fixedlength	/fieldspl7char4 /fieldsdl3:5-3:9en /derivedfieldddf1100ls5
variablelengthpositionalfield	/fieldsp272char
fixednumberofdelimitedfields	/fieldsd210:-15:,d318:-18:en
unknown numberofdelimitedfields	/fieldsd425:

Table 3. Reference Type Examples

Default Data Type and Length

If you omit the data type and length of the derived field, SyncSort attempts to determine these from the field definition as follows:

- When the definition is not a conditional value or an arithmetic expression, the data type and length are the same as that of the source.

-
- When the source is another field, the derived field has the same data type and length as the source field.
 - When the value is an integer number, the derived field is an unsigned decimal for a positive number and a leading separate sign decimal for a negative number. The length is sufficient to hold the number of significant digits in the integer.
 - When the value is a floating point number, the derived field is edited numeric. The length is sufficient to hold a sign, the number of significant digits before the decimal point, and, when there are non- zero digits after the decimal point, the decimal point and the following significant digits.
 - When the definition is a conditional value, SyncSort determines the default data type and length from the values appearing in the **THEN** and **ELSE** clauses. Data types must be compatible. See “Table 1. Data Types Resulting from Paired Data Types” on page 3.52 for a summary of data type compatibility, and also for the data type resulting from pairings of data types in the **THEN** and **ELSE** clauses. When the lengths of the values differ, SyncSort assigns to the derived field the length necessary to contain the longest value.
 - When the definition is an arithmetic expression, SyncSort determines the default data type and length from the values appearing in the expression. Be aware that the default length may not be sufficient to hold the result of the arithmetic expression for all records. Overflow or underflow occurs for this situation. It is preferable that you provide the data type and length for the derived field, as well as the precision for edited numeric type.
 - When two values (operands) are combined via an arithmetic operation (addition, subtraction, multiplication or division), the result has a data type that is dependent on the data type of the two operands. See “” on page 2.126 for a summary of the resultant data types. The length of the result is sufficient to hold the larger of the two operands.

Data Conversion

When a value of one numeric format and length is converted to another numeric format and/or length, the value is unchanged when the target can

accommodate the source value. When the target cannot hold the source value, you are informed of the overflow.

When a character value of one length is converted to another length, the value is padded on the right with the record pad byte when the target is longer than the source. The rightmost characters are truncated when the target is shorter than the source.

When a bit value of one length is converted to another length, the value is padded on the right with 0 bits when the target is longer than the source. The rightmost bits are truncated when the target is shorter than the source.

Extracting Parts of a Field

Sometimes you may want to extract a portion of a field that does not have the same location in the field for all records. This could happen, for example, for a field containing a date and time. The field might contain any of the following: '5/7/1999 3:12:45', '12/22/1999 15:55:12', '4/15/1999 6:10:46'. You cannot predict where a particular component of the date/time starts within the field, or what its length is. However, the **pattern** of components within the field is constant from record to record; that is, the month is followed by a slash, the day, another slash, the year, a space, the hour, a colon, etc. Under these conditions you can extract portions of the field by supplying the pattern of the components through **EXTRACT /pattern_match/** and the component(s) you want through **fmt_string**. For example, **EXTRACT /[0-9]+:[0-9]+:[0-9]+/** extracts the time portion and **EXTRACT /[0-9]+\/[0-9]+\\[0-9]+\/"\1'** extracts the year from the date/time field described earlier. Note that backslash characters precede the slash characters within the mask. See “Masks to Extract Sub-fields” on page E.15 for full details.

When you extract components and supply a format string, SyncSort inserts the extracted components into the format string. For example,

```
EXTRACT /([0-9]+)\/([0-9]+)\/([0-9]+)/ 'year: \3 month: \1
day: \2'
```

yields a value such as **year: 1999 month: 5 day: 7**. If you exclude the format string, the resulting value is the portion of the field matched by the complete mask. For example,

```
EXTRACT /[0-9]+:[0-9]+:[0-9]+/
```

yields a value such as **3:12:45**.

A derived field extracted from a character field also has character type. If you

know that the result of the extraction is numeric, and you need it to be treated as numeric, convert the derived field to numeric by supplying the EN keyword. For example, the date/time field described earlier is character (it contains non-digits). If you want to sort the records by month, you need to specify the extracted portion of the field as numeric, otherwise month **10** will sort before month **5**. The following extraction will cause the derived field to contain the month and to be treated as numeric

```
EXTRACT / ([0-9]+) \/[0-9]+\/[0-9]+/ '\1' EN
```

When a sub-field is extracted from a field with numeric format, the field is first converted to edited numeric type. The resulting derived field has edited numeric type unless the format string contains non-digit characters. In that case, the derived field is character. If you want a numeric derived field to be treated as character, supply the CHARACTER keyword. A sub-field cannot be extracted from a bit field.

The reference types of values derived by extracting sub-fields are summarized in Table 5.

ReferenceType of Base	Reference Type of Derived Field
fixedlength	variable length
variablelengthfield	variable length
fixednumberofdelimitedfields	unknown number of delimited fields
unknown numberofdelimitedfields	unknown number of delimited fields

Table 4. Reference Type of Value Derived by Sub-field Extraction

When Records are Short

If a record is too short to supply all the data required for the derived field, SyncSort pads the record with the record pad byte before extracting the data.

Compression and Truncation

Some simple editing of values with displayable data types is possible through the **COMPRESS** and **TRUNCATE** arguments. Compression and truncation provide simple editing functions to remove blanks (spaces and tabs).

Data Type	Editing Applied for Compression	Editing applied for Truncation
character	removal of all leading and trailing blank characters	removal of all trailing blank characters
edited numeric	removal of all leading blanks and leading zeroes following any sign character	none
leading separate signed decimal	removal of all leading zeroes following any sign character	none
unsigned decimal	removal of all leading zeroes	none
all others	none	none

Table 5. Editing Resulting from Compression and Truncation

Precision of Edited Numeric Fields

When you do not supply a precision for a numeric derived field with edited numeric format, SyncSort attempts to derive the precision from the source:

- When the source is a numeric constant, SyncSort assigns a precision that is sufficient to represent the numeric constant.
- When the source is a field of **float** or **dfloat** type, the precision is as follows:
 - for the integral portion, the precision is sufficient to hold the maximum integral digits in **float** or **dfloat**
 - for the fractional portion, the precision is 2.
- When the source is another edited numeric field with a precision, the derived field inherits the precision of the source field.
- When the source is an arithmetic expression, the precision is determined from the operands in the expression. See “Data Type and Precision of the Result of Arithmetic Expressions” on page F.4 for details. When the result of the

arithmetic expression is placed in a field, the type of field determines the final precision.

- delimited field: the delimited field is of sufficient length to hold all integral digits, the decimal point, and all fractional digits.
- fixed-position field: the field is of fixed length, so if there is insufficient room to accommodate all integral digits and fractional digits, trailing fractional digits are truncated. If there is insufficient room to accommodate the integral digits, an overflow results.
- In all other cases, SyncSort assigns no precision to the derived field.

When an edited numeric derived field has no attached precision specification, the field does not contain thousands separators.

Examples

```
/derivedfield country if in_usa then "USA" else "outside  
USA" char 12
```

The option creates a new field, **country**, which is 12 bytes long and contains character data. If the condition **in_usa** is true, the character string **USA** is placed in the field. Otherwise, the string **outside USA** is placed in the field.

```
/derivedfield cold "COLD"
```

The option requests that the character string **COLD** be placed in the new field **cold**, which is a character field of length 4 bytes.

```
/derivedfield c_total if out_of_stock then total_on_order  
else 0 int 2
```

The field **c_total** is a 2-byte long integer field. If the **out_of_stock** condition is true, the **c_total** field receives the value of the field **total_on_order**. Otherwise, the value appearing in **c_total** is zero.

```
/derivedfield state if in_usa then address_state else  
2 x"FF" fascii
```

If the condition **in_usa** evaluates as true, the value of the **address_state** field is assigned to the derived field **state**. If **in_usa** is false, the **state** field is assigned the value of hexadecimal FFFF. The values in the **state** field are to be ordered according to the folded ASCII sequence. **state** is a character field whose length is equal to the length of **address_state**, or 2, whichever is the larger.

```
/derivedfield print_date today (mm/dd/year hh:mi0)
```

The derived field **print_date** contains a value dependent on the current system date. **print_date** is 16 characters long. The result of applying the edit mask is left justified in the field. The editing consists of creating a value containing the current month in numeric form with any leading zero removed (e.g. **3** for **March**, **10** for **October**), followed by a slash, the current day of the month with any leading zero removed, another slash, and the four digit year. The date portion is followed by a space and the time portion, consisting of the current hour from the 24-hour clock with leading zero removed, a colon and the current minute with no zero removed. For example, 3 o'clock in the afternoon of February 14, 1998 appears as **2/14/98 15:00** with a single trailing space. For details on edit masks for dates, see "Edit Specifications for Date Fields" on page E.11.

```
/derivedfield e_total if out_of_stock then total_on_order else 0  
(ZZ9)
```

When the condition **out_of_stock** is true, the value assigned to the field **e_total** is the value of the field **total_on_order**. When **out_of_stock** is false, **e_total** contains zero. The value in **e_total** contains 3 characters and is displayable. Leading zeroes are replaced by spaces, but when the **e_total** value is zero, a '0' character appears, preceded by 2 spaces. For details on edit masks for numeric fields, see "Edit Specifications for Numeric Fields" on page E.1.

```
/derivedfield page pagenumber (z9)
```

The field **page** is intended for use in a report in headers and footers. It cannot be referenced for other uses, for example, as a key, as a column, or in reformatting. The field contains the number of the current page of the report. **page** is 2 characters long, as indicated by the numeric edit mask Z9. A page number less than 10 appears as a space followed by a digit. For details on edit masks for numbers, see "Edit Specifications for Numeric Fields" on page E.1.

```
/derivedfield comments remainder truncate
```

The derived field **comments** is a character field of variable length. It contains the contents of the **remainder** field, which contains character data, with all trailing blanks (spaces and tabs) removed.

```
/derivedfield test_bits if switch_bits then bit_fl else b"111"  
bit 3
```

The derived field **test_bits** is a bit field of length 3 bits. When the condition **switch_bits** is true, **test_bits** contains the first 3 bits of the bit field **bit_fl**. If **bit_fl** is less than 3 bits long, the missing trailing bits are defaulted to 0 bits. When **switch_bits** is false, all bits in **test_bits** are set to 1.

```
/derivedfield percent_vested  
if years_lt_5 then if years_lt_2 then 0.0  
                                else percent_complete  
else 100.0 en 6 3/1
```

The value appearing in the derived field **percent_vested** is dependent on the evaluation of one of both of the conditions **years_lt_5** and **years_lt_2**. If both conditions are true, **percent_vested** contains the value zero. If **years_lt_5** is true but **years_lt_2** is not true, **percent_vested** contains the value from the field **percent_complete**, which has a numeric data type. If **years_lt_5** is false, **percent_vested** contains the value 100. **percent_vested** is 6 bytes long and is of edited numeric type. The precision specification **3/1** ensures that the derived field has no more than 3 decimal digits prior to any decimal point, and no more than one digit after the decimal point. So, for example, when **percent_complete** contains the value 57.352 and **years_lt_5** is true but **years_lt_2** is not true, **percent_vested** contains the value 57.3, with two leading spaces.

```
/derivedfield integral_part en_field extract /(+|-)?[0-9,]*  
9,]*/compress
```

The value of **integral_part** is extracted from the field **en_field**. The extracted portion of **en_field** is defined by the pattern **/(+|-)?[0-9,]***/. The first part of the pattern **(+|-)?** matches the first plus or minus sign in **en_field**. There does not have to be a sign, as indicated by the zero-or-one repetition operator **?**. The final part of the pattern **[0-9,]*** matches all digits and commas immediately following any sign. There do not have to be digits or commas, as indicated by the zero-or-more repetition operator *****, but if there are, all such characters are matched. Since there is no accompanying format string, the portion of **en_field** matched by the

/DERIVEDFIELD

complete pattern is extracted. The resulting sub-field is compressed. The following table shows the intermediate extracted portion before compression and the **integral_part** values that result from various **en_field** values.

en_field	Extracted Portion	integral_part
-123,456.76	-123,456	-123,456
+3,400	+3,400	3,400
.99		0
3522.768	3522	3522
0	0	0
-0.45	-0	0
000723.	000723	723

```
/derivedfield file1_recnumber recordnumber an 8 compress
```

The field **file1_recnumber** is intended for use in the reformatting associated with a target file. It cannot be referenced for other uses, for example, as a key, as a column, or in a condition. The field contains the number of the current record in the target file. **file1_recnumber** is 8 digits long, so it can contain up to 99,999,999. Any leading zeroes are removed, as indicated by **compress**.

```
/derivedfield centigrade (fahrenheit - 32) * 5 / 9 en 5 2/1
```

If value of **centigrade** is determined from the **fahrenheit** field. The precedence of the arithmetic operations (multiplication and division before addition and subtraction) is overruled by the parentheses surrounding the expression **fahrenheit - 32**. Hence, the contents of **fahrenheit** are first reduced by 32. Binary operations with equal precedence (multiplication and division) are applied left to right. Hence, the result of the subtraction is multiplied by 5, and the result of that multiplication is divided by 9. The final result is converted to edited numeric type and formatted so that there is a single digit after the decimal point, as indicated by the precision **2/1**.

/DISPLAY

Purpose

To display the current application definition.

Format

`/DISPLAY`

Location

The option may appear anywhere in the application definition.

Notes

`/DISPLAY` is valid only for applications invoked through the **syncsort** command in a line-by-line interactive session.

While you are creating the application definition in an interactive line-by-line session at a terminal, you can display the current definition at any time by using the `/DISPLAY` option. The option itself does not become a permanent part of the definition but is removed automatically.

Example

`/display`

This is a request to display the current application definition.

Notes**Documenting Your SyncSort Application**

Use the /DOCUMENTATION option to add descriptive text to your application. The documentation appears in the log that SyncSort produces when you run the application.

NEWLINE creates a line break. Each leading NEWLINE produces a blank line at the start of the documentation. For example, **3 NEWLINE** at the start of the documentation option results in 3 leading blank lines. Each non-leading NEWLINE, i.e. those that appear after text, starts a new line. Thus, if you want blank lines in the middle or end of the documentation specify one NEWLINE to start a new line and one NEWLINE for each blank line you want. For example, **3 NEWLINE** anywhere other than at the start of the documentation option results in 2 blank lines.

When the last element is not a NEWLINE, SyncSort automatically appends a NEWLINE so that the next portion of the log starts on a new line.

Example

```
/documentation 'Title:', newline,
                '    Summary report for web log activity', 2 newline,
                'Author:', newline,
                '    Ian Black (original)', newline,
                '    Bernard Taylor (updated)', 2 newline,
                'Company:', newline,
                '    ABC Incorporated', 2 newline,
                'Comments:', newline,
                '    Summarizes web log activity for a single day by client.',
                2 newline,
```

The option requests SyncSort to document the log with 4 pieces of text, each with its own heading. An example of the resulting log follows.

```
[SyncSort HP-UX/LFS Copyright (c) 2001 Syncsort Inc.]
[For use by ABC Incorporated.]
Title:
    Summary report for web log activity
Author:
    Ian Black (original)
    Bernard Taylor (updated)
Company:
    ABC Incorporated
Comments:
    Summarizes web log activity for a single day by client.
SyncSort options validated. Processing continues.
```

SyncSort statistics

Records read:	n	Data read (bytes):	n
Records sorted:	n	Data sorted (bytes):	n
Records output:	n	Data output (bytes):	n
Input record length:	n	Output record length:	n
Work space used (bytes):	n		
Elapsed time: hh:mm:ss.ss		CPU time: hh:mm:ss.ss	
SyncSort has completed			

/EDIT

Purpose

To invoke the system editor to amend the current application definition.

Format

/EDIT

Location

The option may appear anywhere in the application definition.

Notes

/EDIT is valid only for applications invoked through the **syncsort** command in a line-by-line interactive session.

You can invoke the system editor during an interactive SyncSort session to add or amend text in the current definition. To enter the editor, simply issue the /EDIT option at any stage after SyncSort starts execution. SyncSort creates a text file containing the current definition (which may not yet have any options). You are taken into the editor where you can create or amend the text. When you terminate the editor, you are returned to the SyncSort interactive session, where the updated definition is displayed.

Example

```
/edit
```

This activates the editor.

/END

/END

Purpose

To terminate the application definition.

Format

/END

Location

The option must appear after all other options in the application definition. Any text after */END* is ignored.

Notes

The */END* option (or Ctrl-D) is required to terminate the application definition when SyncSort is invoked through the **syncsort** command.

Example

/end

This ends the application definition.

/FIELDS

Purpose

To describe the layout(s) of the input records in terms of named positional or delimited fields.

Format

```
/FIELDS field [, field . . . ]
```

where

<i>field</i>	=	<i>name</i>	<i>[fixed_position_field]</i> <i>[delimited_field]</i>	<i>[precision sequence]</i>
--------------	---	-------------	---	-----------------------------

<i>fixed_position_field</i>	=	<i>[byte_position]</i> <i>[bit_position]</i>	<i>[format_length]</i>
-----------------------------	---	---	------------------------

<i>format_length</i>	=	<i>[format [length]]</i> <i>[length format]</i>
----------------------	---	---

<i>format</i>	=	<i>[[CHARACTER]]</i> <i>[numeric_format]</i> <i>[DATETIME (datetime_pattern)]</i> <i>[BIT]</i>
---------------	---	---

<i>precision</i>	=	<i>integral_digits / fractional_digits</i>
------------------	---	--

<i>bit_position</i>	=	<i>byte_position B bit_location</i>
---------------------	---	-------------------------------------

<i>delimited_field</i>	=	<i>start [NOBLANK] [-end [NOBLANK]] format</i>
------------------------	---	--

<i>start</i>	=	<i>start field num: [start_byte_loc]</i>
--------------	---	--

<i>end</i>	=	<i>end field num : [end_byte_loc]</i>
------------	---	---------------------------------------

Arguments

name the name of the field in the input record, which you will use to reference the field in other SyncSort options.

The name assigned to a field must adhere to the rules described for an identifier in “Identifiers” on page B.7.

byte_position the position at which the field starts in the input record. The first byte in the record is position 1.

sequence the sequence in which values are to be ordered when the field is involved in comparisons. The sequence is used when the field is defined as a sort key, when the field appears in a condition, and when the field is used as the control for a section of a report.

An acceptable value for the *sequence* argument is one of the following:

- A standard collating sequence:

Keyword	Collating Sequence
ASCII EBCDIC	ASCIIsequence
FASCII FEBCDIC	EBCDICsequence
MULTINATIONAL	foldedASCIIsequence foldedEBCDICsequence MULTINATIONALsequence

Table 6. Standard Collating Sequences Supported by SyncSort

- The name of an alternative collating sequence defined through /COLLATINGSEQUENCE.

Only a character field can have an associated sequence. The default sequence is the collating sequence of the current locale unless the default is defined through /COLLATINGSEQUENCE DEFAULT. See “Appendix A. Appendix C. Collating Sequences” on page C.1 for a description of the standard collating sequences.

<i>length</i>	the length of the field. The length is in bytes for all data types except BIT, when the length is in bits. The acceptable values for the length of a field with one of the recognized data types can be found in “Table 1. `Data Types Supported by SyncSort” on page A.1. The length may be omitted only for a variable length character field at the end of the input record or when the datatype is DFLOAT, FLOAT, BIT, or DATETIME. The default date/time field length required for the longest date/time value corresponding to the date/time pattern and the default bit field length is 1 bit.
<i>numeric_format</i>	the type of numeric data held in the field. The default data type is character unless the position is a bit position, when the default data type is bit. A list of acceptable values for the data type argument can be found in “Table 1. `Data Types Supported by SyncSort” on page A.1.
<i>integral_digits</i>	the maximum number of significant digits to appear before any decimal point in an edited numeric value. The minimum number is 0. The maximum number is the maximum length supported for edited numeric fields. See “Table 1. `Data Types Supported by SyncSort” on page A.1. The default is 15 digits for a delimited field. The default for a positional field is dependent on the field length.
<i>fractional_digits</i>	the maximum number of digits to appear after any decimal point in an edited numeric value. The minimum number is 0. The maximum number is one less than the maximum length supported for edited numeric fields. See “Table 1. `Data Types Supported by SyncSort” on page A.1.
<i>bit_location</i>	the location of the first bit in a bit field, within the byte at position <i>byte_position</i>. The first bit in a byte is bit 1. The range of valid values is 1 - 8.

<i>start_field_num</i>	the delimited field in the record containing the first byte of the field being defined. Delimited fields are separated by the field separator character and are numbered from left to right, beginning with 1.
<i>start_byte_loc</i>	the location in <i>start_field_num</i> of the first byte of the field being defined. You can omit the byte location to indicate the first byte of the field being defined is the first byte of <i>start_field_num</i> , though the colon is still required. When the NOBLANK argument is not specified, the first character in <i>start_field_num</i> is in location 1. When the NOBLANK argument is specified, leading blanks (spaces and tabs) are ignored and the first nonblank character is in location 1.
<i>end_field_num</i>	the delimited field in the record containing the last byte of the field being defined. Delimited fields are separated by the field separator character and are numbered from left to right, beginning with 1.
<i>end_byte_loc</i>	the location in <i>end_field_num</i> of the last byte of the field being defined. You can omit the byte location to indicate that the last byte of the field being defined is the last byte of <i>end_field_num</i> , though the colon is still required. When the NOBLANK argument is not specified, the first character in <i>end_field_num</i> is in location 1. When the NOBLANK argument is specified, leading blanks (spaces and tabs) are ignored and the first nonblank character is in location 1.
<i>date_pattern</i>	the pattern describing the contents of a date/time field. Source field date patterns are described in “Source Date Patterns” on page E.27.

Location

The option may appear anywhere in the application definition.

Notes

Using Delimited Fields

When the input records to your application are text with fields separated by a field separator character, you have to define all the portions of the record to which you need to refer in terms of delimited fields. Each portion of the record which you define through /FIELDS has a start and an end. At a minimum, you have to specify the start of a portion. Both the start and the end are specified in terms of a numbered input record field and a character location within the numbered field. Fields are numbered from 1, starting at the first field in the record. Within a field, the characters are numbered from left to right, starting at position 1. The first significant character assigned the location number 1 is dependent on whether you specify the NOBLANK argument. When NOBLANK is not specified, the first character in the field is location 1. When NOBLANK is specified, all blanks (spaces and tabs) are ignored and the first non-blank character is location 1. NOBLANK can be specified independently for the start and the end. The end must not precede the start.

When specifying the start of a portion of a record you can omit the byte location to indicate the first character in the field. If you omit the byte location when specifying the end location, this indicates that the portion of the record being defined ends at the last character in the field. If you omit the byte location of either the start or the end you must still specify the colon after the field number.

When you specify the start of a portion of the record but omit the end specification this is taken to refer to the remainder of the record.

To ensure that the values in a field containing edited numeric data compare as arithmetic values you must specify the field as EN (edited numeric). If the field is not used in a comparison (i.e. in /KEYS, /CONDITION, or /SECTION) you can safely specify it as a character field. Edited numeric data consists of optional leading blanks, an optional minus sign and zero or more digits with optional embedded thousands separator characters and the radix character.

Multiple delimited fields are allowed only when the field is in character format.

Bit Fields

When you need to refer to the values in consecutive bits of your input record and the bits do not start at a byte boundary or are not a multiple of 8 bits, describe the bits as a bit field through the BIT argument. Describe the starting position by providing both the byte in which the bit field starts and the bit number within that byte (the first bit of the field). The length you specify is the number of bits in the field. BIT format is not supported for delimited fields.

Date/Time Fields

A date/time elementary field consists of a calendar date component and/or a time component. The contents are in printable form. For example, the date component may appear as **month/day/year** with the month as two digits, the day as 2 digits and the year as 4 digits. The time component may appear as **hour:minute:second** with the hour as two digits using the 24-hour clock, the minutes as 2 digits and the seconds as 4 digits. 5 minutes and 9 seconds after 8 o'clock in the morning of August 4, 2000 appears as 08/04/2000 08:05:09 for this format. You would represent this date/time date_pattern as **(MM0/DD0/YEAR HH0:MI0:SE0)**. See “Source Date Patterns” on page E.27 for a description of how to describe a date in a source field.

Examples

```
/fields cust_id 3 integer 4, cust_name 7 character 40  
fascii
```

The option defines two fields within an input record. The first field, **cust_id**, starts at position 3, and contains integer data of length 4 bytes. The second field, **cust_name**, is a 40-byte long character field which starts at position 7; the values in this field are ordered according to the folded ASCII (**FASCII**) sequence.

```
/fields po_number 1 char 10, item_code 15 char 5, num_ordered  
23 int 2
```

Two character fields and one numeric field are defined. One character field starts in the first position and is 10 bytes long; the other character field starts at position 15 and is 5 bytes long. The numeric field starts at position 23, is 2 bytes long and of type integer.

```
/fields company_id 1 int 4, zip_code 54 char 9 zip_seq,  
comment 128 char
```

The field **company_id** is a 4-byte long integer field with the starting point at position 1. **zip_code** starts at position 54 and contains character data of length 9 bytes; the values in this field are ordered according to the alternative collating sequence **zip_seq**. **comment** is a variable length field that contains character data; it starts at position 128 and occupies the rest of the input record.

```
/fields zip 12:1 - 12:5
```

The field **zip** is a 5-byte character field that starts at the first character of the twelfth field in the record and ends at the fifth character of the same field.

```
/fields first_name 3:1 - 3: fascii, full_name 3: noblank - 5:
```

This describes two fields. **first_name** is the third field in the input record and is collated according to the folded ASCII (**FASCII**) sequence. **full_name** consists of fields three through five of the input record. Any blanks (spaces or tabs) at the beginning of the third field are not part of **full_name**.

```
/fields total_amount 25: - 25: en
```

The field **total_amount** contains edited numeric data; it is the twenty-fifth field in the input record.

```
/fields comment 13: noblank
```

The field **comment** begins at the thirteenth field in the input record and contains all characters to the end of the record. Any blanks at the beginning of the thirteenth field are not part of **comment**.

```
/fields invoice_balance 21 en 9 4/2
```

The **invoice_balance** field contains edited numeric data. The field starts at position 21 of the input record and occupies the next 9 bytes of the record. The precision specification **4/2** indicates that there are a maximum of four significant digits before the decimal point and a maximum of two digits after the decimal point. This means that the maximum value contained by **invoice_balance** is 9,999.99 and the minimum value is -9,999.99.

/FIELDS

```
/fields on_off 17b4 bit 4
```

The **on_off** field contains bit data. The field is 4 bits long, starting at the 4th bit of the 17th byte.

```
/fields date_sent 3: - 3: datetime (mm/dd/year)
```

date_sent is the third field in a delimited-field record. It contains a date consisting of the month with any leading zero removed, the day with suppressed leading zero and the 4-digit year. The three components of the date are separated by slashes.

```
/fields date_received 3 DATETIME (year/mm0/dd0)
```

date_received is a field starting at position 3. It contains a date consisting of a 4-digit year, a 2-digit month, and a 2-digit day. The three components of the date are separated by slashes. The field is 10 bytes long, as determined from the date pattern.

/IGNORE

Purpose

To specify that a number of input records are to be bypassed after input selection before any records are processed.

Format

/IGNORE *number*

Arguments

number the number of records to be bypassed after input selection.

Location

The option may appear anywhere in the application definition.

Notes

You can specify **/IGNORE** for a sort or copy if the input records originate from files.

There are several options which you can use to limit the records which are released to SyncSort. These are:

/SKIPRECORD

/SKIPBLANK

/STOPAFTER

/INCLUDE and **/OMIT**

/IGNORE

/COLLECT

/IGNORE

The order in which the options appear in the above list is the order in which SyncSort applies them. Thus, the **/IGNORE** option determines the number of records surviving input selection that SyncSort discards before it starts sorting or copying.

If you request both **/SKIPRECORD** and **/IGNORE**, SyncSort will discard records according to both options. For example, for **/SKIPRECORD 10 /IGNORE 20**, SyncSort will discard the first 10 records from the first input file and read from the 11th record onwards. As each record is retrieved, any input selection requested through **/INCLUDE** and **/OMIT** is applied. Each record surviving the selection is counted towards the **/IGNORE** count. SyncSort starts releasing records to the application when 20 records have survived selection. Thus, a minimum of 30 records from the start of the first input file are discarded.

Example

```
/ignore 6
```

This requests that SyncSort bypass the first six records that survive the input selection criteria specified through **/INCLUDE** and **/OMIT**.

//INCLUDE

Purpose

To select records for processing or for an output destination.

Format

```
/INCLUDE      =      [condition]  
                  [ALL      ]
```

The first format is used to define a group of records which satisfy a particular condition. The second format can be used to define that group of records which remains after assigning records to all other groups.

Arguments

condition the name of a condition defined in a /CONDITION option or a level 88 in a COBOL copybook supplied through a /DATADictionary option. All records which satisfy the condition belong to the same record group.

Location

The option may appear anywhere in the application definition. However, the order in which /INCLUDE and /OMIT options are defined is important. So also is the position of these options in relation to the first output destination option (/OUTFILE). Any /INCLUDE ALL must appear last in a series of /INCLUDE and /OMIT options.

Notes

Input Record Selection

Record selection options /INCLUDE and /OMIT specified **before** any output specification will target and determine if a record from all input sources is included or omitted.

The following rules determine how a record from all input sources is selected:

- If no record selection options are specified, the record is included.
- If record selection options are specified, the record is tested against each condition until a condition is satisfied. If the satisfied condition is /INCLUDE, the record is included. If the satisfied condition is /OMIT, the record is omitted. All records satisfy the condition /INCLUDE ALL.
- If a record fails all conditions and the last condition is /INCLUDE, the record is omitted. If a record fails all conditions and the last condition is /OMIT, the record is included.

Output Record Selection

Record selection options /INCLUDE and /OMIT specified **after** each output specification will target and determine if a record from all input sources is included or omitted.

A record is tested for each output target independently. Therefore, one record can be included in multiple targets.

For an output target, the following rules determine how a record is selected:

- If no record selection options are specified, the record is included.
- If record selection options are specified, the record is tested against each condition in the specified order until a condition is satisfied. If the satisfied condition is /INCLUDE, the record is included. If the satisfied condition is /OMIT, the record is omitted. All records satisfy the condition /INCLUDE ALL.

- If a record fails all conditions and the last condition is **/INCLUDE**, the record is omitted. If a record fails all conditions and the last condition is **/OMIT**, the record is included.

Examples

```
/include in_usa
```

The records to be processed further are those that meet the condition **in_usa**.

```
/include all
```

The option defines the group of records that do not meet any of the conditions for preceding **/INCLUDE** and **/OMIT** options.

//INFILE

//INFILE

Purpose

To define a file as a source of input records.

Format

```
/INFILE          file_identifier [alias] [file_attribute ...] [ record_limit ...]
```

where

```
file_identifier  =    [file_name          ]  
                   [file_descriptor OPEN]  
                   [-                    ]
```

```
file_attribute  =    [organization          ]  
                   [record_fmt             ]  
                   [maximum_length         ]  
                   [minimum_length         ]  
                   [field_separator        ]  
                   [ENCLOSEDBY leading_character [,trailing_character]]  
                   [record_alignment       ]  
                   [file_size              ]  
                   [block_size             ]  
                   [compress_attribute     ]
```

```
organization    =    [SEQUENTIAL]  
                   [INDEXED   ]  
                   [VSAM      ]
```

```
file_size        =    [r_number RECORDS  ]  
                   [b_number BYTES    ]  
                   [k_number KILOBYTES]  
                   [m_number MEGABYTES]
```



```

field_separator      =  ["a"   ]
                      =  [X"xx" ]
                      =  [O"ooo"]

record_alignment     =  [ALIGNED2 ]
                      =  [ALIGNED4 ]
                      =  [UNALIGNED]

block_size           =  BLOCKSIZE b_size

record_limit         =  FSKIPRECORDskip_number
                      =  FSTOPAFTERstop_number
                      =  FSKIPNUMBER

compress_attribute   =  [COMPRESSED ]
                      =  [UNCOMPRESSED]

```

Arguments

<i>file_name</i>	the pathname of the file which provides input records for the application. If the pathname contains a space, surround the name with double quotation marks.
<i>file_descriptor</i>	the file descriptor of the already opened file which provides input records for the application.
—	indicator for standard input. Tells SyncSort that the source records are piped from standard input.
<i>alias</i>	a name you assign to the file, and which you will use to refer to the file in other SyncSort options. The name assigned to a file must adhere to the rules described for an identifier in “Identifiers ” on page B.7.
<i>maximum_length</i>	the length in bytes of the longest record in the file. The default is 256 bytes.
<i>minimum_length</i>	the length in bytes of the shortest record in the file.

//INFILE

<i>record_fmt</i>	the format of the records in the file. Record formats are described in “Record Formats” on page D.1. Any of the record format keywords can be used.
<i>r_number</i>	the number of logical records in the file.
<i>b_number</i>	the amount of data in bytes in the file.
<i>k_number</i>	the amount of data in the file, in kilobytes (1,024 bytes).
<i>m_number</i>	the amount of data in the file, in megabytes (1,048,576 bytes).
<i>b_size</i>	the block size in bytes for a character special file other than a terminal device.
<i>a</i>	the character value of the field separator character.
<i>xx</i>	the hexadecimal value of the field separator character.
<i>ooo</i>	the octal value of the field separator character.
<i>skip_number</i>	the number of records from the beginning of the file to be bypassed, and not to be released to the application.
<i>stop_number</i>	the maximum number of records to be read from the file for release to the application.
<i>leading_character</i>	The character in the field-delimited text records that optionally precedes the data in each delimited field.
<i>trailing_character</i>	The character in the field-delimited text records that optionally follows the data in each delimited field.

Location

The option may appear anywhere in the application definition.

Defaults

When no /INFILE option is present, SyncSort takes its input from standard input (file descriptor 0).

Organization

When no input file organization is provided, SyncSort assumes that the file has sequential organization.

Record Format

When no record format is specified SyncSort processes records as stream linefeed (text).

Record Length

When no maximum record length is specified SyncSort uses 256 bytes as the maximum length for records with other than a fixed format.

When no minimum record length is specified SyncSort uses 0 bytes as the minimum length for records with other than a fixed format.

Field Separator

When you provide neither a field delimiter nor an enclosing character, SyncSort treats a sequence of one or more spaces and tabs (the UNIX default) as the delimiter in field-delimited text records.

When you do not provide a field delimiter but you do supply an enclosing character, SyncSort assumes that the delimiter in field-delimited text records is the comma.

Compression

When no compression attribute is specified, the default for the file will be the UNCOMPRESSED attribute.

Notes

Number of Input Files

The number of files which can be input to a sort, merge or copy is not limited by SyncSort. For each input file, use a separate */INFILE* option.

Organization

The input file for SyncSort can reside on disk, tape, and unit-record devices. For files on disk the supported types of organization are sequential, indexed, and VSAM. An indexed file can be a MicroFocus indexed file or a C-ISAM indexed file. The organization of files on tapes and unit-record devices is sequential.

SyncSort supports all types of VSAM files that are supported by SUN Mainframe Transaction Processing (formerly known as TRANS and Unikix). Currently those types are KSDS, ESDS, and RRDS files. SyncSort accesses VSAM files through the SUN™ Mainframe Batch Manager (MBM). See SUN™ Mainframe Transaction Processing (MTP) and Mainframe Batch Manager documentation for more information on the VSAM file organization. An indexed or VSAM file must not be supplied to SyncSort as an open file using the *file_descriptor* OPEN file identifier.

File Size

SyncSort obtains the file size from the file system, whenever possible. In some circumstances, the value is not available or is incorrect. When you know this is the case, you should provide the information in the */INFILE* option.

Record Length

If one of the record lengths is provided, SyncSort assumes that it is the maximum record length. Therefore, if you wish to specify the minimum record length, you must specify also the maximum record length (the order is unimportant).

Records longer than the maximum record length are truncated. Records shorter than a specified minimum record length are discarded. In both cases, the application proceeds.

See “Appendix A. Appendix D. Record Formats ” on page D.1 for the maximum record lengths associated with various record formats.

Record Alignment

If all records are aligned on 2-byte boundaries, you have to supply the `ALIGNED2` keyword except for fixed format records of even length. If all records are aligned on 4-byte boundaries, you have to supply the `ALIGNED4` keyword except for fixed format records whose length is a multiple of 4 bytes. The default is that records are not aligned.

Field Separator Character

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When the field delimiter is the UNIX default, each blank in a sequence of blanks is part of the following field.

When the field delimiter is not the default, specify it through the `field_separator` argument. Each delimiter separates two adjacent fields; therefore, contiguous delimiter characters indicate a null field (one whose length is zero). A non-default delimiter is not part of the following field.

When the field separator character for your input is not the default you must specify it through the `field_separator` argument. Each separator is considered to separate two adjacent fields; therefore, consecutive separator characters indicate a null field (one whose length is zero). A non-default separator is not considered part of the following field.

Field Enclosing Character

When you cannot generate the character from your keyboard, provide the hexadecimal or octal representation specifying the same syntax used for the field delimiter. You can find the hexadecimal representation in the table supplied for the ASCII collating sequence in Appendix C.

The leading and trailing enclosing characters can be the same. When they are the same, you do not need to specify the trailing character.

Each field in a field-delimited text record may be enclosed by two enclosing characters. When fields are enclosed, specify the ENCLOSEDBY argument. Any leading and trailing enclosing characters are not part of the field. Two consecutive trailing enclosing characters within the field are treated as a single enclosing character. The leading enclosing character cannot be a blank (tab or space). The trailing enclosing character cannot be the same as the field delimiter.

SyncSort ignores any characters between an enclosing character terminating a field and the following delimiter or the end of the record. When the first nonblank character following a delimiter (or the first nonblank character in the record for the first field) is not a leading enclosing character, SyncSort assumes that the field is not enclosed.

File Alias

When you wish to include or omit records based on which input file they originate from, you must give aliases to some or all of the input files. This alias is simply a shorthand way of referring to an input file. You then code the alias as part of a comparison in a /CONDITION definition.

Compression

The compressed file format that SyncSort can read is gzip, standardized in RFC 1952.

Compatibility with Other SyncSort Options

The file level record-limiting criteria FSKIPRECORD and FSTOPAFTER cannot be specified in the same application as /SKIPRECORD and /STOPAFTER. The compress attribute cannot be specified with the INDEXED, VSAM file organization, or MFLINESEQUENTIAL record format.

Examples

```
/infile sort1.in
```

The option specifies that the input records come from the file **sort1.in** with the maximum record length equal to the default of 256 bytes, and the default record format (stream LF).

```
/infile 0 open fixed 80
```

The option specifies that the input records come from standard input. The format of the records is fixed. The length of each record is 80 bytes.

```
/infile year90/payables/current/trans.dat transactions  
240 120 variable
```

This option assigns an alias, **transactions**, to the input file **trans.dat** in the directory **year90/payables/current**. The file contains variable length records. The length of the longest record is 240 bytes, that of the shortest 120 bytes.

```
/infile tape1.in 1024 fixed 210 kilobytes
```

The name of the input file is **tape1.in**. It contains fixed format records of length 1024 bytes. The size of the file is 210 kilobytes.

```
/infile report.dat fskiprecord 5 fskipblank
```

The option specifies that the first five records and all blank records in the file **report.dat** are to be bypassed. The file **report.dat** has a maximum record length equal to the default, which is 256, and the default record format (stream LF).

```
/infile sort.in2 fixed 79 aligned2
```

The option specifies that the input records are aligned on a 2 byte boundary. The format of the records is fixed. The length of each record is 79 bytes.

```
/infile sort.in ":"
```

The option specifies that the fields of the input records are separated by a colon.

```
/infile corporate_db indexed
```

The file **corporate_db** is an indexed file to be used to supply input records.

```
/infile sortdelim.dat "," enclosedby """"
```

//INFILE

The file sortdelim.dat is a text delimited file with the comma used as the field separator character and the double quote (") used as the starting and ending enclosing character.

```
/infile sortdelim.dat "," enclosedby ":", ";"
```

The file sortdelim.dat is a text delimited file with the comma used as the field separator character with the starting enclosing character colon (:) and a different ending field enclosing character (;).

/INSERT

Purpose

To augment an application definition with text contained in a file.

Format

/INSERT file_name

Arguments

<i>file_name</i>	the pathname of the file which contains a portion of the application definition. If the pathname contains a space, surround the name with double quotation marks.
------------------	---

Location

The option may appear anywhere in the application definition except where a file identifier is expected.

Notes

Options text can also be inserted via the SYNCSORT_INSERT environment variable. (See below.)

SyncSort replaces an /INSERT option with the contents of the /INSERT file when the application is executed. For example, when a SyncSort application with an /INSERT option is part of a daily batch job, the application definition can be changed from day to day simply by editing and changing the contents of the /INSERT file.

The /INSERT keyword may be placed anywhere in the definition, even after a partially completed option, except where a file identifier is expected (though that

option must be completed by the text at the start of the `/INSERT` file). Consider the following fragment from an application definition: `... /keys /insert keys.ins ...`. Here, the introduction of the `/INSERT` option leaves the specification of `/KEYS` incomplete. The insert file **keys.ins**, however, begins with **key1, key2 desc, key3 ...**, that is, with pieces of information that complete the interrupted specification of the `/KEYS` option.

An `/INSERT` option can be specified as part of the contents of another `/INSERT` file. There is no limit on the level of nesting of `/INSERT`s.

You cannot amend the contents of an `/INSERT` file during an interactive SyncSort session. If there is an error within the text, terminate the SyncSort session, edit the `/INSERT` file and re-execute SyncSort.

When you first key an `/INSERT` option, SyncSort displays the contents of the inserted file. This is to enable you to check that you have picked the correct file.

Since an application definition is terminated by the first `/END` option, `/INSERT` is inoperative if it appears after `/END`.

An insert file must be a text file.

If an interactive session containing `/INSERT` is saved, the saved file contains the `/INSERT` option but not the actual contents of the insert file.

One of the uses of `/INSERT` is to include a common record layout into an application definition. One master copy of the record layout file can be inserted in many applications. Suppose you are in the middle of an interactive session and you have used a record layout file to define your fields. When you use the `/SAVE` option, your saved file will contain the `/INSERT file_name` option. You will not get multiple copies of the record layout file on disk.

Using the SYNC_SORT_INSERT Environment Variable

Instead of using the `/INSERT` option, you can specify that a file of SyncSort options be used by supplying its name as the value of the `SYNC_SORT_INSERT` environment variable. For example, if the file **srt1.ins** contains the SyncSort options to be used for all sorts, specify on the command line:

```
SYNCSORT_INSERT=srt1.ins  
export SYNCSORT_INSERT
```

This method of specifying SyncSort options is particularly useful if you want certain memory or workspace allocations to be used for all sorts.

Example

```
/insert srt1.ins
```

The option specifies that the next application options are to be taken from the file **srt1.ins**.

/JOIN

/JOIN

Purpose

To specify the disposition of paired and unpaired records in a join.

Format

`/JOIN UNPAIRED [LEFTSIDE] [RIGHTSIDE] [ONLY]`

Location

The `/JOIN` option may appear anywhere in the application definition.

Defaults

When you do not provide a `/JOIN` option in an application that has `/JOINKEYS` options, SyncSort writes all paired records to the target file. All unpaired records from both sides of the join are discarded.

When you provide a `/JOIN` option without specifying either `LEFTSIDE` or `RIGHTSIDE`, SyncSort writes the unpaired records from both sides of the join to the target file.

Notes

When `/JOIN` appears in an application, `/JOINKEYS` options must also appear.

Retaining unpaired records

When joining files, a record from one file may or may not have a match in the other file(s). A match occurs when the contents of the join keys in the record from the first file equal the contents of the join keys in the record from the second file.

You can discard unpaired records from one or both files, or retain unpaired records from both files.

To retain unpaired records from the first side, specify **/JOIN UNPAIRED LEFTSIDE**. To retain unpaired records from the second side, specify **/JOIN UNPAIRED RIGHTSIDE**. To retain unpaired records from the both sides, specify either **/JOIN UNPAIRED LEFTSIDE RIGHTSIDE** or simply **/JOIN UNPAIRED**.

Discarding paired records

You have the option of discarding the paired records from a join and keeping the unpaired ones. To do this, specify **/JOIN UNPAIRED ONLY**. If you want to keep only the unpaired records from one of the sides, add either the **LEFTSIDE** or **RIGHTSIDE** argument.

Example

```
/join unpaired leftside only
```

This option tells SyncSort to discard all paired records from the join and to keep unpaired records from the first file (leftside). Unpaired records from the second file are also to be discarded.

As an example of joining files, consider the following:

- Two transaction files from different sources contain one record per customer transaction. Each record holds information that includes the customer number, the transaction date and amount, as well as other information. There can be multiple records per customer.
- A master customer name and address file holds the current name, address and phone number for every customer, as well as the customer number. There is one record per customer.

We want to create a file containing one record per transaction, with each record holding the customer number, the customer's name and phone number and the transaction amount. The SyncSort application to do this is as follows:

```
syncsort

/fields   trans_customer_number 1: - 1:,
          /* customer number in transaction file */
trans_amount 10: - 10: en,
          /* transaction amount */
na_customer_number 1: - 1:,
          /* customer number in name & address file */
na_full_name 2: - 2:,
          /* customer name */
na_phone 8: - 8:
          /* customer phone number */

/* the left side of the join */
/infile   transactions-1 ":" 240
          /* first transaction file */
/infile   transactions-2 ":" 240
          /* second transaction file */
/joinkeys trans_customer_number
          /* join on customer number */

/* the right side of the join */
/infile   name-address ":" 200
          /* name & address file */
/joinkeys sorted na_customer_number
          /* join on customer number */
          /* file is pre-sorted */

/join     unpaired leftside
          /* include transactions for unknown customers */

/outfile  joined-file
          /* file for joined records */
/reformat leftside: trans_customer_number,
          /* customer number */
          rightside: na_full_name, na_phone,
          /* customer name and phone number */
          leftside: trans_amount
          /* transaction amount */
```

/JOINKEYS

Purpose

To identify the fields used to join records from one side to records from the other side in a join application.

Format

`/JOINKEYS [SORTED] field [, field ...]`

Arguments

<i>field</i>	the name of a field defined through /DATADictionary, /FIELDS, /DELIMITEDRECORDLAYOUT, or RECORDLAYOUT. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.
--------------	---

Location

The /JOINKEYS option for the left side must appear after the /INFILE option(s) defining the left side and before the /INFILE option(s) defining the right side. The /JOINKEYS option for the right side must appear after the /INFILE option(s) defining the right side.

Defaults

When you specify a /JOINKEYS option without providing a /REFORMAT, Sync-Sort joins the records so that the join key appears as the first field, followed by all remaining fields from left side record, followed by the remaining fields from the right side record.

Notes

Joining the records from files

To join the records from multiple files, you need to provide an /INFILE option for every source file. A record from the first file(s) (the left side) is joined to a record from the second file(s) (the right side) when the contents of specific fields from the first record equal the contents of specific fields from the second record. For each side, you need to supply a /JOINKEYS option to identify the fields used in the join.

Each record from the left side with a given value in its join keys is joined to each record from the right side with the identical value in its join keys. Thus, if m records from the left side have a given join keys value, and n from the right side have the same join keys value, the join results in $m * n$ records with the keys value.

Prior to running SyncSort to perform the join records on either side may be pre-sorted according to the join keys. If the records in the files of a side are already sorted on the join keys, you can obtain a performance boost by indicating that the records are pre-sorted. To do this, add the SORTED argument to the /JOINKEYS option, prior to the first join key.

Record filtering is not valid with a join application.

The join keys

The fields used as join keys can be either single delimited fields or fixed position fields containing character data. All join keys must be the same field format, single delimited or fixed position.

The NOBLANK argument is only valid with fields that are delimited by one or more blank characters (spaces and tabs), and can only be specified in the start field part of the field definition. For example:

```
/fields joinkey1 1:noblink - 1:
```

Examples

```
/joinkey third_field
```


This option specifies that SyncSort is to sort the records from the side associated with the join key such that the values in `third_field` are in ascending order. The records with a specific value in `third_field` are to be joined to the records in the other side with the same value in their join key.

```
/joinkeys sorted cust_number, trans_date
```

This `/joinkeys` option indicates to SyncSort that the records from the side associated with the join key are pre-sorted. The records with a specific value in `cust_number` and `trans_date` are to be joined to the records in the other side with the same value in their join keys.

/KEYS

/KEYS

Purpose

To specify the key fields used for ordering the input records.

Format

```
/KEYS field [, field ...]
```

where

field = *fieldname* [*sense*]

sense = [ASCENDING]
[DESCENDING]

Arguments

fieldname the name of a field defined through /DATADictionary, /DERIVEDFIELD, /FIELDS /DELIMITEDRECORDLAYOUT, or /RECORDLAYOUT. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.

Location

The option may appear anywhere in the application definition unless there is input record selection. When you want to specify different keys for different input record groups, the placing of the /KEYS option is important. The /KEYS option for a particular record group must appear between that /INCLUDE option which defines the group and the next /INCLUDE or /OMIT option, if any. If all groups are to have the same keys, the /KEYS option must appear before the first input /INCLUDE or /OMIT option. If several but not all groups of records are to have the same keys, the /KEYS options for those groups can be omitted and replaced by a single /KEYS statement before the first /INCLUDE or /OMIT option.

Defaults

When no **/KEYS** option is present for a sort or merge, SyncSort uses the complete data portion of the record as a single key of type character collated according to the sequence of the current locale.

The default *sense* is ASCENDING.

Notes

You must define all key fields for an application in the same **/KEYS** option unless you need different keys for different record groups. In that case, all key fields for a given record group must appear in the same **/KEYS** option.

You must specify the major key as the first field in the **/KEYS** option. Subsequent key fields should be specified in decreasing order of priority.

When you need to sort a character field using a collating sequence other than the current locale, you must specify the sequence when defining the field. For example, to sort on a character field in ASCII sequence such that the uppercase and lowercase characters are equivalent, define the key field as **/FIELDS chkey 20 CHAR 4 FASCII**.

If you need a sequence other than the DEFAULT collating sequence as specified in the **/COLLATINGSEQUENCE** option, or collating sequence in the current locale for a field specified through **/DATADictionary**, you must redefine the field using **/FIELDS** or **/DERIVEDFIELD** so that you can specify the alternative collating sequence.

/KEYS is not valid for a copy.

Examples

```
/keys volume_id
```

The option specifies that the records are to be sorted such that the values in the **volume_id** field are in ascending order (the default).

```
/keys state ascending, county ascending, amount descending
```

The option specifies that the records are to be sorted according to the values in three fields, **state**, **county** and **amount**. The fields are specified in the /KEYS option in decreasing order of priority. Thus, when SyncSort compares two records to determine which is first in order, the values in the **state** field in the two records are compared first. If the values differ, the record with the smaller value is placed before the record with the larger value (**state** is sorted in ascending order). However, if the values are the same, SyncSort compares the values in the second key field, **county**. Again, if the values differ, the record with the smaller value is placed before the record with the larger value (**county** is sorted in ascending order). Only if both **state** and **county** values are the same in the two records does SyncSort compare the values in the third key field, **amount**. If the **amount** values differ, the record with the larger value is placed before the record with the smaller value (**amount** is sorted in descending order). When all three key fields contain the same value in the two records the order of the records is not predictable, unless the /STABLE option has also been specified.

```
/keys cust_id, trans_date desc, trans_amount
```

The option specifies that the records are to be sorted according to the values in three fields, **cust_id**, **trans_date** and **trans_amount**. The fields are specified in the /KEYS option in decreasing order of priority. Thus, when SyncSort compares two records to determine which is first in order, the values in the **cust_id** field in the two records are compared first. If the values differ, the record with the smaller value is placed before the record with the larger value (**cust_id** is sorted in ascending order by default). However, if the values are the same, SyncSort compares the values in the second key field, **trans_date**. If the values differ, the record with the larger value is placed before the record with the smaller value. (**trans_date** is sorted in descending order). Only if both **cust_id** and **trans_date** values are the same in the two records does SyncSort compare the values in the third key field, **trans_amount**. If the **trans_amount** values differ, the record with the smaller value is placed before the record with the larger value (**trans_amount** is sorted in ascending order by default). When all three key fields contain the same value in the two records the order of the records is not predictable, unless the /STABLE option has also been specified.

/MEMORY

Purpose

To change the amount of memory SyncSort uses for data storage.

Format

/MEMORY amount

where

amount = $\begin{bmatrix} k_number \text{ KILOBYTES} \\ m_number \text{ MEGABYTES} \end{bmatrix}$

Arguments

k_number the maximum amount of memory SyncSort is to use for data, in kilobytes (1,024 bytes).

m_number the maximum amount of memory SyncSort is to use for data, in megabytes (1,048,576 bytes).

Location

The option may appear anywhere in the application definition.

Notes

SyncSort uses an amount of main memory sufficient to sort the input data efficiently without impacting other users on the system. There are two situations where you may want to use the /MEMORY option to change SyncSort's default behavior.

/MEMORY

- If your system has limited memory and there is the possibility of excessive paging, you can specify a limit to the amount of memory that SyncSort is able to use. Be aware that limiting memory in this manner will probably increase sort time and may also increase the disk space that SyncSort uses.
- If your input data is less than the total main memory on your system, you may be able to reduce sort time and disk space usage by specifying a memory value greater than your data size but less than your system's physical memory. In this case, SyncSort attempts to perform the sort in memory.

It is recommended that you do not specify a memory value greater than the available physical memory on your processor.

Example

```
/memory 1.2 megabytes
```

The option specifies that SyncSort can only use 1.2 megabytes to store data.

```
/memory 75 megabytes
```

The option specifies that SyncSort can use 75 megabytes to store data.

/MERGE

Purpose

To specify that SyncSort is to perform a merge (rather than a sort, join, or copy).

Format

/MERGE [*number*]

Arguments

<i>number</i>	the number of threads of input records to be merged when the records are supplied to SyncSort through a user exit. This argument is valid only when SyncSort is initiated through subroutine calls.
---------------	---

Location

The option may appear anywhere in the application definition.

Notes

/MERGE specifies a merge on sorted data without using any work files. The number of output records will be the same as the number of input records unless there is record selection (**/INCLUDE**, **/OMIT**, **/CONDITION**) or record summarization (**/SUMMARIZE**).

Example

```
/merge
```

This is a request for SyncSort to merge the threads of input records.

/NOEXECUTE

/NOEXECUTE

Purpose

To request that SyncSort should only validate the options, not execute them.

Format

`/NOEXECUTE`

Location

The option may appear anywhere in the application definition.

Notes

When you specify the `/NOEXECUTE` option, SyncSort validates your application definition, but terminates after the validation. This is useful when you are first developing an application and you do not want any of the input data to be processed.

Example

`/noexecute`

SyncSort is only to validate the options but not to execute the application.

/NOINTERRUPT

Purpose

To disable SyncSort's interrupt trap.

Format

/NOINTERRUPT

Location

The option may appear anywhere in the application definition.

Notes

SyncSort has an interrupt trap that is in operation after options are validated. Normally, when you issue an interrupt request (usually Ctrl-C), SyncSort suspends the application, displays a menu of options and prompts you with a menu of options and waits for you to take action. This is described in "2.7 Interrupting SyncSort" on page 2.xxx. If you want to disable this behavior and revert to the operating system interrupt, specify the /NOINTERRUPT option.

You can also modify SyncSort's behavior through the /NOPROMPT option. When you issue an interrupt request the behavior under the two options is as follows:

- If neither option is specified, SyncSort prompts for action.
- If /NOINTERRUPT is specified, then SyncSort reverts to the operating system interrupt.
- If /NOPROMPT is specified, SyncSort terminates and cannot be restarted.
- If both /NOINTERRUPT and /NOPROMPT is specified then SyncSort reverts to the operating system interrupt.

/NOINTERRUPT

Example

```
/nointerrupt
```

This is a request that SyncSort's interrupt trap be disabled.

/NOPROMPT

Purpose

To disable SyncSort's prompts.

Format

/NOPROMPT

Location

The option may appear anywhere in the application definition.

Notes

In certain situations, such as a disk full condition, SyncSort suspends the application and asks you to take action by displaying one or more prompts. The /NOPROMPT option disables this behavior and instructs SyncSort to return control to the operating system or to the program invoking SyncSort when such a situation arises.

Disk Full

Normally, when SyncSort is unable to write to a disk because it is full, SyncSort will prompt you with a menu of options and ask you to take action to resolve the problem. This is to provide you an opportunity to recover and to allow the application to continue. This is described in "Disk Full Recovery" on page 2.131.

If you specify /NOPROMPT and SyncSort hits a disk full condition, SyncSort issues an exception message and stops immediately.

Interrupting SyncSort

SyncSort has an interrupt trap that is in operation after options are validated. Normally, when you issue an interrupt request (usually Ctrl-C), SyncSort suspends the application, prompts you with a menu of options and waits for you to take action. This is described in “Interrupting SyncSort” on page 2.129.

You can modify this behavior when interrupted through the `/NOPROMPT` and `/NOINTERRUPT` options. When you issue an interrupt request the behavior under these options is as follows:

- If neither option is specified, SyncSort prompts for action.
- If `/NOINTERRUPT` is specified, then SyncSort reverts to the operating system interrupt.
- If `/NOPROMPT` is specified, SyncSort terminates and cannot be restarted.

If both `/NOINTERRUPT` and `/NOPROMPT` is specified then SyncSort reverts to the operating system interrupt.

Example

```
/noprompt
```

This instructs SyncSort not to issue prompts on a interrupt request or when an exception occurs.

/OMIT

Purpose

To omit records from further processing or from an output destination.

Format

/OMIT condition

Arguments

<i>condition</i>	the name of a condition defined in a /CONDITION option or a level 88 in a COBOL copybook supplied through a /DATADictionary option. All records which satisfy the condition are in the omitted record group.
------------------	--

Location

The option may appear anywhere in the application definition. However, the order in which /INCLUDE and /OMIT options are defined is important. So also is the position of these options in relation to the first output destination option (/OUTFILE).

Notes

Input Record Selection

Record selection options /INCLUDE and /OMIT specified **before** any output specification will target and determine if a record from all input sources is included or omitted.

The following rules determine how a record from all input sources is selected:

- If no record selection options are specified, the record is included.
- If record selection options are specified, the record is tested against each condition until a condition is satisfied. If the satisfied condition is /INCLUDE, the record is included. If the satisfied condition is /OMIT, the record is omitted. All records satisfy the condition /INCLUDE ALL.
- If a record fails all conditions and the last condition is /INCLUDE, the record is omitted. If a record fails all conditions and the last condition is /OMIT, the record is included.

Output Record Selection

Record selection options /INCLUDE and /OMIT specified **after** each output specification will target and determine if a record from all input sources is included or omitted.

A record is tested for each output target independently. Therefore, one record can be included in multiple targets.

For an output target, the following rules determine how a record is selected:

- If no record selection options are specified, the record is included.
- If record selection options are specified, the record is tested against each condition in the specified order until a condition is satisfied. If the satisfied condition is /INCLUDE, the record is included. If the satisfied condition is /OMIT, the record is omitted. All records satisfy the condition /INCLUDE ALL.
- If a record fails all conditions and the last condition is /INCLUDE, the record is omitted. If a record fails all conditions and the last condition is /OMIT, the record is included.

Example

```
/omit rejects
```

The records that meet the condition **rejects** are to be omitted from further processing.

/OUTFILE

Purpose

To specify a file as a destination of output records.

Format

```
/OUTFILE file_identifier [file_qual ...]
```

where

<i>file_identifier</i>	=	[<i>file_name</i> [<i>file_descriptor</i> OPEN] [-
<i>file_qual</i>	=	[<i>maximum_length</i> [<i>minimum_length</i> [<i>organization</i> [<i>record_fmt</i> [<i>field_separator</i> [<i>disposition</i> [<i>addrout</i> [<i>record_alignment</i> [<i>block_size</i> [<i>record_numbering</i> [<i>compress_attribute</i>
<i>organization</i>	=	[SEQUENTIAL] [INDEXED] [VSAM]
<i>field_separator</i>	=	["a"] [X"xx"] [O"ooo"]

<i>disposition</i>	=	[OVERWRITE] [APPEND]
<i>addrout</i>	=	ADDRROUT [RFA] [RIDX]
<i>record_alignmen</i>	=	[ALIGNED2] [ALIGNED4] [UNALIGNED]
<i>block_size</i>	=	BLOCKSIZE b_size
<i>record_numberinG</i>	=	RECORDNUMBER [START recnum_start]
<i>compress_attribute</i>	=	[COMPRESSED [HIGHCOMPRESSION]] [uncompressed]

Arguments

<i>file_name</i>	the pathname of the file which is to receive records from the application. If the pathname contains a space, surround the name with double quotation marks.
<i>file_descriptor</i>	the file descriptor of the already opened file which is to receive records from the application.
–	indicator for standard output. Tells SyncSort to pipe target records to standard output.
<i>maximum_length</i>	the length in bytes of the longest record in the file.
<i>minimum_length</i>	the length in bytes of the shortest record in the file.
<i>record_fmt</i>	the format of the records in the file. Record formats are described in “Record Formats” on page D.1. Any of the record format keywords can be used.
<i>b_size</i>	the block size in bytes for a character special file other than a terminal device.

<i>recnum_start</i>	the number assigned to the first record in the file, for record numbering. The minimum value is zero. The default is 1.
<i>a</i>	the character value of the field separator character.
<i>xx</i>	the hexadecimal value of the field separator character.
<i>ooo</i>	the octal value of the field separator character.

Location

The option may appear anywhere in the application definition.

Defaults

When no /OUTFILE option is present, SyncSort writes its output to standard output (file descriptor 1).

Organization

When you do not provide the organization of an output file, the default is determined as follows:

- When the input records are provided in files, the output file organization is the same as that of the first input file. When the default organization is indexed, the output file must exist.
- When records are supplied through an input buffer, the output file has sequential organization.
- When records are supplied through an input procedure, the output file has sequential organization.
- An addrout file always has sequential organization.
- A report file always has sequential organization.
- When the default organization is indexed or VSAM, the output file must exist.

Record Format

When you do not provide the record format, the default is determined as follows:

- When the input records are provided in files, the output format is the same as for the first input file.
- When records are supplied through an input buffer, the output format is the same as for the input records.
- When records are supplied through an input procedure, the output format is stream linefeed.
- An addnout file always contains fixed format records.
- A report file contains stream linefeed format (text) records.

Record Length

If no maximum record length is specified in */OUTFILE*, SyncSort uses the maximum from the first input file, the input buffer, or the input procedure or user exit.

If no record length is specified at all, SyncSort uses a maximum of 256 bytes as the maximum length for records other than fixed format when the output file is not a report. When the output file is a report, the maximum record length assigned is large enough to hold all lines specified for headers and footers, and the detail lines.

See “Record Formats” on page D.1 for the maximum record lengths associated with various record formats.

Field Separator

A field separator cannot be specified for a report. The Report Writer options are the following: */COLUMNS*, */PAGEFOOTER*, */PAGEHEADER*, */REPORT*, */SECTION*, */SECTIONFOOTER*, */SECTIONHEADER*, */TITLE*, and */TRAILER*.

When an output record consists of fields separated by a special character (the field separator character), the character used by default is the field separator for the

first input file (or input buffer, input procedure or user exit) for which you define a separator character, or a space if you have not defined an input field separator character. If you wish to have anything other than this default you must specify it through the *field_separator* argument.

Record Alignment

When you do not provide the record alignment, the default is determined as follows:

- When the input records are provided in files, the output alignment is the same as for the first input file.
- When records are supplied through an input buffer, the output record alignment is the same as the input record alignment.
- When records are supplied through an input procedure, the output record is unaligned.

Compression

When no compression attribute is specified, the default for the file will be the UNCOMPRESSED attribute. The HIGHCOMPRESSION option, when specified with the COMPRESSED attribute, will optimize the data output size. If optimal data output size is not required, then do not specify the HIGHCOMPRESSION option.

Notes

Number of Output Files

SyncSort writes the output records to one or more files. The number of output files is not restricted by SyncSort. For each output file, use a separate /OUTFILE option.

Organization

The output file for SyncSort can reside on disk, tape, and unit-record devices. For files on disk the supported types of organization are sequential, indexed, and

VSAM. An indexed file can be a MicroFocus indexed file or a C-ISAM indexed file. The organization of files on tapes and unit-record devices is sequential.

SyncSort supports all types of VSAM files that are supported by SUN Mainframe Transaction Processing (formerly known as TRANS and Unikix). Currently those types are KSDS, ESDS, and RRDS files. SyncSort can work with VSAM files only through SUN™ Mainframe Batch Manager (MBM). See SUN™ Mainframe Transaction Processing (MTP) and Mainframe Batch Manager documentation for more information on the VSAM file organization. An indexed or VSAM file must not be supplied to SyncSort as an open file using the *file_descriptor* OPEN file identifier.

Report Writer

Report Writer options must follow the /OUTFILE option that defines the output file for the report. The Report Writer options are the following: /COLUMNS, /PAGEFOOTER, /PAGEHEADER, /REPORT, /SECTION, /SECTIONFOOTER, /SECTIONHEADER, /TITLE, and /TRAILER.

Splitting Output Files

When output is too large to be held in a single file on disk, the SyncSort split utility can be used to split output data into multiple files. The split utility is invoked by the **syncsplit** command, which reads data from standard input. The utility writes the data to multiple files, each of which, except possibly the last, is *ulimit* blocks in size. (For a description of *ulimit*, refer to your UNIX system documentation.)

The format of the **syncsplit** command is:

syncsplit *name*

where *name* is the base name of output files that will be generated by **syncsplit** (for example, /user/tmp/sorted_part). The **syncsplit** command will sequentially add a numeric suffix to the base name to distinguish up to 1,000 output files. The suffix is a period followed by three digits (000 through 999). Thus, the command **syncsplit sorted_part** would create the output files sorted_part.000, sorted_part.001, etc.

For an example /OUTFILE option with **syncsplit**, see “Example syncsplit Command” on page 3.123.

If **syncsplit** attempts to create an output file that already exists (for example, `sorted_part.001`), **syncsplit** terminates.

Record Format

An output file does not need to have the same record format as the input file(s).

Existence of Output File

Sequential.

If the sequential file you specify to hold the output from SyncSort does not exist, a new file is created. If the sequential file exists, you must specify either **OVERWRITE** or **APPEND**. If **OVERWRITE** is specified, the output records will overwrite whatever data may exist in the file. If **APPEND** is specified, the output records will be added to the end of the file.

Indexed or VSAM.

For indexed or VSAM output files, the output file must exist. SyncSort does not support overwriting records that are already in the file. You cannot specify **OVERWRITE**. SyncSort accesses VSAM files through the SUN™ Mainframe Batch Manager. See SUN™ Mainframe Transaction Processing (MTP) and Mainframe Batch Manager documentation for more information on the VSAM file organization.

Record Length

If one of the record lengths is provided, SyncSort assumes that it is the maximum record length. Therefore, if you wish to specify the minimum record length, you should specify also the maximum record length (the order is unimportant). Records longer than the maximum record length are truncated. Records shorter than the minimum record length are padded.

See “Appendix D. Record Formats” on page D.1 for the maximum record lengths associated with various record formats.

Field Separator Character

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When you want the field delimiter to be different to the one used in the input file, specify the output delimiter through the `field_separator` argument.

You cannot specify a field delimiter for a report.

Addrout Files

Ordinarily, the records in the output file(s) contain only data from the input file(s) and any strings added by record reformatting. The `ADDRROUT` qualifier, however, allows you to request a different type of output file, known as an "addrout" file. This is a sequentially organized file that can be created only from a sort. The input files to the sort must be on disk. The sorted output must go to a single addrout file.

There are two addrout file formats, the address format (obtained by specifying `RFA` as the value of the `ADDRROUT` argument) and the address + key format (`RIDX`). An address file is a list of binary record file addresses, ordered by the specified sort key. The `RFA` is 6 bytes, the first 4 of which contain a binary integer that identifies the block number of a 512-byte block in which a record is contained. The last 2 bytes of the `RFA` contain a binary integer that specifies the offset of the record within the block. When multiple input files are sorted, the output record includes also a binary file number.

A record in an address + key addrout file contains an `RFA`, the key field values (in descending order of priority) and a file number (if multiple input files are sorted). The key fields can include both byte fields, occupying an integral number of full bytes, and bit fields. When a byte key field follows a bit key field, SyncSort automatically aligns the byte field value on the next available byte boundary in the key portion of the addrout record. When the prior bit field does not end on a byte boundary, SyncSort fills the unused bits up to the byte boundary with '0' bits. When a bit field follows another bit field, SyncSort places the second bit field immediately after the first bit field in the addrout record.

For a single input file, the format of an address file record is:

	Start Position	Length (bytes)
RFA	1	6

For multiple input files, the format of an address file is:

	Start Position	Length (bytes)
RFA	1	6
file number	7	1

For a single input file, the format of an address + key record is:

	Start Position	Length (bytes)
RFA	1	6
key	7	varies

For multiple input files, the format of an address + key record is:

	Start Position	Length (bytes)
RFA	1	6
file number	7	1
key	8	varies

Record Alignment

If you want all records in an output file to be aligned on 2-byte boundaries, supply the **ALIGNED2** keyword except for fixed format records of even length. If you want records to be aligned on 4-byte boundaries, supply the **ALIGNED4** keyword except for fixed format records whose length is a multiple of 4 bytes.

/OUTFILE

When you want to add a record number to the records in the file, specify the **RECORDNUMBER** argument. To start the numbering at other than 1, provide **START** *recnum_start*.

If you do not specify a **/REFORMAT** option for the file SyncSort will automatically add a record number to the beginning of each record. The number appears as a 10-digit, unsigned decimal number. When the records in the file are field-delimited text records, leading zeroes in the number do not appear and a delimiter character follows the number.

If you do provide a **/REFORMAT**, SyncSort does not automatically add a record number. See **/REFORMAT** for details on how to add the record number to a reformatting.

Record numbering is not available for a report.

Compression

The compression file format that is output by SyncSort is compatible with gzip, standardized in RFC 1952.

Compatibility with Other SyncSort Options

The **compress** attribute cannot be specified with the **INDEXED**, **VSAM** file organization, or **MFLINESEQUENTIAL** record format.

Examples

```
/outfile sort1.out
```

The option specifies that the output from the application is to be written to the file **sort1.out**.

```
/outfile 1 open fixed 80
```

The option specifies that the output from the application is to be placed in standard output. The format of the records is fixed. The length of each record is 80 bytes.

```
/outfile year95/payables/current/trans_sorted.dat  
variable 240 120
```

The option requests that the output file **trans_sorted.dat** be created in the directory **year95/payables/current**. The format of the records in the file is to be variable. The length of the longest record is 240 bytes, that of the shortest 120 bytes.

```
/outfile tape1.out 1024 fixed
```

The name of the output file is **tape1.out**. It is to contain fixed format records of length 1024 bytes.

```
/outfile customers.out append
```

This is a request for SyncSort to append output records to the file **customers.out** without overwriting the current contents of the file.

```
/outfile prod.in addrout ridx
```

The output file **prod.in** is to be an addrout file. It will contain sorted record file addresses and key fields.

```
/outfile out.dat variable 201 aligned2
```

The output file **out.dat** is to contain variable length records aligned on a 2 byte boundary.

```
/outfile corporate_db indexed
```

The file **corporate_db** is an existing indexed file to be used to hold the output records. It exists and is empty.

```
/outfile numbered.dat recordnumber
```

The output file **numbered.dat** is to contain numbered records. When the records in the file are not reformatted, the record number field appears at the start of the record. By default, the number is an unsigned decimal field of 10 digits.

Example syncsplit Command

The following shell script illustrates **syncsplit**:

/OUTFILE

```
rm sorted_part.* rm /usr/tmp/sorted_part.*
syncsort /infile 0 open /keys 1 char 5 \
/outfile 1 open /end | syncsplit /usr/tmp/sorted_part
```

The output from the SyncSort application is to be written in the directory /usr/tmp to the file(s) sorted_part.000, sorted_part.001, etc. All the sorted_part files (sorted_part.*) are removed before **syncsplit** runs to ensure that it completes successfully.

/PADBYTE

Purpose

To specify the value of the character for padding short records and keys.

Format

`/PADBYTE pad_char`

where

$$\text{pad_char} = \begin{bmatrix} \text{"a"} \\ \text{X"xx"} \\ \text{O"ooo"} \end{bmatrix}$$

Arguments

<i>a</i>	the character value of the pad byte.
<i>xx</i>	the hexadecimal value of the pad byte.
<i>ooo</i>	the octal value of the pad byte.

Location

The option may appear anywhere in the application definition.

Notes

When you specify a pad byte through the /PADBYTE option, SyncSort uses that character to pad both short keys and short records which are encountered during the application. Key fields are padded only for the duration of the application but record padding survives in the records output from SyncSort.

If short keys and short records need to be padded with different characters, use the /PADKEY and /PADRECORD options, instead of /PADBYTE.

/PADBYTE

You must specify a single character as a general pad byte. A double character cannot be used, even if it is defined to collate as a single character in an alternative collating sequence.

If you do not specify a pad byte, SyncSort uses the null character X"00" to pad both short keys and short records.

Examples

```
/padbyte "a"
```

The lowercase a will be used to pad short keys and records.

```
/padbyte x"20"
```

The character whose hexadecimal value is 20 will be used to pad short keys and records.

/PADKEY

Purpose

To specify the value of the character for padding short keys.

Format

`/PADKEY pad_char`

where

<code>pad_char</code>	=	<code>["a"]</code>
		<code>[X "xx"]</code>
		<code>[O "ooo"]</code>

Arguments

<code>a</code>	the character value of the pad byte.
<code>xx</code>	the hexadecimal value of the pad byte.
<code>ooo</code>	the octal value of the pad byte.

Location

The option may appear anywhere in the application definition.

Notes

When you specify a pad byte through the `/PADKEY` option, SyncSort uses that character to pad short keys which are encountered during the application. Key fields are padded only for the duration of the application.

You must specify a single character as a key pad byte. A double character cannot be used, even if it is defined to collate as a single character in an alternative collating sequence.

/PADKEY

If you do not specify a pad byte, SyncSort uses the null character X"00" to pad short keys.

Example

```
/padkey o"040"
```

The character whose octal value is 040 will be used to pad short keys.

/PADRECORD

Purpose

To specify the value of the character for padding short records.

Format

```
/PADRECORD pad_char
```

where

<i>pad_char</i>	=	["a"]
		[X "xx"]
		[O "ooo"]

Arguments

<i>a</i>	the character value of the pad byte.
<i>xx</i>	the hexadecimal value of the pad byte.
<i>ooo</i>	the octal value of the pad byte.

Location

The option may appear anywhere in the application definition.

Notes

When you specify a pad byte through the /PADRECORD option, SyncSort uses that character to pad short records which are encountered during the application. Record padding survives in the records output from SyncSort.

You must specify a single character as a record pad byte. A double character cannot be used, even if it is defined to collate as a single character in an alternative collating sequence.

/PADRECORD

If you do not specify a pad byte, SyncSort uses the null character X"00" to pad short records.

Example

```
/padrecord o"040"
```

The character whose octal value is 040 will be used to pad short records.

/PAGEFOOTER

Purpose

To describe the contents of the page footer for a report.

Format

/PAGEFOOTER *field* [, *field* ...] [NOSKIP]

where

<i>field</i>	=	[<i>fieldname</i> [TRUNCATE]] [[<i>repeat</i>] <i>string</i>] [[<i>nl_number</i>] NEWLINE] [PAGENUMBER] [TODAY]
--------------	---	---

Arguments

- | | |
|------------------|---|
| <i>fieldname</i> | the name of a field defined through /AGGREGATE, /DATADictionary, /DERIVEDFIELD, /DELIMITEDRECORDLAYOUT, /RECORDLAYOUT, or /FIELDS. TRUNCATE can be used with a character field only. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names. |
| <i>string</i> | a character string. The string can be specified in any of the formats described in “Specifying Character String Constants” on page B.1. |
| <i>repeat</i> | the number of times a character string is to be repeated. |
| <i>nl_number</i> | the number of new lines to be inserted into the footer. The minimum value is 1. The default is 1. |

/PAGEFOOTER

Location

The `/PAGEFOOTER` option must appear after the `/OUTFILE` option which defines the report file containing the page footer. When there are multiple output files, any `/PAGEFOOTER` option for a particular report must appear between the `/OUTFILE` option for that report and the next `/OUTFILE` option, if any.

Notes

There can be only one `/PAGEFOOTER` option for a report.

The page footer is placed at the bottom of each page unless you specify the `NOSKIP` argument, when the footer is placed immediately after the last detail line or last section footer on the page.

Using delimited fields in a report

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When you use a field from such a record in a page footer, the field delimiter does not appear in the footer.

Enclosed delimited fields

Each field in a field-delimited text record may be enclosed by two enclosing characters. When you use a single or partial delimited field in a page footer, any enclosing characters do not appear in the footer. When you use a multiple delimited field in a page footer, the field appears in the footer with all enclosing characters.

Footer Contents

All trailing blanks (spaces and tabs) are removed from a character footer field which you specify with the `TRUNCATE` argument. This allows you to embed values in sentences without having large gaps. Remember, however, that all spaces are truncated; you need to ensure that the following field begins with at least one space if you do not want the fields to run together.

NEWLINE creates a line break. Each leading NEWLINE in a page footer option produces a blank line at the start of the footer. For example, **3 NEWLINE** at the start of a footer option results in 3 leading blank lines. Each non-leading NEWLINE, i.e. those which appear after other types of arguments such as a field name, starts a new line. Thus, if you want blank lines in the middle or end of a footer, specify one NEWLINE to start a new line and one NEWLINE for each blank line you want. For example, **3 NEWLINE** anywhere other than at the start of a footer option results in 2 blank lines.

A PAGENUMBER field contains the current page number edited using the default page number mask (ZZZ9). A TODAY field contains the current system date edited with the default date mask (MM0/DD0/YEAR). For details on changing edit masks, see “Appendix E. Masks” on page E.1 and “/DERIVEDFIELD ” on page 3.48.

A Footer Field

When a page footer field is a field in the input record, the value assigned to the footer field is the value of the field in the **last** detail line on the page.

The field which you specify in a footer must be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on the footer field. However, when you specify input reformatting, you must not discard a field which you want to appear in a footer. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want the **fielda** value to appear in a footer.

Examples

```
/pagefooter newline, "p ", pagenumber, 5" ", today
```

This /PAGEFOOTER option creates the following two line footer:

p 32 05/10/1996

- The first line is blank (caused by the leading NEWLINE).

/PAGEFOOTER

- The second line starts with the string "p " which is followed by the number of the current page edited according to the default mask ZZZ9. The page number is separated from the current system date by 5 spaces. The date is edited according to the default date mask (MM0/DD0/YEAR). See “Edit Specifications for Date Fields ” on page E.11. The second line of the footer appears on the last line of the page.

```
/pagefooter "Summary for ", sales_rep truncate, " of the ",  
local_office truncate, " division for this year.",  
newline, "Total sales: ", dollar_amount,  
" Number of sales: ", sales_count, 8" " , "--Page ",  
pagenumber, "--", newline noskip
```

This /PAGEFOOTER option creates the following sample two line footer:

```
Summary for Brian Jones of the El Paso division for this year.  
Total sales: 1435892 Number of sales: 855 --Page 10--
```

- The first line begins with the text "Summary for " followed by the contents of the **sales_rep** field in the last detail appearing in the current page. Any spaces and tabs at the end of the **sales_rep** value are removed.

The first line continues with the text " of the ", the truncated contents of **local_office** from the last detail and the text " division for this year."

- The second line (caused by the first NEWLINE) starts with the text "Total sales: ". This is followed by the contents of the **dollar_amount** field in the last detail on the page.

Since no edit mask is provided with **dollar_amount**, the appearance of the field in the footer depends on the data type of the originating field. For example, when **dollar_amount** is a field with unsigned integer format (UNINTEGER) and is 4 bytes long, the footer contains the value in **dollar_amount** converted to edited numeric type containing 11 characters.

The second line continues with the text " Number of sales: ", the contents of the **sales_count** field from the last detail record converted to the default printable format, 8 spaces, the text "--Page " and the current report page number edited according to the default mask ZZZ9.

The second line ends with two hyphens. The final NEWLINE terminates the second line.

The footer appears immediately after the last detail line on the page since the NOSKIP argument is used.

/PAGEHEADER

/PAGEHEADER

Purpose

To describe the contents of the page header for a report.

Format

```
/PAGEHEADER field [, field ... ]
```

where

<i>field</i>	=	<i>[fieldname [TRUNCATE]]</i> <i>[[repeat] string]</i> <i>[[nl_number] NEWLINE]</i> <i>[PAGENUMBER]</i> <i>[TODAY]</i>
--------------	---	---

Arguments

<i>fieldname</i>	the name of a field defined through <i>/DATADictionary</i> , <i>/DERIVEDFIELD</i> , <i>/FIELDS</i> or <i>/DELIMITEDRECORDLAYOUT</i> , <i>/RECORDLAYOUT</i> . <i>TRUNCATE</i> can be used for character fields only. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.
------------------	--

<i>repeat</i>	the number of times a character string is to be repeated.
---------------	---

<i>nl_number</i>	the number of new lines to be inserted into the header. The minimum value is 1. The default is 1.
------------------	---

Location

The */PAGEHEADER* option must appear after the */OUTFILE* option which defines the report file containing the page header. When there are multiple output

files, any /PAGEHEADER option for a particular report must appear between the /OUTFILE option for that report and the next /OUTFILE option, if any.

Notes

There can be only one /PAGEHEADER option for a report.

Generally, the header is positioned at the top of each page. The only exception is when you provide the report with a title (through the /TITLE option) and the title does not end with a NEWPAGE. In this case, the header on the page containing the last portion of the title immediately follows the title.

Using delimited fields in a report

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When you use a field from such a record in a page header, the field delimiter does not appear in the header.

Enclosed delimited fields

Each field in a field-delimited text record may be enclosed by two enclosing characters. When you use a single or partial delimited field in a page header, any enclosing characters do not appear in the header. When you use a multiple delimited field in a page header, the field appears in the header with all enclosing characters.

Header Contents

All trailing blanks (spaces and tabs) are removed from a header field when you specify the TRUNCATE argument. This allows you to embed values in sentences without having large gaps. Remember, however, that all spaces are truncated; you need to ensure that the following field begins with at least one space if you do not want the fields to run together.

NEWLINE creates a line break. Each leading NEWLINE in a page header option produces a blank line at the start of the header. For example, **3 NEWLINE** at the start of a header option results in 3 leading blank lines. Each non-leading NEWLINE, i.e. those which appear after other types of arguments such as a field name, starts a new line. Thus, if you want blank lines in the middle or end of a

/PAGEHEADER

header, specify one NEWLINE to start a new line and one NEWLINE for each blank line you want. For example, **3 NEWLINE** anywhere other than at the start of a header option results in 2 blank lines.

A PAGENUMBER field contains the current page number edited using the default page number mask (ZZZ9). A TODAY field contains the current system date edited with the default date mask (MM0/DD0/YEAR). For details on changing edit masks, see “Appendix E. Masks” on page E.1 and “/DERIVEDFIELD ” on page 3.48.

A Header Field

When a page header field is specified as a field in the input record, the value of the header field is the value of the field in the first detail on the page.

Every field in a header must be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on a header field. However, when you specify input reformatting, you must not discard a field which you want to appear in a header. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want the **fielda** value to appear in a header.

Examples

```
/pageheader 5" ", " Production report for ", today, " ", 5 " "
```

The following header appears as the first line on each page.

```
***** Production report for 05/10/1996 *****
```

The page header described by this /PAGEHEADER option consists of a single line. The line starts with a string of five asterisks followed by the text " Production report for " and the current system date. The date is edited according to the default date mask (MM0/DD0/YEAR). The line is completed by a space and five asterisks.

```
/pageheader "Sam's Auto Repair", 30" ", "Page ",  
pagenumber, 2 newline, "Schedule for ", employee_name  
truncate, " for ", todays_date
```

This /PAGEHEADER option creates the following sample three line header:

Sam's Auto Repair	Page 21
Schedule for Kevin Costello for January 6, 1996	

- The first line begins with the text "Sam's Auto Repair" followed by 30 spaces, the text "Page " and the current report page number edited according to the default mask ZZZ9.
- The second line is a blank line. The first line is terminated by one of the 2 NEWLINES.
- The third line begins with the text "Schedule for " followed by the contents of the **employee_name** field in the first detail appearing in the current page. Any spaces and tabs at the end of the **employee_name** value are removed.

The third line continues with the text " for " and ends with the contents of **todays_date** (possibly a derived field containing today's date with an edit mask).

/QUIT

/QUIT

Purpose

To terminate an interactive session.

Format

`/QUIT`

Location

The option may appear anywhere in the application definition.

Notes

`/QUIT` can be used only when SyncSort is invoked through the **syncsort** command in a line-by-line interactive session.

You can use the `/QUIT` option to terminate a line-by-line SyncSort session at a terminal during the creation of the application definition. You are immediately returned to the operating system and nothing will be saved unless you issued a `/SAVE` option previously.

Since an application definition is terminated by the first `/END` option, `/QUIT` is not recognized if it appears in the definition after `/END`.

Example

```
/quit
```

This option causes SyncSort to terminate the session underway and return you to the operating system immediately.

/RECORDLAYOUT

Purpose

To describe the layout of the input record in terms of single fields.

Format

```
/RECORDLAYOUT field [, field . . . ]
```

where

```
field = [ composite_field ]  
       [ elementary_field ]
```

```
composite_field = name { field [, field ... ] } [REPEAT number]
```

Note that the braces ({ }) in *composite_field* have to appear as shown. They do not indicate a choice of argument in this case.

```
elementary_field = [ name [ format_length ] [ precision/sequence ] [REPEAT number] ]  
                   [ FILLER length [BITS] ]
```

```
format_length = [ format [ length ] ]  
                [ length format ]  
                [ CHARACTER ]
```

```
format = [ numeric_format ]  
          [ DATETIME ( datetime_pattern ) ]  
          [ BIT ]
```

```
precision = integral_digits / fractional_digits
```

Arguments

<i>name</i>	the name of a field in the input record layout, which you will use to reference the field in other SyncSort options. The name assigned to a field must adhere to the rules described for an identifier in “Identifiers ” on page B.7.
<i>number</i>	the number of times a field is repeated consecutively in the input record.
<i>length</i>	the length of a field. The length is in bytes for all datatypes except BIT, in which case the length is in bits. The acceptable values for the length of a field with one of the recognized data types can be found in “Table 1. `Data Types Supported by SyncSort” on page A.1. The length may be omitted only for a variable length character field at the end of the input record, when the data type is DFLOAT, FLOAT, BIT, or DATETIME. The default date/time field length specified by the date/time pattern and the default bit field length is 1 bit.
<i>sequence</i>	the sequence in which values must be ordered when the field is involved in comparisons. The sequence is used when the field is defined as a sort key, when the field appears in a condition, and when the field is used as the control for a section of a report. An acceptable value for the sequence argument is one of the following: • A standard collating sequence. See Table 6. on page 3.70. • The name of an alternative collating sequence defined through /COLLATINGSEQUENCE. Only a character field can have an associated sequence. The default sequence is the collating sequence of the current locale unless the default is defined through /COLLATINGSEQUENCE DEFAULT. See “Appendix C. Collating Sequences”

on page C.1 for a description of the standard collating sequences.

integral_digits the maximum number of significant digits to appear before any decimal point in an edited numeric value. The minimum number is 0. The maximum number is the maximum length supported for edited numeric fields. See “Table 1. `Data Types Supported by SyncSort” on page A.1. The default is 15 digits for a delimited field. The default for a positional field is dependent on the field length.

fractional_digits the maximum number of digits to appear after any decimal point in an edited numeric value. The minimum number is 0. The maximum number is one less than the maximum length supported for edited numeric fields. See “Table 1. `Data Types Supported by SyncSort” on page A.1.

numeric_format the type of numeric data held in the field. The default data type is character unless the position is a bit position, in which case the default data type is bit.

A list of acceptable values for the data type argument can be found in “Table 1. `Data Types Supported by SyncSort” on page A.1.

Location

The option may appear anywhere in the application definition.

Notes

Describing the Record Layout

With the /RECORDLAYOUT option you can describe your input fields without explicitly specifying their positions within the record. SyncSort calculates the starting position of each field from the size of the fields that precede it. You can then refer to fields defined through /RECORDLAYOUT in the same way as fields

/RECORDLAYOUT

defined through /FIELDS, /DATADictionary, DELIMITEDRECORDLAYOUT, etc.

The record layout consists of elementary fields and composite fields. For example, if your records contain dates in the format yyyy/mm/dd, you could describe this data via elementary fields **year**, **month**, **day** and composite field **date**, as follows:

```
/recordlayout date {year      char 4,  
                    filler    char 1,  
                    month     char 2,  
                    filler    char 1,  
                    day       char 2}  
                    date_sent char 6
```

If all the elementary fields of a composite field have data type bit, then the composite field has data type bit, otherwise the composite field has data type character. In the example above therefore, the field **date** has data type character and length 10. Note that the fields **date**, **year**, **month** and **day** can all be referenced similarly in the application. The filler fields cannot be referenced in the application. A composite field can contain other composite fields.

You can create a second type of composite field by using the REPEAT number clause to define an array. You can then refer to a single member of the array by following the field name by the appropriate subscript, in square brackets. A reference to the field name without any subscript in square brackets references the entire array as a composite field. The data type of an array is determined in the same way as other composite fields, i.e., if all the fields of the array have data type bit, then the array has data type bit, otherwise the array has data type character. References to arrays and array members are described more fully in “Specifying Field Names ” on page B.9.

Bit Fields

When you need to have access to individual bits, or strings of consecutive bits, in the input record you can specify fields with a data type of BIT. The length you specify is the number of bits in the field. When two elementary bit fields follow one another in /RECORDLAYOUT without an intervening FILLER, SyncSort assumes that the fields are contiguous in the record. When an elementary non-bit field follows a bit field which does not end on a byte boundary SyncSort assumes that the non-bit field starts on the next byte boundary.

Date/Time Fields

A date/time elementary field consists of a calendar date component and/or a time component. The contents are in printable form. For example, the date component may appear as **month/day/year** with the month as two digits, the day as 2 digits and the year as 4 digits. The time component may appear as **hour:minute:second** with the hour as two digits using the 24-hour clock, the minutes as 2 digits and the seconds as 4 digits. 5 minutes and 9 seconds after 8 o'clock in the morning of August 4, 2000 appears as 08/04/2000 08:05:09 for this format. You would represent this date/time date_pattern as **(MM0/DD0/YEAR HH0:MI0:SE0)**. See “Source Date Patterns” on page E.27 for a description of how to describe a date in a source field.

Examples

```
/recordlayout filler 2, emp_id integer 4, filler 23,  
emp_name character 40
```

This option describes an input record layout. **filler 2** and **filler 23** cover the input record fields that will not be referenced in other options. The option identifies by name only those fields that will be needed elsewhere in the application. The first of these, **emp_id**, starts at position 3 and is a 4-byte binary integer. The second field, **emp_name**, is a 40-byte character field starting at position 30.

```
/recordlayout location char 14 fascii, filler 8, store_code char  
4, manager_code char 4, emp_count int 2, filler 34, p_ind char 1
```

The option describes an input record layout with five named fields. The first record field, **location**, contains character data and is 14 bytes long. The values in **location** are collated according to the folded ASCII sequence. **filler 8** specifies an 8-byte long unnamed portion of the record starting at position 15. Next, starting at position 23, is a 4-byte long character field **store_code**. Another character field, **manager_code**, which is also 4 bytes long, starts at position 27. It is followed at position 31 by a 2-byte integer field, **emp_count**. **filler 34** identifies a 34-byte long unnamed stretch starting at position 33. The last field in the record, **p_ind**, starts at position 67 and is one byte long.

```
/recordlayout cust_code int 2, regular_payment uint 2,  
month_amount uint 2 repeat 12, month_paid bit repeat 12,
```

/RECORDLAYOUT

```
filler 4 bits, date_received char 6 repeat 12,  
reminder_sent bit repeat 12
```

The record layout described by this option consists of seven fields, six named fields and one filler. The first field, **cust_code**, starts at the first byte of the record and is a 2-byte long signed binary integer. This is followed at byte 3 by the second field, **regular_payment**, which is a 2-byte unsigned integer. The remaining named fields define arrays in the record layout. The **month_amount** array defines an array containing twelve items, as indicated by repeat 12. Each item in the array is a 2-byte unsigned binary integer. The first item in the array immediately follows **regular_payment** at byte 5. The complete array occupies bytes 5 through 28. This is followed by the **month_paid** array, which consists of twelve items each of which is 1 bit long and occupies from the first bit of the 29th byte through the 4th bit of the 30th byte. The remaining 4 bits of the 30th byte are described by **filler 4 bits**. Bytes 31 through 102 of the record are occupied by the **date_received** array, consisting of twelve 6-byte character items. The record layout is completed by the **reminder_sent** array, which contains 12 items, each of which is 1 bit. The **reminder_sent** field occupies from the first bit of the 103rd byte through the 4th bit of the 104th byte. The remaining 4 bits of the 104th byte are a filler.

```
/recordlayout book_id an 6, author {first_name 20,  
middle_initial 1, last_name 20 }, title 50, publisher 40,  
date_published datetime (mm/dd/year) 10
```

This /RECORDLAYOUT option describes an input record containing information on one book in a library. The record starts with a unique identifier for the book (**book_id**, a 6-byte unsigned decimal number). Starting in byte 7 of the record is the author information. This consists of the 20-character **first_name** field, the 1-character **middle_initial** field and the 20-character **last_name** field. The author information can also be referenced as a 41-character field, **author**, which is a composite field consisting of the first name, middle initial and last name. The 50-character **title** field, the 40-character **publisher** field, and the 10-byte **date_published** field complete the record layout. **date_published** contains a date consisting of the month with any leading zero removed, the day with suppressed leading zero and the 4-digit year. The three components are separated by slashes.


```
/recordlayout cust_code int 2, regular_payment uint 2,
    monthly_account
    {month_amount uint 2,
      date_received datetime (mm0/dd0/year),
      month_paid bit,
      reminder {reminder_sent bit,
                filler 7 bits,
                date_sent datetime(mm0/dd0/year) 10
              }repeat 2,
    }repeat 12
```

The **/RECORDLAYOUT** option in this example describes a record containing one year's payment records for a customer. The first field in the record is a customer's identifier (**cust_code**), held as a 2-byte binary integer. The amount of the customer's regular payment amount (**regular_payment**) is held in bytes 3 and 4 as an unsigned binary integer.

Starting in byte 5 is the 12-entry array **monthly_account**. Each **monthly_account** entry contains the payment record for a single month. Each month's entry is 35 bytes long and consists of the amount received for the monthly payment (**month_amount**, a 2-byte unsigned binary integer), the date payment was received (**date_received**, a 10-byte date field), a 1-bit flag indicating if the monthly payment is completely paid (**month_paid**), and a 2-entry **reminder** array. Each of the **reminder** array entries is 11 bytes long and consists of a 1-bit **reminder_sent** flag, a 7-bit filler and a 10-byte **date_sent** field. The length of the record represented by this record layout is 424 bytes.

/REFORMAT

/REFORMAT

Purpose

To specify alterations in the layout of records.

Format

```
/REFORMAT [side:] field [, field...] [, side: field [, field...]...]
```

where

<i>side</i>	=	[LEFTSIDE] [RIGHTSIDE]
<i>field</i>	=	[<i>fieldname</i>] [RECORDNUMBER]

Arguments

<i>fieldname</i>	the name of a field defined through /DATADictionary, /DERIVEDFIELD, /FIELDS, /DELIMITEDRECORDLAYOUT, or /RECORDLAYOUT. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.
------------------	---

Location

/REFORMAT may appear anywhere in the application definition. However, its position both in relation to the first destination option (/OUTFILE) and any /INCLUDE options is important.

If you want to reformat input records before releasing them to the sort, merge or copy stage of the application, the /REFORMAT option must appear before the first output destination option. If you want to reformat records before they are sent to a

destination, the /REFORMAT option must appear after the destination option. When there are multiple output files, any /REFORMAT option applicable to a particular file must appear between the /OUTFILE option specifying that file and the next /OUTFILE option, if any.

If all groups of input records are to be reformatted in the same manner, the /REFORMAT option must appear before the first /INCLUDE or /OMIT option. Similarly, if all groups of records for a destination are to be reformatted in the same manner, the /REFORMAT option must appear before the first /INCLUDE or /OMIT option that groups records for that destination.

If different groups of input or output records must be reformatted differently, the /REFORMAT option for a particular group of records must appear between that /INCLUDE option which defines the group and the next /INCLUDE or /OMIT option, if any. If several but not all groups of records are to be reformatted in the same way, the /REFORMAT options for those groups can be omitted and replaced by a single /REFORMAT option before the first /INCLUDE or /OMIT option.

Notes

Side argument

The side argument is applicable only to a join. It is ignored for applications other than a join.

All fields for a reformatting specification must be specified in the same /REFORMAT option.

When you have specified an input file that contains records with delimited fields optionally enclosed by two enclosing characters, you cannot use a partial field when reformatting records for all outputs.

Input Record Reformatting

For input reformatting, the same field cannot be specified more than once in the same /REFORMAT option. If you are reformatting each input record group differently, any field which appears in more than one /REFORMAT option must end up at the same location in the reformatted record.

You can specify a field of indeterminate length only as the last field in an input /REFORMAT option.

Output Record Reformatting

Fields which you specify in an output /REFORMAT option must be available in some form in the records leaving the sort, merge, or copy stage of the application. When you do not specify any input record reformatting, there is no limitation on the output record /REFORMAT option fields. However, when you specify input reformatting, you must not discard any of the data which you need for the output records. For example, if a derived field **fielda** is obtained by converting an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want **fielda** in the output record.

Reformatting Delimited Fields

When you want to reformat text input records that contain fields delimited by a field separator character, you must first use /FIELDS to define the portions of the record as delimited fields before you refer to them with /REFORMAT. Each portion of a record defined with /FIELDS has a start and end. At a minimum, you must specify the start of a portion. Both the start and the end are defined by the delimited field number and character position within the field. At a minimum, you must specify the delimited field number.

The types of delimited fields which you can use in reformatting are as follows:

- one or more complete input fields, such as 3: - 3:, 1: - 5:
- a partial input field such as 3:3 - 3:, 2: - 2:5, 1:6 - 1:8, 4:2 noblank - 4:7
- the portion of the input record from the start of an input field to the end of the record, such as 7:.

When specifying an input /REFORMAT option, you must specify any portions of the input record that have the same length in all records before those portions that vary in length between records. Field portions have the same length when:

- the start and end field numbers are identical,

- neither the start nor the end are specified with NOBLANK or both the start and the end are specified with NOBLANK,
- and the character location of the end is specified.

Examples of fixed length portions are 5:3 - 5:7, 4: - 4:2, 6:1 - 6:1. Examples of variable length portions are 4: - 4:, 5:1 - 5:, 3: - 4:, 3:2 noblank - 3:7, 2:2 - 2:9 noblank, 7:.

After SyncSort has reformatted a record according to a /REFORMAT option, all portions of the record with varying length except the last are followed by a field separator character. Portions of fixed length are not followed by a separator character.

Bit Fields in Reformatting

You can include both byte fields occupying an integral number of full bytes and bit fields in a single /REFORMAT option. When a byte field follows a bit field, SyncSort automatically aligns the byte field on the next available byte boundary in the reformatted record. When the prior bit field does not end on a byte boundary, SyncSort fills the unused bits up to the byte boundary with '0' bits.

When a bit field follows another bit field, SyncSort places the second bit field immediately after the first bit field in the reformatted record.

Record Numbering

When you want to add a record number to the records in an output destination, you can specify the RECORDNUMBER argument to the /OUTFILE option. This results in SyncSort adding a record number to the beginning of each record in the output destination. The number appears as a 10-digit, unsigned decimal number. If you want to have the record number in a location other than the beginning of the record, create a reformatting for the output destination and use the RECORDNUMBER form for one of the reformat fields.

If you need to change the appearance of the record number from the default, specify a /DERIVEDFIELD to describe the form of the record number you want. Use the derived field in the /REFORMAT.

/REFORMAT

The record number field is treated in the same way as all other fields in a reformatting. Since it is a fixed length field, it is not automatically delimited during reformatting of field-delimited text records.

Manual numbering occurs when at least one reformatting for the output destination explicitly includes a record number field. If there are multiple reformats for an output destination, there is no restriction on location of record numbers, and it is not mandatory for all reformats to include a record number. Record numbering may or may not be requested on the `/OUTFILE` option. If the starting number is other than 1, you need to specify this through the `START` option of `/OUTFILE`.

`RECORDNUMBER` is valid only for output reformatting.

Reformatting Joined Records

When a record from one side of a join is joined with a record from the other side, you can choose what information from the two records is placed in the joined record through the `/REFORMAT` option. Prefix each field name in the reformat option by either `LEFTSIDE:` or `RIGHTSIDE:` to indicate which of the two records is the source of the field. When several consecutive fields in the reformatting come from the same record, you can omit the side argument for the second and subsequent fields. For example, `leftside: f1, leftside: f2, leftside: f3, rightside: g1` is equivalent to `leftside: f1, f2, f3, rightside: g1`. If you omit a side argument prior to the first field, SyncSort assumes that all fields up to the first field with a side prefix are from the record from the left side.

Only fields defined through `/DATADictionary`, `/FIELDS`, `/DELIMITEDRECORDLAYOUT`, or `/RECORDLAYOUT` can appear in a reformat for a join.

A join application must not have more than one `/REFORMAT` option.

Defaults

If you do not provide a `/REFORMAT` option for a join, SyncSort joins the records so that the join keys appears first, followed by all remaining fields from the left side record, followed by the remaining fields from the right side record.

Examples

```
/reformat cust_id, cust_name, address, city, state, zip
```

This reformatting option represents a case in which the size of records is reduced to six fields and/or the left-to-right order of the fields is rearranged.

```
/reformat vol_id, bl4, author_name, bl4, book_title,  
publisher, bl4, pub_date, bl4, price
```

This output reformatting option can be interpreted in a manner similar to the previous example with the exception that the fields are to be separated from each other by the field **bl4**.

```
/reformat recordnumber, trans_date, trans_time, trans_amount
```

This reformatting option takes three fields from the source record for the target layout (trans_date, trans_time, trans_amount) and precedes them with a record number. The first field in a record contains the number of that record in the target. It is an unsigned decimal field of 10 digits.

```
/reformat leftside:trans_customer_number,  
rightside:na_full_name, na_phone, leftside:trans_amount
```

This reformatting option is part of a join application. The first and fourth fields in the output record are created from the trans_customer_number and trans_amount fields in the first input file (left side). The second and third fields in the output record are created from the na_full_name and na_phone fields in the second input file (right side).

/REPORT

/REPORT

Purpose

To describe general controls and defaults for a report.

Format

```
/REPORT report_qual [report_qual ... ]
```

where

```
report_qual      = [In_number LINES          ]  
                  [PAGESTART pg_number      ]  
                  [NOCOLUMNHEADINGS         ]  
                  [COLUMNSPACING col_spacing]  
                  [NODETAIL                  ]  
  
col_spacing      = [before_space [,after_space]]  
                  [,after_space              ]
```

Arguments

<i>ln_number</i>	the number of lines to make up a page of the report. The default is 60.
<i>pg_number</i>	the starting page number for the report. The minimum value is 0. The default is 1.
<i>before_space</i>	the number of spaces to be inserted before each column of the report, unless explicitly overridden by a <i>/COLUMNS</i> argument.
<i>after_space</i>	the number of spaces to be inserted after each column of the report, unless explicitly overridden by a <i>/COLUMNS</i> argument.

Location

/REPORT is optional but if you use it, it must appear after the /OUTFILE option which defines the report file. When there are multiple output files, the /REPORT option for a particular report must appear between the /OUTFILE option for that report and the next /OUTFILE option, if any.

Notes**Page Numbering**

You can choose to start page numbering for a report at some point other than 1. This is useful, for example, if you wish to combine reports from several SyncSort executions into a single report. You can also use this facility if your report has a separate title page, but you wish to number the detail pages from 1. In this case, you would specify PAGESSTART 0, so that the title page is numbered zero.

Column Headings

Column headings are produced on all detail pages, except if you specify NOCOLUMNHEADINGS. In this case, the headings appear on the first detail page only.

Column Spacing

When you do not explicitly specify the column spacing, either through the COLUMNSPACING argument or the /COLUMNS option, SyncSort spreads the columns as evenly as possible within the record length for the file containing the report, if provided. When possible, equal numbers of spaces (a maximum of 10, a minimum of 1) are inserted between columns, and half as many at the beginning and end of a detail line. When you do not specify a maximum record length for the report file, one space is placed after each column.

Summary Report

When you require a summary report consisting only of section footers, specify the NODETAIL argument to suppress production of the detail records on the report.

/REPORT

You can have column headings without column entries by specifying headers in `/COLUMNS` and using `NODETAIL`.

Page Length

You can provide the number of lines to appear on a page of the report through the `LINES` argument. When you do not specify the line count, it defaults to 60. The page must be long enough to contain all of the following components which apply:

- The last page of the title, when the last title specification is not `NEWPAGE`
- The page header
- The column headings
- All section headers
- The detail line for a single record, when `NODETAIL` is not specified
- The page footer

The page must be long enough to contain all of the following components which apply:

- The page header
- The column headings, when `NOCOLUMNHEADINGS` is not specified
- All section footers
- The page footer
- The first page of the trailer, when the first trailer argument is not `NEWPAGE` and the page footer is not forced to the bottom of the page

Examples

```
/report 58 lines nocolumnheadings
```

This /REPORT option describes a report with 58 lines per page. Any column headings are printed once after any page header on the first detail page but not on any succeeding pages.

```
/report pagestart 0 columnspacing 0, 2
```

The first page of the report associated with this option is assigned a page number of zero. The columns are spaced such that there are no spaces prior to the first column and all succeeding columns are separated by 2 spaces. The final column is followed by 2 spaces. This spacing may be overridden for specific columns by the COLUMNSPACING argument in the /COLUMNS option.

```
/report 65 lines nodetail
```

The maximum number of lines in any page of the report associated with this option is 65. The NODETAIL argument suppresses detail lines generated by records, resulting in a summary report.

/SAVE

/SAVE

Purpose

To save the current application definition to a file.

Format

`/SAVE file_name`

Arguments

<code>file_name</code>	the pathname of the file where the application definition is to be written. If the pathname contains a space, surround the name with double quotation marks.
------------------------	--

Location

The option may appear anywhere in the application definition.

Notes

`/SAVE` can be used only when SyncSort is invoked through the **syncsort** command in a line-by-line interactive session.

You can use the `/SAVE` option during an interactive line-by-line SyncSort session to save the current application definition to a file. The `/SAVE` option itself does not become a permanent part of the definition but is removed automatically.

Since an application definition is terminated by the first `/END` option, `/SAVE` is inoperative if it appears after `/END`.

The file specified cannot be an existing file.

Once you have saved the application definition, you can terminate the SyncSort session with **/QUIT** and amend the file in an editor.

SyncSort may be executed via the saved application definition by issuing the command **syncsort /INSERT save_spec**. This command may take you back into the interactive session. If the saved definition contains **/END**, SyncSort will execute. If there is no **/END**, you can add more options to the definition, though you cannot alter the portion of the definition from the saved file.

Example

```
/save save1.srt
```

The option specifies that the current application definition is to be saved to a file named **save1.srt**.

/SECTION

/SECTION

Purpose

To describe a section of a report.

Format

```
/SECTION fieldname [ONBREAK [ln_number [LINES]]  
[NEWPAGE ] ]
```

Arguments

<i>fieldname</i>	the name of a field defined through /DATADictionary, /DERIVEDFIELD, /FIELDS, /DELIMITEDRECORDLAYOUT, or /RECORDLAYOUT. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.
<i>ln_number</i>	the number of blank lines to be inserted at the end of the section.

Location

The /SECTION option must appear after the /OUTFILE option which defines the report file containing the section. When there are multiple output files, a /SECTION option for a particular report must appear between the /OUTFILE option specifying that report and the next /OUTFILE option, if any.

Notes

Multiple Sections

When you wish to report on several levels of detail, e.g. by **division**, **office** within the **division**, **salesperson** within the **office**, you need to specify a /SECTION

option for each level of detail. The order of the /SECTION options indicates the priority of the sections. The first /SECTION option is for the highest priority section, with other sections following in decreasing order of priority. In the preceding example, the /SECTION option for **division**, the highest priority section, should appear first, followed by the /SECTION options for **office** and **salesperson** respectively.

Action on a Section Break

You must specify a /SECTION option for a field when you want to do something special each time the value in the field changes. The actions that are possible are:

- Insert a section footer for the previous value of the field (see /SECTIONFOOTER).
- Insert a number of blank lines, or skip to the top of a new page.
- Insert a section header for the new value of the field (see /SECTIONHEADER).

A section break occurs when the value in the section control field changes from one record to the next. Any alternative collating sequence defined for the field is applied when making the decision that the control field value has changed. For example, if FOLDED is specified or implied in the collating sequence for this break field, two records whose break fields differ only in capitalization belong to the same section.

When there are multiple sections in a report, a break in one of the higher priority sections also causes a break in all lower priority sections. When a section break occurs, the section footers for all the breaking sections, defined by /SECTIONFOOTER options, are generated. The footers are generated from lowest priority section up to the highest priority breaking section. After any required /SECTIONFOOTER summary has been generated for a particular section, the break action from the /SECTION option is applied. Any section headers specified for the breaking sections through /SECTIONHEADER options follow the final section footer. The section header for the highest priority breaking section is generated first, followed by the section headers for all other breaking sections in decreasing order of priority.

The Section Break Control

/SECTION

The field which you specify as a section break control must be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on the field which can be specified to control a section break. However, when you specify input reformatting, you must not discard a field which you want to use as a section break. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want to use the **fielda** value to control a section break.

Examples

```
/section company onbreak 2 lines
```

The details making up the report are to be sectioned according to the value of the **company** field. When the value in **company** changes, two blank lines are to be inserted in the report (after any footer for the previous value of **company**) before starting the section for the subsequent value.

```
/section category onbreak newpage
```

The details making up the report are to be sectioned according to the value of the **category** field. When the value in **category** changes, a new page is to be started for the report (after any footer for the previous value of **category**) before starting the section for the subsequent value.

/SECTIONFOOTER

Purpose

To describe the contents of a section footer for a report.

Format

/SECTIONFOOTER *field* [, *field* ...]

where

		[<i>fieldname</i> [TRUNCATE]]
		[[<i>repeat</i>] <i>string</i>]
<i>field</i>	=	[[<i>nl_number</i>] NEWLINE]
		[PAGENUMBER]
		[TODAY]

Arguments

- | | |
|------------------|---|
| <i>fieldname</i> | the name of a field defined through /AGGREGATE, /DATADictionary, /DERIVEDFIELD, /RECORDLAYOUT, /DELIMITEDRECORDLAYOUT, or /FIELDS. TRUNCATE can be used for character fields only. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names. |
| <i>string</i> | a character string. The string can be specified in any of the formats described in “Specifying Character String Constants” on page B.1. |
| <i>repeat</i> | the number of times a character string is to be repeated. |
| <i>nl_number</i> | the number of new lines to be inserted into the section footer. The minimum value is 1. The default is 1. |

/SECTIONFOOTER

Location

The `/SECTIONFOOTER` option must appear after the `/OUTFILE` option which defines the report file containing the section footer. When there are multiple output files, any `/SECTIONFOOTER` option for a particular report must appear between the `/OUTFILE` option specifying that report and the next `/OUTFILE` option, if any. When the report is divided into several sections, the `/SECTIONFOOTER` option must appear between the `/SECTION` option describing the section to which it applies, and the next `/SECTION` option for the report, if any.

Notes

There can be only one `/SECTIONFOOTER` for each section of a report.

The section footer is placed immediately after:

- the last detail line in the section or
- the section header for the section if there are no detail lines or
- the section footer for the prior section if there is a lower priority section with a footer.

Using delimited fields in a report

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When you use a field from such a record in a section footer, the field delimiter does not appear in the footer.

Enclosed delimited fields

Each field in a field-delimited text record may be enclosed by two enclosing characters. When you use a single or partial delimited field in a section footer, any enclosing characters do not appear in the footer. When you use a multiple delimited field in a section footer, the field appears in the footer with all enclosing characters.

Section Footer Contents

All trailing spaces are removed from a section footer field which you specify with the TRUNCATE argument. This allows you to embed values in sentences without having large gaps. Remember, however, that *all* spaces are truncated; you need to ensure that the following field begins with at least one space if you do not want the fields to run together.

NEWLINE creates a line break. Each leading NEWLINE in a section footer option produces a blank line at the start of the footer. For example, **3 NEWLINE** at the start of a footer option results in 3 leading blank lines. Each non-leading NEWLINE, i.e. those which appear after other types of arguments such as a field name, starts a new line. Thus, if you want blank lines in the middle or end of a footer, specify one NEWLINE to start a new line and one NEWLINE for each blank line you want. For example, **3 NEWLINE** anywhere other than at the start of a footer option results in 2 blank lines.

A PAGENUMBER field contains the current page number edited using the default page number mask (ZZZ9). A TODAY field contains the current system date edited with the default date mask (MM0/DD0/YEAR). For details on changing edit masks, see “Appendix E. Masks” on page E.1 and “/DERIVEDFIELD ” on page 3.48.

A Section Footer Field

When a section footer field is a field in the input record, the value assigned to the section footer field is the value from the **last** detail in the section.

The field which you specify to supply a section footer value must be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on the field which can be specified for a section footer field. However, when you specify input reformatting, you must not discard a field which you want to appear in a section footer. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want the **fielda** value to appear in a section footer.

/SECTIONFOOTER

Examples

```
/sectionfooter newline, "End of details for ", city,  
" on ", rept_date
```

Following is a sample of the two line section footer produced by this /SECTIONFOOTER option:

```
End of details for San Francisco           on 4th April 1996
```

- The first line is blank (caused by the leading NEWLINE).
- The second line starts with the text "End of details for ". Next are the contents of the **city** field from the last detail which appears in the section. The **city** value is followed by the text " on " and the contents of the field **rept_date** (possibly a derived field containing today's date with an edit mask).

```
/sectionfooter "Summary for the ", county truncate,  
" county office", newline, 20" "  
"Total sales: ", dollar_amount,  
" Number of sales: ", sales_count
```

This /SECTIONFOOTER produces the following two line section footer:

```
Summary for the Middlesex county office  
      Total sales:           57463874 Number of sales:  12756
```

- The first line begins with the text "Summary for the " followed by the contents of the **county** field in the last detail appearing in the section. Any spaces and tabs at the end of the **county** value are removed. The first line ends with the text " county office".
- The second line starts with 20 spaces and the text "Total sales: ". These are followed by the contents of the **dollar_amount** field in the last detail in the section. Since no edit mask is provided with **dollar_amount**, the appearance of the field in the footer depends on the data type of the originating field. For example, when **dollar_amount** is a field with unsigned

integer format (INTEGER) and is 4 bytes long, the footer contains the value in **dollar_amount** converted to an 11 character edited numeric value.

The second line continues with the text " Number of sales: " and ends with the contents of the **sales_count** field from the last detail converted to the default printable format.

```
/sectionfooter "Totals for ", manager truncate, "'s  
group",2 newline, 10" ", total_hours_spent, 10" ",  
total_hours_charged, 10" ",total_overtime_hours, 10" ",  
total_charged, newline, 10" ", "Number of employees: ",  
emp_count
```

This /SECTIONFOOTER produces the following sample four line section footer:

Totals for Terry Smith's group			
207	204	14	\$12,760
Number of employees: 5			

- The first line begins with the text "Totals for " followed by the contents of the **manager** field in the last detail appearing in the section. All spaces and tabs at the end of the **manager** value are removed.

The first line ends with the text "'s group".

- The second line is blank (the first line is terminated by one of the 2 NEWLINEs).
- The third line begins with 10 spaces followed by the contents of **total_hours_spent** (probably an aggregate containing the total hours spent for all members of the group).

The third line continues with 10 spaces, the contents of **total_hours_charged** (probably an aggregate containing the total hours charged for all members of the group), 10 spaces, the contents of **total_overtime_hours** (probably an aggregate containing the total number of overtime hours worked by all members of the group), and ends with another

10 spaces and the contents of **total_charged** (probably an aggregate containing the total amount charged for all members of the group).

- The fourth line of the footer starts with 10 spaces followed by the text "Number of employees: " and ends with the contents of **emp_count** (probably an aggregate containing a count of the members of the group).

/SECTIONHEADER

Purpose

To describe the contents of a section header for a report.

Format

/SECTIONHEADER *field* [, *field* ...] [PAGETOP]

where

		[<i>fieldname</i> [TRUNCATE]]
		[[<i>repeat</i>] <i>string</i>]
<i>field</i>	=	[[<i>nl_number</i>] NEWLINE]
		[PAGENUMBER]
		[TODAY]

Arguments

- | | |
|------------------|---|
| <i>fieldname</i> | the name of a field defined through /DATADictionary, /DERIVEDFIELD, /FIELDS, /DELIMITEDRECORDLAYOUT, or /RECORDLAYOUT. TRUNCATE can be used for character fields only. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names. |
| <i>string</i> | a character string. The string can be specified in any of the formats described in “Specifying Character String Constants” on page B.1. |
| <i>repeat</i> | the number of times a character string is to be repeated. |
| <i>nl_number</i> | the number of new lines to be inserted into the section header. The minimum value is 1. The default is 1. |

/SECTIONHEADER

Location

The /SECTIONHEADER option must appear after the /OUTFILE option which defines the report file containing the section header. When there are multiple output files any /SECTIONHEADER option for a particular report must appear between the /OUTFILE option specifying that report and the next /OUTFILE option, if any. When the report is divided into several sections, the /SECTIONHEADER option must appear between the /SECTION option describing the section to which it applies, and the next /SECTION option for the report, if any.

Notes

There can be only one /SECTIONHEADER for each section of a report.

For the first section on a page, the section header is placed immediately after the column headings, or after the page header if there are no column headings, or at the top of the page if there are neither column headings nor page header. For a subsequent section, the section header appears after the section header for the section at the prior level, or after the section footer for the prior value of the section at the same and lower levels.

When you specify the PAGETOP argument and details with a particular value of a section control appear on more than one page of the report, the section header is repeated on the subsequent pages. The section header for each lower level section is, of necessity, also repeated. When you do not specify PAGETOP, the section header appears only when the section control value changes.

Using delimited fields in a report

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When you use a field from such a record in a section header, the field delimiter does not appear in the header.

Enclosed delimited fields

Each field in a field-delimited text record may be enclosed by two enclosing characters. When you use a single or partial delimited field in a section header, any enclosing characters do not appear in the header. When you use a multiple delim-

ited field in a section header, the field appears in the header with all enclosing characters.

Section Header Contents

All trailing spaces are removed from a section header field which you specify with the TRUNCATE argument. This allows you to embed values in sentences without having large gaps. Remember, however, that all spaces are truncated; you need to ensure that the following field begins with at least one space if you do not want the fields to run together.

NEWLINE creates a line break. Each leading NEWLINE in a section header option produces a blank line at the start of the header. For example, **3 NEWLINE** at the start of a header option results in 3 leading blank lines. Each non-leading NEWLINE, i.e. those which appear after other types of arguments such as a field name, starts a new line. Thus, if you want blank lines in the middle or end of a header, specify one NEWLINE to start a new line and one NEWLINE for each blank line you want. For example, **3 NEWLINE** anywhere other than at the start of a header option results in 2 blank lines.

A PAGENUMBER field contains the current page number edited using the default page number mask (ZZZ9). A TODAY field contains the current system date edited with the default date mask (MM0/DD0/YEAR). For details on changing edit masks, see “Appendix E. Masks” on page E.1 and “/DERIVEDFIELD ” on page 3.48.

A Section Header Field

When a section header field is a field in the input record, the value assigned to the section header field is the value of the field in the **first** detail line in the section.

The field which you specify to supply a section header value must be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on the field which can be specified for a section header field. However, when you specify input reformatting, you must not discard a field which you want to appear in a section header. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input

/SECTIONHEADER

reformatting specification if you want the **fielda** value to appear in a section header.

Examples

```
/sectionheader newline, "Attendance record of ",  
student_name
```

This /SECTIONHEADER produces the following sample two line section header.

```
Attendance record of Dorothy Donegan
```

- The first line is blank (caused by the leading NEWLINE).
- The second line starts with the text "Attendance record of ". Next are the contents of the **student_name** field from the first detail which appears in the section.

The header appears at most once for each value in the section control.

```
/sectionheader "Current inventory of ", mech_assembly  
truncate," parts, at warehouse location ", wh_loc, newline page  
top
```

This /SECTIONHEADER produces the following single line section header.

```
Current inventory of terminator parts, at warehouse location 75
```

The header line starts with the text "Current inventory of ". Next are the contents of the **mech_assembly** field from the first detail appearing in the section. Any spaces and tabs at the end of the **mech_assembly** value are removed. The truncated **mech_assembly** value is followed by the text " parts, at warehouse location ". The line ends with the contents of the **wh_loc** field from the first detail.

Since PAGETOP is specified, the section header appears at the top of each detail page as well as at the beginning of each new value of the section control.

/SILENT

Purpose

To request that no status messages be printed.

Format

/SILENT

Location

The option may appear anywhere in the application definition.

Notes

When you specify /SILENT in the definition of an application invoked through a command interface, SyncSort will not display any status messages. You will still see final statistics if you request them using /STATISTICS. Also, messages reporting any problems will be shown.

Example

```
/silent
```

This requests that status messages be suppressed.

/SKIPBLANK

/SKIPBLANK

Purpose

To request the removal of blank records.

Format

/SKIPBLANK

Location

The option may appear anywhere in the application definition.

Notes

When /SKIPBLANK is specified, SyncSort discards all blank records from all input sources. A blank record is defined as follows for the various record formats:

- A fixed length record is blank if all bytes in the record are space characters.
- A text record is blank if the record consists of the terminator character only or if all bytes in the record prior to the terminator are space characters.
- A variable format record is blank if the record length is zero or if all bytes in the record are space characters.
- A record in stream format is blank if the record consists of the terminator character only or if all bytes in the record prior to the terminator are space characters.
- A Micro Focus variable record is blank if the record length is zero or if all bytes in the record are space characters.

- A Micro Focus line sequential record is blank if the record length is zero or if all bytes in the record are space characters.
- A Fortran Unformatted record is blank if the record length is zero or if all bytes in the record are space characters.

Example

`/skipblank`

This is a request to remove the blank records from the input.

/SKIPRECORD

/SKIPRECORD

Purpose

To specify that a number of records are to be bypassed before any records are processed.

Format

/SKIPRECORD number

Arguments

<i>number</i>	the number of input records to be bypassed before any input selection is applied and records are released to SyncSort.
---------------	--

Location

The option may appear anywhere in the application definition.

Notes

You can specify */SKIPRECORD* for a sort or copy and only if the input records originate from files.

There are several options which you can use to limit the records which are released to SyncSort. These are:

- /SKIPRECORD*
- /SKIPBLANK*
- /STOPAFTER*
- /INCLUDE* and */OMIT*
- /IGNORE*
- /COLLECT*

The order in which the options appear in the above list is the order in which SyncSort applies them. Thus, the **/SKIPRECORD** option determines the number of input records which SyncSort discards before applying additional record-limiting options. SyncSort discards records starting with the first record in the first input file. If the first file does not satisfy the **/SKIPRECORD** requirement, records are discarded from the second input file, and so on.

If you request both **/SKIPRECORD** and **/IGNORE**, SyncSort will discard records according to both options. Given the definition **/SKIPRECORD 10 /IGNORE 20**, SyncSort will discard the first 10 records from the first input file and read from the 11th record onwards. As each record is retrieved, SyncSort applies any input selection that may have been requested through **/INCLUDE** and **/OMIT**. Each record surviving the selection is counted towards the **/IGNORE** count. SyncSort starts releasing records to SyncSort when 20 records have survived selection. Thus, a minimum of 30 records from the start of the first input file are discarded.

/SKIPRECORD and the file level record-limiting criteria (**FSKIPRECORD** and **FSTOPAFTER** arguments in the **/INFILE** option) cannot be specified in the same application.

Example

```
/skiprecord 100
```

100 records must be bypassed before any input selection criteria are applied and any records are released to SyncSort.

/STABLE

/STABLE

Purpose

To indicate that the order of equal-keyed records is to be maintained.

Format

`/STABLE`

Location

The option may appear anywhere in the application definition.

Notes

The terms stable and non-stable refer to the order in which records with equal key values leave the application.

When an application is stable, the order of the equal-keyed records is completely predictable. For a stable application with a single input file, the original order of equal-keyed records is maintained, i.e. the order of these records is the same in the output file as in the input file. For a stable application with more than one input file, equal-keyed records from a specific input file appear together in the output file. They appear in the same sequence as they did in the input file. The collections of records from the different input files appear in the order in which the input files are specified in the application definition.

When an application is not stable, the order of equal-keyed records is not predictable; this is the default.

Preserving the original order of equal-keyed records adds a performance overhead to an application. Thus, if you do not require a stable application, do not specify `/STABLE`.

/STABLE is ignored for a copy.

Example

```
/stable
```

This option requests that the original order of equal-keyed records be kept intact in the output file.

/STATISTICS

Purpose

To request statistics concerning SyncSort's execution.

Format

/STATISTICS

Location

The option may appear anywhere in the application definition.

Notes

When invoking SyncSort through a program, you can obtain statistics from the SyncSort subroutine call `sync_get_statistic`. See “`sync_get_statistic`” on page 2.43 for details.

When you request statistics for an application invoked through the command interface, SyncSort writes the following information to standard error when it completes:

`SyncSort statistics`

<code>Records read:</code>	<code>n</code>	<code>Data read (bytes):</code>	<code>n</code>
<code>Records sorted:</code>	<code>n</code>	<code>Data sorted (bytes):</code>	<code>n</code>
<code>Records merged:</code>	<code>n</code>	<code>Data merged (bytes):</code>	<code>n</code>
<code>Records copied:</code>	<code>n</code>	<code>Data copied (bytes):</code>	<code>n</code>
<code>Records output:</code>	<code>n</code>	<code>Data output (bytes):</code>	<code>n</code>
<code>Input records deleted:</code>	<code>n</code>	<code>Data deleted (bytes):</code>	<code>n</code>
<code>Input record length:</code>	<code>n</code>	<code>Output record length:</code>	<code>n</code>
<code>Work space used (bytes):</code>	<code>n</code>		

```
Output records deleted: n      Data deleted (bytes): n
Elapsed time: hh:mm:ss.ss  CPU time: hh:mm:ss.ss
```

Note that only one of the lines `Records sorted...`, `Records merged...` and `Records copied...` is printed, depending on the kind of application. The lines `Input records deleted...` and `Output records deleted...` are printed only when records have been deleted because of data errors.

The information provided is:

Records read	the number of records read from the input file(s).
Data read	the amount in bytes of data read from the input file(s).
Records sorted	the number of records that were sorted.
Data sorted	the amount of data in bytes which survived all input record selection.
Records merged	the number of records that were merged.
Data merged	the amount of data in bytes which survived all input record selection.
Records copied	the number of records that were copied.
Data copied	the amount of data in bytes which survived all input record selection.
Records output	the number of records written to an output file.
Data output	the amount of data in bytes written to the output file.
Input records deleted	the number of input records deleted because of data errors.
Data deleted	the amount in bytes of input data deleted because of data errors.
Input record length	the length in bytes of the longest input record.

/STATISTICS

Output record length	the length in bytes of the longest record written to an output file.
Workspace used	the amount of disk space used for work files, in bytes.
Output records deleted	the number of output records deleted because of data errors.
Data deleted	the amount in bytes of output data deleted because of data errors.
Elapsed time	the amount of time taken to execute the application.
CPU time	the amount of CPU time used during the application.

When there is more than one output file, the number of records, the amount of data and the maximum record length are provided for each file.

Example

```
/statistics
```

The option requests that SyncSort provide statistics.

/STOPAFTER

Purpose

To specify the maximum number of input records to be processed.

Format

/STOPAFTER *number*

Arguments

number the maximum number of records to be passed to SyncSort.

Location

The option may appear anywhere in the application definition.

Notes

You can specify **/STOPAFTER** for a sort or copy and only if the input records originate from files.

There are several options which you can use to limit the records which are released to SyncSort. These are:

- /SKIPRECORD**
- /SKIPBLANK**
- /STOPAFTER**
- /INCLUDE** and **/OMIT**
- /IGNORE**
- /COLLECT**

/STOPAFTER

The order in which the options appear in the above list is the order in which SyncSort applies them. Thus, the */STOPAFTER* option determines the maximum number of records which can be sorted or copied. The selection options */INCLUDE* and */OMIT* are then applied, if specified, and may further limit the number of records.

If you request additional record-limiting options, the number of records which are finally released to your application may be less than the number you request in the */STOPAFTER* option, but it will not exceed that number.

/STOPAFTER and the file level record-limiting criteria (*FSKIPRECORD* and *FSTOPAFTER* arguments in the */INFILE* option) cannot be specified in the same application.

Example

```
/stopafter 200
```

The option specifies that 200 records are to be passed to SyncSort.

/SUMMARIZE

Purpose

To summarize the contents of a set of equal-keyed records into a single record.

Format

```
/SUMMARIZE [sum_unit [, sum_unit ...]]
```

where

```
sum_unit          =    TOTAL s_field
```

Arguments

s_field the name of a numeric field defined through /DATADictionary, /DERIVEDFIELD, /FIELDS, /DELIMITEDRECORDLAYOUT, or /RECORDLAYOUT. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.

The named field is used as the source of the data to be summarized for a set of equal-keyed records. A field specified through /DERIVEDFIELD must also appear in a /REFORMAT statement to make it available for summarization.

Location

The /SUMMARIZE option must appear before the first output destination option. If input records are grouped, the /SUMMARIZE option must appear before the first input /INCLUDE or /OMIT option.

Notes

/SUMMARIZE is invalid for a copy.

Summarizing EN Fields

When SyncSort summarizes an edited numeric field with no precision specification, digits are not truncated and exact totals will be output. SyncSort will not apply any default precision. The thousands separator, as defined in the locale will not be output in an EN sum total. To output the thousands separator defined in the locale, derive a new field from the sum totaled field, specifying the precision, and reformat the new derived field into the record in place of the original sum totaled field. If the total of an EN summation is negative, a leading sign will be output. If the last delimited EN field is summarized and totaled, SyncSort will not put a delimiter at the end of the record.

For example, adding 1.23456 and 230 will result in 231.23456. After each addition, the running total is checked to make sure that it fits within the field. If there is a precision specified for the field, it is applied to the running total for computing its length. For example, with a precision of 3/2, the length of running total 231.23456 is 6. For a delimited EN field, the final total is moved to the location of the original field in the record that survives. If there is not enough space for the total (after removing leading blanks, leading zeros, and trailing zeros), the record is expanded. If the maximum input record length is exceeded due to expansion, an overflow is declared and duplicate records are output. This implies that for fixed length input, record expansion is not possible. For a positional EN field, expansion is never possible. So if the total doesn't fit within the field in the record, an overflow is declared.

When SyncSort summarizes a delimited edited numeric field with a precision specified, the length of the field output is determined by the precision. Fractional digits are rounded as follows. If n fractional digits are to survive and more than n fractional digits exist, the $n+1$ st fractional digit is greater than 4, a 1 is added to the n th fractional digit. If $n+1$ st fractional digit is greater than 4, a 1 is added to the n th fractional digit. Integral digits are truncated starting from the most significant digit. For example, if the sum total is 99.934 and there are only two bytes available for the field, 00 is output. This is because after rounding, the value becomes 100. Since only two bytes are available, the most significant digit is dropped and 00 is output. A warning message of the overflow is displayed.

A delimited edited numeric field which spans field separators cannot be summarized. For example, fields defined as 2: - 3: EN, 3:4 - 4:1 EN and

7: EN cannot be summarized, but fields defined as 2: - 2: EN, 3:4 - 3: EN and 7: - 7:8 EN can be summarized.

The Surviving Record

When you specify **/SUMMARIZE** for an application, SyncSort retains only one record from each set of records with equal key values. The retained record contains any summarized values requested.

When an application is stable, the record which is retained is predictable. It is the first record of the set of equal-keyed records encountered as the input files are read in order. When an application is not stable, the record which is retained is not predictable. For more information see the **/STABLE** option.

If one of the summary fields is too small to hold the summarized value, you will be warned that overflow has occurred, and more than one record will survive from the set of records being summarized. At the point overflow occurs, SyncSort releases the current summary record (i.e. in the state before overflow occurs), creates another summary record and continues.

If you know that a field to be totalled is too small to hold the summarized value, use **/DERIVEDFIELD** to increase the length of the field. Change the recordlayout with **/REFORMAT** to replace the field with the derived field, and summarize the derived field.

Examples

```
/summarize
```

This option specifies that all but one of a set of equal-keyed records must be discarded.

```
/summarize total amt_paid
```

This option specifies that all but one of each set of equal-keyed records must be discarded and that data in **amt_paid** will be summed for that set and placed in the surviving record.

/TITLE

/TITLE

Purpose

To describe the contents of the title for a report.

Format

`/TITLE field [, field ... ]`

where

		<code>[fieldname [TRUNCATE]]</code>
		<code>[repeat] string</code>
		<code>[nl_number] NEWLINE</code>
<code>field</code>	<code>=</code>	<code>[np_number] NEWPAGE</code>
		<code>[PAGENUMBER</code>
		<code>[TODAY</code>

Arguments

<i>fieldname</i>	the name of a field defined through /DATADictionary, /DERIVEDFIELD, /FIELDS, /DELIMITEDRECORDLAYOUT, or /RECORDLAYOUT. TRUNCATE can be used for character fields only. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.
<i>string</i>	a character string. The string can be specified in any of the formats described in “Specifying Character String Constants” on page B.1.
<i>repeat</i>	the number of times a character string is to be repeated.
<i>nl_number</i>	the number of new lines to be inserted into the title. The minimum value is 1. The default is 1.

np_number the number of new pages to be inserted into the title. The minimum value is 1. The default is 1.

Location

A /TITLE option must appear after the /OUTFILE option which defines the report file containing the title. When there are multiple output files, the /TITLE option for a particular report must appear between the /OUTFILE option for that report and the next /OUTFILE option, if any.

Notes

There can be only one /TITLE option for a report.

The report title is placed on the first line of the first page of the report. If you do not want the page header, section header or detail lines to appear on the title page, you must specify NEWPAGE at the end of the /TITLE option. If the first title field is NEWPAGE there will be a blank page at the beginning of the report.

Using delimited fields in a report

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When you use a field from such a record in a title, the field delimiter does not appear in the title.

Enclosed delimited fields

Each field in a field-delimited text record may be enclosed by two enclosing characters. When you use a single or partial delimited field in a title, any enclosing characters do not appear in the title. When you use a multiple delimited field in a title, the field appears in the title with all enclosing characters.

Title Contents

All trailing spaces are removed from a title field which you specify with the TRUNCATE argument. This allows you to embed values in sentences without having large gaps. Remember, however, that all spaces are truncated; you need to

/TITLE

ensure that the following field begins with at least one space if you do not want the fields to run together.

NEWLINE creates a line break. Each leading NEWLINE in a title option produces a blank line at the start of the title. For example, **3 NEWLINE** at the start of a title option results in 3 leading blank lines. Each non-leading NEWLINE, i.e. those which appear after other types of arguments such as a field name, starts a new line. Thus, if you want blank lines in the middle or end of a title, specify one NEWLINE to start a new line and one NEWLINE for each blank line you want. For example, **3 NEWLINE** anywhere other than at the start of a footer option results in 2 blank lines.

A title field specified by the PAGENUMBER argument contains the current page number edited using the default page number mask. A title field specified by the TODAY argument contains the current system date edited with the default date mask.

A Title Field

When a report title field is a field in the input record, the value assigned to the title field is the value of the field in the **first** detail line in the report.

The field which you specify in a title must be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on the title field. However, when you specify input reformatting, you must not discard a field which you want to appear in a title. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want the **fielda** value to appear in a title.

Examples

```
/title 28 newline, 56"*, newline, "Sales Report : ",  
today, " for ", country, newline, 56"*, newpage
```

This title option produces the following single page containing 31 lines of text (including the leading 28 blank lines).

```
*****  
Sales Report : 11/17/1996 for Switzerland  
*****
```

- The first 28 lines in the title page are blank.
- The 29th line contains 56 asterisks.
- The 30th line contains the text "Sales Report : ", followed by the current system date. The date is edited according to the default date mask (MM0/DD0/YEAR). The line ends with the text " for " and the value from the **country** field of the first detail record in the report.
- The 31st line holds 56 asterisks. The next line of the report is at the top of the second page (caused by the NEWPAGE argument).

```
/title "Quality control details for ", report_month  
truncate,". Printed: ", print_date
```

This title option produces the following single line which appears at the top of the first page of the report, before any page header.

```
Quality control details for April. Printed: 4/1/96
```

The title line begins with the text "Quality control details for ", followed by the contents of the field **print_month** (probably a derived field containing the month component of the current system date). All trailing spaces and tabs are removed from the value of **print_month**. The title line ends with the text ". Printed: " and the value in the field **print_date** (probably a derived field containing the current system date with a suitable edit mask).

/TRAILER

/TRAILER

Purpose

To describe the contents of the trailer for a report.

Format

```
/TRAILER field [, field ... ]
```

where

<i>field</i>	=	<i>[fieldname [TRUNCATE]]</i>
		<i>[[repeat] string]</i>
		<i>[[nl_number] NEWLINE]</i>
		<i>[[np_number] NEWPAGE]</i>
		<i>[PAGENUMBER]</i>
		<i>[TODAY]</i>

Arguments

<i>fieldname</i>	the name of a field defined through <i>/DATADictionary</i> , <i>/DERIVEDFIELD</i> , <i>/FIELDS</i> , <i>/DELIMITEDRECORDLAYOUT</i> , or <i>/RECORDLAYOUT</i> . TRUNCATE can be used for character fields only. See “Specifying Field Names ” on page B.9 for details concerning the specification of field names.
<i>string</i>	a character string. The string can be specified in any of the formats described in “Specifying Character String Constants” on page B.1.
<i>repeat</i>	the number of times a character string is to be repeated.
<i>nl_number</i>	the number of new lines to be inserted into the trailer. The minimum value is 1. The default is 1.

np_number the number of new pages to be inserted into the trailer. The minimum value is 1. The default is 1.

Location

A /TRAILER option must appear after the /OUTFILE option which defines the report file containing the trailer. When there are multiple output files, the /TRAILER option for a particular report must appear between the /OUTFILE option for that report and the next /OUTFILE option, if any.

Notes

There can be only one /TRAILER option for a report.

The report trailer is placed immediately after the last detail line, or last section footer, or the last page footer. You can control the location of the trailer by use of NEWLINE and NEWPAGE.

When the trailer starts with a NEWPAGE, the trailer does not share a page with any other text. In this case, the page prior to the start of the trailer contains a page footer if there is a footer.

When the trailer does not start with a NEWPAGE, the trailer text immediately follows the last item produced (e.g. a detail line or a section footer) unless a section break action has caused the trailer to be written to a separator page. No page footer is added to the page(s) containing the trailer.

Using delimited fields in a report

A field-delimited text record consists of consecutive fields, with adjacent fields separated by the delimiter character. When you use a field from such a record in a trailer, the field delimiter does not appear in the trailer.

Enclosed delimited fields

Each field in a field-delimited text record may be enclosed by two enclosing characters. When you use a single or partial delimited field in a trailer, any enclosing

characters do not appear in the trailer. When you use a multiple delimited field in a trailer, the field appears in the trailer with all enclosing characters.

Trailer Contents

All trailing spaces are removed from a trailer field which you specify with the TRUNCATE argument. This allows you to embed values in sentences without having large gaps. Remember, however, that all spaces are truncated; you need to ensure that the following field begins with at least one space if you do not want the fields to run together.

NEWLINE creates a line break. Each leading NEWLINE in a trailer option produces a blank line at the start of the trailer. For example, **3 NEWLINE** at the start of a trailer option results in 3 leading blank lines. Each non-leading NEWLINE, i.e. those which appear after other types of arguments such as a field name, starts a new line. Thus, if you want blank lines in the middle or end of a trailer, specify one NEWLINE to start a new line and one NEWLINE for each blank line you want. For example, **3 NEWLINE** anywhere other than at the start of a trailer option results in 2 blank lines.

A trailer field specified by the PAGENUMBER argument contains the current page number edited using the default page number mask (ZZZ9). A trailer field specified by the TODAY argument contains the current system date edited with the default date mask (MM0/DD0/YEAR). For details on changing edit masks, see “Masks” on page E.1 and “/DERIVEDFIELD ” on page 3.48.

A Trailer Field

When a report trailer field is a field in the input record, the value assigned to the trailer field is the value from the **last** detail record in the report.

Every field which you specify in a trailer value be available in some form in the records leaving the sort, merge or copy phase of the application. When you do not specify any input reformatting, there is no limitation on a trailer field. However, when you specify input reformatting, you must not discard a field which you want to appear in a trailer. For example, if the value of a derived field **fielda** is obtained from an input record field **fieldb**, you must specify either **fielda** or **fieldb** in any input reformatting specification if you want the **fielda** value to appear in a trailer.

Examples

```
/trailer 6" ", " End of report for ", today, " ", 6" "
```

This trailer option produces the following single line which appears in the line after the last detail line or section footer in the report.

```
***** End of report for 04/19/1996 *****
```

The trailer line begins with six asterisks and the text " End of report for " and is followed by the current system date. The date is edited according to the default date mask (MM0/DD0/YEAR). The line is completed by a space and six asterisks.

```
/trailer newpage, 8" ", "Final Summary For ", pr_year,  
2 newline, 12" ", "Budget Amount : ", final_budget,  
newline, 12" ", "Actual Amount : ", final_actual
```

This trailer option produces the following report trailer, consisting of four lines, which appears on a page by itself (indicated by the leading NEWPAGE spec).

```
Final Summary For 1995
```

```
Budget Amount : $1,067,845
```

```
Actual Amount : $1,097,011
```

- The first line of the trailer begins with 8 spaces and the text "Final Summary For " followed by the contents of **pr_year** (possibly a derived field containing the report year in printable format derived from the last detail record in the report).
- The second line is blank (the first line is terminated by one of the 2 NEWLINEs).
- The third line consists of 12 spaces and the text "Budget Amount : " followed by the contents of **final_budget** (possibly an aggregate containing the total of a budget amount field for all records in the report, with an edit mask).

/TRAILER

- The fourth line consists of 12 spaces and the text "Actual Amount : " followed by the contents of **final_actual** (possibly an aggregate containing the total of an amount field for all records in the report, with an edit mask).

/WARNINGS

Purpose

To limit the number of warning messages that are issued.

Format

`/WARNINGS number`

Arguments

<i>number</i>	the maximum number of warning messages to be issued during the application.
---------------	---

Location

The option may appear anywhere in the application definition.

Notes

By default, SyncSort will issue a maximum of 100 warnings, not including validation warnings.

When you specify `/WARNINGS` in an application invoked through the command interface, SyncSort will stop issuing warning messages after it has reached the limit you set. The limit on the number of warnings does not include those that result from validation. These warnings are issued regardless of the number requested with `/WARNINGS`. You will still see final statistics if you request them using `/STATISTICS`. Also, messages reporting any problems with severity greater than warning will continue to be issued.

/WARNINGS

Limiting the number of warning messages is useful when you expect that Sync-Sort will issue a warning for each record, e.g., when you expect that every record will be truncated.

Example

```
/warnings 20
```

The option specifies that no more than 20 warning messages are to be issued.

/WORKSPACE

Purpose

To specify the disk space to be used by SyncSort for work files.

Format

/WORKSPACE file_name

Arguments

<i>file_name</i>	the pathname of a directory which SyncSort can use as work space. If the pathname contains a space, surround the name with double quotation marks.
------------------	--

Location

The option may appear anywhere in the application definition.

Defaults

When no /WORKSPACE option is present, SyncSort writes its sortwork files to the directory specified in the environment variable TMPDIR. If no /WORKSPACE option is present and the environment variable TMPDIR is not defined, SyncSort writes to the directories /usr/tmp, /var/tmp, and/or /tmp.

Notes

For the majority of sorts, the amount of sortwork space needed is either zero (if the data to be sorted will fit into memory) or approximately the same as the amount of input data to be sorted.

/WORKSPACE

When there is insufficient sortwork space for a regular sort, SyncSort may attempt a tag sort. The sortwork space needed for each record for a tag sort is 7 bytes plus the length of the sort keys. A tag sort may take significantly longer than a non-tag sort so you should provide SyncSort with sufficient sortwork space whenever possible.

In cases where special processing is performed, the amount of sortwork space required varies:

- For stream format records, the additional amount of work space that may be needed is 2 bytes per record.
- For sorts using one or more alternative collating sequences, the additional amount of work space needed for each record is (a) the number of bytes in each key that has a single character alternative collating sequence and (b) two times the number of bytes in each key that has a double character alternative collating sequence or uses tie-break.
- For sorts using input reformatting (not reformatting for a specific output file), the amount of sortwork space needed is proportional to the size of the record after reformatting.
- For sorts using input record selection, the amount of sortwork space needed is proportional to the number of records that survive selection.
- For very large sorts (where the amount of data to be sorted is more than 50 times the size of memory available), the additional amount of sortwork space needed is a maximum of two thirds of the amount for a regular sort.
- For sorts using the /STABLE option, the additional amount of sortwork space needed is 10 bytes per record.
- For sorts using delimited fields as sort keys, the additional space needed is 4 bytes per key.

The method of specifying the work areas is to provide the pathname of a directory to SyncSort, e.g.

```
syncsort ... /WORKSPACE scratch1 /WORKSPACE scratch2 ...
```

Only one work area can be specified per /WORKSPACE option.

You must have read, write, and delete access for the work area.

Example

```
/workspace /usr/workspace
```

This option specifies that the directory `/usr/workspace` is to be used for SyncSort temporary files.

Chapter 4. The SyncSort Message Utility

The SyncSort Message Utility provides information on exceptions that may be encountered when running SyncSort. The utility conveys this information in the form of either:

- on-line help for one exception at a time
- or
- a file that contains a full listing of the information on all exceptions.

You can invoke the SyncSort Message Utility by issuing the **syncmsg** command at the shell prompt. The format of the **syncmsg** command is as follows:

```
syncmsg  [mnemonic          ]  
         [mnumber          ]  
         [/LIST [file_name]]
```

where

<i>mnemonic</i>	exception mnemonic, the mnemonic appearing in parentheses preceding the message text seen when a message is printed (e.g. SEQER).
<i>mnumber</i>	an exception number as returned in the external variable errno from an invoked sort (e.g. 1413).
<i>file_name</i>	the pathname of the file to receive the full listing of information on all exceptions.

On-Line Help

When you enter a valid exception mnemonic or exception number with the **syncmsg** command, the message associated with the mnemonic or number is displayed, along with an explanation. In some cases, you are also given information for correcting the condition that caused the exception. Here is an example of on-line help information for a single exception condition:

SYNCSORT: (SEQER) a subroutine has been called out of sequence

EXPLANATION: One of the subroutines which you have called from your application program is not called in the right sequence. You may be attempting to call the correct subroutines, but in the wrong order. Or, you may have omitted a call to a mandatory subroutine.

ACTION: Refer to the section entitled "The SyncSort Subroutines" in the SyncSort Reference Guide. There you will find a description of the order in which the SyncSort subroutines must be called to execute a sort, a merge, and a copy. In the same section is a description of the subroutines which are mandatory.

On completion of the display, you are given an opportunity to obtain information about another exception:

Enter a mnemonic or number for additional help, or blank to exit:

> _

The process is repeated until you press <enter> without keying any characters.

Exception Listing

If you specify the /LIST option with the **syncmsg** command, the message utility produces a complete listing of all exception conditions. As in the case of on-line help for single exception, the listing provides for each exception

- an exception code

-
- a description of the exception
 - an explanation of the exception
 - an action for the correction of the exception.

The listing is in alphabetical order by exception mnemonic. It is written to a file in printable and editable format. If you do not specify a file name (*file_name*) in the /LIST option, the listing is written to **syncmsg.lis** in the current directory. The file specified cannot be an existing file.

Appendix A. Data Types

Data Types and Lengths Supported by SyncSort

Data Type	Keyword	Length (bytes)
ASCII numeric - leading embedded sign decimal, Micro Focus format	LZ	1-31
ASCII numeric - leading embedded sign decimal, alternative format	LP	1-31
ASCII numeric - trailing embedded sign decimal, Micro Focus format	ZD	1-31
ASCII numeric - trailing embedded sign decimal, alternative format	TP	1-31
ASCII numeric - leading separate sign decimal	LS	2-32
ASCII numeric - trailing separate sign decimal	TS	2-32
ASCII numeric - unsigned decimal (positional)	AN	1-31
ASCII numeric - unsigned decimal (delimited)	AN	1-65,535
Binary - floating point - single precision	FLOAT	4
Binary - floating point - double precision	DFLOAT	8
Binary - signed integer	INTEGER	1-8
Binary - unsigned integer	UNINTEGER	1-8
Bit	BIT	1-65,535 bits
Character	CHARACTER	1-65,535
Date/Time	DATETIME	1-65,535
Edited numeric	EN	1-65,535
Packed decimal	PD	1-15

Table 1. Data Types Supported by SyncSort

	AN	BIT	CH	DFLOAT	EN	FLOAT	INT	LP	LS	LZ	PD	TP	TS	UINT	ZD
AN	*				*		*	*	*	*	*	*	*	*	*
BIT		*													
CH			*												
DFLOAT				*	*	*	*							*	
EN	*			*	*	*	*	*	*	*	*	*	*	*	*
FLOAT				*	*	*	*							*	
INT	*			*	*	*	*	*	*	*	*	*	*	*	*
LP	*				*		*	*	*	*	*	*	*	*	*
LS	*				*		*	*	*	*	*	*	*	*	*
LZ	*				*		*	*	*	*	*	*	*	*	*
PD	*				*		*	*	*	*	*	*	*	*	*
TP	*				*		*	*	*	*	*	*	*	*	*
TS	*				*		*	*	*	*	*	*	*	*	*
UINT	*			*	*	*	*	*	*	*	*	*	*	*	*
ZD	*				*		*	*	*	*	*	*	*	*	*
Note: * Indicates valid comparisons															

Table 2. Valid Comparisons between Data Types

<i>TO → FROM:</i>	AN	BIT	CH	DFLOAT	EN	FLOAT	INT	LP	LS	LZ	PD	TP	TS	UINT	ZD
AN	*				*		*	*	*	*	*	*	*	*	*
BIT		*													
CH			*												
DFLOAT				*	*	*									
EN				*	*	*	*	*	*	*	*	*	*		*
FLOAT				*	*	*									
INT				*	*	*	*	*	*	*	*	*	*		*
LP					*		*	*	*	*	*	*	*		*
LS					*		*	*	*	*	*	*	*		*
LZ					*		*	*	*	*	*	*	*		*
PD					*		*	*	*	*	*	*	*		*
TP					*		*	*	*	*	*	*	*		*
TS					*		*	*	*	*	*	*	*		*
UINT	*			*	*	*	*	*	*	*	*	*	*	*	*
ZD					*		*	*	*	*	*	*	*		*

Table 3. Valid Conversions between Data Types

Notes

A field of a given data type with one length can be converted to the same data type with a different length. If the original length is shorter than the converted length, data is not lost. However, if the converted length is shorter than the original length, data is lost if:

- the value in the field is larger than the maximum value which can be stored in the converted field
- for float, the precision of the converted field is insufficient to hold the significant digits of the original value.

When a field is truncated, the resulting value is unpredictable.

Description of Supported Data Types

The following sections describe the data types supported by SyncSort.

ASCII Numeric

ASCII numeric data can hold up to 31 decimal digits and a sign. Each decimal digit occupies a single byte and is represented by its 8-bit ASCII value. The first byte of an ASCII numeric field can be on an arbitrary byte boundary. The sign value is represented in a number of different ways, as described below. When data is represented in a delimited format, unsigned ASCII numeric data holds up to 65,535 digits with no sign value, and optional leading spaces are permitted.

Unsigned

The field contains only decimal digits. All values are positive.

Exceptions: Any value greater than X"9" in the numeric bits of a byte containing a digit is invalid. Any value other than X"3" in the zone bits of a byte containing a digit is invalid.

Example:

The number 123 in a 3-byte unsigned decimal field, represented in hexadecimal:

31	32	33
----	----	----

Separate sign

The sign is held as a separate single character, additional to the digits, either ASCII + or -. A space is also a valid sign character for input data. For leading separate sign decimal data, the sign occupies the first byte of the field, immediately followed by the decimal digits. For trailing separate sign decimal data, the sign occupies the last byte of the field, immediately following the decimal digits.

Exceptions: Any value greater than X"9" in the numeric bits of a byte containing a digit is invalid. Any value other than X"3" in the zone bits of a byte containing a

digit is invalid. Any sign byte containing a value other than X"2B" or X"2D" is invalid.

Examples:

The number +123 in a 4-byte leading separate sign decimal field, represented in hexadecimal:

2B	31	32	33
----	----	----	----

The number -123 in a 4-byte trailing separate sign decimal field, represented in hexadecimal:

31	32	33	2D
----	----	----	----

Embedded sign

The sign is incorporated into the first or last decimal digit, represented by the values in the tables below. The digit with the embedded sign can be in one of two formats, Micro Focus format (LZ or ZD) or alternative format (LP or TP). For the alternative format, other supported values are recognized in processing input data, but not used in generating output data from a conversion or summarization operation.

Exceptions: Any value greater than X"9" in the numeric bits of a byte containing a digit is invalid. Any value other than X"3" in the zone bits of a byte containing a digit is invalid. Any value in the byte containing the embedded sign other than those listed for Micro Focus format in Table 4. on page A.6 or alternative format in Table 5. on page A.6 is invalid.

First or Last Digit	Positive Value	Negative Value
0	0	p
1	1	q
2	2	r
3	3	s
4	4	t
5	5	u
6	6	v
7	7	w
8	8	x
9	9	y

Table 4. *Embedded Sign, Micro Focus Format (Leading Sign, LZ, Trailing Sign, ZD)*

First or Last Digit	Positive Value	Other Supported Positive Values	Negative Value	Other Supported Negative Values
0	{	0[?	}	] :!
1	A	1	J	
2	B	2	K	
3	C	3	L	
4	D	4	M	
5	E	5	N	
6	F	6	O	
7	G	7	P	
8	H	8	Q	
9	I	9	R	

Table 5. *Embedded Sign, Alternative Format (Leading Sign, LP, Trailing Sign, TP)*

Examples:

The number +123 in a 3-byte Micro Focus format, trailing embedded sign decimal field represented in hexadecimal:

31	32	33
----	----	----

The number -123 in a 3-byte Micro Focus format, leading embedded sign decimal field represented in hexadecimal:

71	32	33
----	----	----

The number +123 in a 3-byte alternative format, leading embedded sign decimal field represented in hexadecimal:

41	32	33
----	----	----

The number -123 in a 3-byte alternative format, trailing embedded sign decimal field represented in hexadecimal:

31	32	4C
----	----	----

Binary Floating Point

The floating point data type represents approximations to numeric quantities using the IEEE standard notation consisting of a sign, a biased exponent, and a fraction (*IEEE Standard for Binary Floating-Point Arithmetic, ANSI/IEEE Std 754-1985*). The sign is represented by the first bit; a sign bit of zero indicates a positive number and a sign bit of 1 indicates a negative number. The next bits contain the biased exponent in unsigned binary format. The true exponent value is equal to the biased exponent value minus the bias. For normal numbers, the range of values of the true exponent are $-(\text{bias} - 1)$ through bias. The remainder of the floating point data type is used to represent the fraction in unsigned binary format. The significand of the floating point number consists of an implicit leading bit to the left of an implicit binary radix point together with the fraction to the right of the radix point. The implicit leading bit for normal numbers has a value of 1. The value of a normal floating point number is the sign applied to the significand multiplied by 2 to the power of the value of the true exponent. A value of zero is represented by a biased exponent value of zero with a fraction value of zero and

either sign. A non-zero floating point number with a biased exponent value of zero is termed a 'denormalized' number and the implicit leading bit has a value of zero. The value of a denormalized floating point number is the sign applied to the significand multiplied by 2 to the power of $-(\text{bias} - 1)$. The first byte of a floating point number is on an arbitrary byte boundary.

Single precision A single precision, IEEE standard, floating point format has a sign bit, a biased exponent of 8 bits and a fraction of 23 bits, for a total length of 4-bytes. The bias value is +127.

Double precision A double precision, IEEE standard, floating point format has a sign bit, a biased exponent of 11 bits and a fraction of 52 bits, for a total length of 8 bytes. The bias value is +1023.

Example:

The number +12345.67 in a normalized single precision floating point field, on a big-endian machine, represented in hexadecimal:

46	40	E6	AE
----	----	----	----

The number -12345.67 in a normalized double precision floating point field, on a big-endian machine, represented in hexadecimal:

C0	C8	1C	D5	C2	8F	5C	29
----	----	----	----	----	----	----	----

Binary Signed Integer

Signed integer is a numeric data type stored in a binary format and represented in two's complement form. Thus positive numbers have a zero most significant bit and a negative number is represented by one greater than the bit by bit complement of its positive equivalent. A negative number always has a most significant bit of 1. The first byte of a signed integer number is on an arbitrary byte boundary.

Example:

The integer +123456 in a 4-byte signed binary field, on a big-endian machine, represented in hexadecimal:

00	01	E2	40
----	----	----	----

The integer -123456 in a 4-byte signed binary field, on a big-endian machine, represented in hexadecimal:

FF	FE	1D	C0
----	----	----	----

Binary Unsigned Integer

Unsigned integer is stored in binary format. All bit patterns are viewed as positive numbers. The first byte of an unsigned integer number is on an arbitrary byte boundary.

Example:

The integer 654321 in a 4-byte signed binary field, on a big-endian machine, represented in hexadecimal:

00	09	FB	F1
----	----	----	----

Bit

Each bit in the data occupies one bit and is either a *0* or a *1*. The first bit of a bit field is on an arbitrary bit boundary.

Example:

The byte represented in hexadecimal as 'F4' is an 8-bit field whose bit definition is B'11110100'.

Character

Each character in the data occupies one byte and is represented by its 8-bit ASCII code. The first byte of a character field is on an arbitrary byte boundary.

Example:

The string "Example" in a 7-byte character field represented in hexadecimal:

45	78	61	6D	70	6C	65
----	----	----	----	----	----	----

Date/Time

Date/time data consists of any combination of the following components: a year, a month, a day, an hour, a minute, a second, or text (including punctuation). The components can be in any order, and may not all be present.

year	2 or 4 decimal digits
month	1 or 2 decimal digits, full month name, abbreviated month name
day	1 or 2 decimal digits, full day-of-week name, abbreviated day-of-week name
hour	1 or 2 decimal digits
minute	1 or 2 decimal digits
second	1 or 2 decimal digits

The day-of-week names and month-of-year names are defined in the locale.

A decimal digit is represented by its 8-bit ASCII code. The minus sign is the ASCII minus. The plus sign is the ASCII plus. Each of the other characters occupies a single byte and is represented by its 8-bit ASCII code. The first byte of a date/time field is on an arbitrary byte boundary.

Example:

The date/time "6/1/2001 13:15:00 -0500" represented in hexadecimal:

36	2F	31	2F	32	30	30	31	20	31	33	3A	31
35	3A	30	30	20	2D	30	35	30	30			

Edited Numeric

An edited numeric number consists of optional leading spaces and/or tabs, each taking up one byte, followed by an optional minus sign character in a single byte, followed by decimal digits, each occupying a single byte. The digits may be interrupted by a radix character in a single byte. The decimal digits preceding the radix character may be interrupted by one or more thousands separators. A decimal digit is represented by its 8-bit ASCII code. The minus sign character is ASCII minus. An edited numeric value without a sign character is taken to be positive. The radix character is represented by the ASCII decimal point as determined from the locale. The thousands separator is represented by the ASCII value as determined from the locale. The first byte of an edited numeric number is on an arbitrary byte boundary.

Exceptions: Any value greater than X"9" in the numeric bits of a byte containing a digit is invalid. Any value other than X"3" in the zone bits of a byte containing a digit is invalid. Any sign byte containing a value other than X"2B" or X"2D" is invalid. Any byte containing a radix point or thousands separator with a value other than that represented in the locale is invalid. Any edited numeric value with more than one radix character is invalid.

Example:

The number 1123.123 in a 10-byte edited numeric field represented in hexadecimal:

20	20	31	31	32	33	2E	31	32	33
----	----	----	----	----	----	----	----	----	----

The number -1123.123 in a 10-byte edited numeric field represented in hexadecimal:

20	2D	31	31	32	33	2E	31	32	33
----	----	----	----	----	----	----	----	----	----

The number 1123.123 in a 10-byte edited numeric field represented in hexadecimal with thousands separator:

20	31	2C	31	32	33	2E	31	32	33
----	----	----	----	----	----	----	----	----	----

The number -1123.123 in a 10-byte edited numeric field represented in hexadecimal with thousands separator:

2D	31	2C	31	32	33	2E	31	32	33
----	----	----	----	----	----	----	----	----	----

Packed Decimal

Packed decimal is a numeric data type consisting of decimal digits and a sign. Each digit is represented by its 4-bit binary equivalent. Each byte in a packed decimal number contains two digits, except the last byte which contains the least significant digit and the sign. The sign is found in the last 4 bits of the number. A value of X"C" in the trailing 4 bits is recognized as indicating a positive number. A value of X"D" indicates a negative number. The first byte of a packed decimal number is on an arbitrary byte boundary.

Exceptions: Any 4-bit value greater than X"9" in a digit (non-sign) location is invalid. Any 4-bit value less than X"A" in the sign location is invalid.

Example:

The number +123 in a 2-byte packed decimal field represented in hexadecimal:

12	3C
----	----

The number -123 in a 2-byte packed decimal field represented in hexadecimal:

12	3D
----	----

Appendix B. Constants and Identifiers

Specifying Character String Constants

When you need to specify a character string as part of an option, you can use any of the following formats.

Double-Quoted Format

$$[repeat] = \begin{bmatrix} "a" \\ "X"xx" \\ "0"ooo" \end{bmatrix}$$

where

a is any printable character. To include a double quote character in a string, include two double quotes.

xx is the hex representation of any character (range 00 - FF). The first character in the string is represented by the first 2 hexadecimal digits, the second character by the next 2 digits, etc.

ooo is the octal representation of any character (range 000 - 377). The first character in the string is represented by the first 3 octal digits, the second character by the next 3 digits, etc.

repeat the number of times the string is repeated. For example, 2 "ABC" is equivalent to "ABCABC"; 3 X"FF" is equivalent to X"FFFFFF".

Single-Quoted Format

The single-quoted form contains ASCII characters, hexadecimal characters, characters in escaped notation and sub-field symbols. A sub-field symbol is only valid when the string represents a format string in a /DERIVEDFIELD option.

where

c

$=$

$[a$

$[\backslash xhh$

$[escaped_character]$

$[\backslash subfield_number]$

$escaped_character$

$=$

$[\backslash t]$

$[\backslash v]$

$[\backslash b]$

$[\backslash n]$

$[\backslash r]$

$[\backslash f]$

$[\backslash ']$

$[\backslash \backslash]$

horizontal tab

vertical tab

backspace

newline

carriage return

form feed

single quotation mark

backslash

- a

is a printable character. To include a single quotation mark or a backslash in a string, precede the character with a backslash.
- hh

is the hex representation of a character (range 00 - FF).
- $subfield_number$

the number of a sub-field defined by a parenthesized portion of a pattern. The maximum sub-field number is 9. See “Masks to Extract Sub-fields” on page E.15 for details.

Examples

- "sample"

a string of 6 characters
- x"73616D706C65"

a string of 6 characters equivalent to *sample*

<code>o"163141155160154145"</code>	a string of 6 characters equivalent to <i>sample</i>
<code>2 "sam"</code>	a string of 6 characters equivalent to <i>samsam</i>
<code>'sample'</code>	a string of 6 characters
<code>'\x73amp\x6C\x65'</code>	a string of 6 characters equivalent to <i>sample</i>
<code>'sam\tple'</code>	a string with <i>sam</i> separated from <i>ple</i> by a tab

Specifying Numeric Constants

When you need to specify a numeric constant as part of an option, you can use any one of the following formats:

Format

$$\begin{array}{l} [[\textit{sign}]9\dots9] \\ [[\textit{sign}]9\dots9.9\dots9] \\ [[\textit{sign}]9\dots9.9\dots9E[\textit{sign}]9\dots9] \end{array}$$

where

9...9 is one or more digits (range 0 - 9) with no intervening white space.

sign is either '+' or '-'.

You must use the first format when you are specifying an integer constant for comparison to any numeric field other than float or edited numeric or for insertion into a derived field of data type other than float or edited numeric.

Examples

1024

This is a positive integer constant.

-512

This is a negative integer constant.

+256

This is a positive integer constant.

75.03

This is a positive floating point constant.

-128.

This is a negative floating point constant.

.802

This is a positive floating point constant.

32E220

This is a positive floating point constant.

0.70133E-3

This is a positive floating point constant.

-21.35E32

This is a negative floating point constant.

Specifying Bit String Constants

When you need to specify a bit constant as part of an option, use the following format:

[repeat] B"b..."

where

b is a bit value 1 or 0

repeat the number of times the string is repeated. For example, 2 B"1001" is equivalent to B"10011001"

Examples

B"110"

B"10111001"

B"1111111100000000"

4 B"0001"

Identifiers

An identifier in SyncSort may contain letters and digits as well as the underscore and hyphen. The hyphen can neither be the first nor the last character. An identifier cannot consist entirely of digits. Also, an identifier cannot be longer than 32 characters. Uppercase and lowercase characters are considered equivalent. For example, "CUSTOMER", "customer" and "Customer", are treated as the same identifier.

Examples

Valid Identifiers

```
customer
Company
CONVERSION_IND
_temp
s1_help
2nd
daytime
X
zp
A-FIELD
```

Invalid Identifiers

```
this_is_a_much_too_long_identifier
```

This is an invalid identifier because it contains more than 32 characters.

```
conversion ind
```

An identifier must not contain embedded spaces.

```
201
```

An identifier must contain a non-digit character.

-X

First character is invalid.

tmp-

Last character is invalid.

Specifying Field Names

In many SyncSort options you need to refer by name to fields which are defined elsewhere. Such fields may be defined in a `/FIELDS`, `/RECORDLAYOUT`, `/DELIMITEDRECORDLAYOUT`, or `/DERIVEDFIELD` option (or in some cases an `/AGGREGATE` option) or in a COBOL copybook defined through a `/DATADictionary` option.

In the majority of cases, the field can be referred to by a simple, unqualified name such as *customer_name*, *date_received*, *total_amount*, *item_code*, *state*. In two circumstances the unqualified name may not be sufficient to identify a field unambiguously. The first is when the field is a subdivision of some higher level entity. This is the case in the `/RECORDLAYOUT` option when a field is an elementary field which is part of a composite field and when a COBOL elementary item is subordinate to a group item. The second circumstance when an unqualified name is not sufficient is when the field is defined as a repeated field, i.e. it is an array. This is the case for the `REPEAT` argument in the `/RECORDLAYOUT` option and the `OCCURS` clause in a COBOL record description.

Sub-fields

There is effectively no limit to the level of nesting of fields within other fields. The highest level field is referred to by its unqualified name. Fields at all other levels may be qualified. Take as an example the record layout shown in the following figure:

```

/RECORDLAYOUT
    emp_name    { first_name    char20,
                  middle_init   char1,
                  last_name     char20},    /* Employee's name          */
    home_address { street       char30,
                  city          char30,
                  state          char2,
                  zip            char9},    /* Employee's home address */
    telephone   { area_code     an 3,
                  tel_num       an 7 },    /* Employee's phone        */
    mail_address { street       char 30,
                  city          char 30,
                  state          char 2,
                  zip            char 9 },    /* Employee's mail address */
    ss_num       an 9,                    /* Social Security number  */
    manager      { mng_name      { first_name char 20,
                                  middle_init char 1,
                                  last_name  char 20}, /* Manager's name          */
                  telephone     { area_code     an 3,
                                  tel_num       an 7 }, /* Manager's phone        */
                  assigned       char6          /* Date of assignment      */

```

There is a single elementary field in this record layout which is not subordinate to other fields: **ss_num**, the social security number field. All other fields are either composite fields or elementary fields which are components of a composite field. Several fields are subordinate to more than one composite field, such as the manager's first name which is a component of **mng_name**, which in turn is a component of **manager**.

To refer to a field name in an option, you must provide a fully qualified name, or sufficient qualification that the name is not ambiguous. A fully qualified name consists of the name of the field and the names of composite fields of which it is a component, ordered from the highest level composite field to the field itself, left to right. The names are separated by a period. For example, the fully qualified name for the manager's first name is **manager.mng_name.first_name**. You could also refer to this field as **mng_name.first_name**, **manager.first_name** and **first_name**. However, **first_name** is not sufficiently qualified to distinguish the manager's first name from the employee's first name so this form of the name is not acceptable. The minimum qualification of names for each field in the above record layout follow:

```
emp_name.first_name
emp_name.middle_init
emp_name.last_name
home_address.street
home_address.city
home_address.state
home_address.zip
telephone.area_code
telephone.tel_num
mail_address.street
mail_address.city
mail_address.state
mail_address.zip
ss_num
mng_name.first_name
mng_name.middle_init
mng_name.last_name
manager.area_code
manager.tel_num
assigned
```

Arrays

All fields in an array have the same data type, the same length, and the same name, which is referred to here as the array name. Both elementary fields and composite fields can form arrays. SyncSort supports arrays of a fixed size, i.e. the number of fields in the array is the same in all input records described in a record layout. To reference a particular field in an array, you specify the name of the array followed by the number of the individual field, surrounded by brackets. The number portion of the field definition is called the subscript. The fields in the array are numbered sequentially from 1. Hence, the fifth field in the **daily_total** array is referenced by name as **daily_total[5]**. If you refer to a field with a subscript larger than the number of fields in the array, SyncSort treats this as an undefined field.

Since composite fields can be formed into arrays, it follows that arrays can occur inside arrays. The "multi-dimensioned" array complicates referencing individual fields. For a two dimensional array, you need to specify two subscripts to reference

an individual field in the innermost array, e.g. **trans_count[6,31]** . Referencing an individual field in a three dimensional array requires three subscripts, etc. The order for the subscripts is in descending order of composite level, from the outermost composite field.

Consider the record layout in the following example:

```
/RECORDLAYOUT
year_of_record      an 4,                /* year          */
location {location_name char 20,         /* sitelocation   */
  monthly_data {
    total_rain  uint 2,                /* monthlyrainfall */
    daily_data {
      max_temp  an 3,/* daily max temp */
      min_temp  an 3,/* daily min temp */
      rainfall  uint 2/* daily rainfall*/
    }repeat 31
  }repeat 12
}repeat 5
```

In this example, **location** is a one dimensional array with 5 elements, **monthly_data** is a two dimensional array with 5 * 12 elements, and **daily_data** is a three dimensional array with 5 * 12 * 31 elements. To reference the rainfall for the 21st day of the 3rd month at the 2nd location, you could specify **rainfall[2, 3, 21]** as the field name. The fully qualified field name is **location.monthly_data.daily_data.rainfall[2, 3, 21]** . The monthly rainfall for the 12th month at the 1st location is referenced by **total_rain[1, 12]** or **location.monthly_data.total_rain[1,12]** . The name of the 5th location is referenced by **location_name[5]** or **location.location_name[5]** .

Under some circumstances it is sensible to reference the complete array, i.e. all elements of the array. For example, if you wish to reformat a record containing an array, it is more convenient to specify the array as a single field in the /REFORMAT option rather than specifying each element in the array separately. To refer to the complete array, simply omit the subscript for the array. If an array is multi-dimensional, only the innermost array can be referenced as a unit by omitting the final subscript. So, for the record layout given above, **daily_data[2,3]** refers to all elements in the **daily_data** array for the 3rd month at the 2nd location; **monthly_data[5]** refers to all elements in the **monthly_data** array for the 5th location; **location** refers to all elements of the **location** array, i.e. the

complete record except for the **year_of_record** field. Subscripts cannot be omitted for a field which is not an array. For example, **rainfall[2,3]** is an invalid field reference and does not mean all rainfall values for the 3rd month at the 2nd location. Such elements are not contiguous within the record layout.

Components of Date/Time

In some cases you may want to reference only part of a date/time. For example, you may need to summarize transaction records in a file based on the month of the transaction. To do this, you would need to sort the file on the year and month components of the transaction date only. To refer to the full date/time field, use its unqualified name. To reference a component of a date/time field, qualify the field name with a name from the following table:

Name	Referenced component
DATE	the year + the month + the day
YEAR *	the year
MONTH	the month
DAY	the day of the month
TIME	the hour + the minute + the second
HOURL	the hour
MINUTE	the minute
SECOND	the second

Table 1. Components of Date/Time

Notes:

**When used in a comparison, a 2-digit year is evaluated according to the century window specified for the application in the /CENTURYWINDOWSTART option, or on the window 0 to 99 when one is not specified.*

For example, **transaction_date** is a date/time field defined as follows:

```
/field transaction_date 4: - 4: datetime (mm/dd/year hr:mi am)
```

You can reference the month, day, year, hour, and minute components of this field separately, in addition to the date component and the time component, as shown in the following table. The examples are based on a field containing the date/time 5/12/2001 10:23 pm.

		Value of component	
Name	Associated element(s)	For comparison	For display purposes
transaction_date	mm dd year hr mi am	200105122223	5/12/2001 10:23 pm
transaction_date.date	mm dd year	20010512	5/12/2001
transaction_date.year	year	2001	2001
transaction_date.month	mm	05	5
transaction_date.day	dd	12	12
transaction_date.time	hr mi am	2223	10:23 pm
transaction_date.hour	hr am	22	10 pm
transaction_date.minute	mi	23	23
transaction_date.second	invalid name	n/a	n/a

If a date/time field contains two components of the same type, a reference of that component retrieves the first occurrence of the component.

Referencing a composite (**date, time, hour**) for a use other than a comparison gives a result that depends on the order in which components appear in the field. The components contributing to the composite appear in the result in the order in which they appear in the field. If contributing components are adjacent (no intervening non-contributing component), any punctuation between the adjacent components is included in the result. If two components are not adjacent, a single space appears between them in the result. For example, a date/time field is defined as follows:

```
/field end_date 2: - 2: datetime (year hh:mi mm/dd)
```

The **end_date.date** component for the field containing 2001 14:15 3/22 is 20010322 for use in a comparison, and 2001 3/22 otherwise. The year and

month components (contributing) are separated by the hour and minute components (non-contributing), so the result contains a space between the year and month values.

Appendix C. Collating Sequences

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
NUL	Null character	00	000	000
SOH	Start of heading	01	001	001
STX	Start of text	02	002	002
ETX	End of text	03	003	003
EOT	End of transmission	04	004	004
ENQ	Enquiry	05	005	005
ACK	Positive acknowledgement	06	006	006
BEL	Bell	07	007	007
BS	Backspace	08	010	008
HT	Horizontal tabulation	09	011	009
LF	Line feed	0A	012	010
VT	Vertical tabulation	0B	013	011
FF	Form feed	0C	014	012
CR	Carriage return	0D	015	013
LS1	Locking shift one	0E	016	014
LS0	Locking shift zero	0F	017	015
DLE	Data link escape	10	020	016
DC1	Device control 1	11	021	017
DC2	Device control 2	12	022	018
DC3	Device control 3	13	023	019
DC4	Device control 4	14	024	020
NAK	Negative acknowledgement	15	025	021
SYN	Synchronous idle	16	026	022
ETB	End of transmission block	17	027	023
CAN	Cancel	18	030	024
EM	End of medium	19	031	025

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
SUB	Substitute	1A	032	026
ESC	Escape	1B	033	027
FS	Field separator	1C	034	028
GS	Group separator	1D	035	029
RS	Record separator	1E	036	030
US	Unit separator	1F	037	031
SP	Space	20	040	032
!	Exclamation point	21	041	033
"	Quotation marks (double quote)	22	042	034
#	Number sign	23	043	035
\$	Dollar sign	24	044	036
%	Percent sign	25	045	037
&	Ampersand	26	046	038
'	Apostrophe (single quote)	27	047	039
(	Opening parenthesis	28	050	040
)	Closing parenthesis	29	051	041
*	Asterisk	2A	052	042
+	Plus	2B	053	043
,	Comma	2C	054	044
-	Hyphen or minus	2D	055	045
.	Period or decimal point	2E	056	046
/	Slash	2F	057	047
0	Zero	30	060	048
1	One	31	061	049
2	Two	32	062	050
3	Three	33	063	051
4	Four	34	064	052
5	Five	35	065	053
6	Six	36	066	054
7	Seven	37	067	055
8	Eight	38	070	056

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
9	Nine	39	071	057
:	Colon	3A	072	058
;	Semicolon	3B	073	059
<	Less than	3C	074	060
=	Equals	3D	075	061
>	Greater than	3E	076	062
?	Question mark	3F	077	063
@	Commercial at	40	100	064
A	Uppercase A	41	101	065
B	Uppercase B	42	102	066
C	Uppercase C	43	103	067
D	Uppercase D	44	104	068
E	Uppercase E	45	105	069
F	Uppercase F	46	106	070
G	Uppercase G	47	107	071
H	Uppercase H	48	110	072
I	Uppercase I	49	111	073
J	Uppercase J	4A	112	074
K	Uppercase K	4B	113	075
L	Uppercase L	4C	114	076
M	Uppercase M	4D	115	077
N	Uppercase N	4E	116	078
O	Uppercase O	4F	117	079
P	Uppercase P	50	120	080
Q	Uppercase Q	51	121	081
R	Uppercase R	52	122	082
S	Uppercase S	53	123	083
T	Uppercase T	54	124	084
U	Uppercase U	55	125	085
V	Uppercase V	56	126	086
W	Uppercase W	57	127	087

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
X	Uppercase X	58	130	088
Y	Uppercase Y	59	131	089
Z	Uppercase Z	5A	132	090
[	Opening Bracket	5B	133	091
\	Backslash	5C	134	092
]	Closing bracket	5D	135	093
^	Circumflex	5E	136	094
_	Underline (underscore)	5F	137	095
`	Grave accent	60	140	096
a	Lowercase a	61	141	097
b	Lowercase b	62	142	098
c	Lowercase c	63	143	099
d	Lowercase d	64	144	100
e	Lowercase e	65	145	101
f	Lowercase f	66	146	102
g	Lowercase g	67	147	103
h	Lowercase h	68	150	104
i	Lowercase i	69	151	105
j	Lowercase j	6A	152	106
k	Lowercase k	6B	153	107
l	Lowercase l	6C	154	108
m	Lowercase m	6D	155	109
n	Lowercase n	6E	156	110
o	Lowercase o	6F	157	111
p	Lowercase p	70	160	112
q	Lowercase q	71	161	113
r	Lowercase r	72	162	114
s	Lowercase s	73	163	115
t	Lowercase t	74	164	116
u	Lowercase u	75	165	117
v	Lowercase v	76	166	118

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
w	Lowercase w	77	167	119
x	Lowercase x	78	170	120
y	Lowercase y	79	171	121
z	Lowercase z	7A	172	122
{	Opening brace	7B	173	123
	Vertical line	7C	174	124
}	Closing brace	7D	175	125
~	Tilde	7E	176	126
DEL	Delete	7F	177	127
		80	200	128
		81	201	129
		82	202	130
		83	203	131
IND	Index	84	204	132
NEL	Next line	85	205	133
SSA	Start of selected area	86	206	134
ESA	End of selected area	87	207	135
HTS	Horizontal tab set	88	210	136
HTJ	Horizontal tab set with justification	89	211	137
VTs	Vertical tab set	8A	212	138
PLD	Partial line down	8B	213	139
PLU	Partial line up	8C	214	140
RI	Reverse Index	8D	215	141
SS2	Single shift 2	8E	216	142
SS3	Single shift 3	8F	217	143
DCS	Device control string	90	220	144
PU1	Private use 1	91	221	145
PU2	Private use 2	92	222	146
STS	Set transmit state	93	223	147
CCH	Cancel character	94	224	148
MW	Message waiting	95	225	149

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
SPA	Start of protected area	96	226	150
EPA	End of protected area	97	227	151
SOS	Start of significance	98	230	152
		99	231	153
SCI	Single character introducer	9A	232	154
CSI	Control sequence indicator	9B	233	155
ST	String terminator	9C	234	156
OSC	Operating system command	9D	235	157
PM	Privacy message	9E	236	158
APC	Application	9F	237	159
NBSP	No-break space	A0	240	160
¡	Inverted exclamation mark	A1	241	161
¢	Cent sign	A2	242	162
£	Pound sign	A3	243	163
¤	General currency sign	A4	244	164
¥	Yen sign	A5	245	165
¦	Broken Bar	A6	246	166
§	Section sign	A7	247	167
¨	Dieresis	A8	250	168
©	Copyright sign	A9	251	169
ª	Feminine ordinal indicator	AA	252	170
«	Angle quotation mark left	AB	253	171
¬	Logical not	AC	254	172
SHY	Soft hyphen	AD	255	173
®	Registered trademark	AE	256	174
—	Macron	AF	257	175
°	Degree sign	B0	260	176
±	Plus or minus sign	B1	261	177
²	Superscript 2	B2	262	178
³	Superscript 3	B3	263	179
´	Acute accent	B4	264	180

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
µ	Micro sign	B5	265	181
¶	Paragraph sign, pilcrow	B6	266	182
·	Middle dot	B7	267	183
¸	Cedilla	B8	270	184
¹	Superscript 1	B9	271	185
º	Masculine ordinal sign	BA	272	186
»	Angle quotation mark right	BB	273	187
¼	Fraction one quarter	BC	274	188
½	Fraction one half	BD	275	189
¾	Fraction three quarters	BE	276	190
¿	Inverted question mark	BF	277	191
À	Uppercase A with grave accent	C0	300	192
Â	Uppercase A with circumflex	C2	302	194
Ã	Uppercase A with tilde	C3	303	195
Ä	Uppercase A with umlaut	C4	304	196
Å	Uppercase A with ring	C5	305	197
Æ	Uppercase AE diphthong	C6	306	198
Ç	Uppercase C with cedilla	C7	307	199
È	Uppercase E with grave accent	C8	310	200
É	Uppercase E with acute accent	C9	311	201
Ê	Uppercase E with circumflex	CA	312	202
Ë	Uppercase E with umlaut	CB	313	203
Ì	Uppercase I with grave accent	CC	314	204
Í	Uppercase I with acute accent	CD	315	205

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
Î	Uppercase I with circumflex	CE	316	206
Ï	Uppercase I with umlaut	CF	317	207
Ð	Uppercase eth	D0	320	208
Ñ	Uppercase N with tilde	D1	321	209
Ò	Uppercase O with grave accent	D2	322	210
Ó	Uppercase O with acute accent	D3	323	211
Ô	Uppercase O with circumflex	D4	324	212
Õ	Uppercase O with tilde	D5	325	213
Ö	Uppercase O with umlaut	D6	326	214
		D7	327	215
Ø	Uppercase O with slash	D8	330	216
Ù	Uppercase U with grave accent	D9	331	217
Ú	Uppercase U with acute accent	DA	332	218
Û	Uppercase U with circumflex	DB	333	219
Ü	Uppercase U with umlaut	DC	334	220
Ý	Uppercase Y with acute accent	DD	335	221
Þ	Uppercase thorn	DE	336	222
ß	German lowercase sharp s	DF	337	223
à	Lowercase a with grave accent	E0	340	224
á	Lowercase a with acute accent	E1	341	225
â	Lowercase a with circumflex	E2	342	226
ã	Lowercase a with tilde	E3	343	227
ä	Lowercase a with umlaut	E4	344	228
å	Lowercase a with ring	E5	345	229
æ	Lowercase ae diphthong	E6	346	230
ç	Lowercase c with cedilla	E7	347	231
è	Lowercase e with grave accent	E8	350	232
é	Lowercase e with acute accent	E9	351	233
ê	Lowercase e with circumflex	EA	352	234
ë	Lowercase e with umlaut	EB	353	235

ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
ì	Lowercase i with grave accent	EC	354	236
í	Lowercase i with acute accent	ED	355	237
î	Lowercase i with circumflex	EE	356	238
ï	Lowercase i with umlaut	EF	357	239
ð	Lowercase eth	F0	360	240
ñ	Lowercase n with tilde	F1	361	241
ò	Uppercase o with grave accent	F2	362	242
ó	Lowercase o with acute accent	F3	363	243
ô	Lowercase o with circumflex	F4	364	244
õ	Lowercase o with tilde	F5	365	245
ö	Lowercase o with umlaut	F6	366	246
		F7	367	247
ø	Lowercase o with slash	F8	370	248
ù	Lowercase u with grave accent	F9	371	249
ú	Lowercase u with acute accent	FA	372	250
û	Lowercase u with circumflex	FB	373	251
ü	Lowercase u with umlaut	FC	374	252
ý	Lowercase y with acute accent	FD	375	253
þ	Lowercase thorn	FE	376	254
ÿ	Lowercase y with umlaut	FF	377	255

Folded ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
NUL	Null character	00	000	000
SOH	Start of heading	01	001	001
STX	Start of text	02	002	002
ETX	End of text	03	003	003
EOT	End of transmission	04	004	004
ENQ	Enquiry	05	005	005
ACK	Positive acknowledgement	06	006	006
BEL	Bell	07	007	007
BS	Backspace	08	010	008
HT	Horizontal tabulation	09	011	009
LF	Line feed	0A	012	010
VT	Vertical tabulation	0B	013	011
FF	Form feed	0C	014	012
CR	Carriage return	0D	015	013
LS1	Locking shift one	0E	016	014
LS0	Locking shift zero	0F	017	015
DLE	Data link escape	10	020	016
DC1	Device control 1	11	021	017
DC2	Device control 2	12	022	018
DC3	Device control 3	13	023	019
DC4	Device control 4	14	024	020
NAK	Negative acknowledgement	15	025	021
SYN	Synchronous idle	16	026	022
ETB	End of transmission block	17	027	023
CAN	Cancel	18	030	024
EM	End of medium	19	031	025
SUB	Substitute	1A	032	026
ESC	Escape	1B	033	027
FS	Field separator	1C	034	028
GS	Group separator	1D	035	029

Folded ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
RS	Record separator	1E	036	030
US	Unit separator	1F	037	031
SP	Space	20	040	032
!	Exclamation point	21	041	033
"	Quotation marks (double quote)	22	042	034
#	Number sign	23	043	035
\$	Dollar sign	24	044	036
%	Percent sign	25	045	037
&	Ampersand	26	046	038
'	Apostrophe (single quote)	27	047	039
(	Opening parenthesis	28	050	040
)	Closed parenthesis	29	051	041
*	Asterisk	2A	052	042
+	Plus	2B	053	043
,	Comma	2C	054	044
-	Hyphen or minus	2D	055	045
.	Period or decimal point	2E	056	046
/	Slash	2F	057	047
0	Zero	30	060	048
1	One	31	061	049
2	Two	32	062	050
3	Three	33	063	051
4	Four	34	064	052
5	Five	35	065	053
6	Six	36	066	054
7	Seven	37	067	055
8	Eight	38	070	056
9	Nine	39	071	057
:	Colon	3A	072	058
;	Semicolon	3B	073	059

Folded ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
<	Less than	3C	074	060
=	Equals	3D	075	061
>	Greater than	3E	076	062
?	Question mark	3F	077	063
@	Commercial at	40	100	064
A,a	Uppercase A and lowercase a	41,61	101,141	065,097
B,b	Uppercase B and lowercase b	42,62	102,142	066,098
C,c	Uppercase C and lowercase c	43,63	103,143	067,099
D,d	Uppercase D and lowercase d	44,64	104,144	068,100
E,e	Uppercase E and lowercase e	45,65	105,145	069,101
F,f	Uppercase F and lowercase f	46,66	106,146	070,102
G,g	Uppercase G and lowercase g	47,67	107,147	071,103
H,h	Uppercase H and lowercase h	48,68	110,150	072,104
I,i	Uppercase I and lowercase i	49,69	111,151	073,105
J,j	Uppercase J and lowercase j	4A,6A	112,152	074,106
K,k	Uppercase K and lowercase k	4B,6B	113,153	075,107
L,l	Uppercase L and lowercase l	4C,6C	114,154	076,108
M,m	Uppercase M and lowercase m	4D,6D	115,155	077,109
N,n	Uppercase N and lowercase n	4E,6E	116,156	078,110
O,o	Uppercase O and lowercase o	4F,6F	117,157	079,111
P,p	Uppercase P and lowercase p	50,70	120,160	080,112
Q,q	Uppercase Q and lowercase q	51,71	121,161	081,113
R,r	Uppercase R and lowercase r	52,72	122,162	082,114
S,s	Uppercase S and lowercase s	53,73	123,163	083,115
T,t	Uppercase T and lowercase t	54,74	124,164	084,116
U,u	Uppercase U and lowercase u	55,75	125,165	085,117
V,v	Uppercase V and lowercase v	56,76	126,166	086,118
W,w	Uppercase W and lowercase w	57,77	127,167	087,119
X,x	Uppercase X and lowercase x	58,78	130,170	088,120
Y,y	Uppercase Y and lowercase y	59,79	131,171	089,121

Folded ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
Z,z	Uppercase Z and lowercase z	5A,7A	132,172	090,122
[	Opening bracket	5B	133	091
\	Backslash	5C	134	092
]	Closing bracket	5D	135	093
^	Circumflex	5E	136	094
_	Underline (underscore)	5F	137	095
`	Grave accent	60	140	096
{	Opening brace	7B	173	123
	Vertical line	7C	174	124
}	Closing brace	7D	175	125
~	Tilde	7E	176	126
DEL	Delete	7F	177	127
		80	200	128
		81	201	129
		82	202	130
		83	203	131
IND	Index	84	204	132
NEL	Next line	85	205	133
SSA	Start of selected area	86	206	134
ESA	End of selected area	87	207	135
HTS	Horizontal tab set	88	210	136
HTJ	Horizontal tab set with justification	89	211	137
VTs	Vertical tab set	8A	212	138
PLD	Partial line down	8B	213	139
PLU	Partial line up	8C	214	140
RI	Reverse index	8D	215	141
SS2	Single shift 2	8E	216	142
SS3	Single shift 3	8F	217	143
DCS	Device control string	90	220	144
PU1	Private use 1	91	221	145

Folded ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
PU2	Private use 2	92	222	146
STS	Set transmit state	93	223	147
CCH	Cancel character	94	224	148
MW	Message waiting	95	225	149
SPA	Start of protected area	96	226	150
EPA	End of protected area	97	227	151
SOS	Start of significance	98	230	152
		99	231	153
SCI	Single character introducer	9A	232	154
CSI	Control sequence indicator	9B	233	155
ST	String terminator	9C	234	156
OSC	Operating system command	9D	235	157
PM	Privacy message	9E	236	158
APC	Application	9F	237	159
NBSP	No-break space	A0	240	160
¡	Inverted exclamation mark	A1	241	161
¢	Cent sign	A2	242	162
£	Pound sign	A3	243	163
¤	General currency sign	A4	244	164
¥	Yen sign	A5	245	165
¦	Broken bar	A6	246	166
§	Section sign	A7	247	167
¨	Dieresis	A8	250	168
©	Copyright sign	A9	251	169
ª	Feminine ordinal indicator	AA	252	170
«	Angle quotation mark left	AB	253	171
¬	Logical not	AC	254	172
SHY	Soft Hyphen	AD	255	173
®	Registered trademark	AE	256	174
—	Macron	AF	257	175
°	Degree sign	B0	260	176

Folded ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
±	Plus or minus sign	B1	261	177
²	Superscript 2	B2	262	178
³	Superscript 3	B3	263	179
´	Acute accent	B4	264	180
µ	Micro sign	B5	265	266
¶	Paragraph sign, pilcrow	B6	266	182
·	Middle dot	B7	267	183
¸	Cedilla	B8	270	184
¹	Superscript 1	B9	271	185
º	Masculine ordinal sign	BA	272	186
»	Angle quotation mark right	BB	273	187
¼	Fraction one quarter	BC	274	188
½	Fraction one half	BD	275	189
¾	Fraction three quarters	BE	276	190
¿	Inverted question mark	BF	277	191
À,à	A and a with grave accent	C0,E0	300,340	192,224
Á,á	A and a with acute accent	C1,E1	301,341	193,225
Â,â	A and a with circumflex	C2,E2	302,342	194,226
Ã,ã	A and a with tilde	C3,E3	303,343	195,227
Ä,ä	A and a with umlaut	C4,E4	302,344	196,228
Å,å	A and a with ring	C5,E5	305,345	197,229
Æ,æ	AE and ae diphthong	C6,E6	306,346	198,230
Ç,ç	C and c with cedilla	C7,E7	307,347	199,231
È,è	E and e with grave accent	C8,E8	310,350	200,232
É,é	E and e with acute accent	C9,E9	311,351	201,233
Ê,ê	E and e with circumflex	CA,EA	312,352	202,234
Ë,ë	E and e with umlaut	CB,EB	313,353	203,235
Ì,ì	I and i with grave accent	CC,EC	314,354	204,236
Í,í	I and i with acute accent	CD,ED	315,355	205,237
Î,î	I and i with circumflex	CE,EE	316,356	206,238
Ï,ï	I and i with umlaut	CF,EF	317,357	207,239

Folded ASCII Collating Sequence

Char.	Description	Hex	Octal	Decimal
	Uppercase and lowercase eth	D0,F0	320,360	208,240
Ñ,ñ	N and n with tilde	D1,F1	321,361	209,241
Ò,ò	O and o with grave accent	D2,F2	322,362	210,242
Ó,ó	O and o with acute accent	D3,F3	323,363	211,243
Ô,ô	O and o with circumflex	D4,F4	324,364	212,244
Õ,õ	O and o with tilde	D5,F5	325,365	213,245
Ö,ö	O and o with umlaut	D6,F6	326,366	214,246
		D7	327	215
Ø,ø	O and o with slash	D8,F8	330,370	216,248
Ù,ù	U and u with grave accent	D9,F9	331,371	217,249
Ú,ú	U and u with acute accent	DA,FA	332,372	218,250
Û,û	U and u with circumflex	DB,FB	333,373	219,251
Ü,ü	U and u with umlaut	DC,FC	334,374	220,252
	Y and y with acute accent	DD,FD	335,375	221,253
	Uppercase and lowercase thorn	DE,FE	336,376	222,254
ß	German lowercase sharp s	DF	337	223
		F7	367	247
ÿ	Lowercase y with umlaut	FF	377	255

EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
NUL	Null character	00	000	000
SOH	Start of heading	01	001	001
STX	Start of text	02	002	002
ETX	End of Text	03	003	003
HT	Horizontal tabulation	09	011	009
DEL	Delete	7F	177	127
VT	Vertical tabulation	0B	013	011
FF	Form feed	0C	014	012
CR	Carriage return	0D	015	013
SO	Shift out	0E	016	014
SI	Shift in	0F	017	015
DLE	Data link escape	10	020	016
DC1	Device control 1	11	021	017
DC2	Device control 2	12	022	018
DC3	Device control 3	13	023	019
BS	Backspace	08	010	008
CAN	Cancel	18	030	024
EM	End of medium	19	031	025
FS	Field separator	1C	034	028
GS	Group separator	1D	035	029
RS	Record separator	1E	036	030
US	Unit separator	1F	037	031
LF	Line feed	0A	012	010
ETB	End of transmission block	17	027	023
ESC	Escape	1B	033	027
ENQ	Enquiry	05	005	005
ACK	Positive acknowledgement	06	006	006
BEL	Bell	07	007	007
SYN	Synchronous idle	16	026	022

EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
EOT	End of transmission	04	004	004
DC4	Device control 4	14	024	020
NAK	Negative acknowledgement	15	025	021
SUB	Substitute	1A	032	026
SP	Space	20	040	032
[	Opening bracket	5B	133	091
.	Period or decimal point	2E	056	046
<	Less than	3C	074	060
(	Opening parenthesis	28	050	040
+	Plus	2B	053	043
!	Exclamation point	21	041	033
&	Ampersand	26	046	038
]	Closing bracket	5D	135	093
\$	Dollar sign	24	044	036
*	Asterisk	2A	052	042
)	Closing parenthesis	29	051	041
;	Semicolon	3B	073	059
^	Circumflex	5E	136	094
-	Hyphen or minus	2D	055	045
/	Slash	2F	057	047
	Vertical line	7C	174	124
,	Comma	2C	054	044
%	Percent sign	25	045	037
_	Underline (underscore)	5F	137	095
>	Greater than	3E	076	062
?	Question mark	3F	077	063
`	Grave accent	60	140	096
:	Colon	3A	072	058
#	Number sign	23	043	035
@	Commercial at	40	100	064

EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
'	Apostrophe (single quote)	27	047	039
=	Equals	3D	075	061
"	Quotation marks (double quote)	22	042	034
a	Lowercase a	61	141	097
b	Lowercase b	62	142	098
c	Lowercase c	63	143	099
d	Lowercase d	64	144	100
e	Lowercase e	65	145	101
f	Lowercase f	66	146	102
g	Lowercase g	67	147	103
h	Lowercase h	68	150	104
i	Lowercase i	69	151	105
j	Lowercase j	6A	152	106
k	Lowercase k	6B	153	107
l	Lowercase l	6C	154	108
m	Lowercase m	6D	155	109
n	Lowercase n	6E	156	110
o	Lowercase o	6F	157	111
p	Lowercase p	70	160	112
q	Lowercase q	71	161	113
r	Lowercase r	72	162	114
~	Tilde	7E	176	126
s	Lowercase s	73	163	115
t	Lowercase t	74	164	116
u	Lowercase u	75	165	117
v	Lowercase v	76	166	118
w	Lowercase w	77	167	119
x	Lowercase x	78	170	120
y	Lowercase y	79	171	121
z	Lowercase z	7A	172	122

EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
{	Opening brace	7B	173	123
A	Uppercase A	41	101	065
B	Uppercase B	42	102	066
C	Uppercase C	43	103	067
D	Uppercase D	44	104	068
E	Uppercase E	45	105	069
F	Uppercase F	46	106	070
G	Uppercase G	47	107	071
H	Uppercase H	48	110	072
I	Uppercase I	49	111	073
}	Closing brace	7D	175	125
J	Uppercase J	4A	112	074
K	Uppercase K	4B	113	075
L	Uppercase L	4C	114	076
M	Uppercase M	4D	115	077
N	Uppercase N	4E	116	078
O	Uppercase O	4F	117	079
P	Uppercase P	50	120	080
Q	Uppercase Q	51	121	081
R	Uppercase R	52	122	082
\	Backslash	5C	134	092
S	Uppercase S	53	123	083
T	Uppercase T	54	124	084
U	Uppercase U	55	125	085
V	Uppercase V	56	126	086
W	Uppercase W	57	127	087
X	Uppercase X	58	130	088
Y	Uppercase Y	59	131	089
Z	Uppercase Z	5A	132	090
0	Zero	30	060	048

EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
1	One	31	061	049
2	Two	32	062	050
3	Three	33	063	051
4	Four	34	064	052
5	Five	35	065	053
6	Six	36	066	054
7	Seven	37	067	055
8	Eight	38	070	056
9	Nine	39	071	057
		FF	377	255

Notes

Any character not defined in the above table will collate at the position of the SUB character.

Folded EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
NUL	Null character	00	000	000
SOH	Start of heading	01	001	001
STX	Start of text	02	002	002
ETX	End of Text	03	003	003
HT	Horizontal tabulation	09	011	009
DEL	Delete	7F	177	127
VT	Vertical tabulation	0B	013	011
FF	Form feed	0C	014	012
CR	Carriage return	0D	015	013
SO	Shift out	0E	016	014
SI	Shift in	0F	017	015
DLE	Data link escape	10	020	016
DC1	Device control 1	11	021	017
DC2	Device control 2	12	022	018
DC3	Device control 3	13	023	019
BS	Backspace	08	010	008
CAN	Cancel	18	030	024
EM	End of medium	19	031	025
FS	Field separator	1C	034	028
GS	Group separator	1D	035	029
RS	Record separator	1E	036	030
US	Unit separator	1F	037	031
LF	Line feed	0A	012	010
ETB	End of transmission block	17	027	023
ESC	Escape	1B	033	027
ENQ	Enquiry	05	005	005
ACK	Positive acknowledgement	06	006	006
BEL	Bell	07	007	007
SYN	Synchronous idle	16	026	022

Folded EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
EOT	End of transmission	04	004	004
DC4	Device control 4	14	024	020
NAK	Negative acknowledgement	15	025	021
SUB	Substitute	1A	032	026
SP	Space	20	040	032
[	Opening bracket	5B	133	091
.	Period or decimal point	2E	056	046
<	Less than	3C	074	060
(	Opening parenthesis	28	050	040
+	Plus	2B	053	043
!	Exclamation point	21	041	033
&	Ampersand	26	046	038
]	Closing bracket	5D	135	093
\$	Dollar sign	24	044	036
*	Asterisk	2A	052	042
)	Closing parenthesis	29	051	041
;	Semicolon	3B	073	059
^	Circumflex	5E	136	094
-	Hyphen or minus	2D	055	045
/	Slash	2F	057	047
	Vertical line	7C	174	124
,	Comma	2C	054	044
%	Percent sign	25	045	037
_	Underline (underscore)	5F	137	095
>	Greater than	3E	076	062
?	Question mark	3F	077	063
`	Grave accent	60	140	096
:	Colon	3A	072	058
#	Number sign	23	043	035
@	Commercial at	40	100	064

Folded EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
'	Apostrophe (single quote)	27	047	039
=	Equals	3D	075	061
"	Quotation marks (double quote)	22	042	034
~	Tilde	7E	176	126
{	Opening brace	7B	173	123
A,a	Uppercase A and lowercase a	41,61	101,141	065,097
B,b	Uppercase B and lowercase b	42,62	102,142	066,098
C,c	Uppercase C and lowercase c	43,63	103,143	067,099
D,d	Uppercase D and lowercase d	44,64	104,144	068,100
E,e	Uppercase E and lowercase e	45,65	105,145	069,101
F,f	Uppercase F and lowercase f	46,66	106,146	070,102
G,g	Uppercase G and lowercase g	47,67	107,147	071,103
H,h	Uppercase H and lowercase h	48,68	110,150	072,104
I,i	Uppercase I and lowercase i	49,69	111,151	073,105
}	Closing brace	7D	175	125
J,j	Uppercase J and lowercase j	4A,6A	112,152	074,106
K,k	Uppercase K and lowercase k	4B,6B	113,153	075,107
L,l	Uppercase L and lowercase l	4C,6C	114,154	076,108
M,m	Uppercase M and lowercase m	4D,6D	115,155	077,109
N,n	Uppercase N and lowercase n	4E,6E	116,156	078,110
O,o	Uppercase O and lowercase o	4F,6F	117,157	079,111
P,p	Uppercase P and lowercase p	50,70	120,160	080,112
Q,q	Uppercase Q and lowercase q	51,71	121,161	081,113
R,r	Uppercase R and lowercase r	52,72	122,162	082,114
\	Backslash	5C	134	092
S,s	Uppercase S and lowercase s	53,73	123,163	083,115
T,t	Uppercase T and lowercase t	54,74	124,164	084,116
U,u	Uppercase U and lowercase u	55,75	125,165	085,117
V,v	Uppercase V and lowercase v	56,76	126,166	086,118
W,w	Uppercase W and lowercase w	57,77	127,167	087,119

Folded EBCDIC Collating Sequence

Char.	Description	Hex	Octal	Decimal
X,x	Uppercase X and lowercase x	58,78	130,170	088,120
Y,y	Uppercase Y and lowercase y	59,79	131,171	089,121
Z,z	Uppercase Z and lowercase z	5A,7A	132,172	090,122
0	Zero	30	060	048
1	One	31	061	049
2	Two	32	062	050
3	Three	33	063	051
4	Four	34	064	052
5	Five	35	065	053
6	Six	36	066	054
7	Seven	37	067	055
8	Eight	38	070	056
9	Nine	39	071	057
		FF	377	255

Notes

Any character not defined in the above table will collate at the position of the SUB character.

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
NUL	Null character	00	000	000
SOH	Start of heading	01	001	001
STX	Start of text	02	002	002
ETX	End of text	03	003	003
EOT	End of transmission	04	004	004
ENQ	Enquiry	05	005	005
ACK	Positive acknowledgement	06	006	006
BEL	Bell	07	007	007
BS	Backspace	08	010	008
HT	Horizontal tabulation	09	011	009
LF	Line feed	0A	012	010
VT	Vertical tabulation	0B	013	011
FF	Form feed	0C	014	012
CR	Carriage return	0D	015	013
SO	Shift out	0E	016	014
SI	Shift in	0F	017	015
DLE	Data link escape	10	020	016
DC1	Device control 1	11	021	017
DC2	Device control 1	12	022	018
DC3	Device control 3	13	023	019
DC4	Device control 4	14	024	020
NAK	Negative acknowledgement	15	025	021
SYN	Synchronous idle	16	026	022
ETB	End of transmission block	17	027	023
CAN	Cancel	18	030	024
EM	End of medium	19	031	025
SUB	Substitute	1A	032	026
ESC	Escape	1B	033	027
FS	File separator	1C	034	028
GS	Group separator	1D	035	029

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
RS	Record separator	1E	036	030
US	Unit separator	1F	037	031
SP	Space	20	040	032
!	Exclamation point	21	041	033
"	Quotation marks (double quote)	22	042	034
#	Number sign	23	043	035
\$	Dollar sign	24	044	036
%	Percent sign	25	045	037
&	Ampersand	26	046	038
'	Apostrophe (single quote)	27	047	039
(	Opening parenthesis	28	050	040
)	Closing parenthesis	29	051	041
*	Asterisk	2A	052	042
+	Plus	2B	053	043
,	Comma	2C	054	044
-	Hyphen or minus	2D	055	045
.	Period or decimal point	2E	056	046
/	Slash	2F	057	047
0	Zero	30	060	048
1	One	31	061	049
2	Two	32	062	050
3	Three	33	063	051
4	Four	34	064	052
5	Five	35	065	053
6	Six	36	066	054
7	Seven	37	067	055
8	Eight	38	070	056
9	Nine	39	071	057
:	Colon	3A	072	058
;	Semicolon	3B	073	059

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
<	Less than	3C	074	060
=	Equals	3D	075	061
>	Greater than	3E	076	062
?	Question mark	3F	077	063
@	Commercial at	40	100	064
a	Lowercase a	61	141	097
A	Uppercase A	41	101	065
à	Lowercase a with grave accent	E0	340	224
À	Uppercase A with grave accent	C0	300	192
á	Lowercase a with acute accent	E1	341	225
Á	Uppercase A with acute accent	C1	301	193
â	Lowercase a with circumflex	E2	342	226
Â	Uppercase A with circumflex	C2	302	194
ã	Lowercase a with tilde	E3	343	227
Ã	Uppercase A with tilde	C3	303	195
ä	Lowercase a with umlaut	E4	344	228
Ä	Uppercase A with umlaut	C4	304	196
b	Lowercase b	62	142	098
B	Uppercase B	42	102	066
c	Lowercase c	63	143	099
C	Uppercase C	43	103	067
ç	Lowercase c with cedilla	E7	347	231
Ç	Uppercase C with cedilla	C7	307	199
d	Lowercase d	64	144	100
D	Uppercase D	44	104	068
e	Lowercase e	65	145	101
E	Uppercase E	45	105	069
è	Lowercase e with grave accent	E8	350	232

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
È	Uppercase E with grave accent	C8	310	200
é	Lowercase e with acute accent	E9	351	233
É	Uppercase E with acute accent	C9	311	201
ê	Lowercase e with circumflex	EA	352	234
Ê	Uppercase E with circumflex	CA	312	202
ë	Lowercase e with umlaut	EB	353	235
Ë	Uppercase E with umlaut	CB	313	203
f	Lowercase f	66	146	102
F	Uppercase F	46	106	070
g	Lowercase g	67	147	103
G	Uppercase G	47	107	071
h	Lowercase h	68	150	104
H	Uppercase H	48	110	072
i	Lowercase i	69	151	105
I	Uppercase I	49	111	073
ì	Lowercase i with grave accent	EC	354	236
Ì	Uppercase I with grave accent	CC	314	204
í	Lowercase i with acute accent	ED	355	237
Í	Uppercase I with acute accent	CD	315	205
î	Lowercase i with circumflex	EE	356	238
Î	Uppercase I with circumflex	CE	316	206
ï	Lowercase i with umlaut	EF	357	239
Ï	Uppercase I with umlaut	CF	317	207
j	Lowercase j	6A	152	106
J	Uppercase J	4A	112	074
k	Lowercase k	6B	153	107
K	Uppercase K	4B	113	075
l	Lowercase l	6C	154	108
L	Uppercase L	4C	114	076
m	Lowercase m	6D	155	109
M	Uppercase M	4D	115	077
n	Lowercase n	6E	156	110

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
N	Uppercase N	4E	116	078
ñ	Lowercase n with tilde	F1	361	241
Ñ	Uppercase N with tilde	D1	321	209
o	Lowercase o	6F	157	111
O	Uppercase O	4F	117	079
ò	Lowercase o with grave accent	F2	362	242
Ò	Uppercase O with grave accent	D2	322	210
ó	Lowercase o with acute accent	F3	363	243
Ó	Uppercase O with acute accent	D3	323	211
ô	Lowercase o with circumflex	F4	364	244
Ô	Uppercase O with circumflex	D4	324	212
õ	Lowercase o with tilde	F5	365	245
Õ	Uppercase O with tilde	D5	325	213
ö	Lowercase o with umlaut	F6	366	246
Ö	Uppercase O with umlaut	D6	326	214
æ	Lowercase oe ligature	F7	367	247
Æ	Uppercase OE ligature	D7	327	215
p	Lowercase p	70	160	112
P	Uppercase P	50	120	080
q	Lowercase q	71	161	113
Q	Uppercase Q	51	121	081
r	Lowercase r	72	162	114
R	Uppercase R	52	122	082
s	Lowercase s	73	163	115
S	Uppercase S	53	123	083
ß	German lowercase sharp s	DF	337	223
t	Lowercase t	74	164	116
T	Uppercase T	54	124	084
u	Lowercase u	75	165	117
U	Uppercase U	55	125	085

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
ù	Lowercase u with grave accent	F9	371	249
Ù	Uppercase U with grave accent	D9	331	217
ú	Lowercase u with acute accent	FA	372	250
Ú	Uppercase U with acute accent	DA	332	218
û	Lowercase u with circumflex	FB	373	251
Û	Uppercase U with circumflex	DB	333	219
ü	Lowercase u with umlaut	FC	374	252
Ü	Uppercase U with umlaut	DC	334	220
v	Lowercase v	76	166	118
V	Uppercase V	56	126	086
w	Lowercase w	77	167	119
W	Uppercase W	57	127	087
x	Lowercase x	78	170	120
X	Uppercase X	58	130	088
y	Lowercase y	79	171	121
Y	Uppercase Y	59	131	089
ÿ	Lowercase y with umlaut	FD	375	253
Ÿ	Uppercase Y with umlaut	DD	335	221
z	Lowercase z	7A	172	122
Z	Uppercase Z	5A	132	090
æ	Lowercase ae diphthong	E6	346	230
Æ	Uppercase AE diphthong	C6	306	198
ø	Lowercase o with slash	F8	370	248
Ø	Uppercase O with slash	D8	330	216
å	Lowercase a with ring	E5	345	229
Å	Uppercase A with ring	C5	305	197
[	Opening bracket	5B	133	091
\	Backslash	5C	134	092
]	Closing Bracket	5D	135	093
^	Circumflex	5E	136	094

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
_	Underline (underscore)	5F	137	095
`	Grave accent	60	140	096
{	Opening brace	7B	173	123
	Vertical line	7C	174	124
}	Closing brace	7D	175	125
~	Tilde	7E	176	126
DEL	Delete, rubout	7F	177	127
		80	200	128
		81	201	129
		82	202	130
		83	203	131
IND	Index	84	204	132
NEL	Next line	85	205	133
SSA	Start of selected area	86	206	134
ESA	End of selected area	87	207	135
HTS	Horizontal tab set	88	210	136
HTJ	Horizontal tab set with justification	89	211	137
VTB	Vertical tab set	8A	212	138
PLD	Partial line down	8B	213	139
PLU	Partial line up	8C	214	140
RI	Reverse index	8D	215	141
SS2	Single shift 2	8E	216	142
SS3	Single shift 3	8F	217	143
DCS	Device control string	90	220	144
PU1	Private use 1	91	221	145
PU2	Private use 2	92	222	146
STS	Set transmit state	93	223	147
CCH	Cancel character	94	224	148
MW	Message waiting	95	225	149
SPA	Start of protected area	96	226	150

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
EPA	End of protected area	97	227	151
SOS	Start of significance	98	230	152
		99	231	153
SCI	Single character introducer	9A	232	154
CSI	Control sequence introducer	9B	233	155
ST	String terminator	9C	234	156
OSC	Operating system command	9D	235	157
PM	Privacy message	9E	236	158
APC	Application	9F	237	159
		A0	240	160
i	Inverted exclamation mark	A1	241	161
¢	Cent sign	A2	242	162
£	Pound sign	A3	243	163
		A4	244	164
¥	Yen sign	A5	245	165
		A6	246	166
§	Section sign	A7	247	167
¤	General currency sign	A8	250	168
©	Copyright sign	A9	251	169
ª	Feminine ordinal indicator	AA	252	170
«	Angle quotation mark left	AB	253	171
		AC	254	172
		AD	255	173
		AE	256	174
		AF	257	175
°	Degree sign	B0	260	176
±	Plus/minus sign	B1	261	177
²	Superscript 2	B2	262	178
³	Superscript 3	B3	263	179
		B4	264	180

Multinational Collating Sequence

Char.	Description	Hex	Octal	Decimal
μ	Micro sign	B5	265	181
¶	Paragraph sign, pilcrow	B6	266	182
·	Middle dot	B7	267	183
		B8	270	184
¹	Superscript 1	B9	271	185
°	Masculine ordinal indicator	BA	272	186
»	Angle quotation mark right	BB	273	187
¼	Fraction one quarter	BC	274	188
½	Fraction one half	BD	275	189
		BE	276	190
	Inverted question mark	BF	277	191
		D0	320	208
		DE	336	222
		F0	360	240
		FE	376	254
		FF	377	255

Notes

When collating with the MULTINATIONAL sequence, characters are ordered as follows:

- All diacritical forms of a character collate at the same position as the character itself and all lowercase characters collate as their uppercase equivalents. For example, Î, ö, ê, ›, ;, ? all collate as character A.
- If two character strings compare equal, they are compared a second time with all diacritical forms of a character taking their places in the collating sequence as given in the table. In addition, NULLCHAR characters are considered as part of a string and any MODIFICATION is ignored. However, all lowercase characters still collate as their uppercase equivalents.

-
- If the character strings still compare equal, they are compared a third time when each character is collated according to the table.

Appendix D. Record Formats

Record Formats

Record Format	Keyword
Fixed length	FIXED
Fortran Unformatted	FORTRANUNFORMATTED
Micro Focus line sequential	MFLINESEQUENTIAL
Micro Focus variable length	MFVARIABLE
Stream	STREAM [<i>standard_terminator</i>] CUSTOMSTREAM [<i>custom_terminator</i>]
Variable length	VARIABLE

Table 1. Record Formats Supported by SyncSort

where

standard_terminator = $\begin{bmatrix} \text{CR} \\ \text{LF} \\ \text{CRLF} \end{bmatrix}$

custom_terminator = $\begin{bmatrix} \text{character} \\ \text{character_pair} \end{bmatrix}$

character a single character. You can specify the character in any of the formats described in “Specifying Character String Constants” on page B.1. For example, X"0B", '\x0c'.

character_pair two characters. You can specify the characters in any of the formats described in “Specifying Character String Constants” on page B.1. For example, X"0D0A", '\x1b\x1b'.

Description of Supported Record Formats

The following sections describe record formats supported by SyncSort.

Fixed length

Records can contain any binary data. All records are the same length. You must provide the record length. The maximum supported record length is 65,535 bytes.

Fortran unformatted

Records can contain any binary data. Records can be of any length from the minimum to the maximum record length. Each record consists of a 4-byte prefix, followed by the data, and a 4-byte suffix, which is identical to the prefix. The prefix and suffix are in the form of a binary signed integer holding the length of the data portion in bytes. The binary integer is represented in machine-specific form, either big-endian or little-endian. The maximum supported record length is 65,535 bytes, which does not include the lengths of the prefix and suffix.

Micro Focus variable

The format is as described in Appendix I of *Micro Focus COBOL for UNIX: COBOL System Reference*, under "File Formats, Variable Structure." The maximum supported record length is 65,526 bytes. The record length does not include the 2- or 4-byte control field that starts each record.

Micro Focus line sequential

The format is as described in Appendix I of *Micro Focus COBOL for UNIX: COBOL System Reference*, under "File Formats, Variable Structure." The maximum record length that can be specified is 65,535 (bytes).

Stream

Records can contain only character data. Records can be of any length from the minimum record length to the maximum record length. A special character or pair

of characters terminates each record. You must identify the terminating character(s) (see Table 2 below). If you do not specify a terminating character, SyncSort assumes that the records have the default stream format described in Table 2 below. The maximum supported record length is 65,535 bytes. The record length does not include the terminating character(s).

Terminator	Description
CR	A carriage return character (X'0D') terminates each record.
LF	A linefeed character (X'0A') terminates each record. This is the format of UNIX text files as generated by most text editors, and the default format of Micro Focus line sequential files.
CRLF	A carriage return-linefeed pair (X'0D0A') terminates each record. This is the format of Windows text files.
default	Any of the following terminate a record: a carriage return, a carriage return followed by a linefeed, a form feed (X'0C'), a vertical tab (X'0B'), or an escape (X'1B').

Table 2. Standard Stream Record Terminators Supported by SyncSort

Terminator	Description
<i>character</i>	The character character terminates each record.
<i>character_pair</i>	The two characters of <i>character_pair</i> terminate each record.

Table 3. Custom Stream Record Terminators Supported by SyncSort

Variable length

Records can contain any binary data. Records can be of any length from the minimum record length to the maximum record length. Each record consists of a 2-byte prefix followed by the data. The 2-byte prefix holds a binary unsigned integer in big-endian representation giving the length of the data portion in bytes. The maximum supported record length is 65,535 bytes. The record length does not include the 2-byte prefix.

Appendix E. Masks

Edit Specifications for Numeric Fields

One of the options you have when using a numeric field in a report is to convert the field value to a printable format and make its contents more readable. You do this by supplying an edit specification. The edit specification, or mask, provides you with a wide variety of ways to reformat the field contents.

A numeric edit mask is a series of editing symbols in a particular pattern with optional embedded characters. The edit mask is enclosed by a pair of parentheses. The component symbols may be any of the following:

9 V Z * P C D + - S E %

You can specify the alphabetic symbols in either upper or lower case. When a mask contains *n* consecutive symbols which are identical, the repeated symbols can be represented by a single symbol followed by [*n*] with no intervening spaces, e.g. Z[5] is equivalent to ZZZZZ.

When an edit mask is applied to a numeric field and the value of the field is too large to be contained in the edited field (other than the fractional portion of a floating point or edited numeric value), you are warned of the overflow and the edited field is filled with pound signs (#).

Replacement

The simplest edit mask consists of any number of the digit replacement symbol '9' and, optionally, a single decimal point symbol 'V' embedded anywhere in the mask. The edited field has the same number of characters as there are symbols in the edit mask. Each position represented in the mask by a digit replacement symbol is replaced by a digit from the value.

When the unedited value is of a type without a fractional component, the significant digits of the value are right justified in the edited field, so that the least significant digit is in the position given by the final '9' in the mask. Any '9' which is not replaced by a significant digit from the value is replaced by a '0'. The position

represented in the mask by 'V' is replaced by the decimal point character from the current locale. Thus, when a mask 9999V999 is applied to a field containing a value 4254711 the resultant field is 8 bytes long (the length of the mask) and has a value of 4254.711. When the same mask is applied to a value 1357, the result is 0001.357.

When the unedited value is of floating point or edited numeric type, the implicit or explicit decimal point is aligned with the 'V' symbol in the edit mask. When the mask does not contain a 'V' symbol, the fractional portion of the original value is discarded. Thus, when a mask 9999V999 is applied to a field containing a value 8463.19 the resultant field is 8 bytes long (the length of the mask) and has a value of 8463.190. When the same mask is applied to a value 3.90167, the result is 0003.901.

Any character in the edit mask which is not one of the listed editing symbols is inserted directly into the edited field. This is referred to as simple replacement. There is no limit to the number of simple replacements in an edit mask, but none can appear between the decimal point symbol and the final digit replacement symbol. If you want to include the character equivalent of one of the editing symbols or the close parenthesis character, specify the symbol between double quote characters. For example, "+s)" causes the simple replacement of the 3-character string enclosed in the double quotes. The double quotes themselves do not appear as part of the edited field. If you want to include a double quote character, specify two double quote characters surrounded by double quote characters. For example, "+"" causes the simple replacement of a plus sign and a single double quote character. Any simple replacement after the final digit replacement symbol appears in the edited field only when the value in the unedited field is negative. Space(s) are substituted when the value is positive. Some examples of simple replacement are:

Mask	Value	Edited Value	Length (bytes)
99/99/99	911230	91/12/30	8
99,999	1043	01,043	6
9,999V99	741380	7,413.80	8
99"- "99"- "99	110387	11-03-87	8
9.9,99	10	0.0,10	7
999#	-25	025#	4
999#	100	100	4
!99,999	2048	!02,048	7
99,999V99	1635.856	01,635.85	9
99999	9,867.43	09867	5

Suppressing Leading Zeroes

You can replace the leading zeroes in an edited field with spaces. To do this, specify the zero suppression symbol 'Z' in the leading positions where the zeroes should be replaced by spaces. Each position represented in the mask by a 'Z' is replaced by a significant digit from the field value, in the same fashion as for '9'. Any 'Z' which is not replaced by a significant digit from the field value is replaced by a space. You can specify as many leading zero suppression symbols as needed, and they can be interrupted by simple replacements. In addition, a maximum of one decimal point symbol can appear, so long as no simple replacements appear between the decimal point symbol and the final zero suppression symbol. Any embedded simple replacement which is not preceded by at least one significant digit in the edited field is replaced by a space. Any simple replacement before the first zero suppression symbol appears in the edited field immediately to the left of the first significant digit. The edited field has the same number of characters as there are symbols in the edit mask.

You can combine the zero suppression symbol 'Z' with the digit replacement symbol '9' as long as no 'Z' appears after the first '9'. For example, ZZZ99Z is not a valid mask. In addition, if a decimal point symbol appears anywhere in the mask,

you cannot mix 'Z' and '9' to the right of the decimal point symbol. Examples of masks used to suppress leading zeroes follow:

Mask	Value	Edited Value	Length (bytes)
ZZZ,ZZZ	123456	123,456	7
ZZZ,ZZZ	5035	5,035	7
ZZZ,ZZZ	100	100	7
ZZ,ZZ9V99	662581	6,625.81	9
ZZ,ZZ9V99	75	0.75	9
ZZ,ZZZV99	0	.00	9
ZZZVZZ	145	1.45	6
ZZZVZZ	3	.03	6
ZZZVZZ	0		6
"*"ZZZVZZ	145	*1.45	7
Z,Z99V999	4.8129	04.812	9
Z,ZZZ,ZZZ	301.78	301	9

As an alternative to replacing leading zeroes in an edited field with spaces, you can replace them with asterisks. To do this, simply use the asterisk replacement symbol '*' in place of the zero suppression symbol. Any embedded simple replacement which is not preceded by at least one significant digit in the edited field is replaced by an asterisk. Several examples follow:

Mask	Value	Edited Value	Length (bytes)
,	123456	123,456	7
,	100	****100	7
,9V99	75	*****0.75	9
,*V99	0	*****.00	9
***V**	145	**1.45	6
V**	0	**	6
@***V**	145	@**1.45	7
,*99V9	897.000	*897.0	9
I*****V99	18.9623	I***18.96	9

Sign Indicator

There are several ways of adding a sign indicator to an edited field (to show if the value is positive or negative). Your choices consist of plus or minus sign, credit or debit abbreviations, or parentheses. You can choose to have the sign in a fixed position in the edited field, or the sign can 'float' so that it appears in the position immediately before the first significant digit.

The sign symbols and their replacement values in an edited field are given in the next table. Also provided is the number of positions taken up by each sign indicator.

Sign symbol	Replaced by:		Length (bytes)
	Positive	Negative	
+	+	-	1
-	space	-	1
C	spacespace	CR	2
D	spacespace	DB	2
P	space...space	(...)	2

The sign symbol 'P' causes a pair of parentheses to be placed around a negative number and a pair of spaces to be placed around a positive number.

In an edit mask you can combine any one of the sign symbols with zero suppression or asterisk replacement symbols, digit replacement symbols, the decimal point symbol and simple replacements. The sign symbol must precede all such symbols, or follow all such symbols.

To have the sign appear in a fixed position independent of the number of digits in the value, specify a single sign symbol. Examples follow:

Mask	Value	Edited Value	Length (bytes)
+ZZ9	349	+349	4
9V99-	-12	0.12-	5
D999	539	539	5
DZ99	-54	DB 54	5
999C	-18	018CR	5
ZZZP	-7	(7)	5
#ZZZP	-10	(#10)	6
**9+	0	**0+	4
-#ZZZ	-5	- #5	5
ZZ9#-	-50	50#-	5
ZZ9#+	50	50 +	5
-99V99	4.3	04.30	6
+Z9V99	-0.879	- 0.87	6

The edited field has the same number of characters as symbols in the edit mask when it contains a fixed sign with length of 1, and one more character for a fixed sign with length of 2.

When you want the sign character to float, specify a string of sign symbols at the beginning of the edit mask. The edited field will contain a single sign indicator located immediately prior to the first significant digit in the field. Any positions before the sign will contain spaces. You cannot combine floating sign symbols with

zero suppression or asterisk replacement symbols, but you can follow floating sign symbols with digit replacement symbols. You can embed simple replacements and the decimal point symbol in the floating sign symbols. No simple replacements can appear between the decimal point symbol and a final following replacement symbol. Any simple replacements before the first floating sign symbol appear in the edited field immediately to the left of the leading sign character.

The edited field has the same number of characters as symbols in the edit mask when it contains a floating sign with length of 1, and one more character when the mask contains a floating sign with length of 2. Several samples of floating sign replacement follow:

Mask	Value	Edited Value	Length (bytes)
+++++9	47188	+47188	6
++++V99	6	+ .06	7
-----9	23	23	6
+ [6]	0		6
CC, CCC	1085	1,085	7
CC, CCC	-200	CR200	7
DDD999	79	079	7
DDDDVDD	-34268	DB342.68	8
PP, PPP	7428	7,428	7
PP, PPP	-2766	(2,766)	7
##+++++9	55	##+55	8
##++++++	0		8
+++ ,+++V++	89647.87	+89,647.87	10
----999V999	-29.3	-029.300	11

Currency indicator

A currency indicator can be added to an edited field in much the same way as a sign indicator. The currency symbol is 'S', which is replaced in the edited field by the currency symbol from the current locale or a blank. The currency indicator can

be inserted in a fixed position, or can float to precede the first significant digit replacement. The same rules apply as for the sign indicator. Here are examples of masks which insert a currency character into the edited field:

Mask	Value	Edited Value	Length (bytes)
S999	775	\$775	4
SZZ9	56	\$ 56	4
S999	8	\$008	4
**9S	50	*50\$	4
SZZ,999	50	\$ 050	7
SSSS99	3	\$03	6
SSSSS9	61	\$61	6
S[6]	0		6
SS,SSSV99	500325	\$5,003.25	9
SS,SSSV99	15	\$.15	9
SSSVSS	0		6
SSSVSS	1	\$.01	6
SSSVSS	205	\$2.05	6
#SSSVSS	205	#\$2.05	7
SSSSSSS	27.901	\$27	7
SS,SSS,999V99	58854.9	\$58,854.90	13

The edited field has the same number of characters as symbols in the edit mask when it contains a single character, currency symbol. An additional character is added for each additional character in a multi-character currency symbol.

Combining Sign and Currency Indicators

You can obtain both a sign indicator and a currency indicator in an edited field by specifying both a sign symbol and a currency symbol in the edit mask for the field.

The symbols can both be at the beginning of the mask, in either order, both can be at the end of the mask, in either order, or one can be at the beginning of the mask and the other at the end.

An additional rule applies when the sign and currency symbols are combined in one mask. If both symbols are at the beginning of the mask, the first symbol must be specified only once. The second symbol can be repeated to cause floating. When the mask is applied to a value, both sign and currency character float together. Thus, a mask +SS,SS9V99 when applied to the value 7044 yields an edited value +\$70.44 with a length of 10 bytes and 3 leading spaces. When both symbols are specified once each, neither floats, so a mask PSZZZ9 applied to the value -40 yields an edited value (\$ 40) with a length of 7 bytes.

Examples of masks which combine the sign and currency symbols follow:

Mask	Value	Edited Value	Length (bytes)
+S999	802	+\$802	5
SZZ9-	-10	\$ 10-	5
S***+	8	\$**8+	5
P99S	-87	(87\$)	5
S999C	-8	\$008CR	6
SZ,999D	-350	\$ 350DB	8
+SSSS99	2	+\$02	7
S+++++99	530	\$+530	8
CSSSSS9	41	\$41	8
SSS,SSS+	0		8
-SS,SSS	2048	\$2,048	7
SS,SSS.99P	500325	\$5,003.25	11
SS,SSS.99P	-1975	(\$19.75)	11
DSS,SSS.99	-60	DB\$.60	11
S#+++ ,+99	4028	\$#+4,028	9
+#SSS,S99	4028	+#\$4,028	9

SSS,SSSV99C	-20966.8	\$20,966.80CR	12
ZZ,ZZZVZZS+	370.0	370.00\$+	11

When the first or second symbol in a mask is the sign symbol 'P' the closing parenthesis for a negative number is always the last character in the edited field. When 'P' is the last symbol, or the symbol before the last, the opening parenthesis for a negative number is the first non-space character in the edited field. Thus a currency symbol at the opposite end of the mask to the 'P' symbol always causes the currency indicator to be enclosed by the parentheses surrounding a negative number. When 'P' is the last symbol, and the mask starts with a floating currency symbol, the opening parenthesis precedes the currency character in the edited field, and floats with it.

Default Numeric Edit Mask

When you use a numeric field in a report, it does not have a printable format (i.e. data type is not LS, TS, EN, AN, LP, TP, ZD, or LZ) and you do not provide an edit mask or data type and length when applicable, the value is converted automatically, as follows:

- Fields with packed decimal data types (PD), are converted to edited numeric (EN), with corresponding length change (twice the original length).
- Fields of signed and unsigned integer type (INT, UINT) are converted to edited numeric (EN), the length of the converted field being dependent on the length of the original integer field (lengths in bytes):

Original Length	Converted Length
1	4
2	6
3	8
4	11
5	13

6	16
7	18
8	20

- Floating point fields currently do not have a default edit mask. You must supply a mask if a floating point field appears in a report.
- For PAGENUMBER, the edit mask applied is ZZZ9.
- For COUNT and RUNCOUNT, the edit mask applied is Z[9]9.

Edit Specifications for Date Fields

When you want to express today's date in one of the supported formats, you supply a date edit specification to indicate the final format of the date. The edit specification, or mask, provides you with a large number of possible date formats.

A date edit mask consists of elements with optional separators in between, surrounded by a pair of parentheses. The component elements may be any of the following:

YEAR, YY, MM, MM0, MMTH, MON, MN, DD, DD0, DDTH, DAY, DY, HH, HH0, HR, HR0, MI, MI0, SE, SE0, AM, A.M.

You can specify the elements in either upper or lower case. The elements cannot be abbreviated. The separators between elements can be any non-alphabetic character or characters, including the space character, and quoted strings (between double quote characters). There is no restriction on the combination of elements and separators. Two elements can appear without a separator.

The date elements cause the edited date to contain the following:

Date Element	Edited Date Contains	Length (bytes)	Example
YEAR	all four digits of the year	4	1991
YY	last two digits of the year	2	90
MM	numeric month of the year, leading zero removed	2	6
MM0	numeric month of the year	2	08
MMTH	numeric month, with suffix, leading zero removed	4	1st
MON	name of the month of the year	9*	November
MN	abbreviated name of the month of the year	3*	Jun
DD	numeric day of the month, leading zero removed	2	5
DD0	numeric day of the month	2	09
DDTH	numeric day, with suffix, leading zero removed	4	4th
DAY	name of the day of the week	9*	Tuesday
DY	abbreviated name of the day of the week	3*	Wed
HH	numeric hour (24-hour clock), leading zero removed	2	2
HH0	numeric hour of the day (24-hour clock)	2	16
HR	numeric hour (12-hour clock), leading zero removed	2	7
HR0	numeric hour of the day (12-hour clock)	2	11
MI	numeric minutes of the hour, leading zero removed	2	0
MI0	numeric minutes of the hour	2	59

SE	numeric seconds of the minute, leading zero removed	2	1
SE0	numeric seconds of the minute	2	25
AM	ante/post meridiem indicator	2*	pm
A.M.	ante/post meridiem indicator with periods	4*	a.m.

The lengths marked by an asterisk are dependent on the current locale. An element length is equal to the length of the longest string returned for the associated element by the system routine "strftime." In the case of am/pm indicator with periods, the length is equal to twice the length of the indicator without periods. The month/date suffix is not available from the locale.

If you want all alphabetic characters in the edited field to be in upper case, specify the argument UPPER immediately following the edit mask.

Separators which you specify between adjacent elements are reproduced in the edited field as they appear, with one exception. Quoted strings are reproduced without the surrounding double quote characters. If you want to include a double quote character, specify two double quote characters surrounded by double quote characters.

When the edited date is constructed, all elements are concatenated and trailing spaces are removed from the date element portions. The length of the final field, however, equals the sum of the lengths of its parts. The resultant string is left aligned in the edited field. Spaces are used to fill the trailing portion. For example, (day, mon dd year) results in a 28-character long field of the form:

Monday, April 9 1990

Several examples of date edit masks and sample resultant edited values follow:

Edit Mask	Sample Result	Length (bytes)
(mm/dd/yy hh:mi)	2/5/91 5:17	14
(yymm0dd0hh0mi0se0)	910813140549	12
(dy mn ddth)	Fri Mar 31st	12
(hr "o'clock in the" am)	8 o'clock in the pm	20
(ddth mn year)	2nd Jan 1990	13
(dd-mon-year hr0:mi0 a.m.)	13-June-1991 05:25 a.m.	28
(day, "the "ddth "of" mon)	Tuesday, the 12th of May	32

Default Date Edit Mask

When you use a date in a report and you do not provide it with an edit mask, SyncSort converts the date automatically. The default date format is (MM0/DD0/YEAR).

Masks to Extract Sub-fields

When you need to extract a portion of a field to use as a key or in a reformatting, for example, you can do one of two things:

1. Define the portion of the field as a field on its own, through `/FIELDS`.
2. Define a `/DERIVEDFIELD` to extract the sub-field.

The first of these two methods is applicable only when the sub-field starts at the same offset from the start of the field in each record. When the sub-field does not start at a fixed offset, you can extract it only if you can describe a pattern of sub-fields that holds for all records. In this case, you need to define a `/DERIVEDFIELD` using the `EXTRACT /pattern_match/ [format_string]` form. The pattern (mask) is in the form of a regular expression, described below.

Regular Expressions

This section describes regular expressions. You can use a regular expression in a condition as one of the operands of a comparison and in a derived field to extract a portion of another field.

A regular expression (RE) is a pattern to match against a string of characters. Matching a string to a pattern is called *pattern matching*. Pattern matching either succeeds or fails.

If you are familiar with Extended Regular Expressions from UNIX, such as used by *regexp*, you already know how to write a RE for SyncSort. One point to note is that REs are *not* wildcards. The RE `b*y` does not match *bay*, *bey*, *boy*, *buy* etc. It matches a string that contains zero or more *b*'s followed by a *y*, so it matches *y*, *by*, *bby* etc.

Single-character matching

The simplest RE component is a single character that matches itself in the compared string. For example, the single-character RE `z` matches any string containing a `z`. Most characters match themselves. These are termed *ordinary* characters. Other characters represent something other than a self-match. For

example, the period (.) represents an any-character match; that is, it matches any character in the compared string. Such characters are termed *special*. The context in which you use a special character sometimes determines if the character usage is special or self-matching. For example, the period is special only when used outside a bracketed list. The context when a character is special is described later in “Context When Characters are Specialized in Regular Expressions” on page E.25.

Character		Description
period	.	Matches any single character
asterisk	*	Matches 0 or more occurrences of the preceding sub-pattern
plus sign	+	Matches 1 or more occurrences of the preceding sub-pattern
question mark	?	Matches 0 or 1 occurrence(s) of the preceding sub-pattern
braces	{ }	Matches multiple occurrences of the preceding sub-pattern
brackets	[]	Designate a list of single characters
hyphen	-	Separates the extremes of a continuous range in a list
parentheses	()	Group a sub-pattern
vertical bar		Separates two sub-pattern that are alternatives
caret	^	1) Matches the beginning of the compared string 2) Complements a list
dollar sign	\$	Matches the end of the compared string
backslash	\	Escapes the following character, removing its special properties

Table 1. Special Characters in Regular Expressions

In general, if you need to remove the special meaning of one of the listed characters, precede the character with a backslash (\) when the character is special in the context it is used. For example, the pattern \$ matches the end of the string whereas the pattern \\$ matches a dollar sign.

Examples of single character matching follow.

Regular Expression	Compared String	Matches
G	regular	no match
	REGULAR	character 3
.	regular	character 1
\.	regular	no match
	regular.	character 8
?		invalid RE
\?	regular?	character 8

List

A list is a sequence of one or more items between left and right square brackets ([]). A list matches a *single* character in a compared string. A list matches a character in the string if that character is part of the list. For example, **[aeiou]** is the list consisting of the characters **a**, **e**, **i**, **o** and **u**. Only one of these characters needs to exist anywhere in the string for pattern matching to succeed.

An item in a list can be a character, a range of characters or a character class.

1. A character can be an ordinary character or a special character. Most special characters lose their special properties within a list. The exceptions are the hyphen (-), the caret (^) and the right bracket (]). Each of these special characters loses its special property when placed at a particular location within the list, as noted below.
2. You can specify a range of ASCII characters by specifying the first and last characters in the range, separating them with a hyphen. For example, the list [a-eiou] consists of the characters a, b, c, d, e, i, o and u.
3. A character class is a predefined set of characters with a specific attribute, but the actual characters can vary from locale to locale. The classes are listed in the table below.

Character Class	Contents
[:alnum:]	alphanumeric characters
[:alpha:]	letters
[:blank:]	space and tab characters
[:cntrl:]	control characters (in the ASCII encoding; codes less than 040, code 0177)
[:digit:]	digits
[:graph:]	characters that are both printable and visible (in the ASCII encoding: codes 041 to 0176 (exclamation point to tilde))
[:lower:]	lowercase letters
[:print:]	printable characters (in the ASCII encoding: codes 040 to 0176 (space to tilde))
[:punct:]	punctuation characters (characters other than controls, blanks, letters, and digits)
[:space:]	characters producing white space in displayed text (space, carriage return, new line, vertical tab, form feed, etc.)
[:upper:]	uppercase letters
[:xdigit:]	hexadecimal digits (0-9, a-f, A-F)

Table 2. Character Classes in Regular Expression Lists

A character class is a predefined set of characters with a specific attribute, but the actual characters can vary from locale to locale. The order in which you specify the items in a list is usually not important. However, there are exceptions. If you need to include the right bracket as a character in a list, enter the bracket as the first character after the initial left bracket that precedes the list. For example, **[]a** forms the list consisting of the right bracket and the letter a. If you need to include the hyphen in a list, enter it as either the first or the last character of the list, for example, **[aeiou-]**.

You can reverse the matching of the list by including a caret prior to the start of the list. For example, the list **[^aeiou]** contains every character except **a**, **e**, **i**, **o** and **u**. When reversing a list that includes a right bracket or hyphen, make the carat the very first character. For example, **[^]a-** forms the list containing everything except the right bracket, the letter **a** and the hyphen. If you need to include

the carat as a character in a list, enter the carat as any character other than the first.

Examples of single character matching involving a list follow.

Regular Expression	Compared String	Matches
[aeiou]	regular	character 2
[xyz.]	regular	no match
[[:punct :]]	regular	no match
	regular?	character 8

Concatenation

The simplest REs that match multiple characters contain concatenated ordinary characters. The order of the component characters is important with multiple-character patterns. For example, **xyz** is a multiple-character RE that matches the letter **x** immediately followed by the letter **y** immediately followed by the letter **z**. If the compared string does not have the three letters in that order somewhere within its length, pattern matching fails.

To remove the special property of a special character in a concatenation, precede it with a backslash (****). For example, the RE **a.** uses the special property of the period (**.**) to match the letter **a** followed by any other character, such as **ab**, **a.**, **aa**, **a5**, and **a%**. The RE **a\.** matches only those strings containing the **a.** sub-string.

A concatenation can contain elements other than ordinary characters and escaped special characters. For example, the RE **b[aeiou]t** matches strings containing **bat**, **bet**, **bit**, **bot** and **but**. A concatenation can also contain more complex elements. These elements are described below.

Examples using concatenation follow.

Regular Expression	Compared String	Matches
rea	retreat	characters 4-6
r..r	retreat	characters 1-4
re\\.t	retreat	no match
r[aeiou][aeiou]	retreat	characters 4-6

Grouping

You can group a portion of a RE by surrounding the portion with parentheses (()). The parenthesized portion is referred to as a group, sub-pattern or sub-expression. The parentheses do not change the pattern - the REs **xy**, **x(y)**, **(x)y**, **(xy)**, **((x)y)** are equivalent.

Repetition

You can create more complex REs that match multiple occurrences of specific characters. There are four possible types of repetition for you to choose from. These are summarized in the table below.

Type of Repetition	Operator		Operation
match zero or more times	asterisk	*	repeats the smallest possible preceding sub-pattern as many times as necessary (including zero) to match the string
match one or more times	plus sign	+	repeats the smallest possible preceding sub-pattern as many times as necessary (at least once) to match the string
match zero or one time(s)	question mark	?	repeats the smallest possible preceding sub-pattern either once or not at all to match the string
match min to max times	braces	{ <i>m</i> }	repeats the smallest possible preceding sub-pattern exactly <i>m</i> times to match the string
		{ <i>m</i> ,}	repeats the smallest possible preceding sub-pattern at least <i>m</i> times to match the string
		{ <i>m</i> , <i>n</i> }	repeats the smallest possible preceding sub-pattern at least <i>m</i> , but no more than <i>n</i> , times to match the string

Table 3. Repetition Operations in Regular Expressions

A repetition operator acts on the *smallest* possible preceding sub-pattern (or sub-expression). For example, the RE **xy+z** matches a single **x** followed by one or more **ys** followed by one **z**. The RE **(xy)+z** matches one or more occurrences of **xy** followed by a single **z**. The parentheses group the **x** and **y**. A repetition operator that operates on nothing is always matched. For example, **^+xy** and **(a|?b)**. An invalid interval operator is matched literally. For example, **x{3,2}y** and **y{2,3z}**.

The order for matches using the repetition operators is longest construct first. Thus, nested constructs are matched from outermost to innermost. Concatenated constructs are matched beginning from the left.

Examples of repetition operations in regular expressions follow.

Regular Expression	Compared String	Matches
xy*	abcxyz	characters 4-5
	zyxabc	character 3
	xyyabc	characters 1-3
xy+	abcxyz	characters 4-5
	zyxcba	no match
	xyyabc	characters 1-3
xy?	abcxyz	characters 4-5
	zyxcba	character 3
	xyyabc	characters 1-2
xy{2,3}	abcxyz	no match
	zyxcba	no match
	xyyabc	characters 1-3
(xy)*z	abcxyz	characters 4-6
	zyxcba	character 1
	xyyabc	no match
	abcxyxyz	characters 4-8
	abcxyxy	no match
a[bc]+	abcxyz	characters 1-3
	ababc	characters 1-2
	aabcb	characters 2-5
	cbaxyz	no match
**	@#***&	characters 3-5

Alternation

Alternation allows you to specify alternative sub-patterns to match against a string. Separate the alternative patterns with a vertical bar (`|`). If **x** and **y** are REs, the RE **x|y** matches all strings matched by **x** plus all the strings matched by **y**. Of course, some strings may be matched by both **x** and **y**.

The alternation operator operates on the *largest* possible surrounding sub-patterns. The only way to delimit its arguments is to group them using parentheses (()).

Examples of the alternation operator in regular expressions follow.

Regular Expression	Compared String	Matches
abc xyz	abcxyz	characters 1-3
	xyzabc	characters 1-3
	abxyz	characters 3-5
ab(c x)yz	abcxyz	no match
	abcyz	characters 1-5
	abxyz	characters 1-5

Anchoring

You can constrain a RE to match only at the start or end of the compared string.

To anchor the pattern to the beginning of the string, use the carat (^) special character as the first character in the RE, or the first character after the alternation operator (|).

Note another use for the carat. The RE **^123** matches only if the compared string begins with **123**. The RE **[^123]** matches any single character in any location in the string, other than the digits **1**, **2** and **3**. The RE **^[123]** matches a string beginning with a **1**, **2** or **3**.

Use the dollar sign (\$) special character to anchor the pattern to the end of the string. The \$ should appear last in the pattern or immediately prior to the alternation operator (|).

Examples involving anchoring follow.

Regular Expression	Compared String	Matches
ret\$	retreat	no match
^rea	retreat	no match
tre ^r..r	retreat	characters 1-4
r[aeiou][aeiou]t\$	retreat	characters 4-7

Precedence

The order of precedence of the RE operators is shown in the following table.

character class	[::]
escaped character	\<special character>
list	[]
grouping	()
duplication	* + ? { <i>m,n</i> }
concatenation	
anchoring	^\$
alternation	

Table 4. *Precedence of Operators in Regular Expressions (High to Low)*

Context for special characters

An RE special character (see “Table 1. Special Characters in Regular Expressions” on page E.16) has a special property in certain contexts. When used outside these contexts, or when escaped by a backslash, such a character matches the character itself. The contexts when the characters are special are summarized in the following table.

Character		Special Context
period	.	Special except in a bracketed list
left bracket	[	
backslash	\	
right bracket	]	Special when matched by a preceding left bracket that it does not immediately follow or is separated from only by a caret
asterisk	*	Special except in a bracketed list Undefined result if appearing first in an RE, or immediately following a vertical bar, carat, or left parenthesis Undefined result if the left brace is not part of a valid repetition operator
plus sign	+	
question mark	?	
left brace	{	
right brace	}	Special outside a bracketed list when matched by a preceding right brace as part of a valid repetition operator
left parenthesis	(	Special except in a bracketed list Undefined result if appearing outside a bracketed list, immediately followed by a right parenthesis
right parenthesis	)	Special outside a bracketed list when matched by a preceding left parenthesis
vertical bar		Special except in a bracketed list Undefined result if appearing first or last in an RE, or immediately following a vertical bar or a left parenthesis, or immediately preceding a right parenthesis
caret	^	Special when appearing first in a bracketed list Special when appearing first in an RE or immediately following a vertical bar
dollar sign	\$	Special when appearing last in an RE or immediately preceding a vertical bar

Table 5. Context When Characters are Specialized in Regular Expressions

Pattern matching

During pattern matching, the search for a match starts at the beginning of a string and stops when the first sequence of characters matched by the RE is found. *First* is defined as *starts earliest in the string*. If the pattern allows a variable number of matching characters and there is more than one such sequence starting at the match point, the longest such sequence is matched. For example, **xy*** matches characters 2 - 5 of **wxyyyz**; **ab(cd | cde)(fg | efgh)** matches all characters of **abcdefgh**, with the first group matching **cd** and the second group matching **efgh**.

Within the string matched by the RE, each group, from left to right, matches the longest possible sub-string.

A group can match an empty sub-string. For example, when **a(b*)** matches **ace**, the group **b*** matches an empty sub-string. A RE can match without using a group. For example, when **(bc) | (b)*** matches the string **bcd**, the **b** group is not used.

During pattern matching, a group can match more than once. For example, when **m(iss)*** matches **mississippi**, the **iss** group matches twice. When this happens, the group is usually assigned to the last part of the string that matched the group. In the example, the **iss** group matches characters 5 - 7. However, the last match is not always the one chosen. The *last opportunity* to match is the one that is considered when assigning groups to sub-strings. This means that when groups are nested, the last match of the outer group affects the match for the inner group. For example, **(m(iss)*[ip]*)*** matches the first 13 characters of the string **mississippi mud**. The last actual match for the **iss** group is characters 5 - 7. However, the outer group **m(iss)*[ip]*** later matches the second **m** of the string. In this stage of the match, the **iss** group is not used. Thus the **iss** group is not assigned to part of the matched string. Similarly, the alternation operator affects group matching. For example, when **(m(iss)*[ip]* | s(t.)*)*** matches **mississippi state**, the **t.** group matches characters 14 - 15 (**ta**) and then characters 16 - 17 (**te**). It is assigned to the last match (**te**). The **iss** group, though it matches in **mississippi**, is not used in the final match of the outer group **m(iss)*[ip]* | s(t.)*** and is not assigned to part of the matched string.

Functions Library

Regular expression support is provided by the PCRE library package, which is open source software, written by Philip Hazel, and copyrighted by the University of Cambridge, England. The FTP site for the source is:

`ftp://ftp.csx.cam.ac.uk/pub/software/programming/pcre/`

Source Date Patterns

When a field contains a date in display format, you supply a date pattern to indicate the layout of the date. The provided elements support a large number of possible date formats.

A source date pattern consists of elements with optional separators in between, surrounded by a pair of parentheses. The component elements may be any from the following list:

YEAR, YY, MM, MM0, MMTH, MON, MN, DD, DD0, DDTH, DAY, DY, HH, HH0, HR, HR0, MI, MI0, SE, SE0, AM, A.M.

You can specify the elements in either upper or lower case. The elements cannot be abbreviated. The separators between elements can be any non-alphabetic character or characters, including the space character, and quoted strings (between double quote characters). There is no restriction on the combination of elements and separators. Two elements can appear without a separator.

The date elements describe a date containing the following:

Date Element	When Source Date Contains	(bytes)	Example
YEAR	all four digits of the year	4	2000
YY	last two digits of the year	2	99
MM	numeric month of the year, leading zero removed	2	6
MM0	numeric month of the year	2	08
MMTH	numeric month, with suffix, leading zero removed	4	21st
MON	name of the month of the year	9*	November
MN	abbreviated name of the month of the year	3*	Jun
DD	numeric day of the month, leading zero removed	2	5
DD0	numeric day of the month	2	09
DDTH	numeric day, with suffix, leading zero removed	4	4th
DAY	name of the day of the week	9*	Tuesday
DY	abbreviated name of the day of the week	3*	Wed
HH	numeric hour (24-hour clock), leading zero removed	2	2
HH0	numeric hour of the day (24-hour clock)	2	16
HR	numeric hour (12-hour clock), leading zero removed	2	7
HR0	numeric hour of the day (12-hour clock)	2	11
MI	numeric minutes of the hour, leading zero removed	2	0
MI0	numeric minutes of the hour	2	59
SE	numeric seconds of the minute, leading zero removed	2	1

Date Element	When Source Date Contains	(bytes)	Example
SE0	numeric seconds of the minute	2	25
AM	ante/post meridiem indicator	2*	pm
A.M.	ante/post meridiem indicator with periods	4*	a.m.

The lengths marked by an asterisk are dependent on the current locale. The maximum length of an element is equal to the length of the longest string returned for the associated element by the system routine "strftime." In the case of the am/pm indicator with periods, the length is equal to twice the length of the indicator without periods. The month/date suffix is not available from the locale.

A separator you specify between adjacent elements does not need to reproduce the field contents exactly. However, the **number of characters** you specify must equal the number of characters in the separator.

Several examples of field contents and possible source date patterns follow:

Field Contents	Example Patterns	Maximum Length (bytes)
2/5/00 5:17	(mm/dd/yy hh:mi) (mm dd yy hh mi)	14
Fri Mar 31st	(dy mn ddth) (dy.mn.dd.. (...mn.dd..)	12
15:31:40	(hh0:mi0:se0)	8
2nd Jan 1999	(ddth mn year)	13
13-June-2000 05:25 a.m.	(dd-mon-year hr0:mi0 a.m.) (dd mon year hr0 mi0 a.m.)	28
Tuesday, the 12th of May	(day, "the "ddth "of" mon) (day. ... dd.. .. mon)	32
20000813140549	(yearmm0dd0hh0mi0se0)	14

Appendix F. Arithmetic Expressions

You use an arithmetic expression in a `/DERIVEDFIELD` option to define a value that needs an arithmetic computation. The evaluation of such an expression produces a numeric value that is assigned to the derived field.

The simplest form of an arithmetic expression is a numeric constant, such as **1024**, or a numeric field reference, such as **month_amount[10]**. You can create more complicated arithmetic expressions by using one or more numeric values (the operands) together with arithmetic operations (the operators) and, perhaps, parentheses.

The arithmetic expression feature is not included in the base licensing of the product and will require an additional license fee and key to activate.

Arithmetic Operators

The arithmetic operators are identified in the following table.

Operator	Represents	Usage	Means
*	multiplication	$x * y$	multiply x and y
/	division	x / y	divide x by y
+	addition identity	$x + y$ $+ y$	add x and y same as y
-	subtraction negation	$X - y$ $- y$	subtract y from x negate y

The multiplication and division operators are binary operators only, i.e. they operate on the pair of operands that they separate. For example, **amount * 0.5** multiplies the contents of the **amount** field by **0.5**.

The addition and subtraction operators can take either a single operand (unary

operator) or two operands (binary operator). For example, in the expression **amount + tax**, the plus sign separates the two operands (it is a binary operator), and the expression is evaluated by adding the contents of the two fields **amount** and **tax**. On the other hand, the expression **–credit** contains a single operand (the minus sign is a unary operator) and is evaluated by negating the contents of the **credit** field. Thus, when **credit** contains 1000, the expression **–credit** evaluates to –1000.

Order of Applying the Operators

When an arithmetic expression contains two or more operators, there is a set of rules that determines in which order the operations are evaluated.

The first rule sets the precedence among the operators. An operator with a higher precedence is applied before an operator with a lower precedence, unless the order of evaluation is changed by the use of parentheses. The precedence of the arithmetic operators is shown in the following table. Operators with higher precedence appear higher in the table. Operators of equal precedence appear in the same row.

Category	Operator	Associativity
unary	+ –	right to left
binary (mult.)	* /	left to right
binary (add.)	+ –	left to right

For example, in the expression **tax + amount * discount**, the multiplication operator has higher precedence than the addition operator does. The expression is evaluated by first multiplying the contents of the **amount** field by the contents of the **discount** field, and adding the result of the multiplication and the contents of the **tax** field.

The second rule determines how an expression is evaluated when it contains two or more operators with the same precedence. The order of the evaluation of operators with equal precedence is determined by their associativity, as given in the above table, unless the order of evaluation is changed by the use of parentheses. For example, the operators in the expression **in_stock + on_order – ordered –**

damaged have the same precedence. The associativity of additive binary operators indicates that they are evaluated left to right. Hence, the contents of **in_stock** and the contents of **on_order** are first added together. The contents of

ordered are subtracted from result of the addition, and the contents of **damaged** are subtracted from the result of the first subtraction.

Data Type and Precision of the Result of Arithmetic Expressions

When an arithmetic expression is evaluated, there is a set of rules to determine the data type and precision of the result if you do not provide this information explicitly in the /DERIVEDFIELD option.

Constant

The form of a constant determines its data type and precision.

When the constant is an integer, the arithmetic expression is unsigned decimal for a positive number and leading separate sign decimal for a negative number. The length is sufficient to hold the number of significant digits in the integer.

When the constant is a floating point number, the arithmetic expression is edited numeric. The length is sufficient to hold a sign, the number of significant digits before the decimal point, and, when there are non-zero digits after the decimal point, the decimal point and the following significant digits.

Operation

The data type and precision of the operands in an arithmetic operation determine the type of the result of the operation. They are identified in Tables 1 and 2, following.

	AN	D-FLOAT	EN	FLOAT	INT	LP	LS	LZ	PD	TP	TS	UINT	ZD	Integer constant	Float constant
AN	an	dfloat	en	float	ls	lp	ls	lz	pd	tp	ts	an	zd	an	dfloat
D-FLOAT	dfloat	dfloat	en	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat
EN	en	en	en	en	en	en	en	en	en	en	en	en	en	en	en
FLOAT	float	dfloat	en	float	float	float	float	float	float	float	float	float	float	float	float
INT	ls	dfloat	en	float	int	lp	ls	lz	pd	tp	ts	int	zd	int	dfloat
LP	lp	dfloat	en	float	lp	lp	lp	lp	pd	tp	ts	lp	lp	lp	dfloat
LS	ls	dfloat	en	float	ls	lp	ls	ls	pd	tp	ts	ls	ls	ls	dfloat
LZ	lz	dfloat	en	float	lz	lp	ls	lz	pd	tp	ts	lz	zd	lz	dfloat
PD	pd	dfloat	en	float	pd	pd	pd	pd	pd	pd	pd	pd	pd	pd	dfloat
TP	tp	dfloat	en	float	tp	tp	tp	tp	pd	tp	tp	tp	tp	tp	dfloat
TS	ts	dfloat	en	float	ts	ts	ts	ts	pd	tp	ts	ts	ts	ts	dfloat
UINT	an	dfloat	en	float	int	lp	ls	lz	pd	tp	ts	uint	zd	uint	dfloat
ZD	zd	dfloat	en	float	zd	lp	ls	zd	pd	tp	ts	zd	zd	zd	dfloat
Integer constant	an	dfloat	en	float	int	lp	ls	lz	pd	tp	ts	uint	zd	int	dfloat
Float constant	dfloat	dfloat	en	float	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat	dfloat

Table 1. Data Types Resulting from Binary Arithmetic Operation

	+	-
AN	an	ls
DFLOAT	dfloat	dfloat
EN	en	en
FLOAT	float	float
INT	int	int
LP	lp	lp
LS	ls	ls
LZ	lz	lz
PD	pd	pd
TP	tp	tp
TS	ts	ts
UINT	uint	int
ZD	zd	zd
Integer constant	an	ls
Float constant	en	en

Table 2. Data Types Resulting from Unary Arithmetic Operation

Appendix G. Technical Support

If you have a maintenance support agreement for SyncSort and you encounter difficulties installing or running SyncSort, contact Syncsort Incorporated in the United States 24 hours a day, seven days a week. Please visit our web site for information on getting technical support outside of North America.

Address

Syncsort Incorporated
50 Tice Boulevard
Woodcliff Lake, NJ 07677
USA
<http://www.syncsort.com>

SyncSort UNIX, Visual SyncSort

Phone:201-930-8270
Fax:201-930-8281
E-mail:techsupport@syncsort.com

SyncSort for Windows

Phone:201-930-8250
Fax:201-930-8255
E-mail:techsupport@syncsort.com

Index

/

/AGGREGATE, 3.2, 3.8
/AGGREGATE, 3.2
/BEGIN, 2.4, 3.9
 See Editing options, 2.4
 See SyncSort options, 2.4
/CENTURYWINDOWSTART, 3.10
/CHECKSEQUENCE, 3.12
/COLLATINGSEQUENCE, 3.14
/COLLECT, 3.21
/COLUMNS, 3.23
/CONDITION, 3.28
/COPY, 3.36
/DATADictionary, 3.37
/DATASIZE, 3.39
/DELETE, 2.4, 3.41
/DELIMITEDRECORDLAYOUT, 3.42
/DERIVEDFIELD, 3.48, 3.133, 3.138,
 3.165, 3.171, 3.194
/DISPLAY, 2.4, 3.63
/DOCUMENTATION, 3.64
/EDIT, 2.4, 3.67
/END, 3.68
/FIELDS, 3.69
/IGNORE, 3.77
/INCLUDE, 3.79, 3.80, 3.111, 3.112, 3.149
 Relation to output options, 3.79
/INFILE, 3.82
/INSERT, 3.91, 3.92
/JOIN, 3.94
/JOINKEYS, 3.97
/KEYS
 Relation with /INCLUDE and
 /OMIT, 3.100
/MEMORY, 3.103
/MERGE, 3.105
/NOEXECUTE, 3.106
/NOINTERRUPT, 3.107
/NOPROMPT, 2.132, 3.109
/OMIT, 3.79, 3.80, 3.111, 3.112, 3.149
 Relation to output options, 3.80,
 3.111, 3.112
/OUTFILE, 3.113
/PADBYTE, 3.125
/PADKEY, 3.127
/PADRECORD, 3.129
/PAGEFOOTER, 3.131
/PAGEHEADER, 3.136
/QUIT, 2.4, 3.140
/RECORDLAYOUT, 3.141, 3.147
/REFORMAT, 3.148
 Relation to input options /INCLUDE
 and /OMIT, 3.148
 Relation to output options, 3.148
/REPORT, 3.154, 3.157
/SAVE, 2.4, 3.158
/SECTION, 3.160, 3.162
/SECTIONFOOTER, 3.163, 3.168
/SECTIONHEADER, 3.169, 3.172
/SILENT, 3.173
/SKIPBLANK, 3.174
/SKIPRECORD, 3.176, 3.177
/STATISTICS, 3.180, 3.182
/STOPAFTER, 3.183
/SUMMARIZE, 3.185
/TITLE, 3.188
/TRAILER, 3.192, 3.196
/WARNINGS, 3.197, 3.198
/WORKSPACE, 3.199
Abbreviating SyncSort keywords, 2.3
ADDRROUT, 3.120
Addrout files, 3.120, 3.121
 Address + key format, 3.120

Address format, 3.120	Defined for different record formats, 3.174
ALIGNED2, 3.87, 3.121	CALLSORT, 2.120
ALIGNED4, 3.87, 3.121	Character, A.10
ALL, 3.79, 3.80, 3.112	CHARACTER, 3.69, A.1, A.3
Alternative format, A.5	Character string constants, 3.24, 3.30, 3.50, 3.51, 3.64, 3.131, 3.163, 3.169, 3.188, 3.192, B.1, D.1
AN, A.1, A.3	Character string matching, 3.31
AND, 3.28	C-ISAM, 3.86
ANSI, 3.38	cob, 2.120
APPEND, 3.113, 3.119	COBOL, 2.8, 3.37, 3.38, 3.79, 3.111
Application definition, 3.158	COBOL
Amending, 2.5	Compiling, 2.8
Errors in, 2.4	Collating sequence, 3.101, C.1, C.9, C.16, C.21, C.22, C.25, D.1
Saving, 3.158	Collating sequence, 3.70
Using a saved definition to execute SyncSort, 3.158	Alternative collating sequence, 3.101
Application statistics, 3.180	ASCII sequence, C.1, C.9
Application statistics, 2.5, 3.180	ASCII sequence, 3.70
Arithmetic expressions, F.1	EBCDIC sequence, C.16, C.21
Arithmetic operators, F.1	EBCDIC sequence, 3.70
Arrays, B.11	Folded ASCII sequence, C.9, C.16
ASCII collating sequence, 3.17, 3.20, 3.70, 3.101, C.1, C.9	Folded ASCII sequence, 3.70
ASCII numeric, described, A.4	Folded EBCDIC sequence, C.22, C.25
Binary floating point, A.7	Folded EBCDIC sequence, 3.70
Binary floating point	Multinational sequence, D.1
Floating point, A.7	MULTINATIONAL sequence, 3.70
Binary signed integer, A.8	Completion status, 2.9
Binary signed integer	Composite field, B.10
Unsigned integer, A.9	COMPRESS, 3.57
Binary unsigned integer, A.9	COMPRESS
Binary unsigned integer	Description, 3.57
Bit, A.9	COMPRESSED, 3.117
Bit, A.9	Compression, 3.57
Bit	Compression, 3.85, 3.117
Character, A.10	Constants, 3.24, 3.30, 3.50, 3.51, 3.64, 3.131, 3.163, 3.169, 3.188, 3.192, B.1, B.4, B.6, D.1
BIT, 3.49, 3.74, 3.142, A.1, A.3	Constants
BIT, 3.71	Bit string, 3.30, 3.50, B.6
Bit fields, 3.74, 3.144, 3.151	Character string, 3.24, 3.30, 3.50, 3.51, 3.64, 3.131, 3.163, 3.169, 3.188, 3.192, B.1, D.1
Bit string constants, 3.30, 3.50, B.6	
Bit string matching, 3.33	
Blank records, 3.174	
Blank records	

Numeric, B.4	Edit masks, 3.133, 3.138, 3.165, 3.171, 3.194, E.1
Constants, specifying, B.1, B.6	Edit Specifications, 3.133, 3.138, 3.165, 3.171, 3.194, E.1
Correcting errors, 4.1	Edited numeric, A.1, A.3, A.11
Ctrl-C, 3.107	Editing options, 3.159
Ctrl-D, 3.68	Editing options /SAVE, 3.159
Currency indicator, E.7	Elementary field, B.10
Currency indicator Combining with sign indicator, E.8	ELSE, 3.48
CUSTOMSTREAM, D.1	Embedded sign, A.5
Data conversion, 3.55	Embedded sign Alternative format, A.5
Data formatting, 3.133, 3.138, 3.165, 3.171, 3.194, E.1	Micro Focus format, A.5
Data length default, 3.54	EN, A.1, A.3
Data type	EN fields, summarizing, 3.186
Arithmetic expressions, 3.58, F.3	ENCLOSEDBY, 3.88, 3.89
default, 3.54	Error messages, 4.1, 4.3
Data types, A.1, A.4	Errors, 2.4
Data types	ESDS, 3.86, 3.118
ASCII numeric, A.1	Exception reporting, 4.1
Binary, A.1	Exception reporting SyncSort messages, 4.1
Bit, A.1	Exceptions, 4.1
Character, A.1	Extracting fields, 3.56
Edited numeric, A.1	FASCII, 3.74, 3.75, 3.145
Floating point, A.1	FASCII, 3.70
Packed decimal, A.1	FEBCDIC, 3.70
Signed decimal, A.1	Field definitions, 3.61, 3.76
Signed integer, A.1	Field definitions /FIELDS, 3.76
Unsigned decimal, A.4	Field names, specifying, 3.3, 3.23, 3.30, 3.30, 3.38, 3.44, 3.49, 3.97, 3.100, 3.131, 3.136, 3.144, 3.148, 3.160, 3.163, 3.169, 3.185, 3.188, 3.192, B.9
Unsigned integer, A.1	Field separator, 3.87, 3.116, 3.120
Data types supported, A.1	File alias, 3.88
Data types, THEN/ELSE, 3.52	File size, 3.86
Date edit masks, 3.49, 3.60, 3.134, E.11	File size Maximum, 3.86
Date formats, 3.49, 3.60, 3.134, E.11	FIXED, D.1, D.3
Date/time, 3.46, 3.47, 3.76, B.13	Fixed length, D.2
Date/time components of, B.13	
DATETIME, 3.70	
Delimited Fields, 3.73, 3.150	
DFLOAT, 3.142, A.1, A.3	
DFLOAT, 3.71	
Double precision, A.8	
EBCDIC, 3.70	
EBCDIC collating sequence, C.16, C.21	

FLOAT, 3.142, A.1, A.3
 Floating point, A.1, A.3, A.7
 Folded ASCII collating sequence, C.9, C.16
 Folded EBCDIC collating sequence, C.22, C.25
 Formatting data, 3.133, 3.138, 3.165, 3.171, 3.194, E.1
 Fortran unformatted, D.1, D.2, D.3
 FSKIPBLANK, 3.88
 FSKIPRECORD, 3.177, 3.184
 FSKIPRECORD, 3.88
 FSTOPAFTER, 3.177, 3.184
 FSTOPAFTER, 3.88
 Help messages, 4.1
 HIGHCOMPRESSION, 3.117
 Identifiers, 3.3, 3.15, 3.29, 3.42, 3.49, 3.70, 3.83, 3.142, B.7
 Identifiers, specifying, 3.3, 3.15, 3.29, 3.42, 3.49, 3.70, 3.83, 3.142, B.7, B.13
 IF, 3.48
 IF statements, 3.52
 IF statements
 Evaluating, 3.52
 Nesting, 3.52
 Including records, 3.149, 3.183
 INDEXED, 3.82, 3.113
Indexed output file, 3.119
 Input files, 3.86
 Input files
 Number of, submitted to application, 3.86
 Input record reformatting, 3.149
 Insert file, 3.91
 INTEGER, A.1, A.3
 Interrupting SyncSort, 3.107
 Interrupting SyncSort
 Disabling the interrupt trap, 3.107
 Invoking SyncSort, 2.1
 With COBOL, 2.8
 With Visual SyncSort, 2.7
 KEYS, 3.100
 Keywords, 3.37, 3.38, 3.42, 3.48, 3.53, 3.64, 3.69, 3.73, 3.74, 3.79, 3.80, 3.82, 3.94, 3.101, 3.111, 3.112, 3.113, **3.119**, 3.120, 3.142, 3.145, 3.151, 3.177, 3.184, A.1, A.2, A.3, D.1, D.3
 Keywords, 3.52, 3.70, 3.71, 3.87, 3.88, 3.94, 3.95, 3.96, 3.97, 3.98, 3.121, 3.122, 3.151, 3.152
 ADDROUT, 3.120
 ALIGNED2, 3.87, 3.121
 ALIGNED4, 3.87, 3.121
 ALL, 3.79, 3.80, 3.112
 AN, 3.53, A.1, A.2, A.3
 ANSI, 3.38
 APPEND, 3.113, **3.119**
 ASCII, 3.101
 ASCII, 3.70
 B, 3.70
 BIT, 3.53, 3.74, 3.142, A.1, A.2, A.3
 BIT, 3.70
 CH, 3.53, A.2, A.3
 CHARACTER, 3.48, 3.69, A.1
 COBOL, 3.37, 3.38, 3.79, 3.111
 COLUMNSPACING, 3.42, 3.64
 COMPRESSED, 3.117
 CT, 3.94
 DFLOAT, 3.53, 3.142, A.1, A.2, A.3
 DFLOAT, 3.71
 EBCDIC, 3.70
 ELSE, 3.48
 EN, 3.53, A.1, A.2, A.3
 ENCLOSEDBY, 3.88
 ESDS, 3.86, 3.118
 FASCII, 3.145
 FASCII, 3.70
 FEBCDIC, 3.70
 FIXED, D.1, D.3
 FLOAT, 3.53, 3.142, A.1, A.2, A.3
 FORTRANUNFORMATTED, D.1, D.3
 FSKIPBLANK, 3.88
 FSKIPRECORD, 3.177, 3.184
 FSKIPRECORD, 3.88
 FSTOPAFTER, 3.177, 3.184
 FSTOPAFTER, 3.88
 HIGHCOMPRESSION, 3.117
 IF, 3.48
 INDEXED, 3.82, 3.113

INT, 3.53, A.2, A.3
 INTEGER, A.1
 KSDS, 3.86, 3.118
 LEFTSIDE, 3.94, 3.96, 3.152
 LP, 3.53, A.1, A.2, A.3
 LS, 3.53, A.1, A.2, A.3
 LZ, 3.53, A.1, A.2, A.3
 MBM, 3.86, 3.118
 MFLINESEQUENTIAL, D.1
 MFVARIABLE, D.1, D.3
 MTP, 3.86, 3.118, **3.119**
 NOBLANK, 3.73, 3.151
 ONLY, 3.94, 3.95
 OPEN, 3.82, 3.113
 OVERWRITE, 3.113, **3.119**
 PAGENUMBER, 3.48
 PD, 3.53, A.1, A.2, A.3
 RECORDNUMBER, 3.52, 3.122, 3.151, 3.152
 RIGHTSIDE, 3.94
 RIGHTSIDE, 3.95, 3.152
 RRDS, 3.86, 3.118
 SEQUENTIAL, 3.113
 SORTED, 3.97, 3.98
 START, 3.122, 3.152
 STREAM, D.1
 THEN, 3.48
 TODAY, 3.48
 TP, 3.53, A.1, A.2, A.3
 TS, 3.53, A.1, A.2, A.3
 UINT, 3.53, A.2, A.3
 UINTEGER, A.1
 UNALIGNED, 3.87, 3.121
 UNCOMPRESSED, 3.85, 3.117
 UNPAIRED, 3.94, 3.95
 VARIABLE, D.1
 VSAM, 3.86, 3.88, 3.115, 3.118, **3.119**, 3.122
 WIDTH, 3.42, 3.64
 ZD, 3.53, A.1, A.2, A.3

Keywords

 BIT, 3.71

Korn shell functions, 2.102

KSDS, 3.86, 3.118

LEFTSIDE, 3.94

, 2.121

LP, A.1, A.3

LS, A.1, A.3

LZ, A.1

Masks, 3.133, 3.138, 3.165, 3.171, 3.194, E.1, E.14

Masks

 Date fields, 3.49, 3.60, 3.134, E.11

 Default numeric edit mask, E.10

 Numeric fields, 3.133, 3.138, 3.165, 3.171, 3.194, E.1

MBM, 3.86, 3.118

Messages, 4.1

MFLINESEQUENTIAL, D.1

MFVARIABLE, D.1, D.3

Micro Focus, 3.86, A.5

Micro Focus

 Alternative format, A.5

 Indexed file, 3.86

Micro Focus format, A.5

Micro Focus line sequential, D.2

Micro Focus variable, D.2

MTP, 3.86, 3.118, **3.119**

MULTINATIONAL collating sequence, D.1

Nesting, B.9

NOBLANK, 3.73, 3.151

Numeric constants, B.4

OMIT, 3.111

Omitting records, 3.80, 3.149, 3.183

ONLY, 3.94

OPEN, 3.82, 3.113

Operators

 Arithmetic, F.1

Organization, 3.115

Organization

 Default, 3.115

Organization, input, 3.86

Output destination, 3.80, 3.112, 3.148

Output files, 3.117, 3.118, 3.119, 3.121

Output files, 3.120

 Addrout files, 3.121

 Addrout files, 3.120

 Existence of, 3.119

 Number of, 3.117

 Record format, 3.119

 Record length, 3.119

- Splitting, 3.118
- Output organization, 3.115
- Output record alignment, 3.117
- Output record format, 3.116
- Output record length, 3.116
- Output record reformatting, 3.150
- Overflow, 3.187
- OVERWRITE, 3.113, **3.119**
- Packed decimal, A.1, A.3, A.12
- Pad byte, 3.125, 3.127, 3.129
- Pad byte
 - Default, 3.125, 3.127, 3.129
- Pad character
 - See¶ Pad byte, 3.125
- PAGENUMBER, 3.48
- Pattern matching, 3.33
- PD, A.1, A.3
- Precision, 3.58
 - Arithmetic expressions, 3.58, F.3
- Precision
 - EN fields, 3.58
- Precision, double, A.8
- Precision, single, A.8
- Problem correction, 2.4
- Problem reporting, 3.173
- Problem reporting
 - See¶ /WARNINGS, 3.197
- Record alignment, 3.87, 3.117
- Record alignment
 - Default, 3.117
- Record file address, 3.113, 3.120
- Record format, 3.86, 3.116, 3.174
- Record format
 - Default, 3.116
 - Fixed, 3.174
 - Stream, 3.174
 - Variable, 3.174
- Record formats, 3.84, 3.87, 3.114, 3.116, 3.119, D.1, D.2, D.4
- Record formats
 - Fixed length, D.2
 - Fortran unformatted, D.2
 - Micro Focus line sequential, D.2
 - Micro Focus variable, D.2
 - Stream, D.2
 - Variable length, D.4
- Record Formats, D.1, D.3
- Record Formats
 - Keyword, D.1, D.3
- Record layout, 3.76, 3.92, 3.143, 3.153
- Record layout
 - Describing, 3.143
- Record length, 3.86, 3.116, 3.119
- Record length, 3.87
 - Default, 3.116
 - Maximum, 3.87
 - Truncating records, 3.86, 3.119
- Record numbering, 3.52**
- Record selection, 3.61
- Record selection
 - See¶ /COLLECT, 3.80
 - See¶ /IGNORE, 3.80
 - See¶ /INCLUDE, 3.80
 - See¶ /OMIT, 3.80
 - See¶ /SKIPBLANK, 3.80
 - See¶ /SKIPRECORD, 3.80
 - See¶ /STOPAFTER, 3.80
 - See¶ FSKIPBLANK, 3.88
 - See¶ FSKIPRECORD, 3.88
 - See¶ FSTOPAFTER, 3.88
- RECORDNUMBER, 3.52
- Reformatting, 3.149, 3.150, 3.151
- Reformatting
 - Bit fields, 3.151
 - Delimited fields, 3.150
 - Input record, 3.149
 - Output record, 3.150
- Reformatting records, 3.149, 3.153
- Reformatting records
 - Input records, 3.149
 - Output records, 3.149
- Regular Expressions, 3.30, 3.32, 3.33, 3.35, E.15, E.17, E.19, E.20, E.22, E.23, E.24, E.26, E.27
 - Alternation, E.22
 - Anchoring, E.23
 - Concatenation, E.19
 - Context for special characters, E.24
 - Functions Library, E.27
 - Grouping, E.20

- List, E.17
- Pattern matching, E.26
- Precedence, E.24
- Repetition, E.20
- Single-character matching, E.15
- Report writer, 3.116, 3.135, 3.139
- RFA, 3.120
- RIGHTSIDE, 3.94
- RRDS, 3.86, 3.118
- Selecting records, 3.61
- Separate sign, A.4
- SEQUENTIAL, 3.113
- Sequential output file, 3.119**
- Sign indicator, E.5
- Sign indicator
 - Combining with currency indicator, E.8
- Signed integer, A.1, A.3, A.8
- Single precision, A.8
- Sort keys, 3.100, 3.101
- Sort order, 3.101, 3.184
- SORTED, 3.97
- Splitting output files, 3.118
- STABLE, 3.178
- standard input, 3.83, 3.85, 3.89, 3.118
- standard output, 3.114
- Statistics, 3.173
- Stream, D.2
- STREAM, D.1
- Sub-fields, B.9
- Subroutine completion status, 2.9
- Summarization of records, 3.185
- Summarizing, 3.186, 3.187
- Summarizing
 - EN fields, 3.186
 - Surviving record, 3.187
- Supported data types, A.1
- Suppressing leading zeroes, E.3
- Surviving record, 3.187
- sync_add_source_file, 2.23
- sync_set_source_file, 2.21
- syncmsg command, 4.1
- syncmsg command
 - /LIST option, 4.1
- syncmsg.lis file, 4.1
- syncsort command, 2.2
- SyncSort message utility, 4.1
- SyncSort options, 3.61, 3.76, 3.89, 3.153
- SyncSort options, 2.2*
 - /FIELDS, 3.76
 - /INFILE, 3.89
 - /REFORMAT, 3.153
- SyncSort subroutines, 2.7, 2.9
- SyncSort subroutines
 - Completion status, 2.9
- SYNCSORT_INSERT, 3.91, 3.92
- syncsplit, 3.118
- Terminating a SyncSort session, 3.107, 3.140
- Terminating a SyncSort session
 - /QUIT, 3.140
 - See ¶ Ctrl-C, 3.107
 - See ¶ Interrupting SyncSort, 3.107
- Terminating an application definition, 3.68
- THEN, 3.48
- TODAY, 3.48
- TOTAL, 3.185
- TP, A.1, A.3
- Truncation, 3.57
- TS, A.1, A.3
- UIINTEGER, A.1, A.3
- UNALIGNED, 3.87, 3.121
- UNCOMPRESSED, 3.85, 3.117
- UNPAIRED, 3.94
- Unsigned decimal, A.4
- Unsigned integer, A.1, A.3, A.9
- VARIABLE, D.1
- Variable length, D.4
- VSAM, 3.86, 3.88, 3.115, 3.118, 3.119, 3.122
- Work files, 3.200
- ZD, A.1, A.3
- Zoned decimal, A.1, A.3
